LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

CS6004NI Application Development

30% Individual Coursework

2025-26 Autumn

Credit: 30

Student Name: Bhishma Pratap Karki

London Met ID: 23049355

College ID: NP01CP4A230361

Assignment Due Date: Wednesday, January 28, 2026

Assignment Submission Date: Wednesday, January 28, 2026

Module Leader: Mr. Bikram Poudel

Word Count: 9332

Project File Links:

Drive Link:	https://drive.google.com/drive/folders/1G3AQ_0Q0MM12LCO8Bnqu-D6SWqsDLh09?usp=sharing
GitHub Link:	https://github.com/Bhishma-Pratap-Karki03/MyJournalProject

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Table of Figures

Table of Tables

1. Introduction

This project involves the design and development of a secure, feature-rick desktop journaling application using C#.NET. The application is created to help user maintain a digital personal journal where they can write daily entries, track their moods, and reflect their daily experiences in an organized and meaningful way.

The system allows users to create, update, and delete one journal entry per day, ensuring consistency and discipline in journaling habits. Each journal entry supports rich text formatting, making it easy for users to write structured and well-formatted content.

In addition to writing, the application supports mood tracking, tagging, categorization, analytics, and security features, making it more than just a simple diary. It also provides tools such as calendar navigation, search, filters, streak tracking, dashboards, and PDF export, which help users analyse their emotional patterns and journaling behavior over time.

The application is designed to provide a modern, organized and secure digital journaling experience, allowing users to store personal information safely while gaining meaningful insights from their daily entries.

1.1. Project Purpose and Scope

1.1.1. Project Purpose

The purpose of this project is to develop a secure and user-friendly desktop journaling application that allows users to record their daily thoughts, emotions, and experiences in a structured digital format. The application is designed to support daily journaling habits, improve self-reflection, and help users understand their emotional patterns through mood tracking and analytics.

The system aims to provide a modern alternative to traditional paper-based journals by offering digital features such as search, filtering and analytics. By using a digital platform, users can easily organize their entries, access past records, and gain meaningful insights from their journaling data.

1.1.2. Project Scope

The scope of this project includes the design and development of a secure desktop journaling application that allows users to create, update, and delete one journal entry per day. Each entry supports rich-text formatting to help users write structured and well-formatted content. The application also includes a mood tracking system, allowing users to select one primary mood and up to two optional secondary moods, along with a tagging system using both custom and predefined tags to categorize entries effectively.

To help users organize and navigate their journal entries, the system provides category-based organizational, a calendar view for date-based navigation, and a paginated journal list view for easy browsing. Users can also take advantage of search and filtering features to find entries by content, date range, moods, or tags. Additionally, the system tracks streaks, including daily streaks, longest streaks, and missed days, providing insights into journaling consistency.

The application includes a dashboard and analytics pages to give users visual insights into mood patterns, tags usage, writing trends, and other journaling behaviours. All data is stored locally using an SQLite database, and the system ensures security through user registration with unique email address and secure login with email and password. Users can also export journal entries as PDF files based on a date range, and the application supports theme customization with light and dark modes for personalized interface.

1.1.3. Technology Stack Selection

The project uses a carefully chosen technology stack to build a secure, feature-rich desktop journaling application. The main framework is .NET MAUI Blazor Hybrid, which enables the creation of modern desktop application with a responsive and interactive user interface. This framework allows combining C# backend logic with Blazor web components for a seamless user experience.

For data storage, the application uses SQLite as the local database. SQLite is lightweight, reliable, and easy to integrate, making it ideal for storing journal entries,

user information, moods, and tags locally on the user's device. To simplify database operations, Entity Framework Core is used as an Object-Relational Mapper (ORM), which allows secure and efficient communication between application and the SQLite database.

The application provides analytics and visual insights using Chart.js, which generates charts for mood distribution, tag usage, and word count trends. For rich-text edition of journal content, Quill.js is implemented, enabling users to format their entries with features such as bold, italics, lists, and headings. To allow users to export their journals, iTextSharp is used to generate PDF files based on selected date ranges. Additionally, JavaScript Interop is used to facilitate communication between C# code and JavaScript components, supporting features like the rich-text editor and chart rendering.

2. Features and Functionalities

2.1. Features Breakdown

Table 1: Features Breakdown

No.	Feature	Description	Status
1	Journal Entry Management	Create, update and delete one journal entry per day	Completed
2	Rich Text Editor	Formatted text writing support	Completed
3	Mood Tracking	Primary and optional secondary mood selection	Completed
4	Tagging System	Custom and predefined tagging support	Completed
5	Category Management	Category-based entry organization	Completed
6	Calendar View	Date-based journal navigation	Completed
7	Paginated Journal View	Paginated browsing of entries	Completed
8	Search and Filter	Content based filtering	Completed
9	Streak Tracking	Daily and longest streak tracking	Completed
10	Dashboard	Summary view of journaling activity	Completed
11	Analytics Page	Visual charts and data insights	Completed
12	User Registration and Login	Secure registration and login system	Completed
13	Security	Password-protected	Completed
14	PDF Export	Date-range PDF export feature	Completed
15	Local Storage	Local data storage using SQLite	Completed
16	Theme Customization	Light and dark mode support	Completed

2.2. Features Description

2.2.1. Journal Entry Management

A complete CRUD system that allows users to create, update, and delete journal entries with strict “one entry per day” rule. Each entry automatically captures system-generated timestamps (CreatedAt, UpdatedAt) and checks for duplicate dates using database constraints. The system prevents creating new entries for past and future dates, while still allowing users to edit or delete existing past entries, and maintains data accuracy and consistency through Entity Framework Core.

Implementation: JournalService.cs, Journal.razor

- JournalService.cs

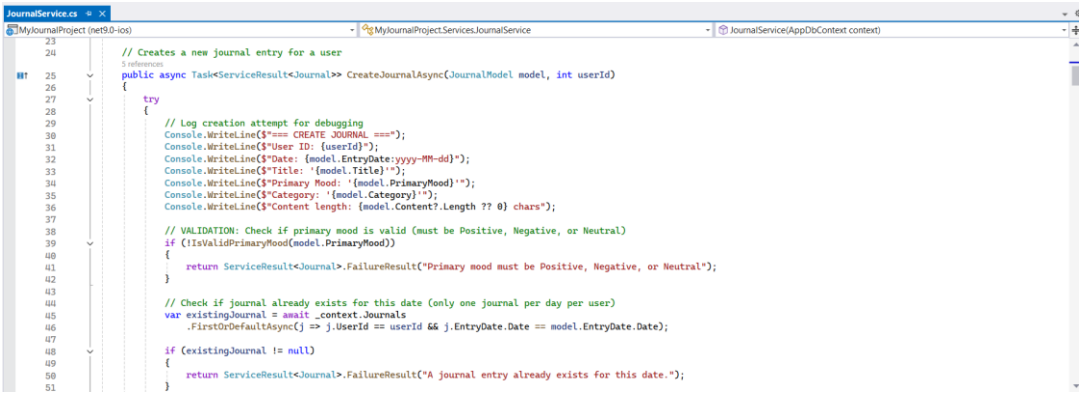


Figure 1: JournalService.cs for Journal Entry Management

- Journal.razor

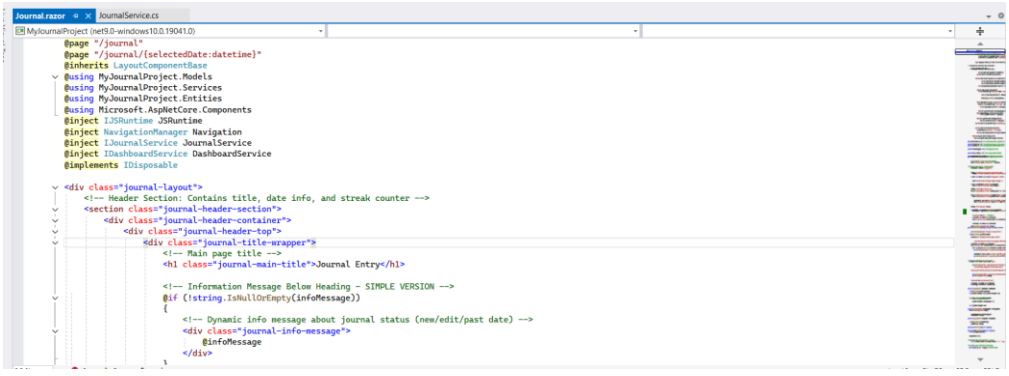


Figure 2: Journal.razor for Journal Entry Management

- Page for Journal Entry:

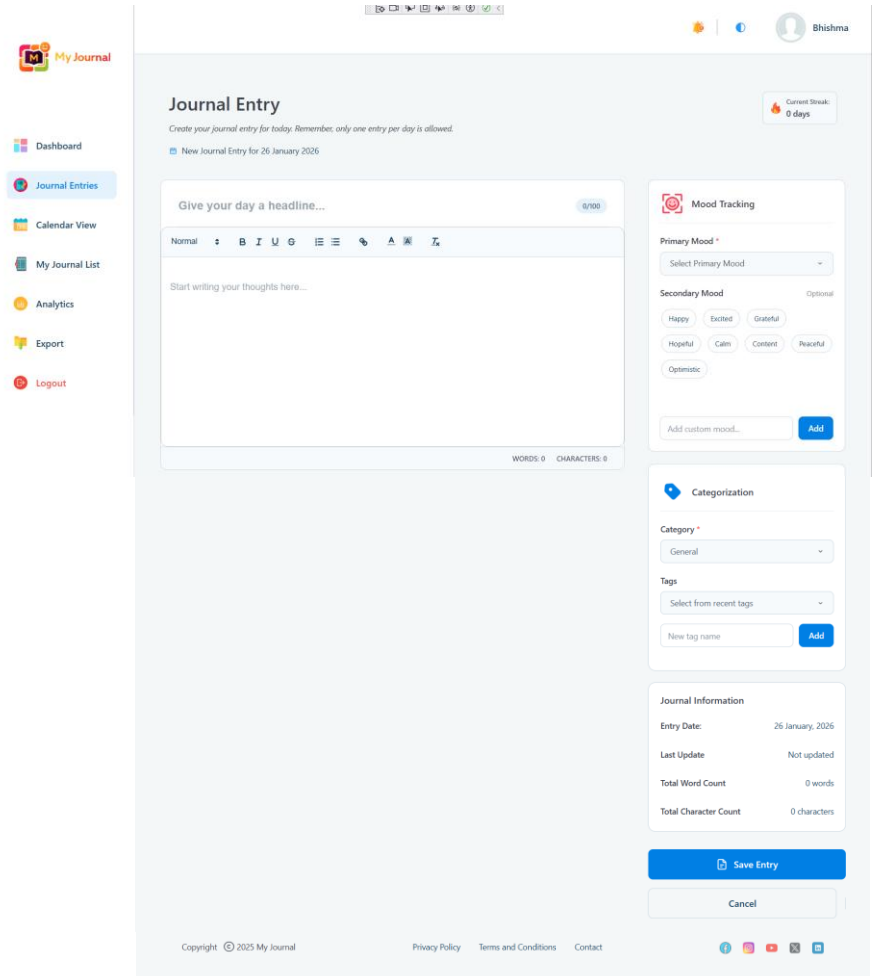


Figure 3: Page for Journal Entry

- Page for updating a journal entry after the user successfully creates the journal for the current date.

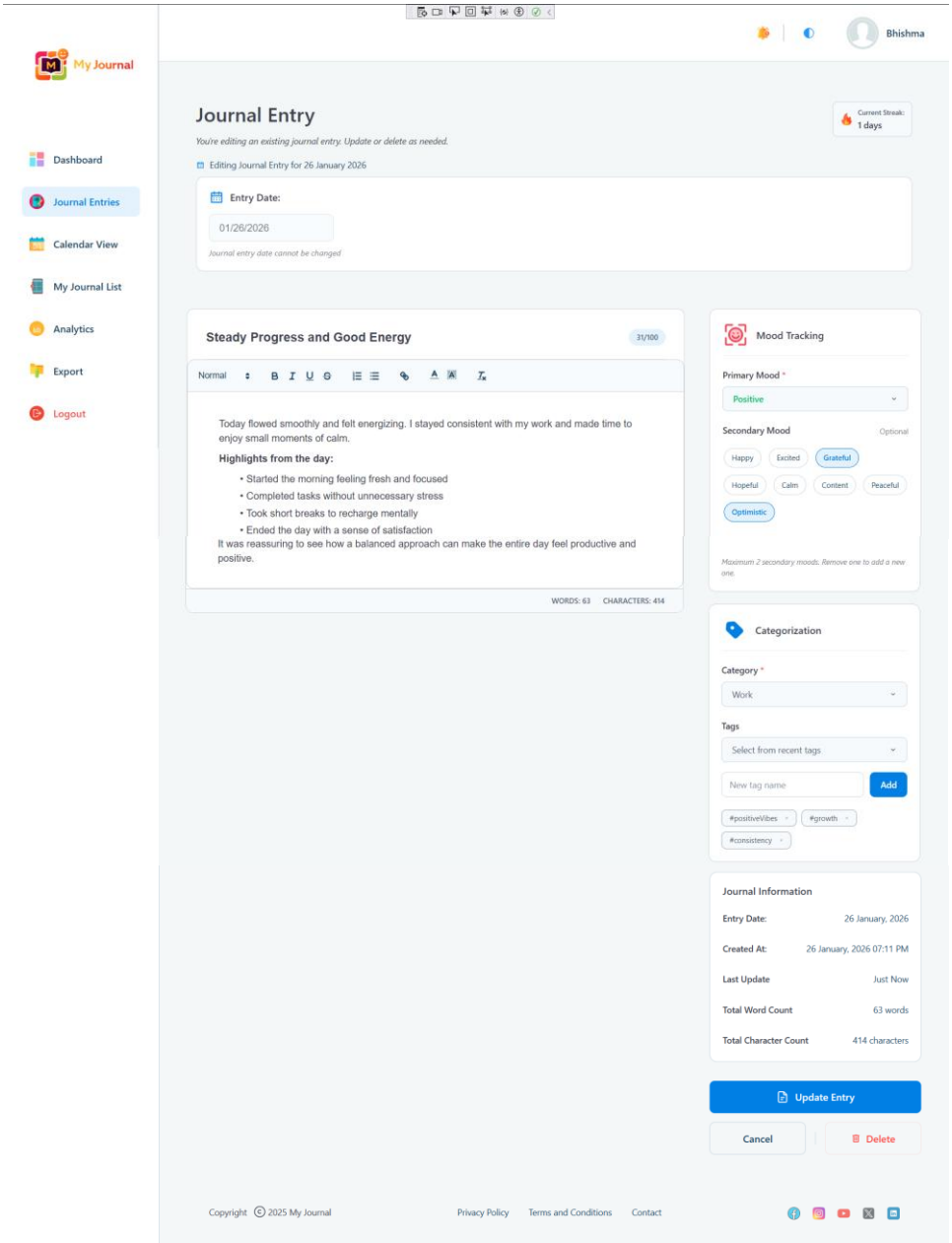


Figure 4: Update Entry

- Journal delete option

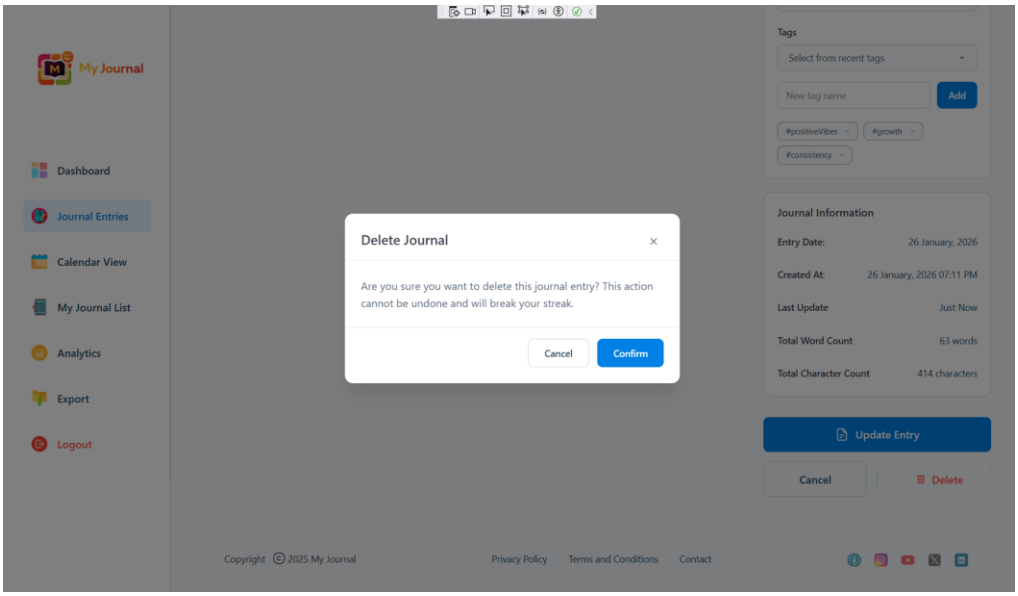


Figure 5: Deleting the entered journal

2.2.2. Rich Text Editor

The application integrates Quill.js as a rich text editor, allowing users to format their journal entries with bold, italics, underline, strikethrough, headers (H1-H3), ordered and bulleted lists, hyperlinks, text colour, and background colour. It also shows real-time word and characters counts and store the content safely in HTML format with sanitization to prevent harmful elements.

Implementation: quill-editor.js, Journal.razor

- quill-editor.js

Figure 6: quill-editor.js Rich Text Editor

- Journal.razor

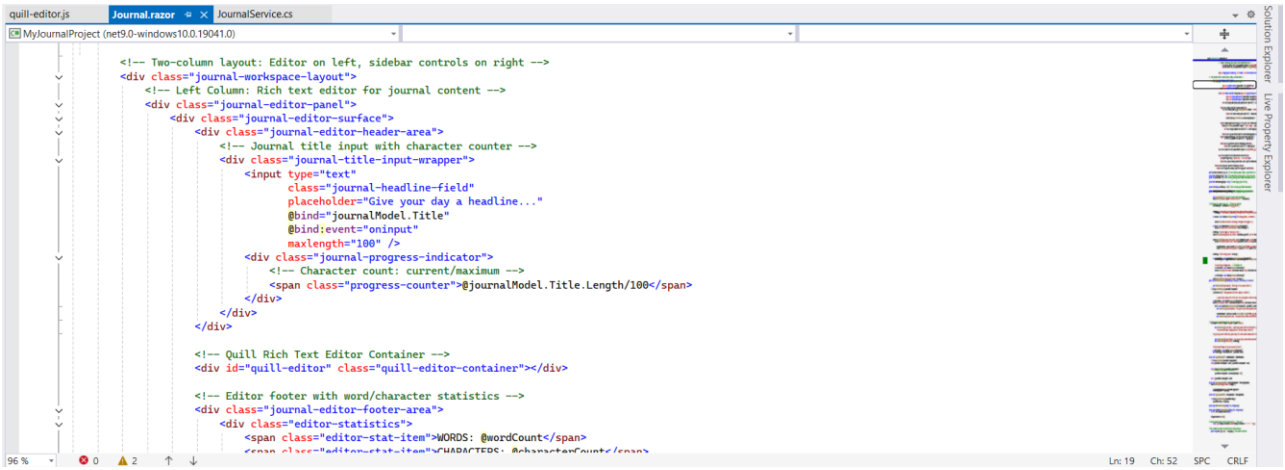


Figure 7: Journal.razor for Rich Text Editor

- Rich text editor interface

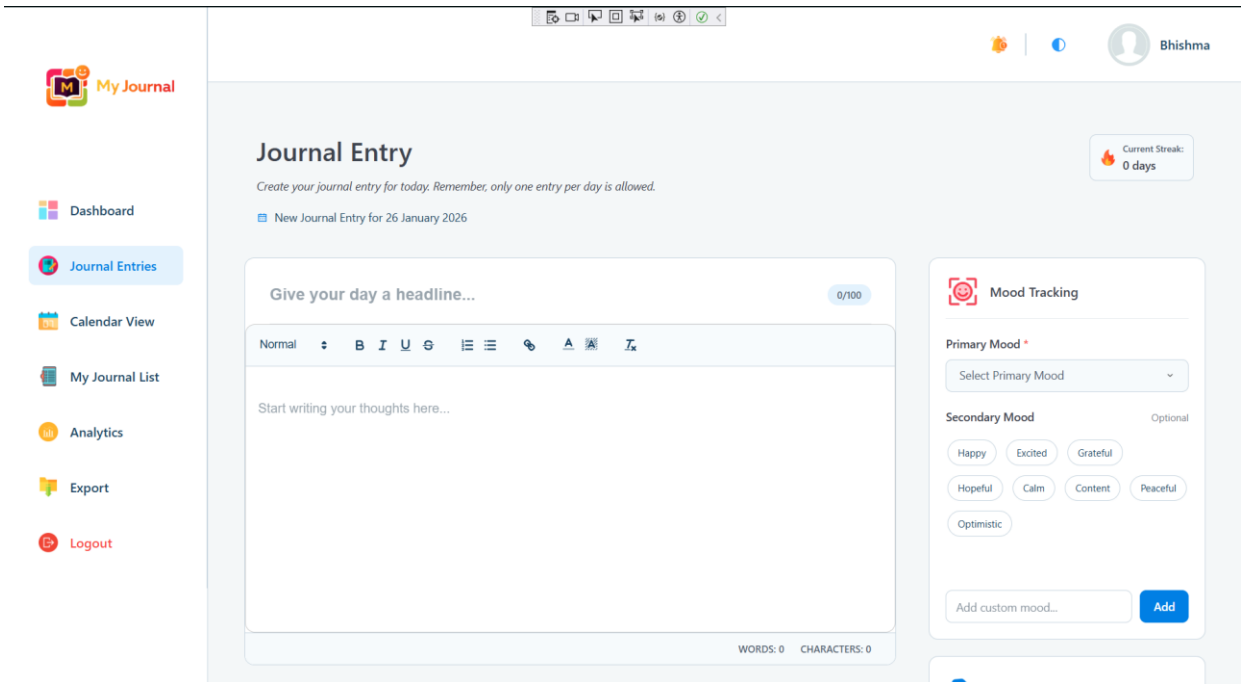


Figure 8: Rich text editor interface

- Example of formatted text in a journal entry

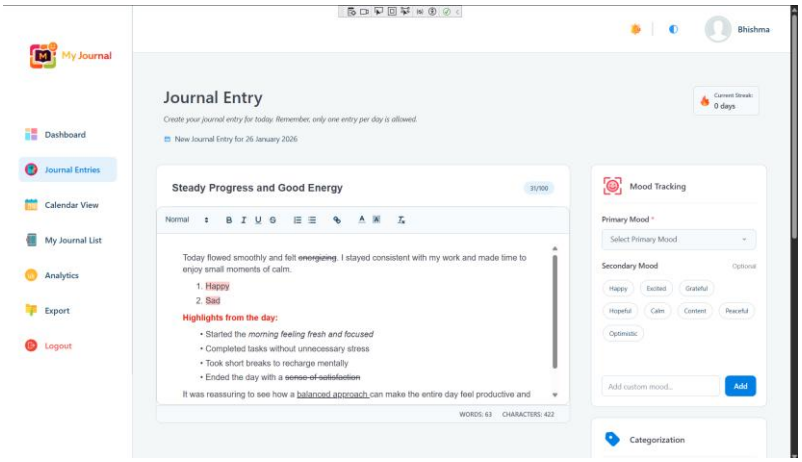


Figure 9: Example of formatted text in a journal entry

2.2.3. Mood Tracking

A dual-layer mood tracking system requiring users to select one primary mood (Positive, Negative, Neutral) and up to two optional secondary moods. Secondary moods include predefined emotions (Happy, Sad, Anxious, etc.) or custom user-defined moods. Mood information is used in calendar views, analytics, dashboards, and PDF export.

Implementation: Journal.razor, JournalService.cs, charts.js, DashboardService.cs, Dashboard.razor, Analytics.razor

- Journal.razor

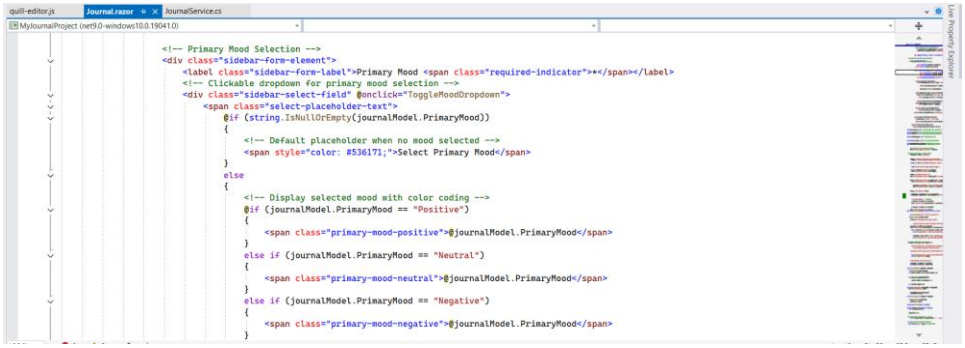
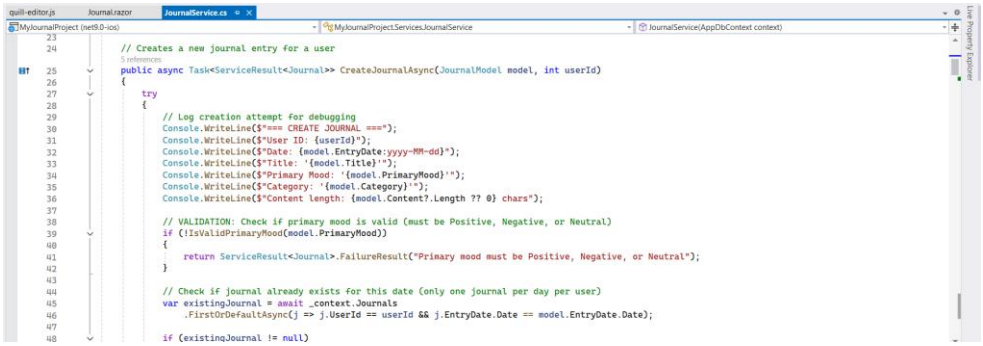


Figure 10: Journal.razor with Mood Tracking or Mood Selection

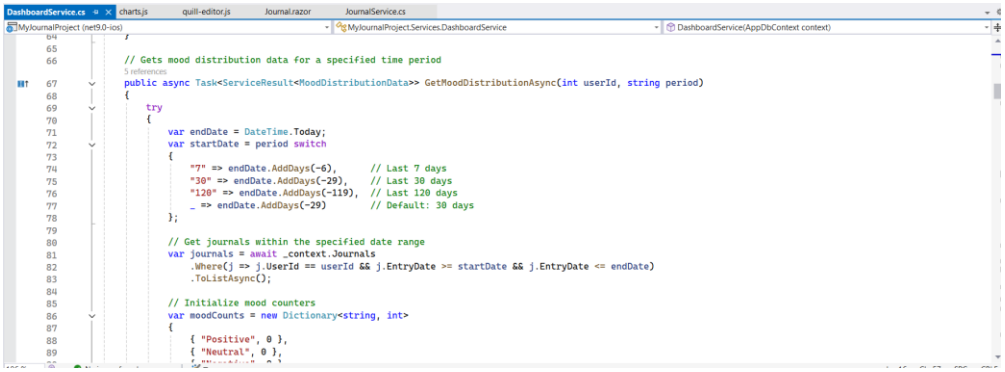
- JournalService.cs



```
// Creates a new journal entry for a user
public async Task<ServiceResult<Journal>> CreateJournalAsync(JournalModel model, int userId)
{
    try
    {
        // Log creation attempt for debugging
        Console.WriteLine($"=== CREATE JOURNAL ===");
        Console.WriteLine($"User ID: {userId}");
        Console.WriteLine($"Date: {model.EntryDate:yyyy-MM-dd}");
        Console.WriteLine($"Title: '{model.Title}'");
        Console.WriteLine($"Primary Mood: '{model.PrimaryMood}'");
        Console.WriteLine($"Category: '{model.Category}'");
        Console.WriteLine($"Content length: {model.Content?.Length ?? 0} chars");

        // VALIDATION: Check if primary mood is valid (must be Positive, Negative, or Neutral)
        if (!IsValidPrimaryMood(model.PrimaryMood))
        {
            return ServiceResult<Journal>.FailureResult("Primary mood must be Positive, Negative, or Neutral");
        }

        // Check if journal already exists for this date (only one journal per day per user)
        var existingJournal = await _context.Journals
            .FirstOrDefaultAsync(j => j.UserId == userId && j.EntryDate.Date == model.EntryDate.Date);

        if (existingJournal != null)
```

Figure 11: JournalService.cs

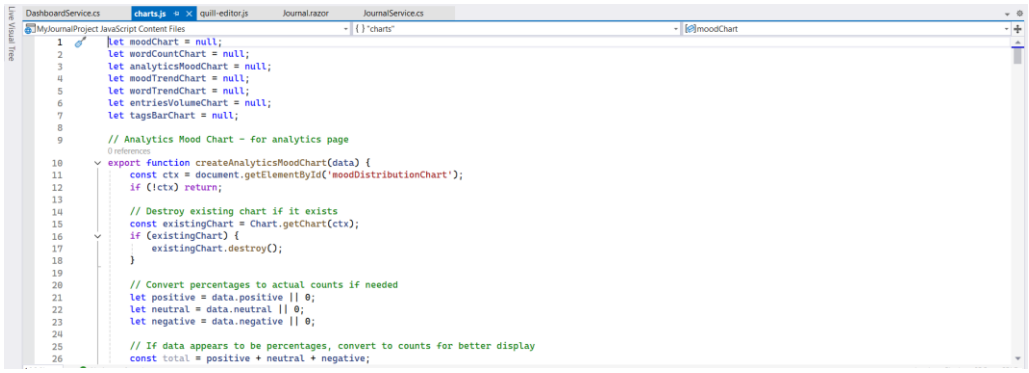
- DashboardService.cs



```
// Gets mood distribution data for a specified time period
public async Task<ServiceResult<MoodDistributionData>> GetMoodDistributionAsync(int userId, string period)
{
    try
    {
        var endDate = DateTime.Today;
        var startDate = period switch
        {
            "7" => endDate.AddDays(-6),      // Last 7 days
            "30" => endDate.AddDays(-29),    // Last 30 days
            "120" => endDate.AddDays(-119),  // Last 120 days
            _ => endDate.AddDays(-29)        // Default: 30 days
        };

        // Get journals within the specified date range
        var journals = await _context.Journals
            .Where(j => j.UserId == userId && j.EntryDate >= startDate && j.EntryDate <= endDate)
            .ToListAsync();

        // Initialize mood counters
        var moodCounts = new Dictionary<string, int>
        {
            { "Positive", 0 },
            { "Neutral", 0 },
```

Figure 12: DashboardService.cs with Mood distribution chart

- charts.js



```
let moodChart = null;
let wordCountChart = null;
let analyticsMoodChart = null;
let moodTrendChart = null;
let wordTrendChart = null;
let entriesVolumeChart = null;
let tagsBarChart = null;

// Analytics Mood Chart - for analytics page
export function createAnalyticsMoodChart(data) {
    const ctx = document.getElementById('moodDistributionChart');
    if (!ctx) return;

    // Destroy existing chart if it exists
    const existingChart = Chart.getChart(ctx);
    if (existingChart) {
        existingChart.destroy();
    }

    // Convert percentages to actual counts if needed
    let positive = data.positive || 0;
    let neutral = data.neutral || 0;
    let negative = data.negative || 0;

    // If data appears to be percentages, convert to counts for better display
    const total = positive + neutral + negative;
```

Figure 13: charts.js

- Dashboard.razor

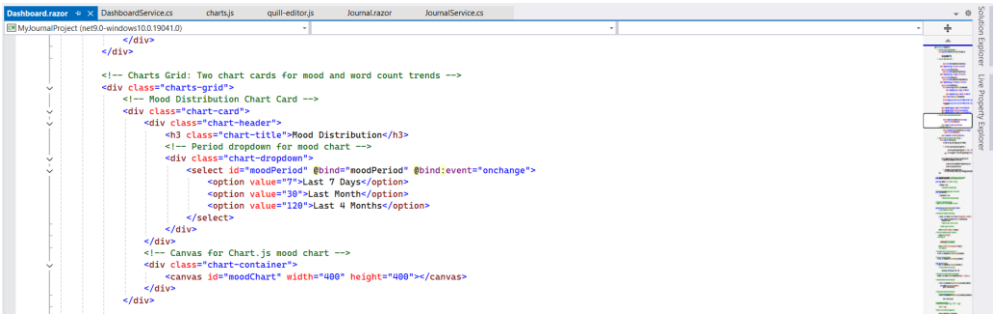


Figure 14: Dashboard.razor with Mood Distribution Chart

- Analytics.razor

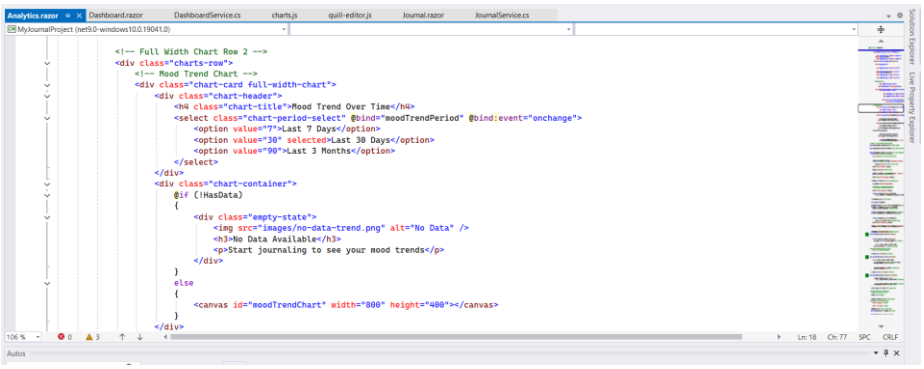


Figure 15: Analytics.razor with Mood Tracking

- Mood visualization in dashboard page

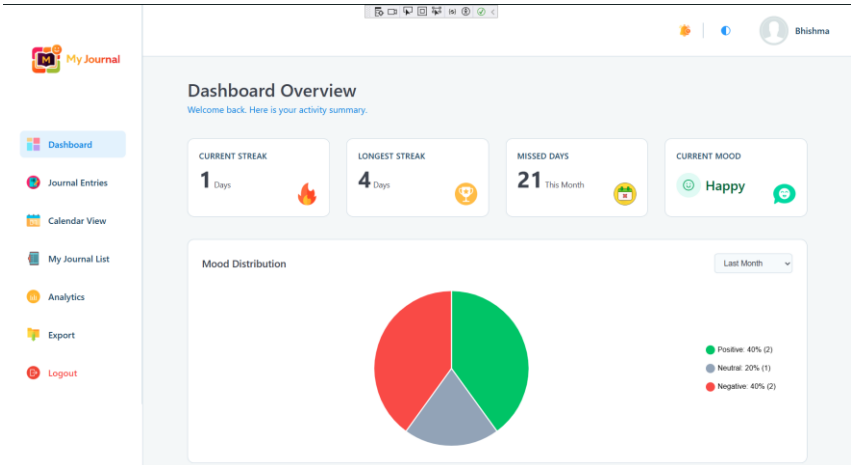


Figure 16: Mood Visualization in dashboard

- Mood selection in journal entry page

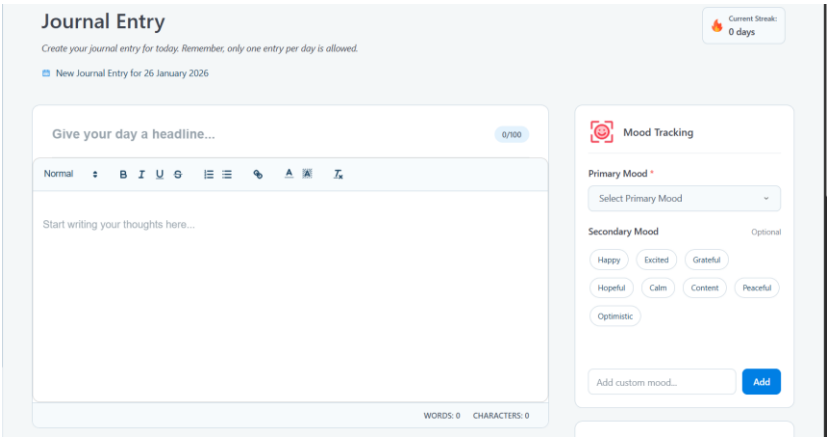


Figure 17: Mood Selection in Journal Entry Page

- Mood visualization in analytics page

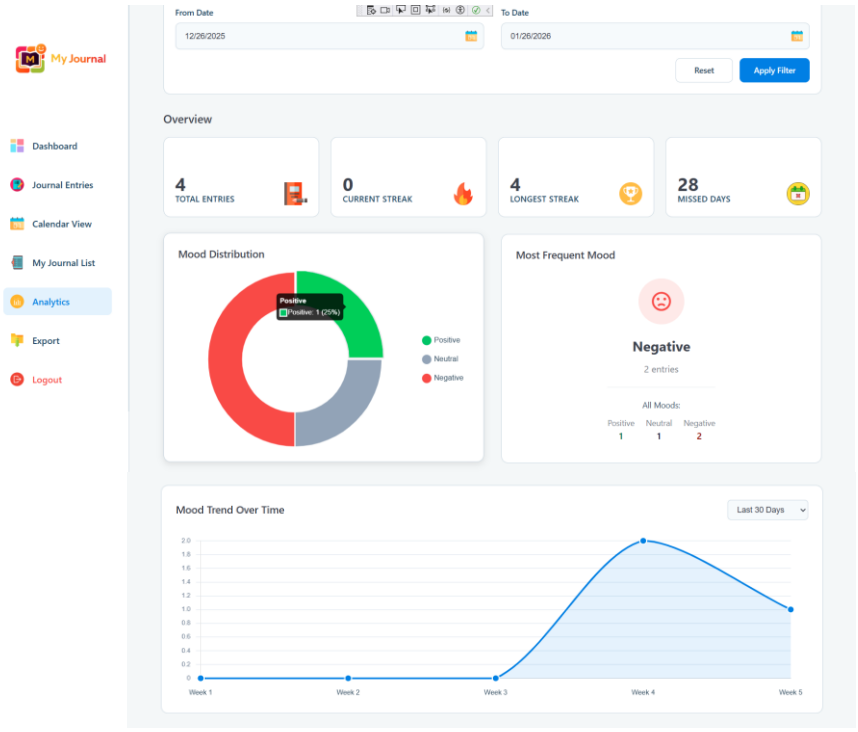


Figure 18: Mood visualization in analytics page

2.2.4. Tagging System

User can add custom tags or select from previously used tags while creating journal entries. Tags are stored as JSON arrays in the database for efficient querying, filtering, and analytics. Auto-suggestions help users pick existing tags to maintain consistency.

Implementation: Journal.razor, JournalService.cs

- Journal.razor

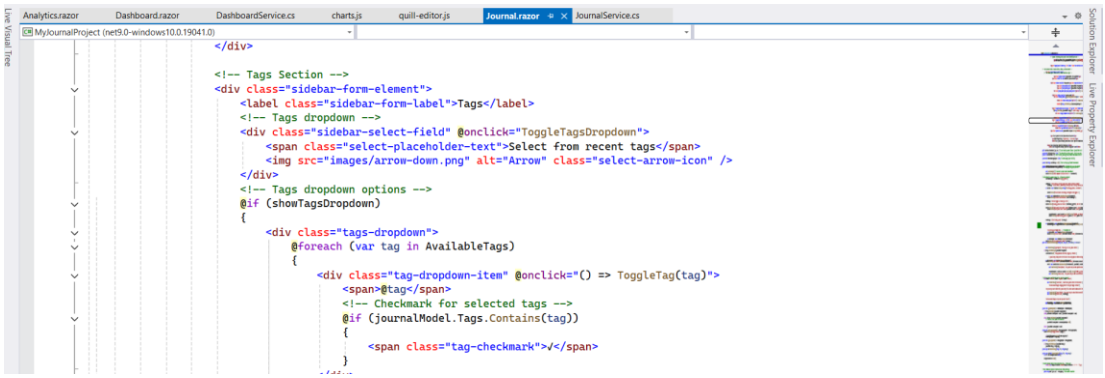


Figure 19: Journal.razor with Tag Selection

- JournalService.cs

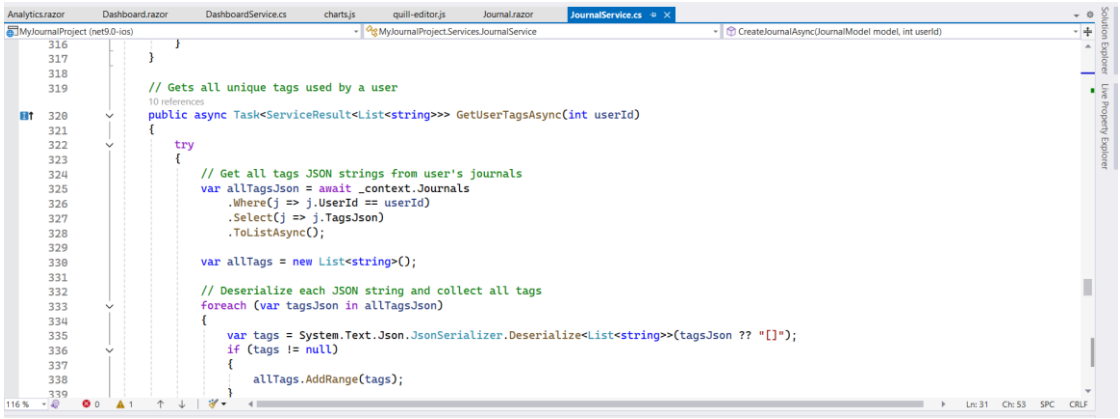


Figure 20: JournalService.cs for getting all unique tags used by a user

- Tag input filed and suggested tags based on previous entries during journal creation

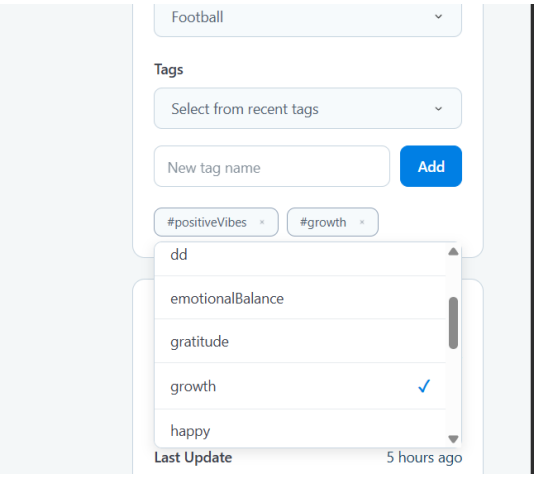


Figure 21: Tag input filed and suggested tags

2.2.5. Category Management

The system provides predefined categories (General, Work, Personal, Health, Travel, etc) and supports custom category creation. Categories help organize journal entries and are used in filtering, analytics and PDF export.

Implementation: Journal.razor, JournalService.cs

- Journal.razor

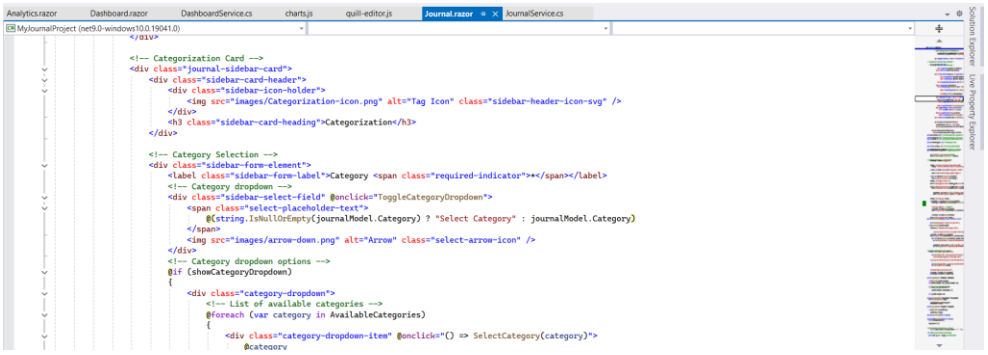


Figure 22: Journal.razor with Category Selection

- JournalService.cs

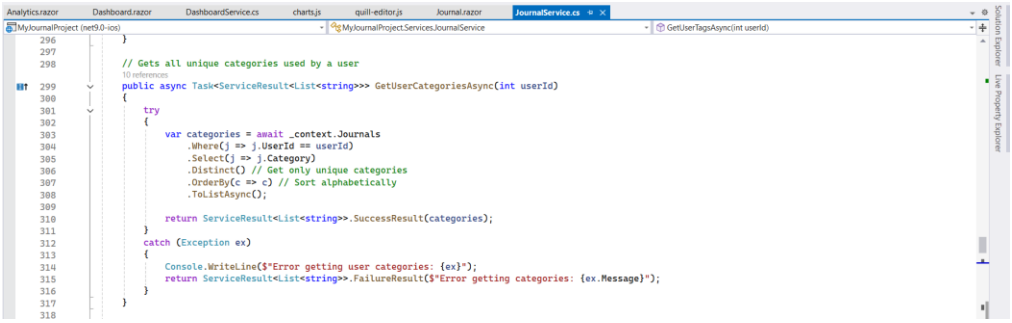


Figure 23: JournalService to get all unique categories used by a users

- Predefined categories selection during journal creation

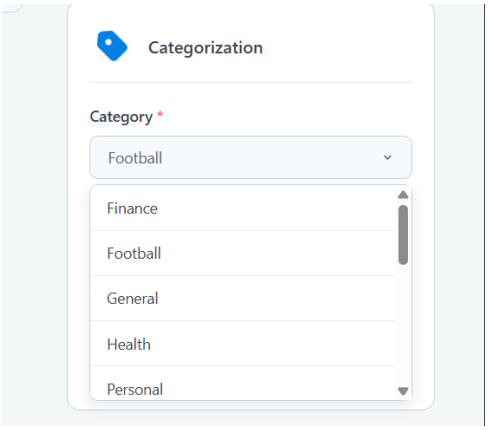


Figure 24: Predefined categories selection

- Custom category creation during journal creation

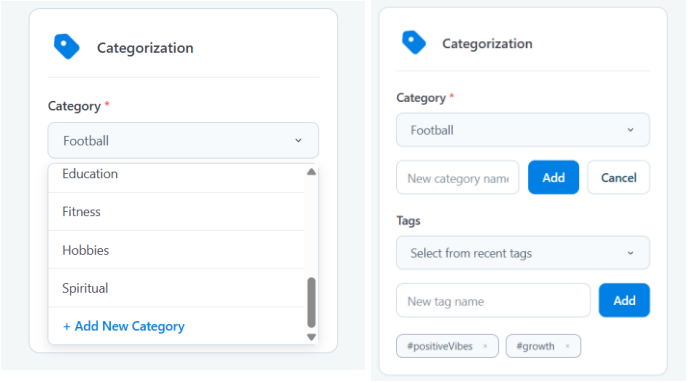


Figure 25: Custom category creation

2.2.6. Calendar View

An interactive calendar shows journal entries with mood-based colour coding (green for positive, red for negative and grey for neutral). User can click on a date to view or edit entries. Missed days are highlighted, and month/year navigation is supported.

Implementation: Calendar.razor

- Calendar.razor

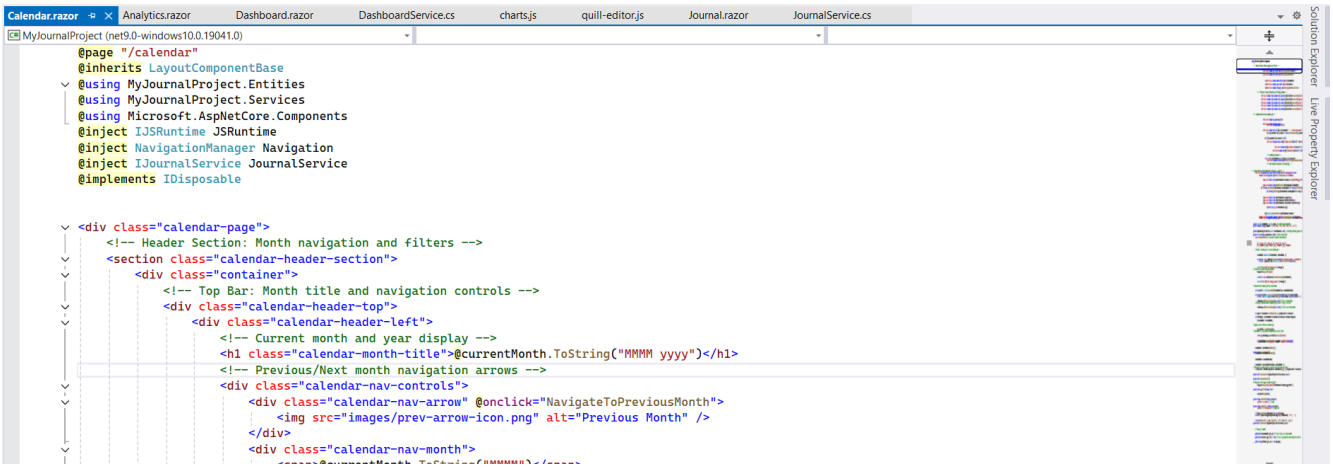


Figure 26: Calendar.razor implementing Calendar View

- Monthly calendar with coloured mood indicators and clickable journal entries for a selected date

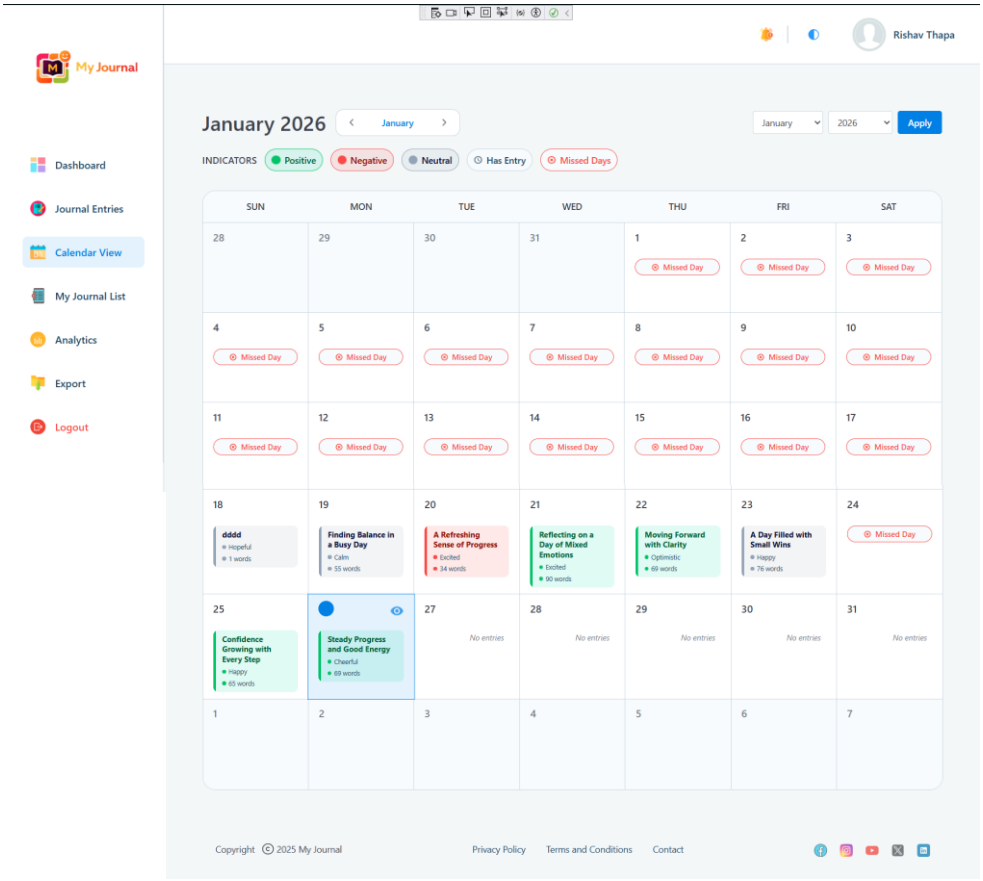


Figure 27: Calendar View

2.2.7. Paginated Journal View

The journal list view is designed for easy browsing with pagination, showing 7 entries per page. Users can navigate using first/previous/next/last buttons and page numbers. Each entry card displays the date, title, mood badges, a content preview, and word count.

Implementation: MyJournalList.razor

- MyJournalList.razor

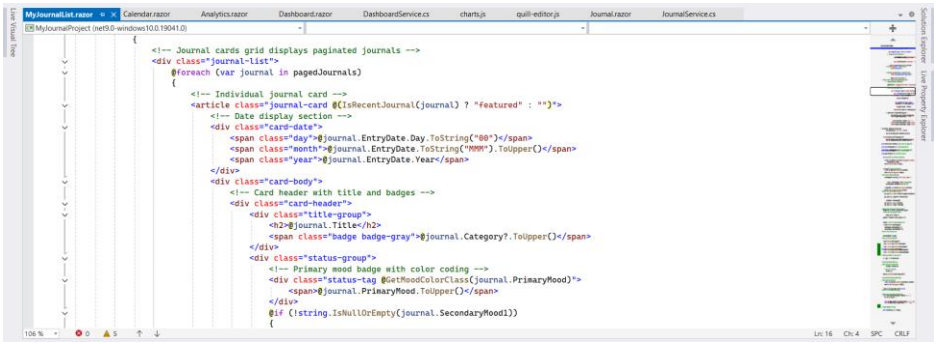


Figure 28: MyJournalList.razor with Paginated Journal View

- Paginated journal list view of page number 1

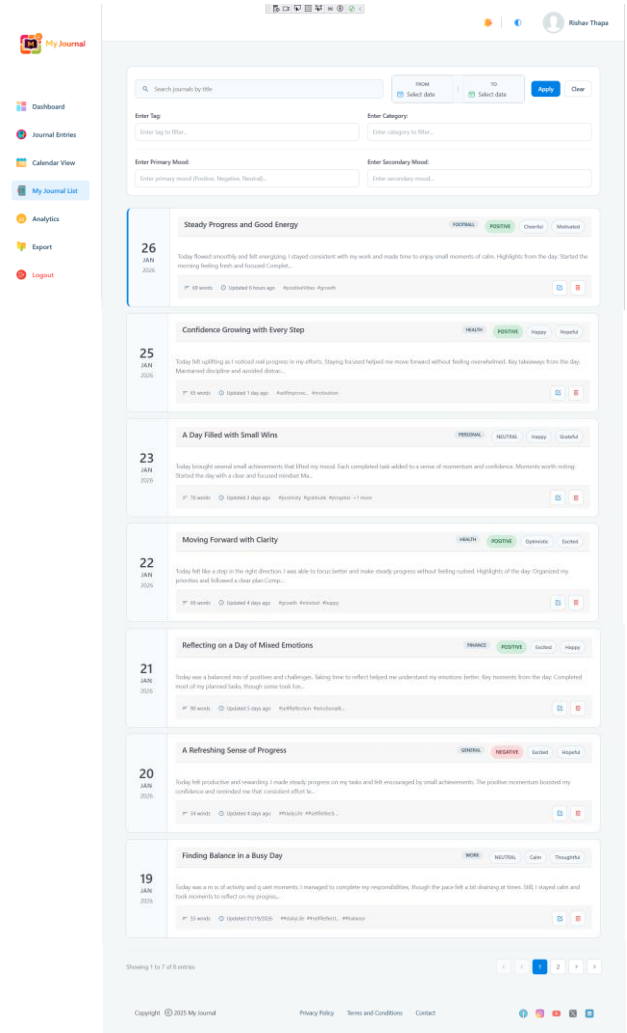


Figure 29: Paginated journal list view of Page 1

- Paginated journal list view of page number 2

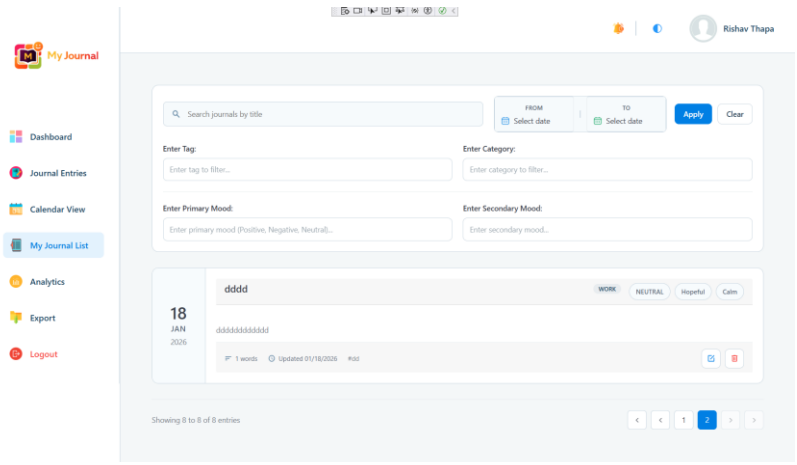


Figure 30: Paginated journal list view of page number 2

2.2.8. Search and Filter

The application provides a powerful search and filtering system that allows users to easily find journal entries using multiple criteria. Users can search by text in journal titles, filter entries by date range, primary mood, secondary mood, category, and tags. Multiple filters can be applied to together to refine results more accurately. The filtering system updates results in real time, making it quick and easy for users to locate specific journal entries without manually browsing through the entire journal list.

Implementation: MyJournalList.razor

- MyJournalList.razor

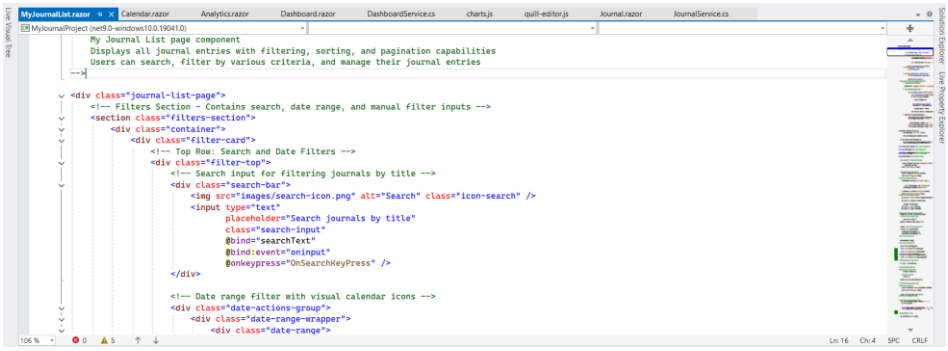


Figure 31: MyJournalList.razor with Search and Filter options

- Searching Journal with the title

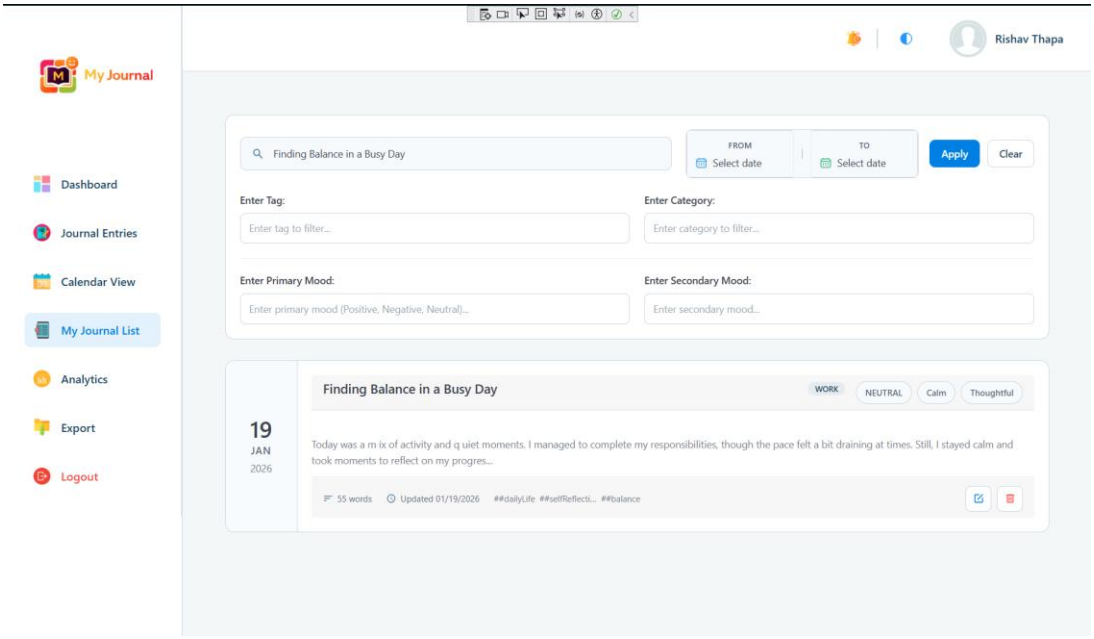


Figure 32: Searching Journal with the title

- Applying Filters with Date Range

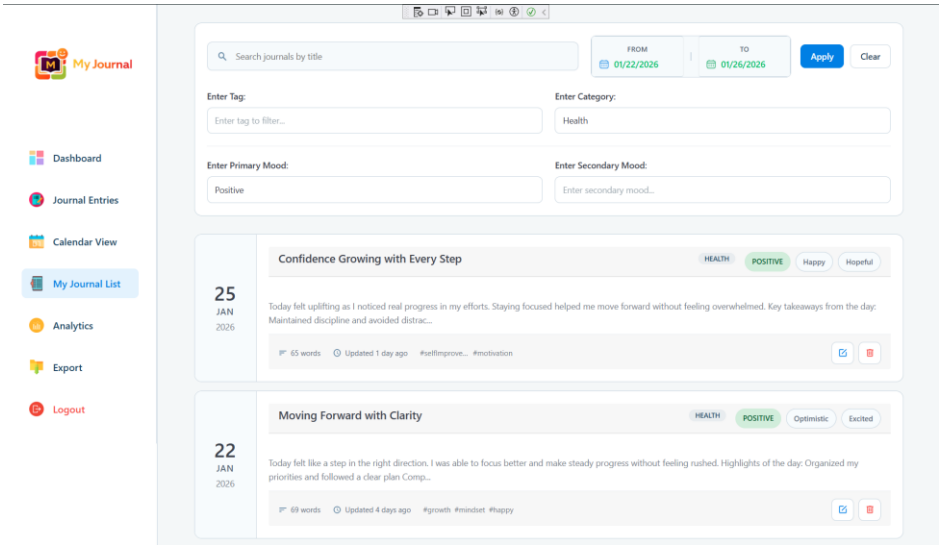


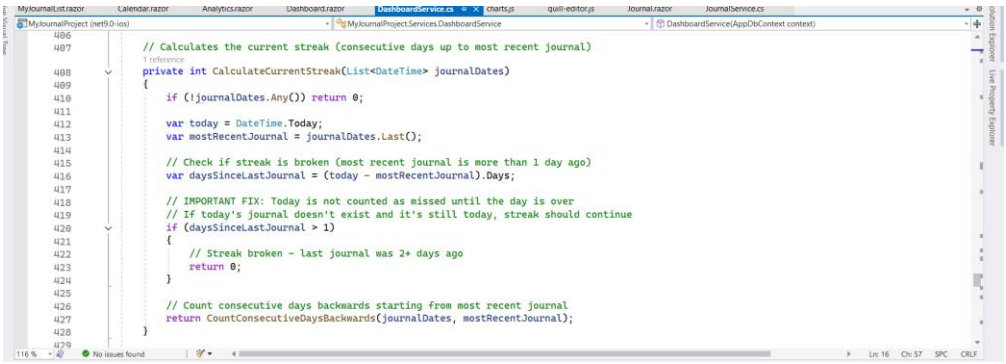
Figure 33: Applying Filters with Date Range

2.2.9. Streak Tracking

Tracks current consecutive journaling days and longest streak. Missed days are calculated and displayed to motivate consistent journaling. The dashboard shows streak with visual indicators. The similar overview is also kept in analytics page. The user can also see the current streak in the Journal Entry Page

Implementation: DashboardService.cs, Dashboard.razor

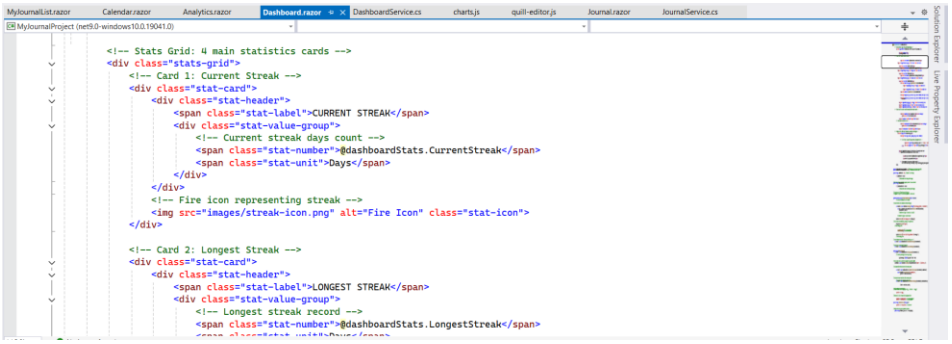
- DashboardService.cs



```
// Calculates the current streak (consecutive days up to most recent journal)
1 reference
private int CalculateCurrentStreak(List<DateTime> journalDates)
{
    if (!journalDates.Any()) return 0;

    var today = DateTime.Today;
    var mostRecentJournal = journalDates.Last();

    // Check if streak is broken (most recent journal is more than 1 day ago)
    var daysSinceLastJournal = (today - mostRecentJournal).Days;

    // IMPORTANT FIX: Today is not counted as missed until the day is over
    // If today's journal doesn't exist and it's still today, streak should continue
    if (daysSinceLastJournal > 1)
    {
        // Streak broken - last journal was 2+ days ago
        return 0;
    }

    // Count consecutive days backwards starting from most recent journal
    return CountConsecutiveDaysBackwards(journalDates, mostRecentJournal);
}
```

Figure 34: DashboardService.cs with Streak Tracking

- Dashboard.razor



```
<!-- Stats Grid: 4 main statistics cards -->
<div class="stats-grid">
    <!-- Card 1: Current Streak -->
    <div class="stat-card">
        <div class="stat-header">
            <span class="stat-label">CURRENT STREAK</span>
            <div class="stat-value-group">
                <!-- Current streak days count -->
                <span class="stat-number">@dashboardStats.CurrentStreak</span>
                <span class="stat-unit">Days</span>
            </div>
        </div>
        <!-- Fire icon representing streak -->
        <img src="images/streak-icon.png" alt="Fire Icon" class="stat-icon">
    </div>

    <!-- Card 2: Longest Streak -->
    <div class="stat-card">
        <div class="stat-header">
            <span class="stat-label">LONGEST STREAK</span>
            <div class="stat-value-group">
                <!-- Longest streak record -->
                <span class="stat-number">@dashboardStats.LongestStreak</span>
```

Figure 35: Dashboard.razor with current streak, longest streak and missed days

- Current and Longest streak display with highlighted missed days

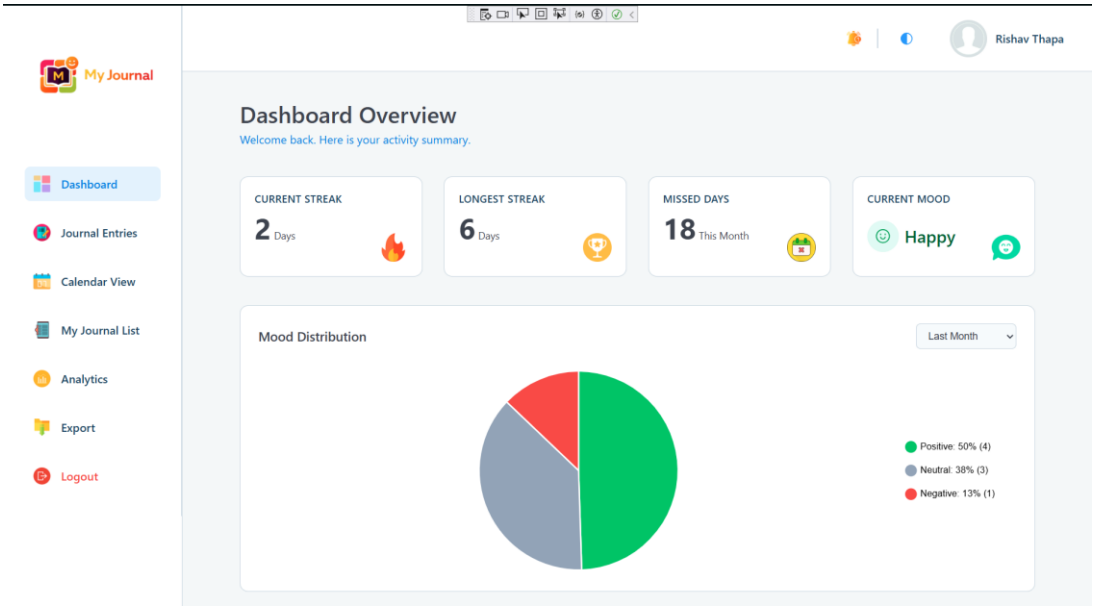


Figure 36: Current and Longest streak display with highlighted missed days

2.2.10. Dashboard

The dashboard provides a central overview of the user's journaling activity and emotional trends in a clear and visual format. It displays the current streak with a fire icon, the longest streak with a trophy icon, the number of missed days, and the user's current mood using visual indicators. The dashboard also includes visual analytics such as mood distribution pie chart for the last 7, 30 and 120 days, a word count trend line chart, and a top tags usage. In addition, motivational message is shown to encourage consistent journaling. This dashboard helps users easily understand their journaling habits, emotional patterns, and overall progress in one place.

Implementation: Dashboard.razor, DashboardService.cs

- Dashboard Showing all analytics sections with mood distribution and word count trend visualization

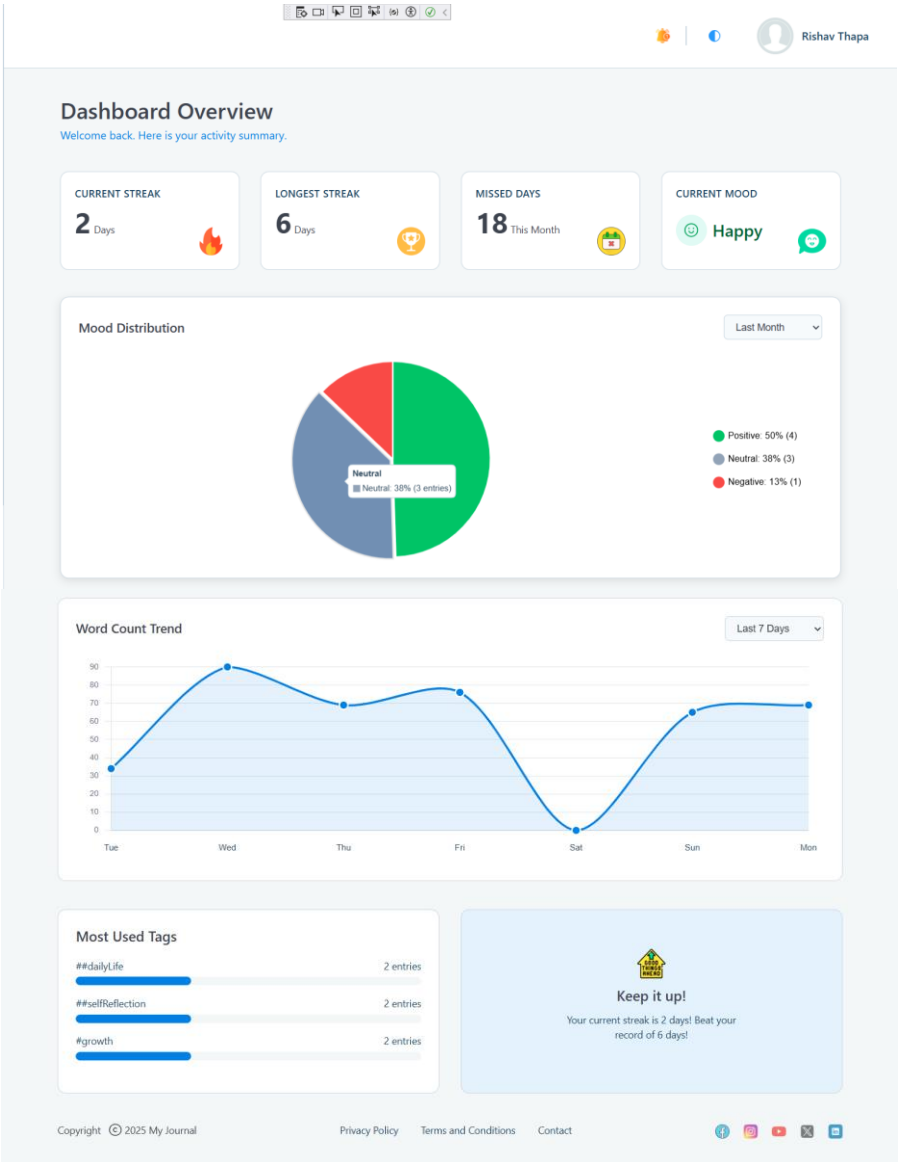


Figure 37: Dashboard Showing all analytics sections

2.2.11. Analytics Page

The analytics page provides detailed visual insights into the user's journaling patterns and emotional trends in a clear and easy to understand format. It displays mood distribution using doughnut charts, shows the most frequent moods, and presents mood trends over selected time periods such as 7, 30 and 90 days. The page also includes word count trend charts, entry volume charts grouped weekly and monthly, and tag usage charts with category-based breakdowns. Users can apply date-range filters to analyse specific time periods, making it easier to understand writing habits, emotional patterns, and journaling consistency.

Implementation: Analytics.razor, DashboardService.cs

- Analytics.razor

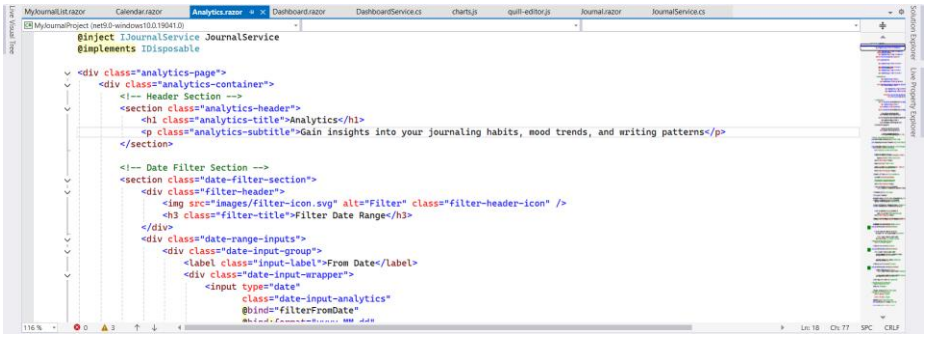


Figure 38: Analytics.razor

- DashboardService.cs to get analytics data

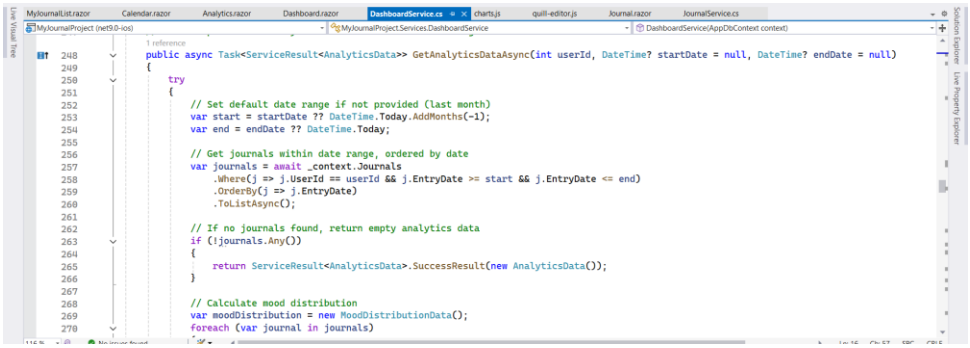


Figure 39: DashboardService.cs to get analytics data

- Analytics page overview and charts showing mood trends and tags usage.

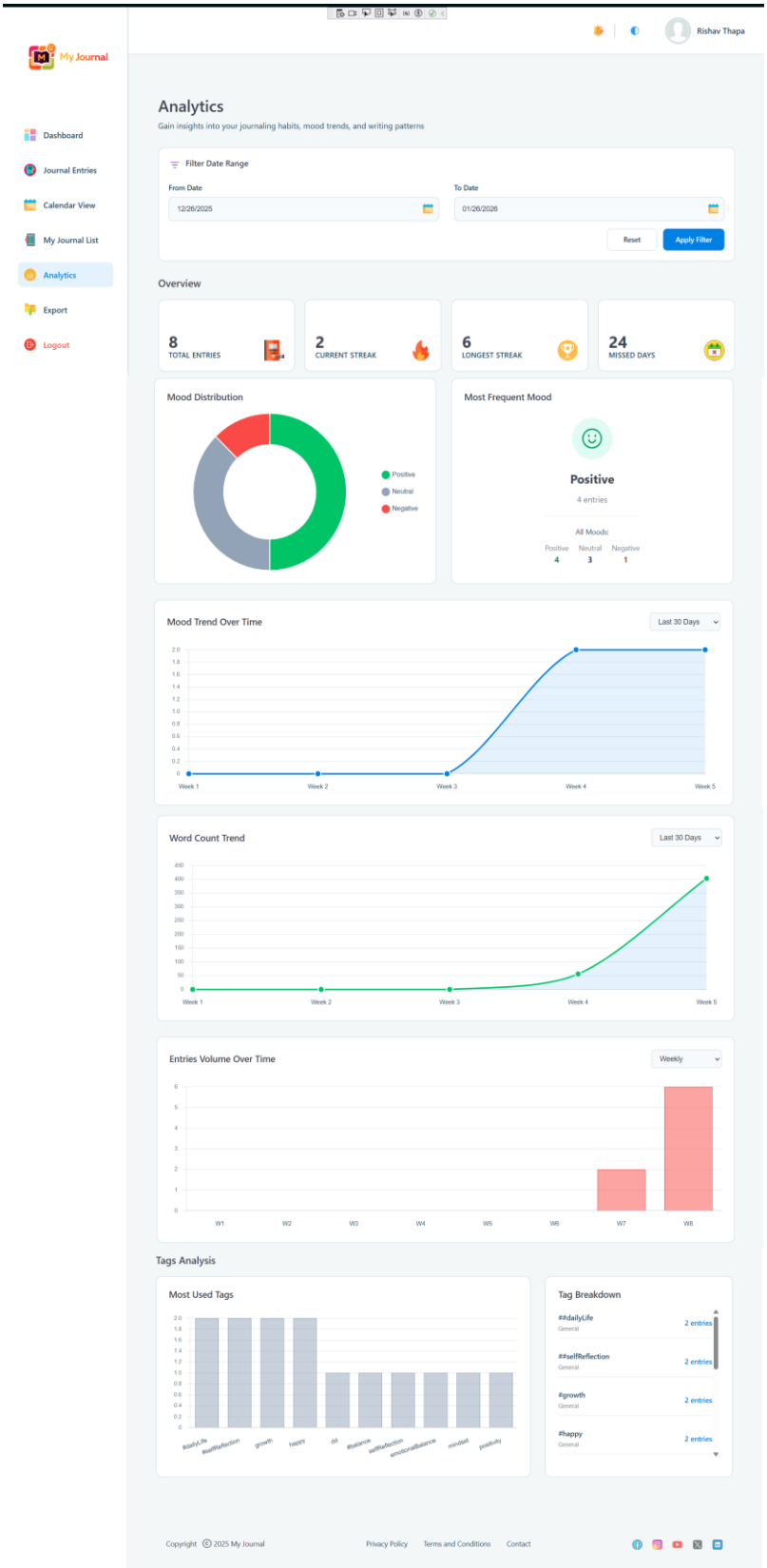


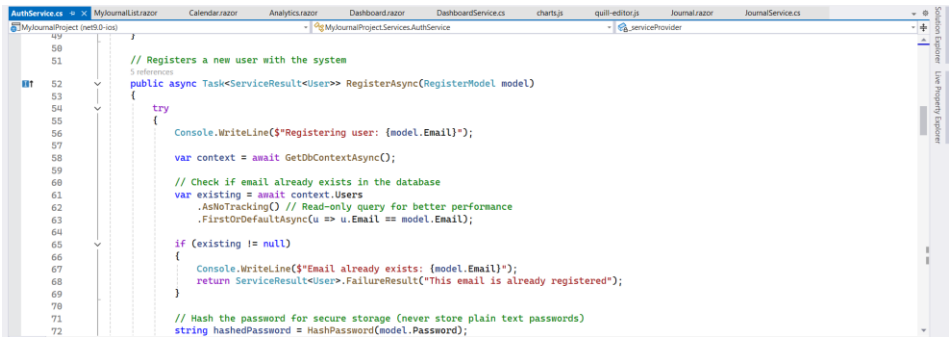
Figure 40: Analytics page

2.2.12. User Registration and Login

The application incudes a secure user registration and login system that allows users to create an account using a unique email address and password. During registration, the system validates the email format, checks for duplicate email addresses, and ensures password confirmation before account creation. Passwords are securely stored using hashing for protection. The login system verifies user credentials before granting access to the application. Protected pages are accessible only after successful login, and users are automatically redirected to the login page if they are not authenticated. This system ensures secure access and proper user identification throughout the application.

Implementation: AuthService.cs, Register.razor, Login.razor

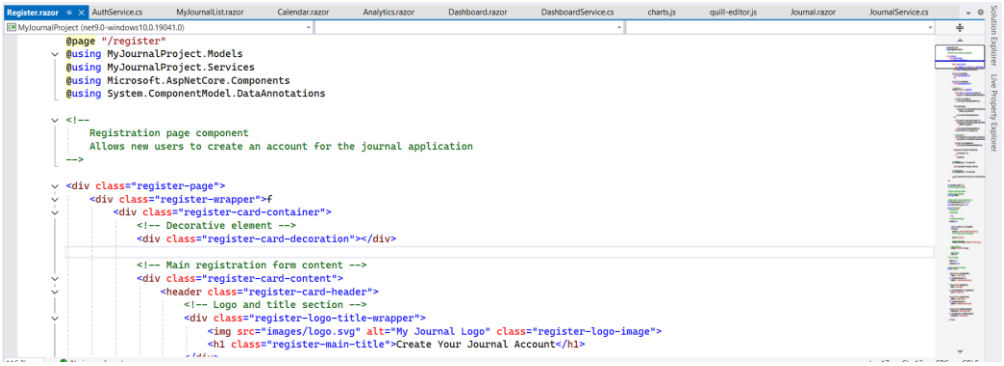
- AuthService.cs



```
// Registers a new user with the system
public async Task<ServiceResult<User>> RegisterAsync(RegisterModel model)
{
    try
    {
        Console.WriteLine($"Registering user: {model.Email}");

        var context = await GetDbContextAsync();

        // Check if email already exists in the database
        var existing = await context.Users
            .AsNoTracking() // Read-only query for better performance
            .FirstOrDefaultAsync(u => u.Email == model.Email);

        if (existing != null)
        {
            Console.WriteLine($"Email already exists: {model.Email}");
            return ServiceResult<User>.FailureResult("This email is already registered");
        }

        // Hash the password for secure storage (never store plain text passwords)
        string hashedPassword = HashPassword(model.Password);
```

Figure 41: AuthService.cs with User Registration and Login

- Register.razor



```
@page "/register"
@using MyJournalProject.Models
@using MyJournalProject.Services
@using Microsoft.AspNetCore.Components
@using System.ComponentModel.DataAnnotations

<!--
    Registration page component
    Allows new users to create an account for the journal application
-->

<div class="register-page">
    <div class="register-wrapper">f
        <div class="register-card-container">
            <!-- Decorative element -->
            <div class="register-card-decoration"></div>

            <!-- Main registration form content -->
            <div class="register-card-content">
                <header class="register-card-header">
                    <!-- Logo and title section -->
                    <div class="register-logo-title-wrapper">
                        <img src="images/logo.svg" alt="My Journal Logo" class="register-logo-image">
                        <h1 class="register-main-title">Create Your Journal Account</h1>
```

Figure 42: Register.razor

- Login.razor

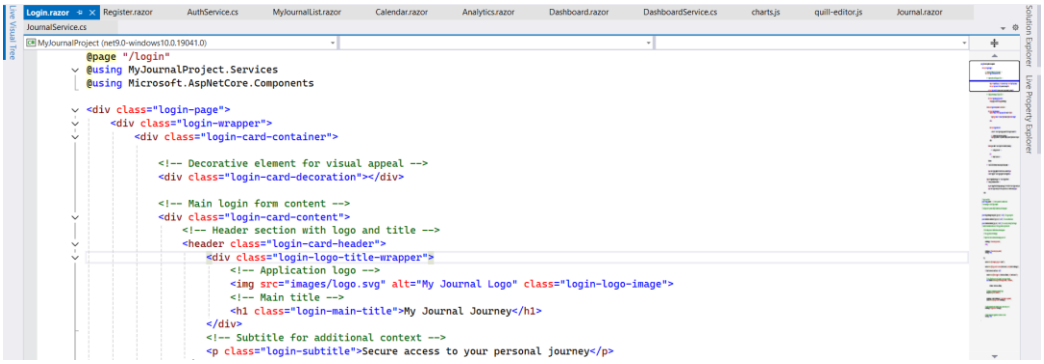


Figure 43: Login.razor

- Register Page

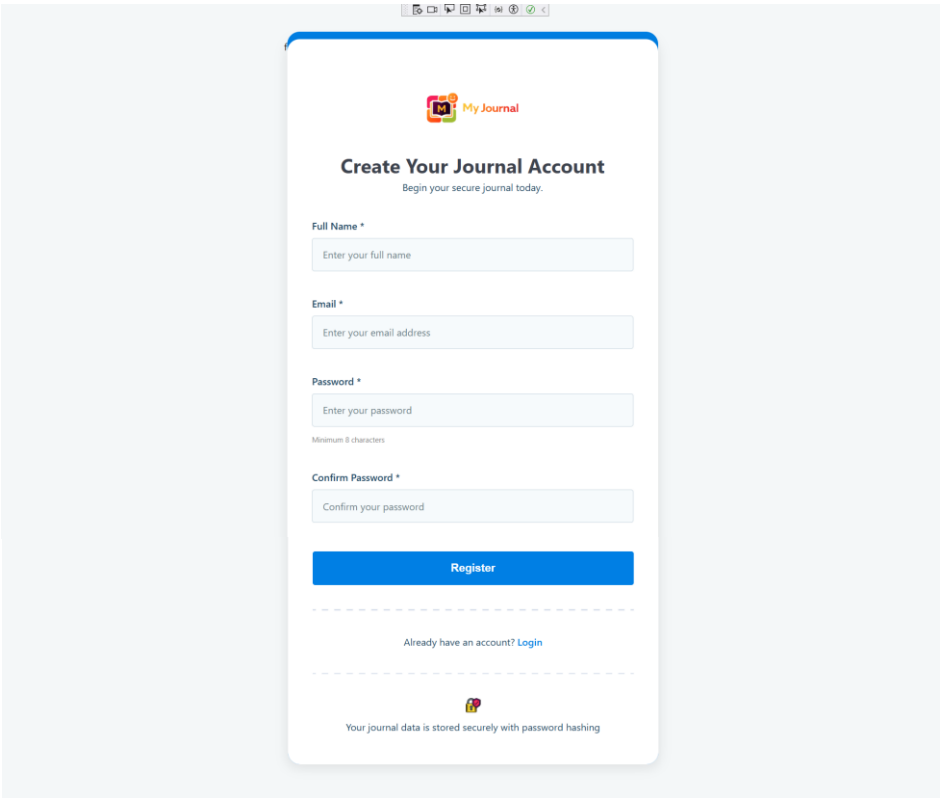


Figure 44: Register Page

- Login Page after successfully registration

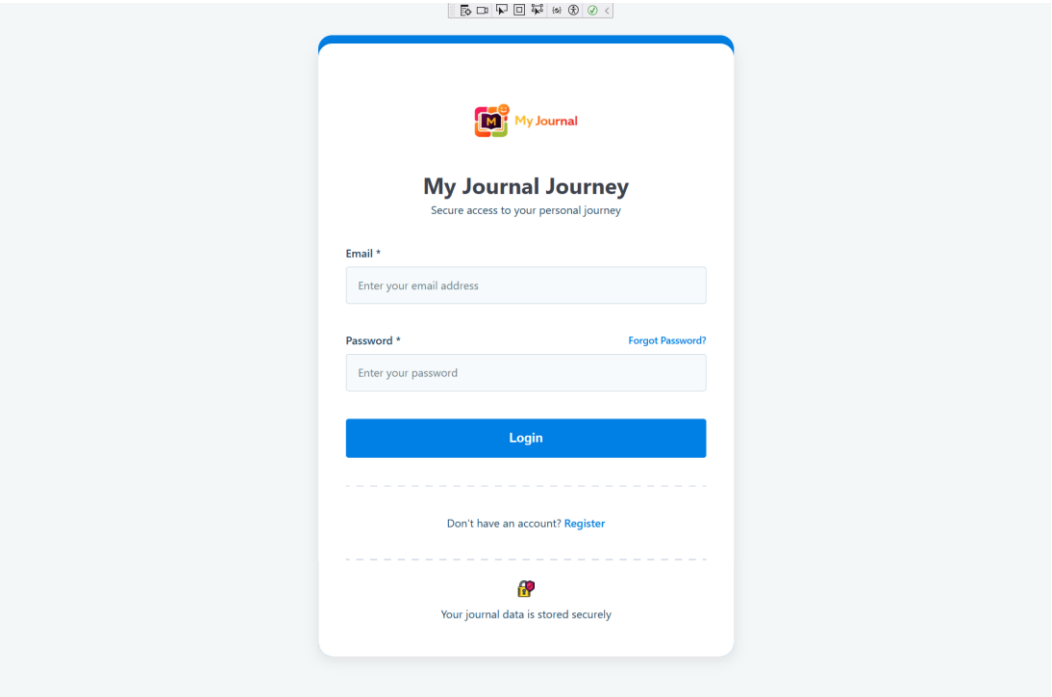


Figure 45: Login Page

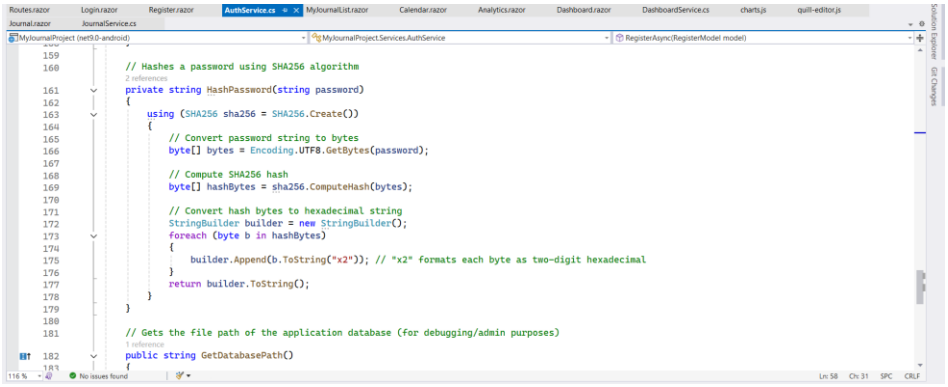
2.2.13. Security (Password Protection)

The application uses secure password protection to ensure user safety. User passwords are not stored in plain text and are securely saved using hashing. The system validates email addresses and prevents duplicate account registration by applying unique email rules in the database. Access to important pages such as the dashboard, journal, calendar, analytics and export pages is restricted to authenticated users only, ensuring that only logged-in users can access protected features.

Implementation:

AuthService.cs, User.cs, AppDbContext.cs, App.razor

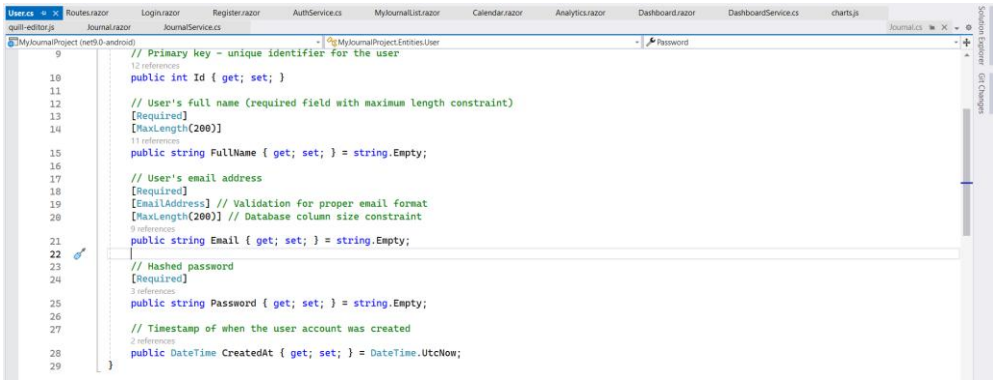
- AuthService.cs



```
// Hashes a password using SHA256 algorithm
private string HashPassword(string password)
{
    using (SHA256 sha256 = SHA256.Create())
    {
        // Convert password string to bytes
        byte[] bytes = Encoding.UTF8.GetBytes(password);

        // Compute SHA256 hash
        byte[] hashBytes = sha256.ComputeHash(bytes);

        // Convert hash bytes to hexadecimal string
        StringBuilder builder = new StringBuilder();
        foreach (byte b in hashBytes)
        {
            builder.Append(b.ToString("x2")); // "x2" formats each byte as two-digit hexadecimal
        }
        return builder.ToString();
    }
}

// Gets the file path of the application database (for debugging/admin purposes)
public string GetDatabasePath()
```

Figure 46: AuthService.cs with Password Security

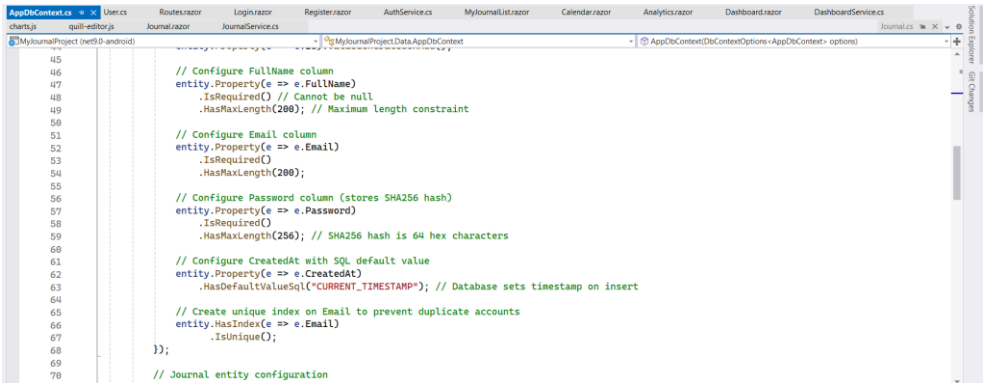
- User.cs



```
// Primary key - unique identifier for the user
public int Id { get; set; }

// User's full name (required field with maximum length constraint)
[Required]
[MaxLength(200)]
public string FullName { get; set; } = string.Empty;

// User's email address
[Required]
[EmailAddress] // Validation for proper email format
[MaxLength(200)] // Database column size constraint
public string Email { get; set; } = string.Empty;

// Hashed password
[Required]
public string Password { get; set; } = string.Empty;

// Timestamp of when the user account was created
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

Figure 47: User.cs with Password Security

- AppDbContext.cs



```
// Configure FullName column
entity.Property(e => e.FullName)
    .IsRequired() // Cannot be null
    .HasMaxLength(200); // Maximum length constraint

// Configure Email column
entity.Property(e => e.Email)
    .IsRequired()
    .HasMaxLength(200);

// Configure Password column (stores SHA256 hash)
entity.Property(e => e.Password)
    .IsRequired()
    .HasMaxLength(256); // SHA256 hash is 64 hex characters

// Configure CreatedAt with SQL default value
entity.Property(e => e.CreatedAt)
    .HasDefaultValueSql("CURRENT_TIMESTAMP"); // Database sets timestamp on insert

// Create unique index on Email to prevent duplicate accounts
entity.HasIndex(e => e.Email)
      .IsUnique();
});

// Journal entity configuration
```

Figure 48: AppDbContext.cs

- App.razor

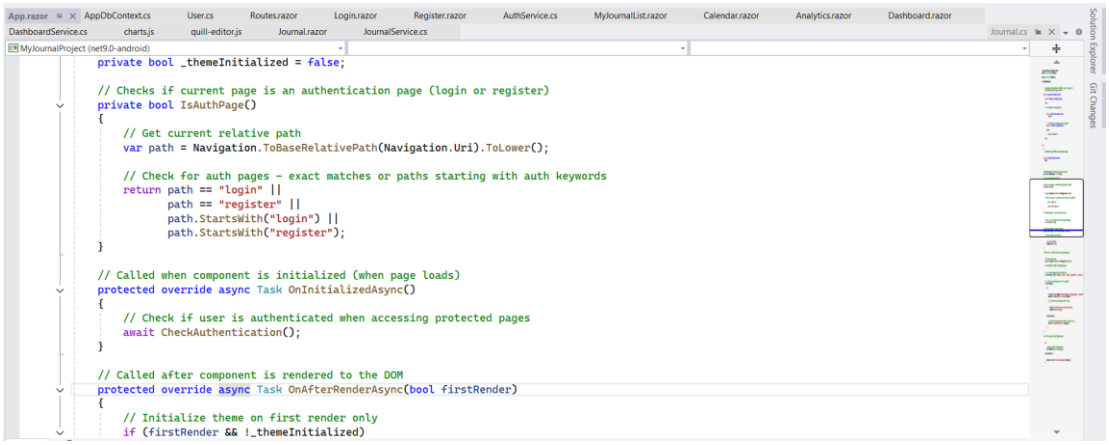


Figure 49: App.razor

- Database with password hashing

Id	FullName	Email	Password	CreatedAt
1	Bhishma	Bhishma2062@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 07:48:14.3511148
2	Rishav Thapa	Rishav@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 09:09:39.1357382
3	Kalyan Bikram Karki	kalyan123@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 10:50:04.0311144
4	John Doe	john@example.com	ff7bd97b1a7789ddd2775122fd6817f3173672da9f802ceec57f284325bf589f	2026-01-22 15:20:44.5091871

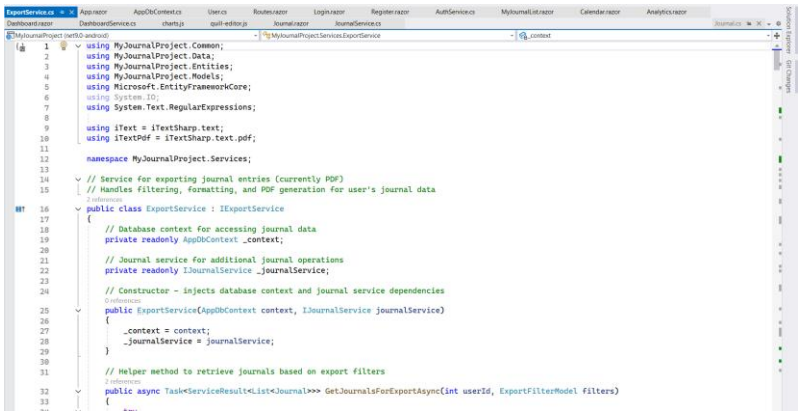
Figure 50: Database with password hashing

2.2.14. PDF Export

The application provides a PDF export feature that allows users to export their journal entries into professionally formatted PDF documents. Users can select a date range and apply filters such as mood, category, and tags before exporting. The export system also allows users to customize the content by choosing whether to include mood details, tags, word count, timestamps, entry date, and category. The generated PDF includes proper formatting with headers, footers, structured layout and muti page support. Files are automatically downloaded with timestamped filenames, making them easy to store and organize. A preview system shows entry counts and word statistics before export, ensuring users know exactly what will be included.

Implementation: ExportService.cs, Export.razor, ExportModel.cs

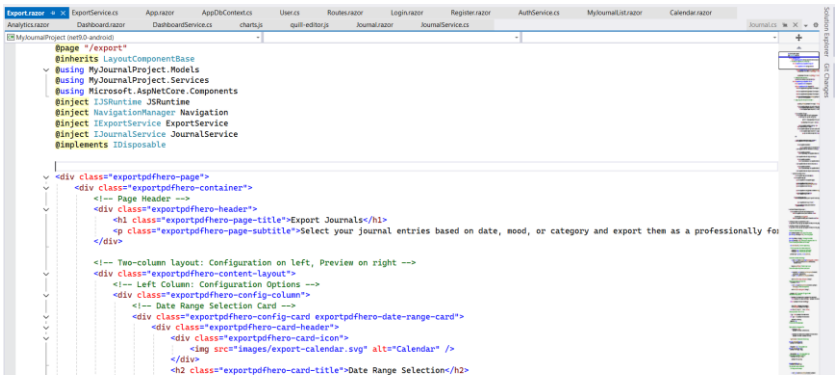
- ExportService.cs



```
using MyJournalProject.Common;
using MyJournalProject.Data;
using MyJournalProject.Entities;
using MyJournalProject.Models;
using Microsoft.EntityFrameworkCore;
using System.IO;
using System.Text.RegularExpressions;

using iText = iTextSharp.text;
using iTextPdf = iTextSharp.text.pdf;

namespace MyJournalProject.Services;

// Service for exporting journal entries (currently PDF)
// Handles filtering, formatting, and PDF generation for user's journal data
public class ExportService : IExportService
{
    // Database context for accessing journal data
    private readonly AppDbContext _context;

    // Journal service for additional journal operations
    private readonly IJournalService _journalService;

    // Constructor - injects database context and journal service dependencies
    public ExportService(AppDbContext context, IJournalService journalService)
    {
        _context = context;
        _journalService = journalService;
    }

    // Helper method to retrieve journals based on export filters
    public async Task<ServiceResult<List<Journal>>> GetJournalsForExportAsync(int userId, ExportFilterModel filters)
    {
```

Figure 51: ExportService.cs with PDF Export

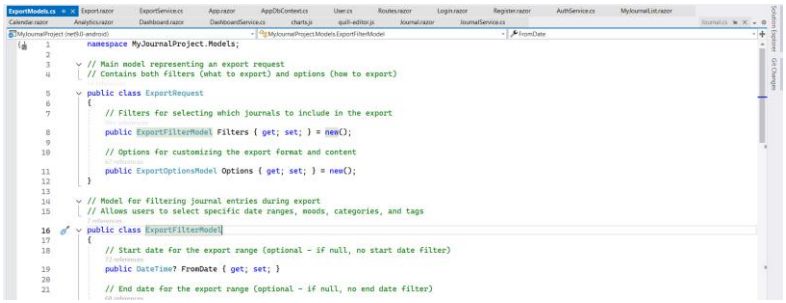
- Export.razor



```
@page "/export"
@inherits LayoutComponentBase
@using MyJournalProject.Models
@using MyJournalProject.Services
@using Microsoft.AspNetCore.Components
@inject IJSRuntime JSRuntime
@inject NavigationManager Navigation
@inject IExportService ExportService
@inject IJournalService JournalService
@implements IDisposable

<div class="exportpdfhero-page">
    <div class="exportpdfhero-container">
        <!-- Page Header -->
        <div class="exportpdfhero-header">
            <h1 class="exportpdfhero-page-title">Export Journals</h1>
            <p class="exportpdfhero-page-subtitle">Select your journal entries based on date, mood, or category and export them as a professionally fo
        </div>

        <!-- Two-column layout: Configuration on left, Preview on right -->
        <div class="exportpdfhero-content-layout">
            <!-- Left Column: Configuration Options -->
            <div class="exportpdfhero-config-column">
                <!-- Date Range Selection Card -->
                <div class="exportpdfhero-config-card exportpdfhero-date-range-card">
                    <div class="exportpdfhero-card-header">
                        <div class="exportpdfhero-card-icon">
                            <img src="images/export-calendar.svg" alt="Calendar" />
                        </div>
                        <h2 class="exportpdfhero-card-title">Date Range Selection</h2>
```

Figure 52: Export.razor with PDF Export

- ExportModel.cs



```
namespace MyJournalProject.Models;

// Main model representing an export request
// Contains both filters (what to export) and options (how to export)
public class ExportRequest
{
    // Filters for selecting which journals to include in the export
    public ExportFilterModel Filters { get; set; } = new();

    // Options for customizing the export format and content
    public ExportOptionsModel Options { get; set; } = new();
}

// Model for filtering journal entries during export
// Allows users to select specific date ranges, moods, categories, and tags
public class ExportFilterModel
{
    // Start date for the export range (optional - if null, no start date filter)
    public DateTime? FromDate { get; set; }

    // End date for the export range (optional - if null, no end date filter)
```

Figure 53: ExportModel.cs with PDF Export

- Export configuration page with pdf option selection

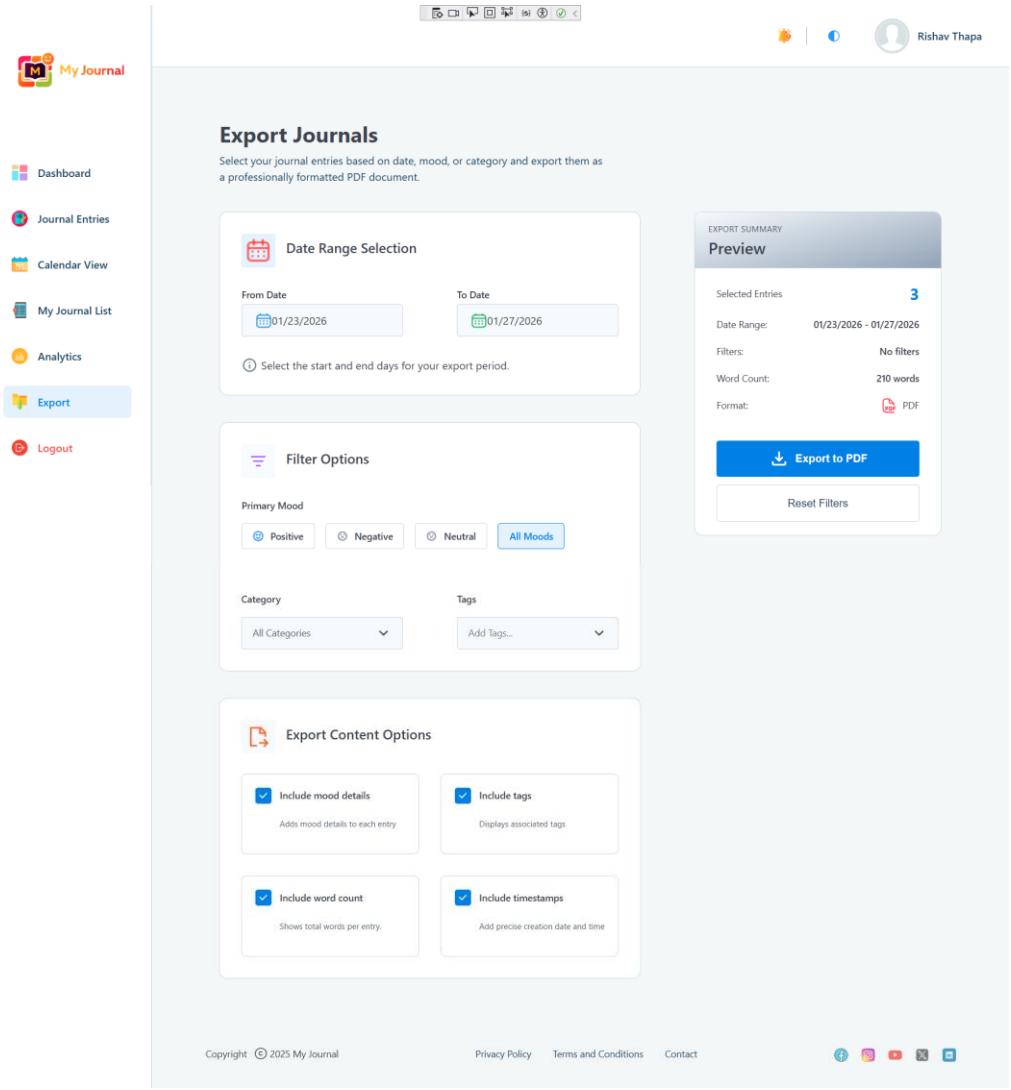


Figure 54: Export configuration page with pdf option selection

- Generated PDF output

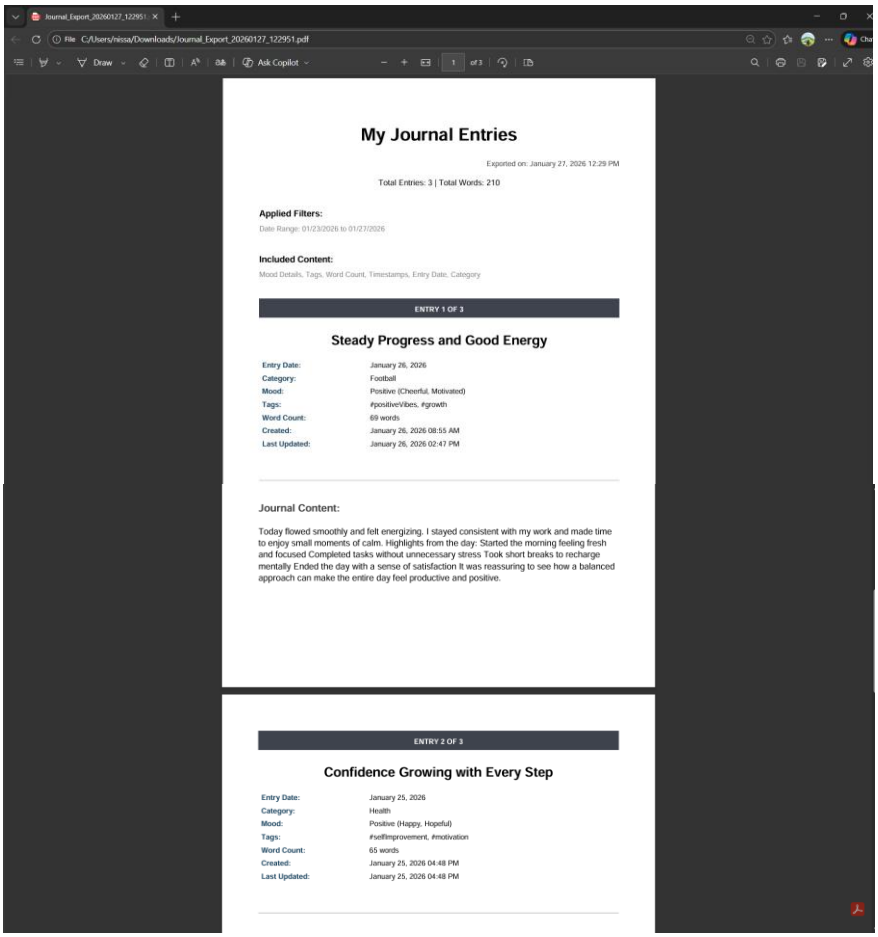


Figure 55: Generated PDF output

2.2.15. Local Storage (SQLite)

The system uses SQLite as a local database to store all user and journal data securely on the user's device. The database is configured using Entity Framework Core, with automatic database creation and schema management. Platform-specific storage paths are used to ensure proper storage on different operating system. The database includes structure tables for users and journals with proper relationships, unique constraints and indexing for better performance. Tags are stored using JSON serialization for flexibility, and timestamps are automatically managed for each entry. This local storage system allows offline access, fast data retrieval, and reliable long-term storage of journal information.

Implementation: MauiProgram.cs, AppDbContext.cs, Entities

- MauiProgram.cs

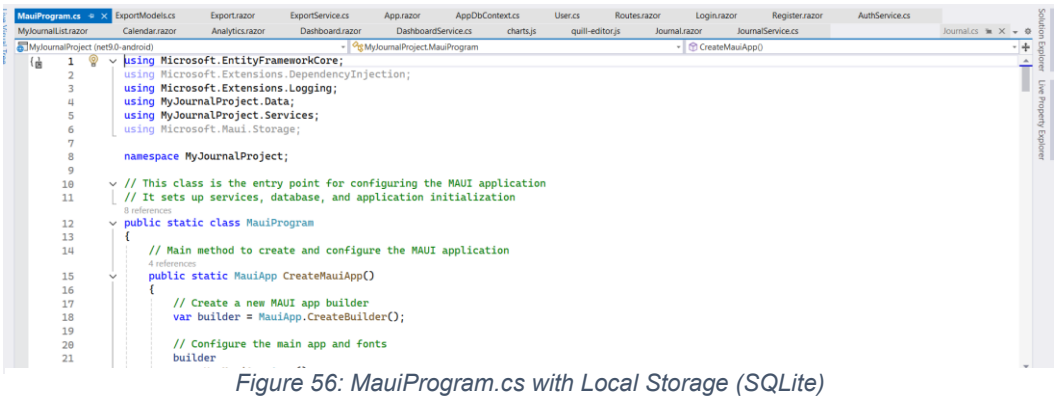


Figure 56: MauiProgram.cs with Local Storage (SQLite)

- AppDbContext.cs

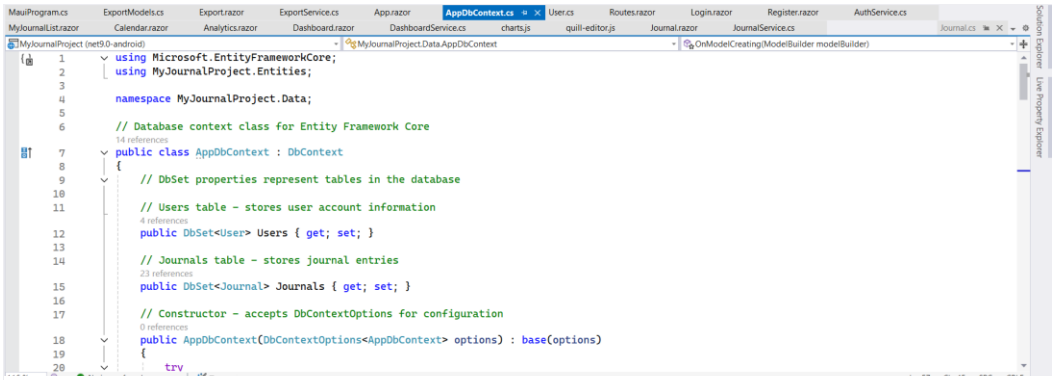
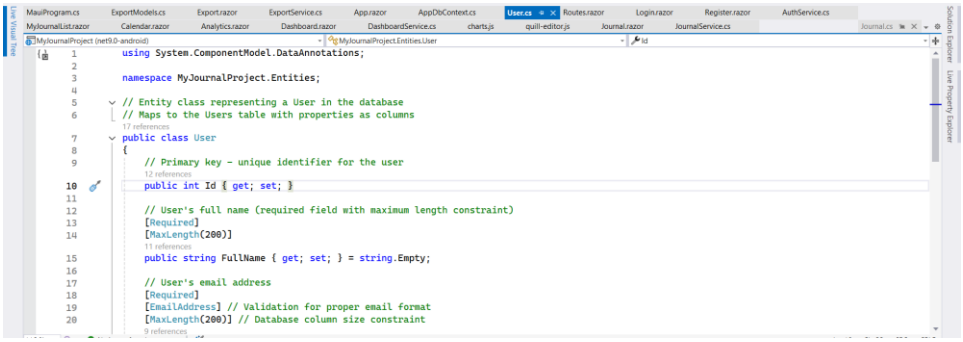


Figure 57: AppDbContext.cs with Local Storage (SQLite)

- User.cs



```
using System.ComponentModel.DataAnnotations;

namespace MyJournalProject.Entities;

// Entity class representing a User in the database
// Maps to the Users table with properties as columns
public class User
{
    // Primary key - unique identifier for the user
    public int Id { get; set; }

    // User's full name (required field with maximum length constraint)
    [Required]
    [MaxLength(200)]
    public string FullName { get; set; } = string.Empty;

    // User's email address
    [Required]
    [EmailAddress] // Validation for proper email format
    [MaxLength(200)] // Database column size constraint
```

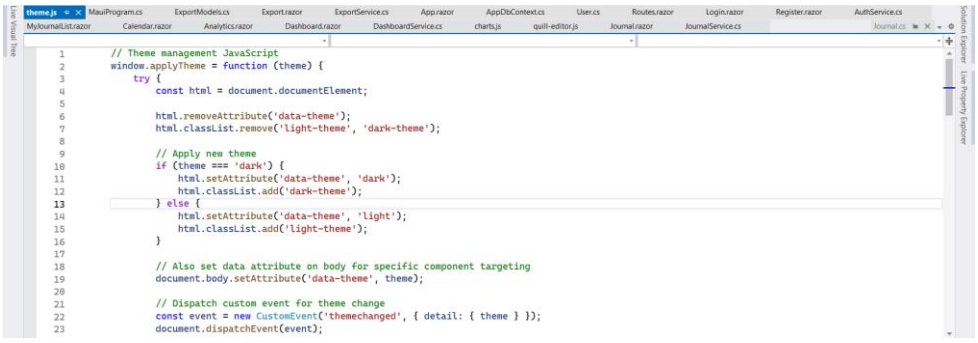
Figure 58: User.cs with Local Storage (SQLite)

2.2.16. Theme Customization

The application supports theme customization with both light and dark modes to improve user comfort and personalization. The system automatically detects the user's system them preference and applies it during the first launch. Users can manually switch between light and dark themes using a theme toggle button in the header. the entire interface updates smoothly without reloading the application. Theme styling is managed using CSS variable and JavaScript integration, providing consistent colours, layouts, and visual appearance across all pages.

Implementation: ThemeService.cs, theme.js, theme.css, Header.razor

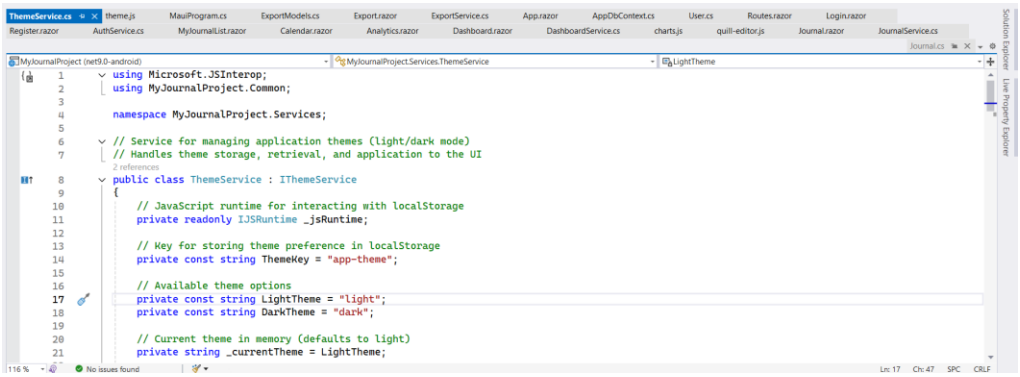
- theme.js



```
// Theme management JavaScript
window.applyTheme = function (theme) {
    try {
        const html = document.documentElement;

        html.removeAttribute('data-theme');
        html.classList.remove('light-theme', 'dark-theme');

        // Apply new theme
        if (theme === 'dark') {
            html.setAttribute('data-theme', 'dark');
            html.classList.add('dark-theme');
        } else {
            html.setAttribute('data-theme', 'light');
            html.classList.add('light-theme');
        }

        // Also set data attribute on body for specific component targeting
        document.body.setAttribute('data-theme', theme);

        // Dispatch custom event for theme change
        const event = new CustomEvent('themechanged', { detail: { theme } });
        document.dispatchEvent(event);
```

Figure 59: theme.js with Theme Customization

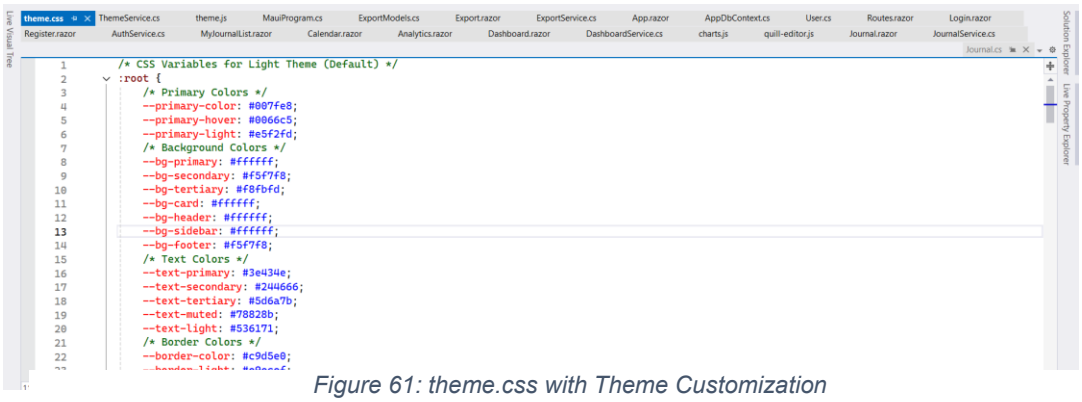
- ThemeService.cs



```
using Microsoft.JSInterop;
using MyJournalProject.Common;

namespace MyJournalProject.Services;

// Service for managing application themes (light/dark mode)
// Handles theme storage, retrieval, and application to the UI
public class ThemeService : IThemeService
{
    // JavaScript runtime for interacting with localStorage
    private readonly IJSRuntime _jsRuntime;

    // Key for storing theme preference in localStorage
    private const string ThemeKey = "app-theme";

    // Available theme options
    private const string LightTheme = "light";
    private const string DarkTheme = "dark";

    // Current theme in memory (defaults to light)
    private string _currentTheme = LightTheme;
```

Figure 60: ThemeService.cs with Theme Customization

- theme.css

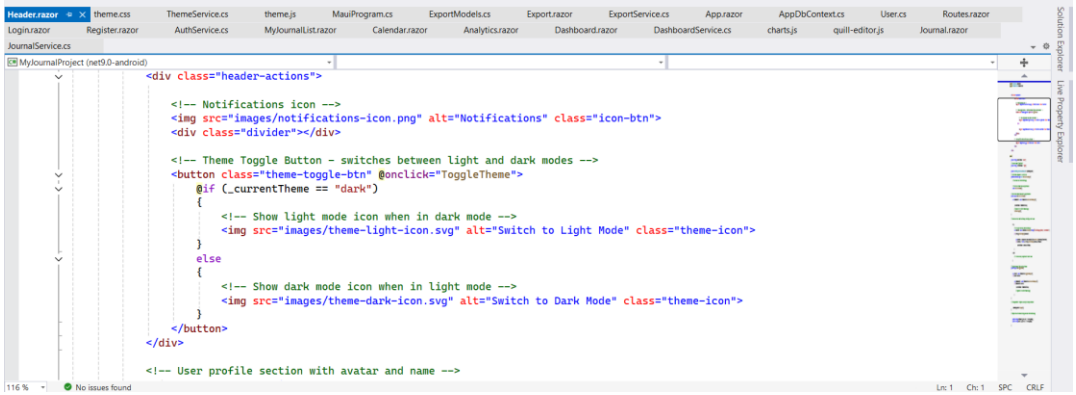


```
/* CSS Variables for Light Theme (Default) */
:root {
    /* Primary Colors */
    --primary-color: #007fe8;
    --primary-hover: #0066c5;
    --primary-light: #e5f2fd;
    /* Background Colors */
    --bg-primary: #ffffff;
    --bg-secondary: #f5f7f8;
    --bg-tertiary: #f8fbfd;
    --bg-card: #ffffff;
    --bg-header: #ffffff;
    --bg-sidebar: #ffffff;
    --bg-footer: #f5f7f8;
    /* Text Colors */
    --text-primary: #3e434e;
    --text-secondary: #2446666;
    --text-tertiary: #5d6a7b;
    --text-muted: #78828b;
    --text-light: #536171;
    /* Border Colors */
    --border-color: #c9d5e0;
```

Figure 61: theme.css with Theme Customization

- Header.razor

,



```
<div class="header-actions">

    <!-- Notifications icon -->
    <img src="images/notifications-icon.png" alt="Notifications" class="icon-btn">
    <div class="divider"></div>

    <!-- Theme Toggle Button - switches between light and dark modes -->
    <button class="theme-toggle-btn" @onclick="ToggleTheme">
        @if (_currentTheme == "dark")
        {
            <!-- Show light mode icon when in dark mode -->
            <img src="images/theme-light-icon.svg" alt="Switch to Light Mode" class="theme-icon">
        }
        else
        {
            <!-- Show dark mode icon when in light mode -->
            <img src="images/theme-dark-icon.svg" alt="Switch to Dark Mode" class="theme-icon">
        }
    </button>
</div>

<!-- User profile section with avatar and name -->
```

Figure 62: Header.razor with Theme Customization

- Light Theme Interface of Journal Entry and Analytics Page

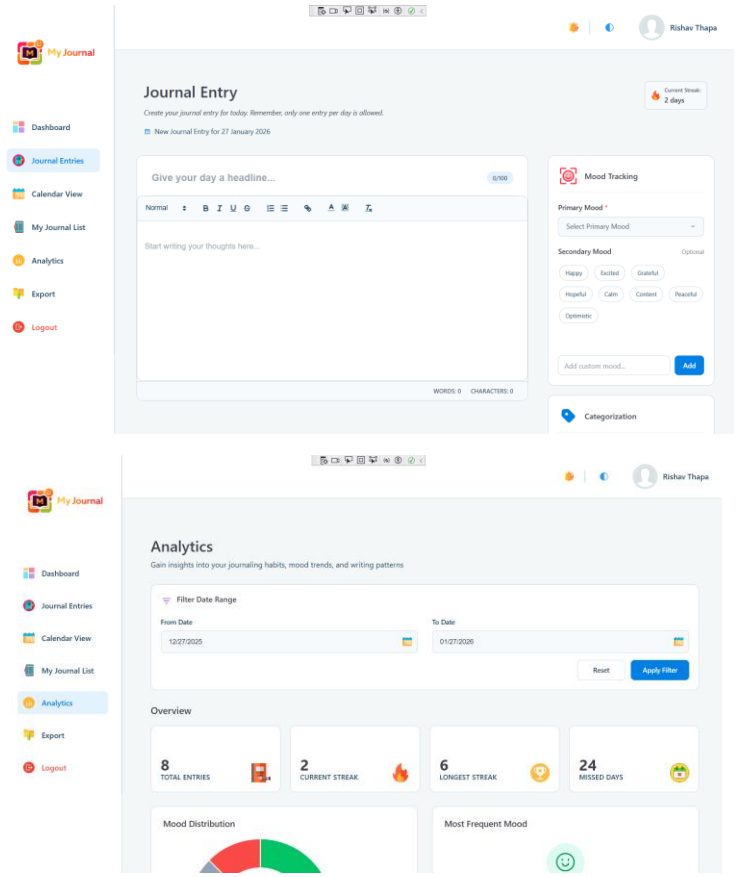


Figure 63: Light Theme Interface of Journal Entry and Analytics Page

- Dark Theme Interface of Journal Entry and Analytics Page

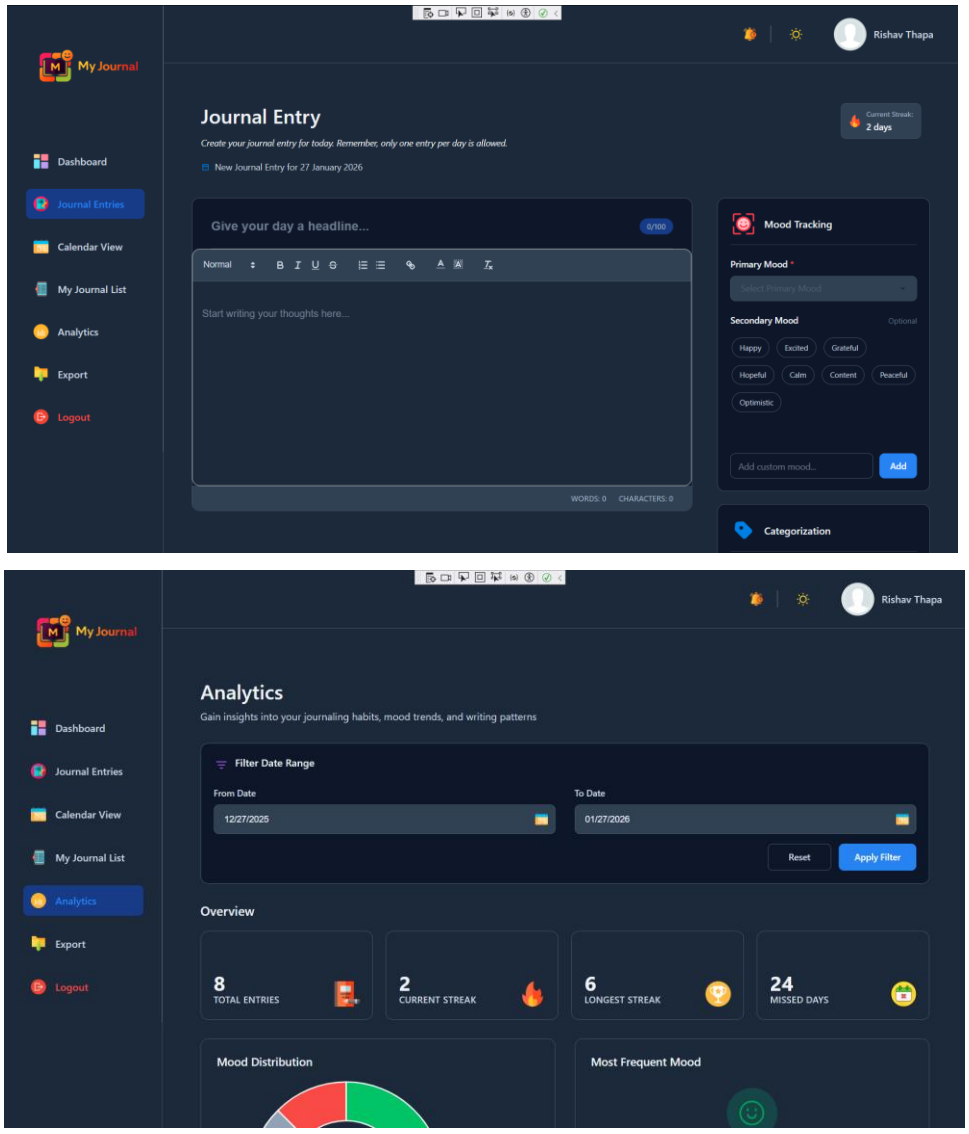


Figure 64: Dark Theme Interface of Journal Entry and Analytics Page

3. Proof of Work

3.1. Wireframes and UI Designs (Figma)

3.1.1. Register Page

- Wireframe of Register Page

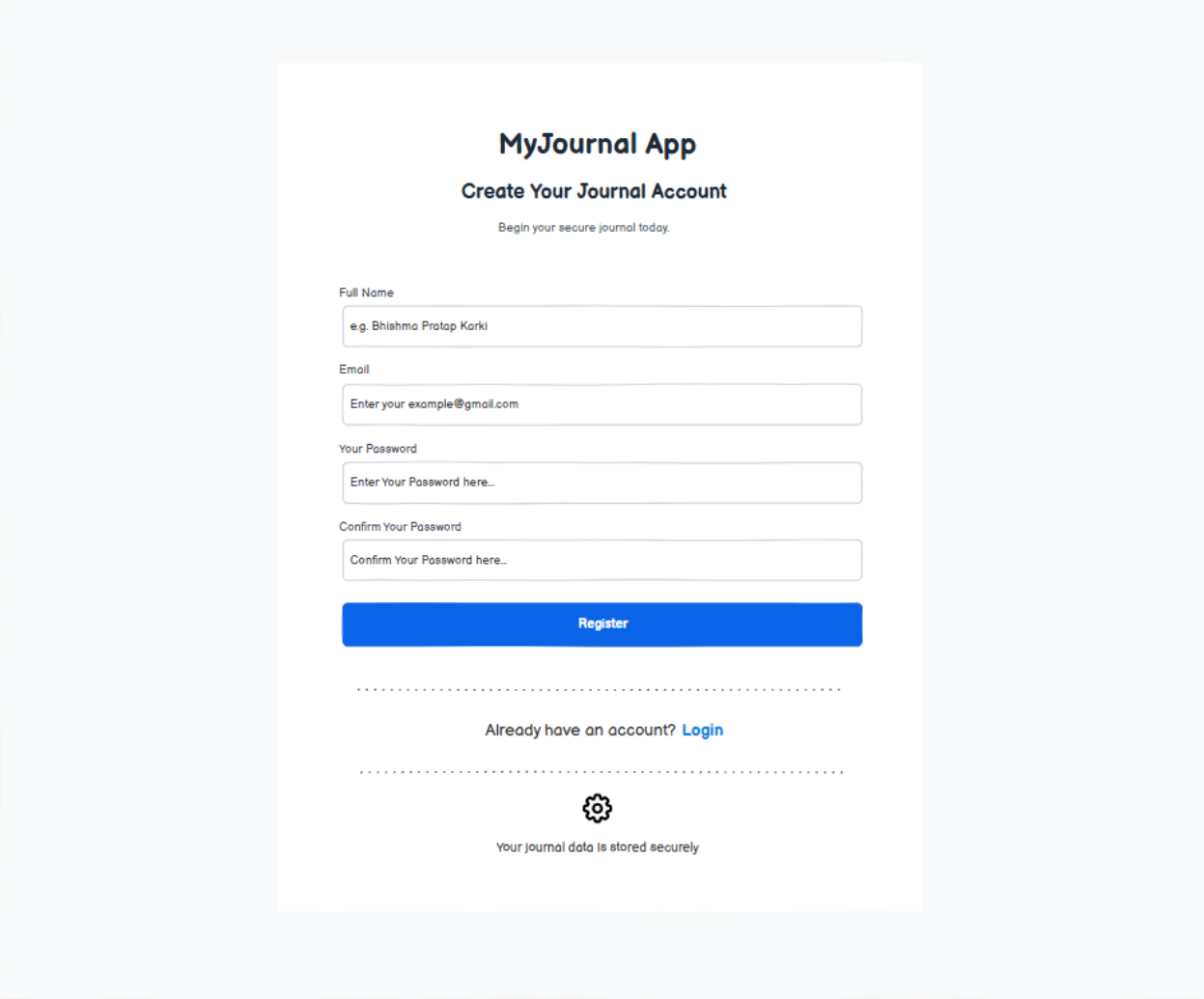


Figure 65: Wireframe of Register Page

- UI Design (Figma) of Register Page

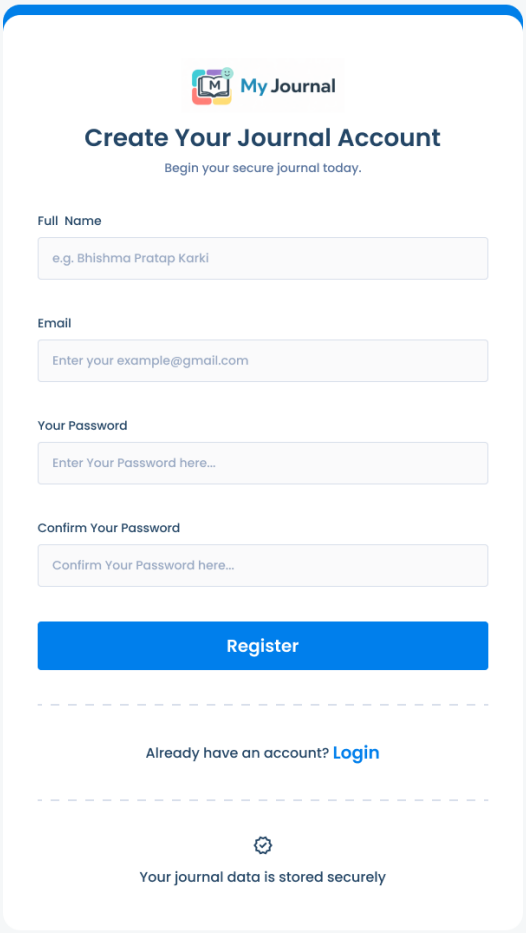


Figure 66: UI Design (Figma) of Register Page

3.1.2. Login Page

- Wireframe of Login Page

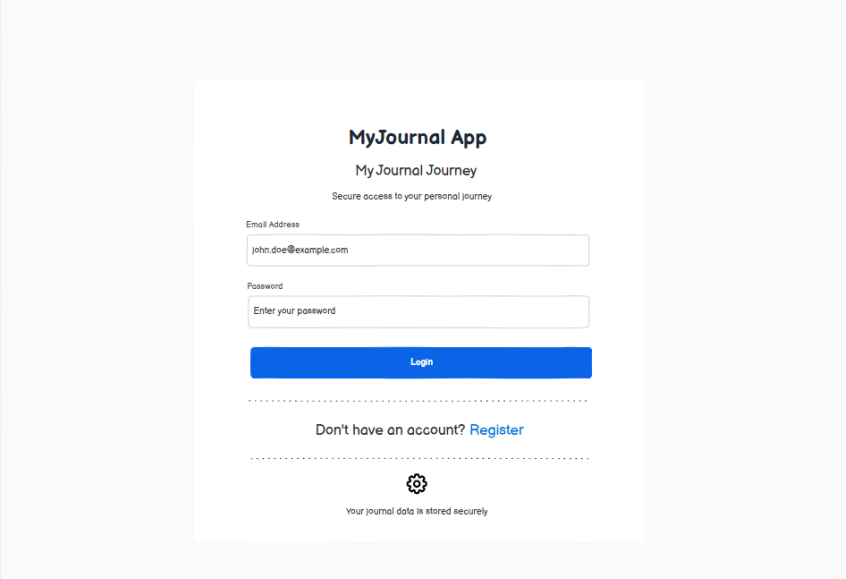


Figure 67: Wireframe of Login Page

- UI Design (Figma) of Login Page

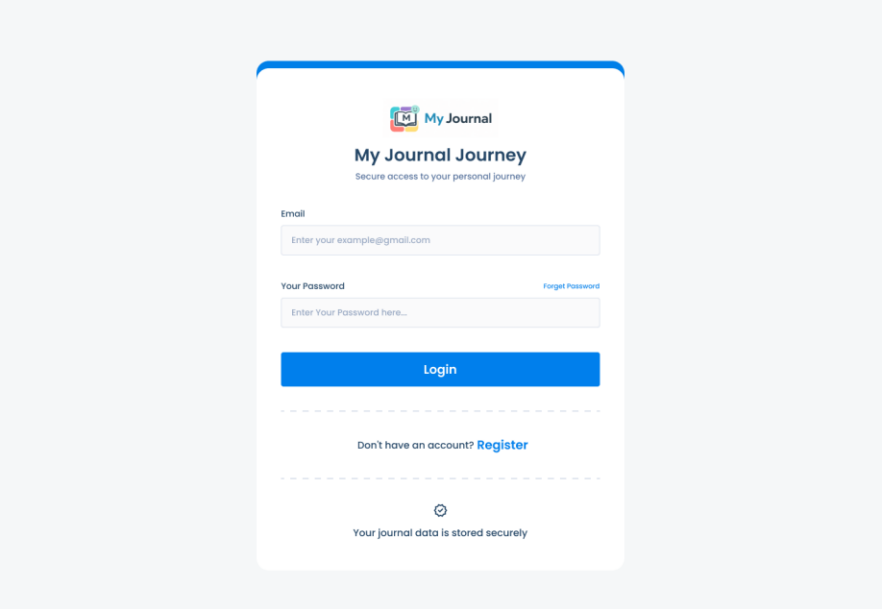


Figure 68: UI Design (Figma) of Login Page

3.1.3. Dashboard Page

- Wireframe of Dashboard Page

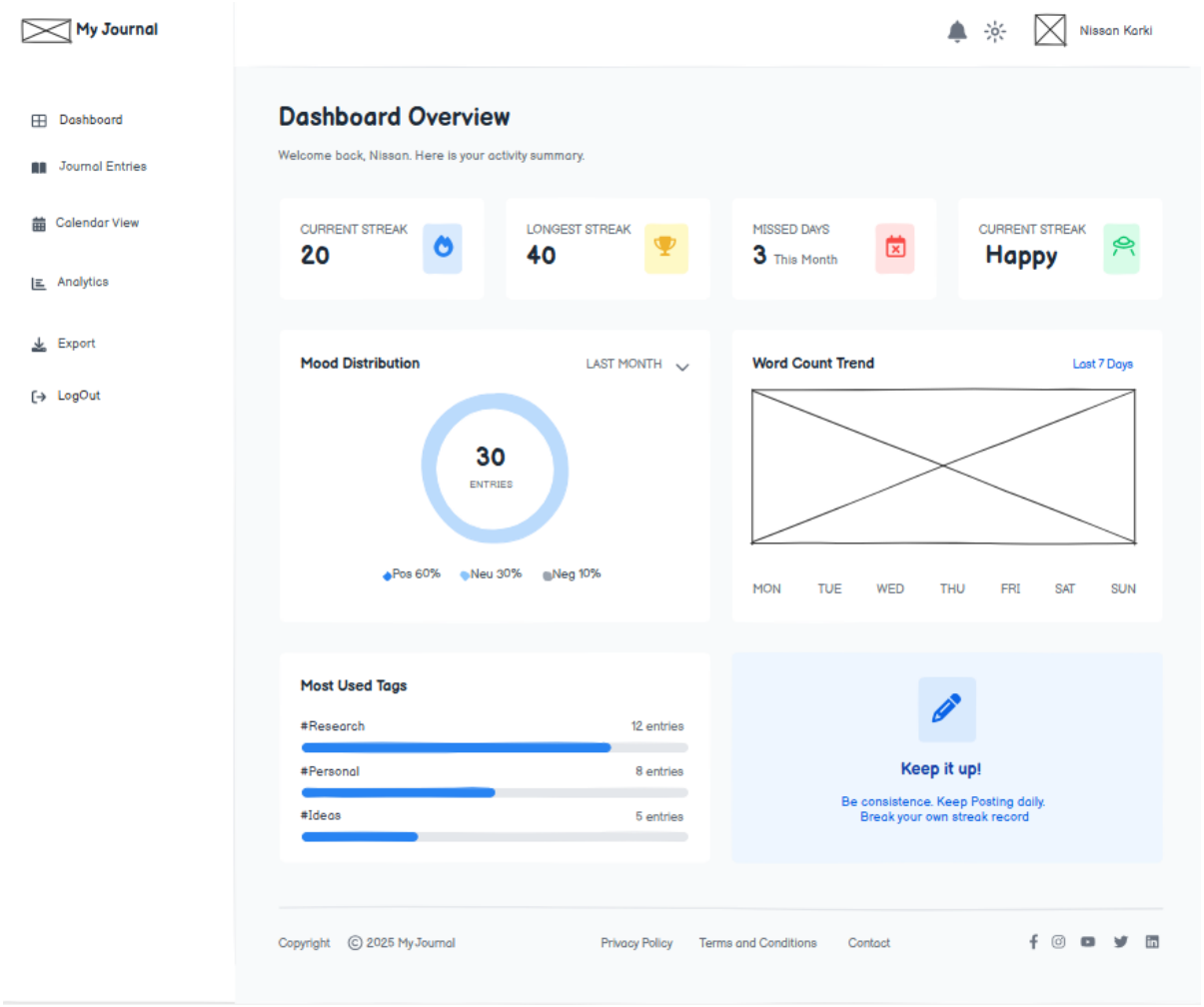


Figure 69: Wireframe of Dashboard Page

- UI Design (Figma) of Dashboard Page

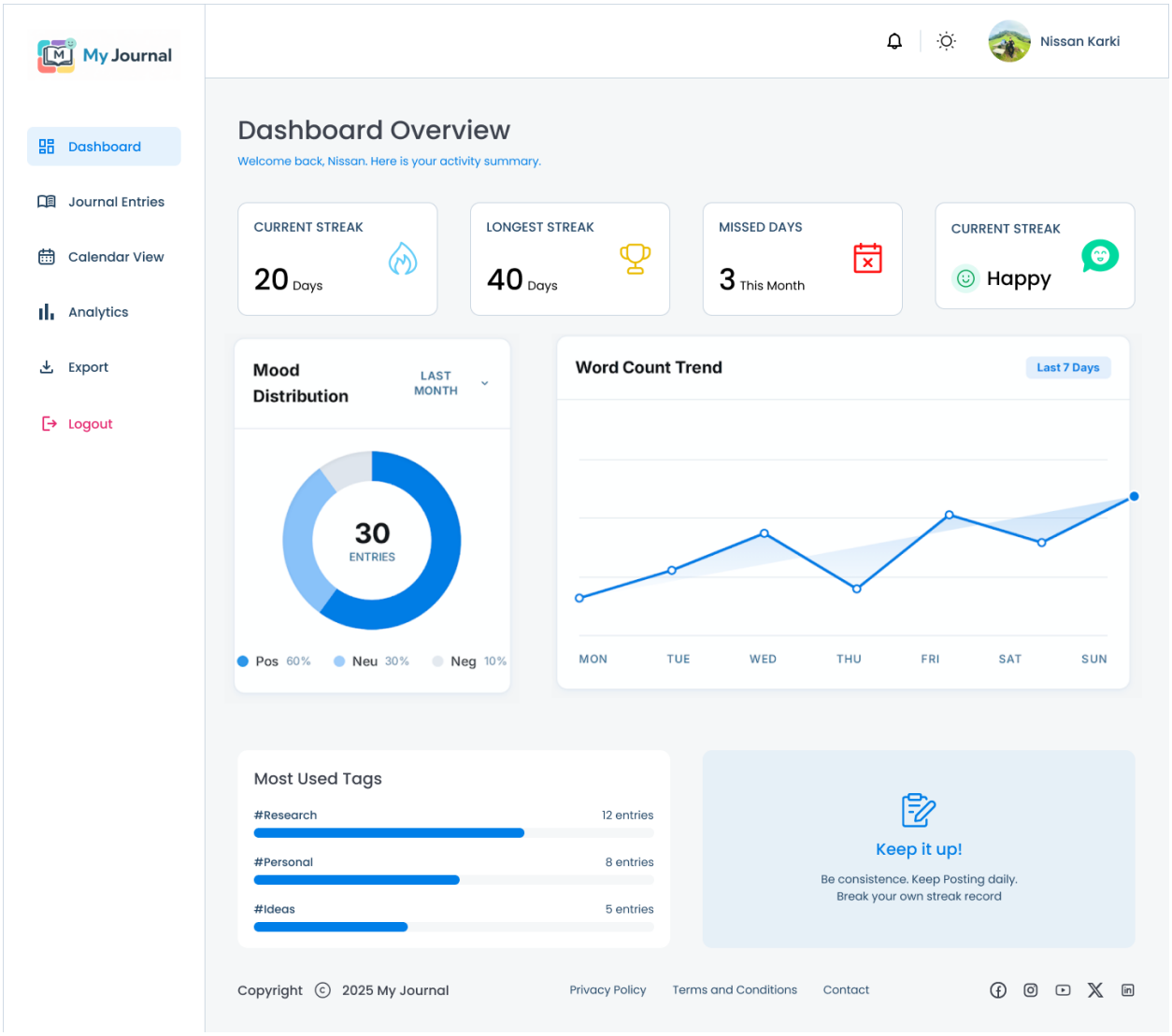


Figure 70: UI Design (Figma) of Dashboard Page

3.1.4. Journal Entry Page

- Wireframe of Journal Entry Page

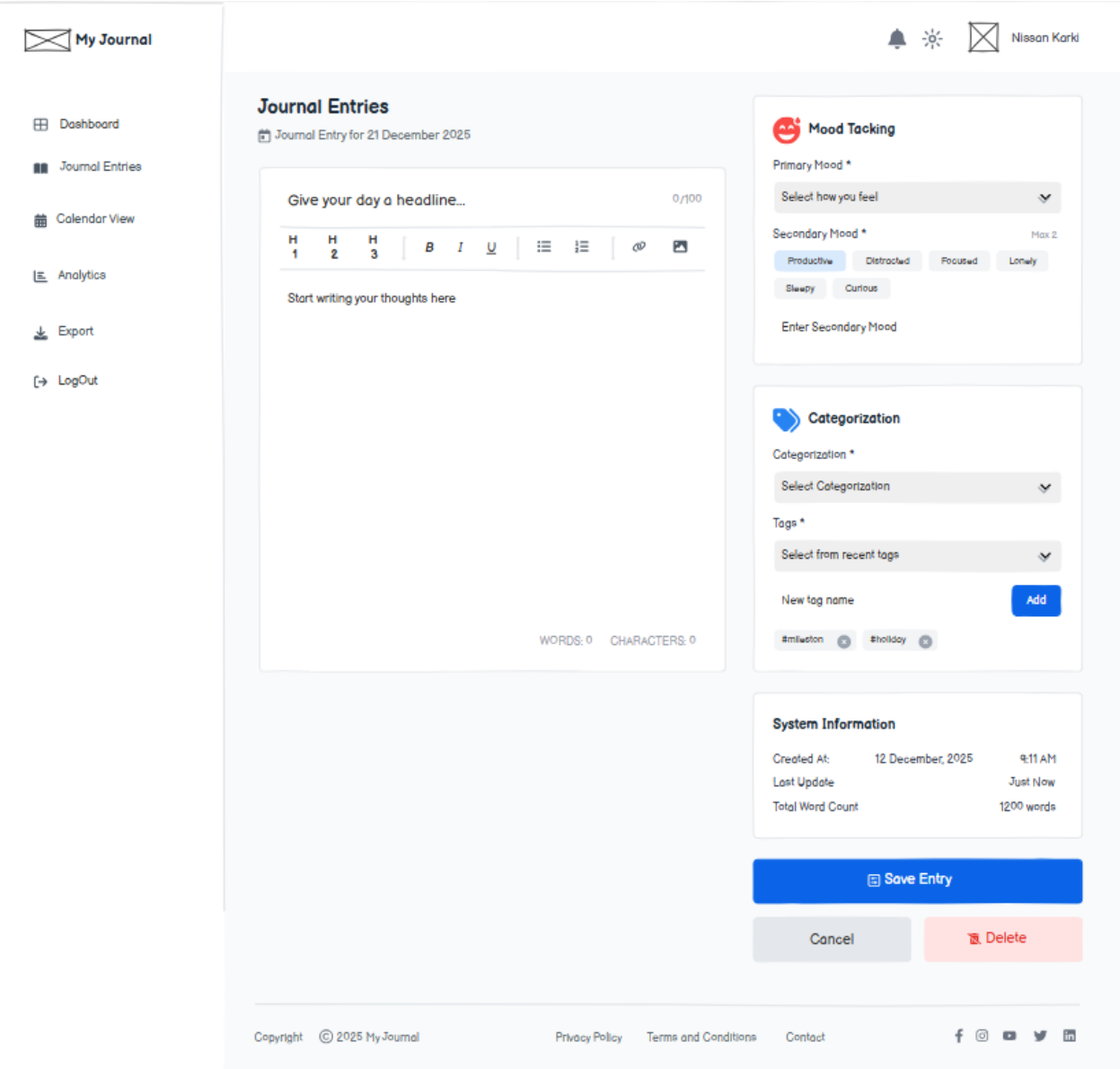


Figure 71: Wireframe of Journal Entry Page

- UI Designs (Figma) for Journal Entry Page

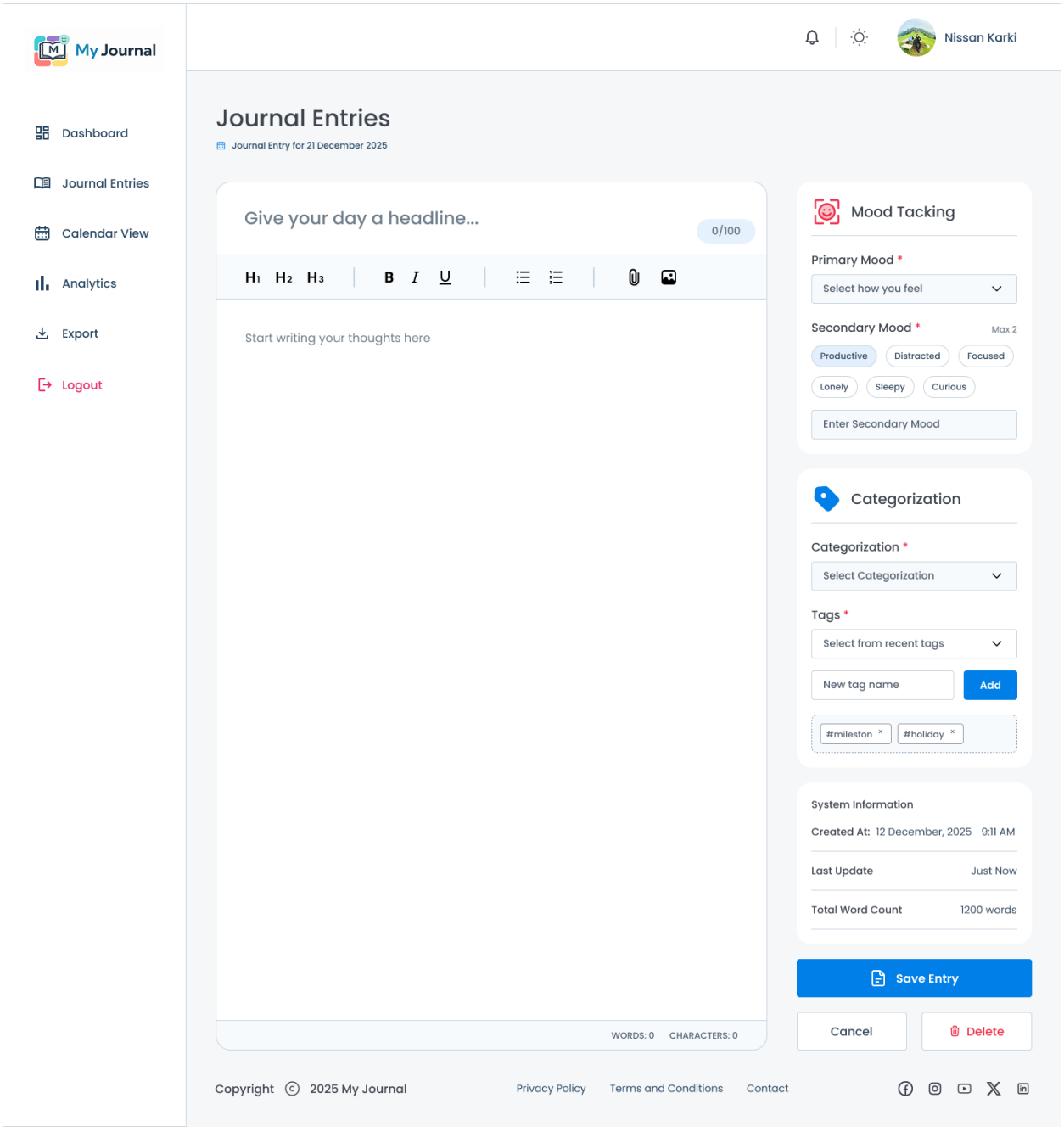


Figure 72: UI Designs (Figma) for Journal Entry Page

3.1.5. Calendar View Page

- Wireframe of Calendar View Page

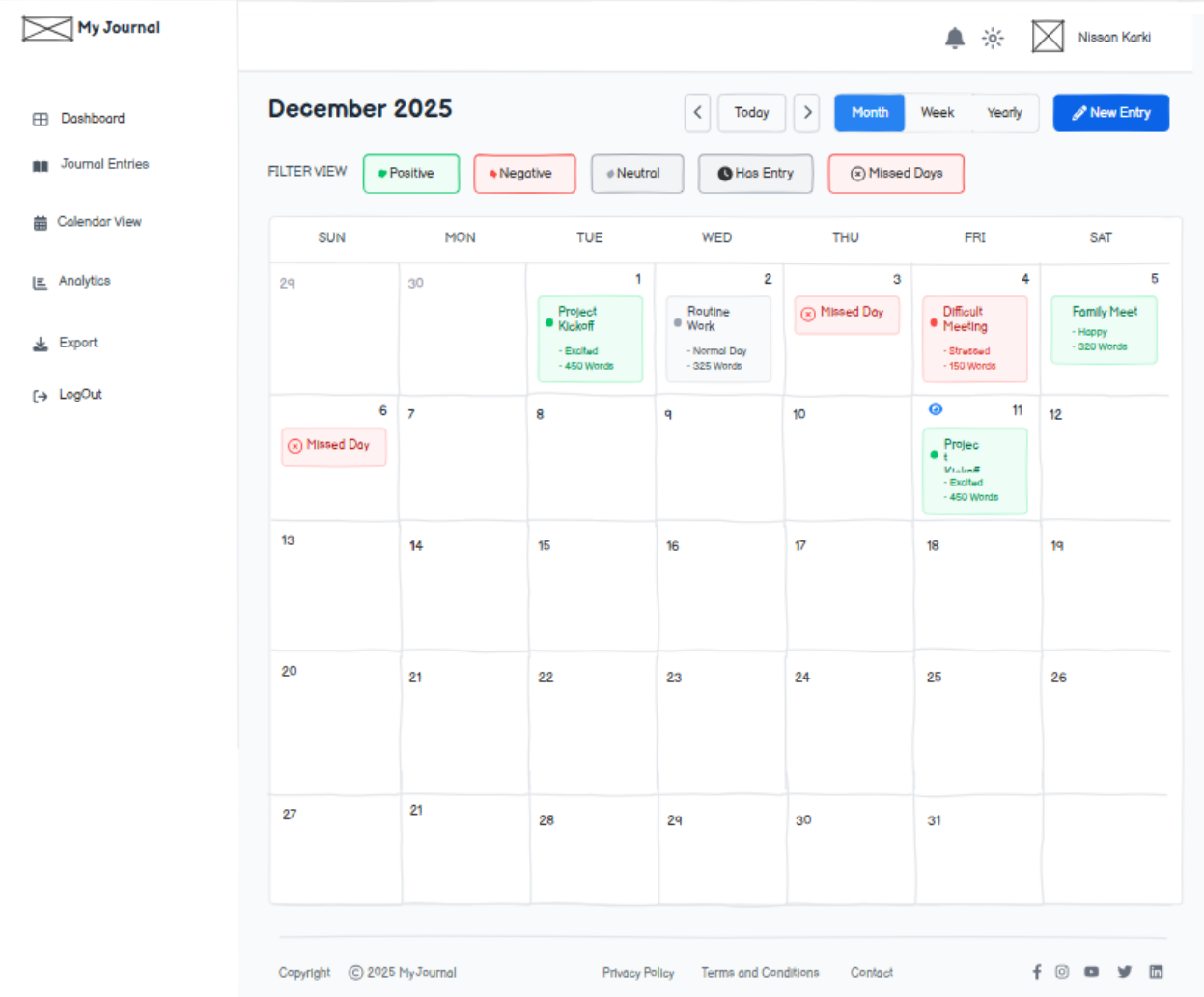


Figure 73: Wireframe of Calendar View Page

- UI Design (Figma) of Calendar View Page

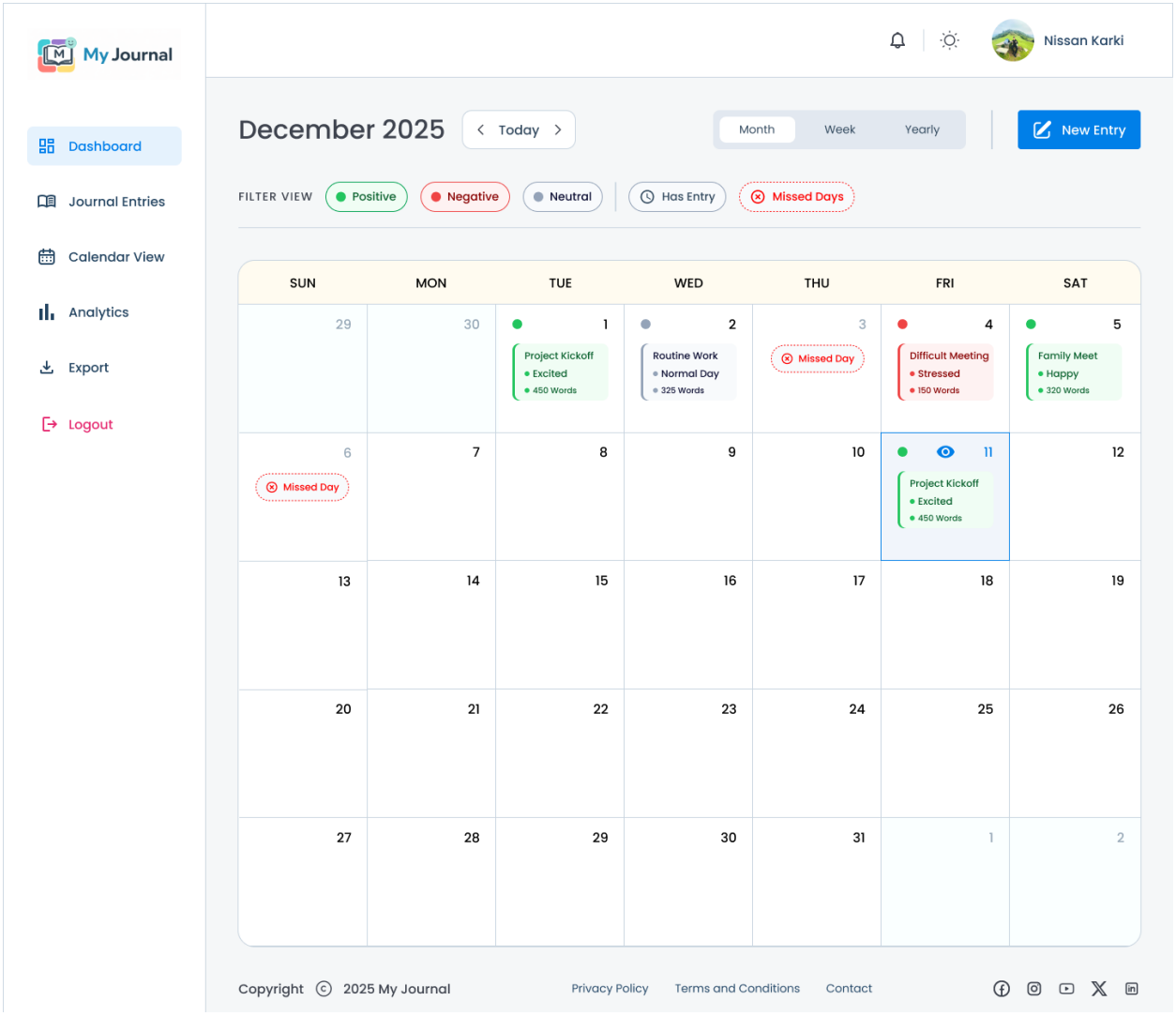


Figure 74: UI Design (Figma) of Calendar View Page

3.1.6. My Journal List Page

- Wireframe of My Journal List Page

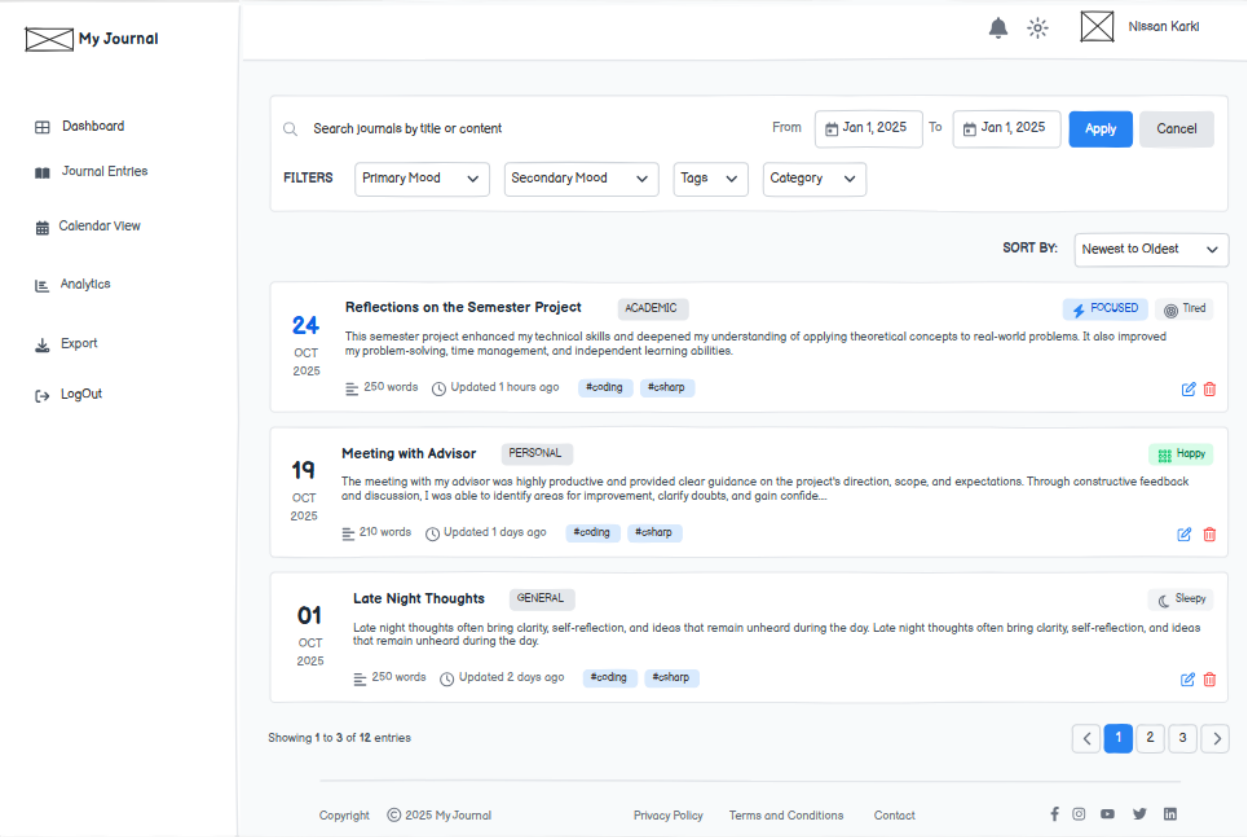


Figure 75: Wireframe of My Journal List Page

- UI Design (Figma) of My Journal List Page

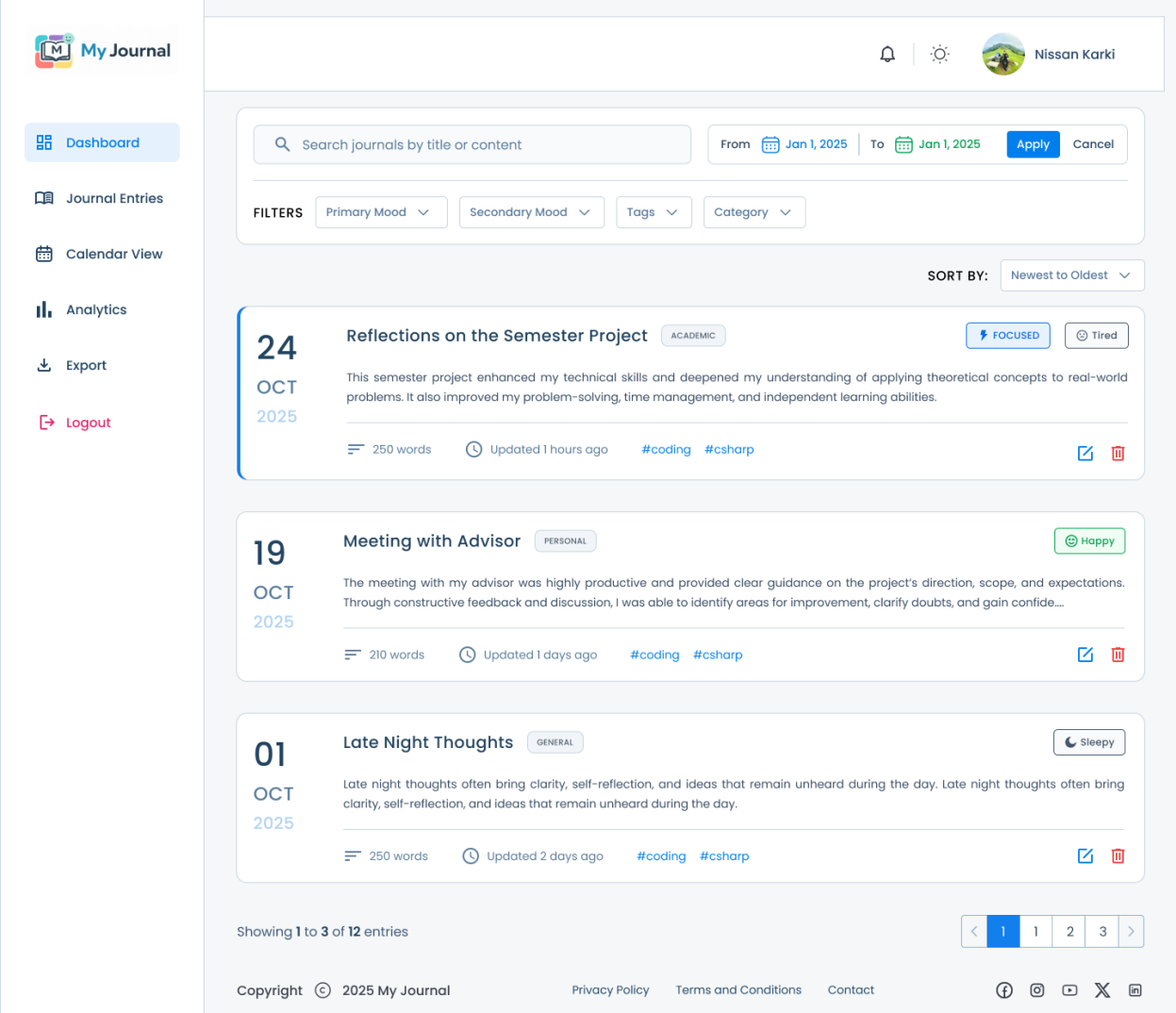


Figure 76: UI Design (Figma) of My Journal List Page

3.1.7. Analytics Page

- Wireframe of Analytics Page

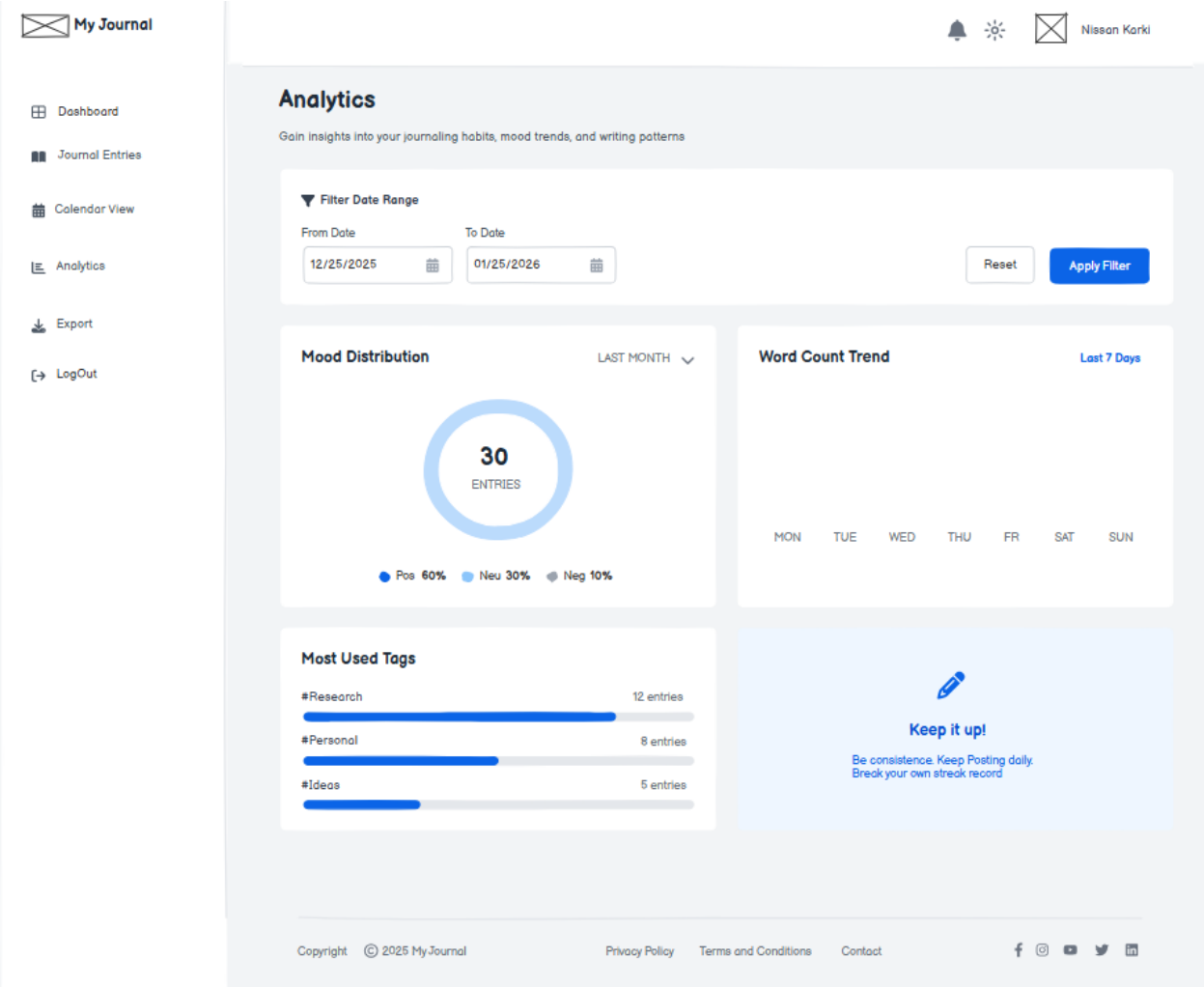


Figure 77: Wireframe of Analytics Page

- UI Design (Figma) of Analytics Page

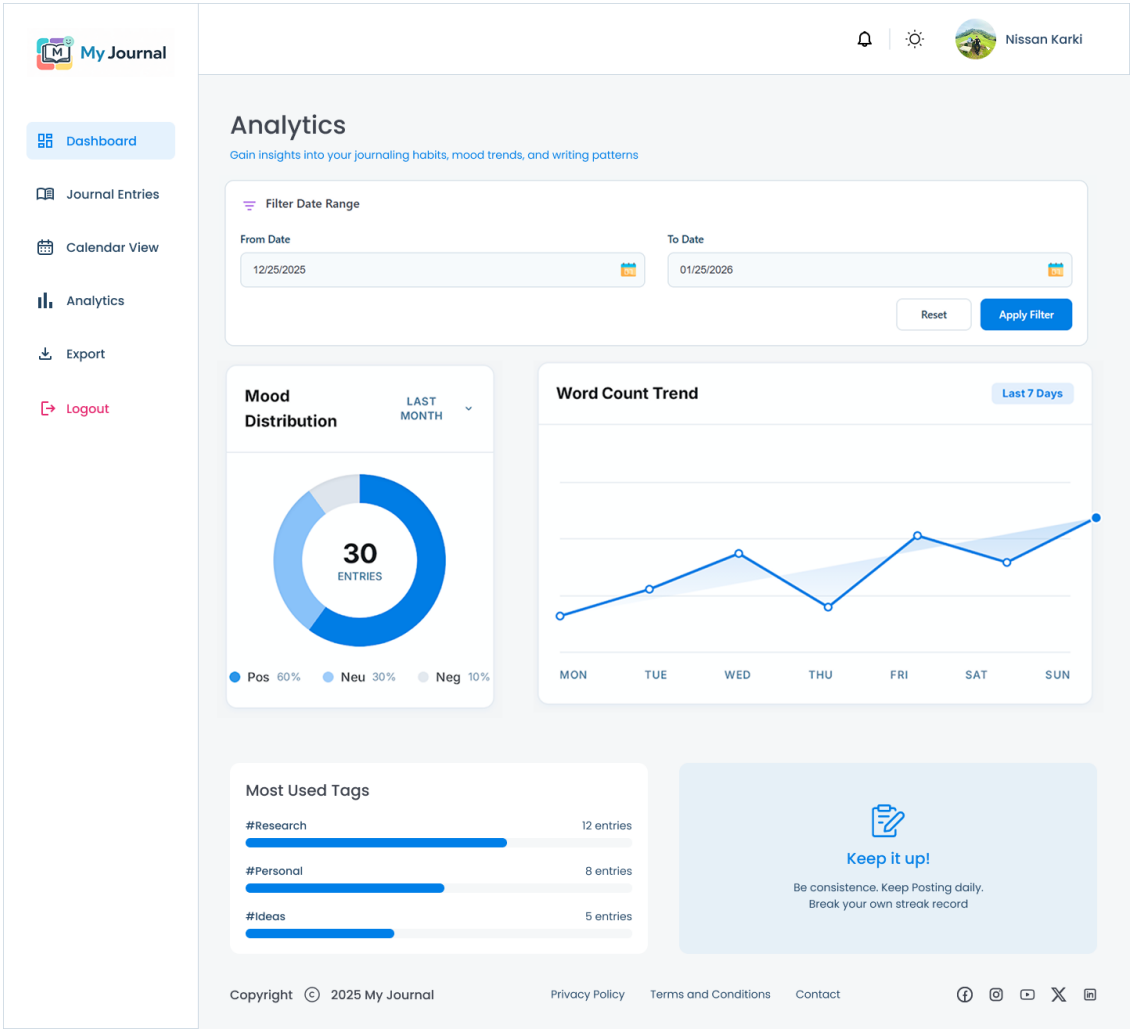


Figure 78: UI Design (Figma) of Analytics Page

3.1.8. Export Page

- Wireframe of Export Page

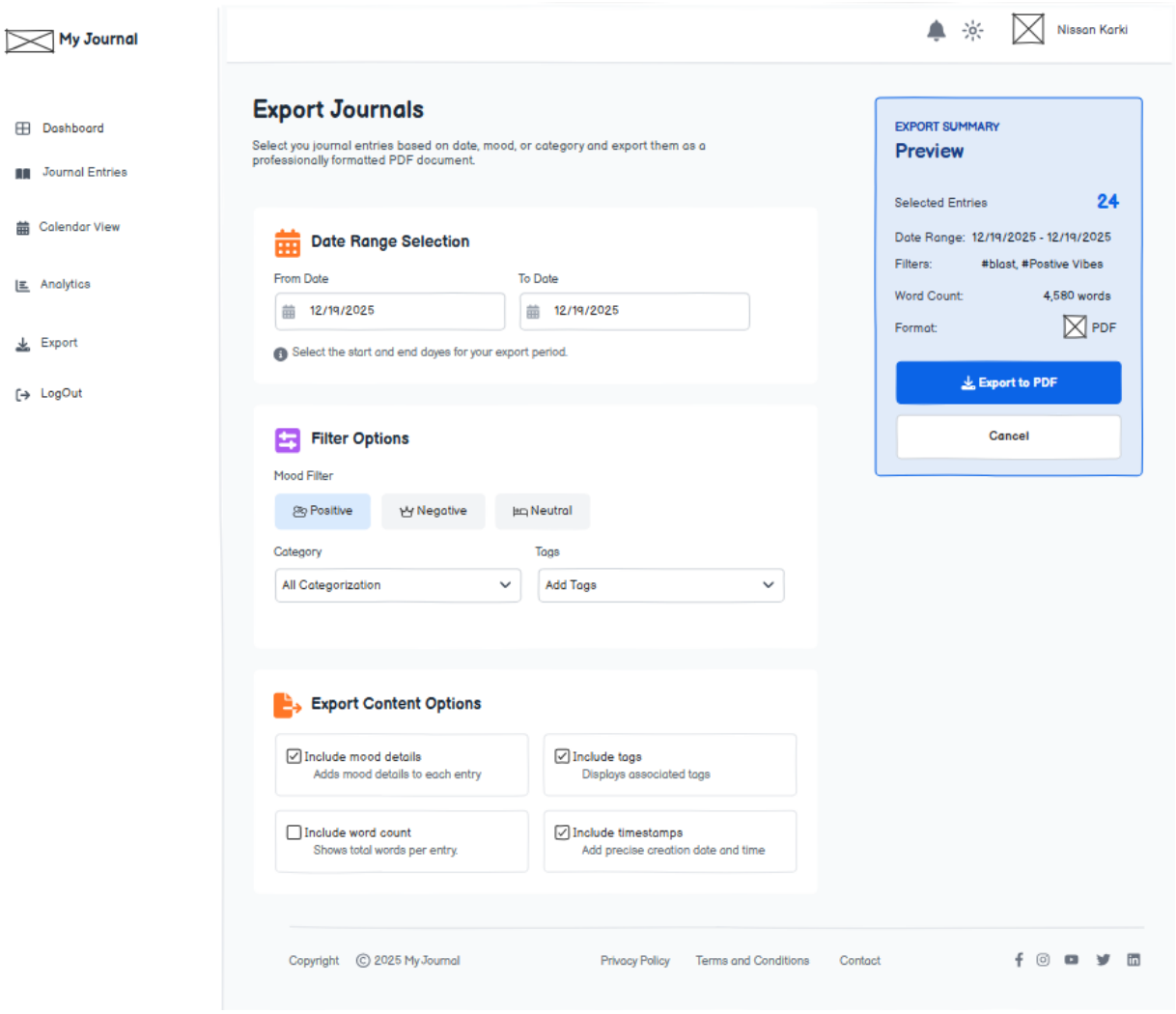


Figure 79: Wireframe of Export Page

- UI Design (Figma) of Export Page

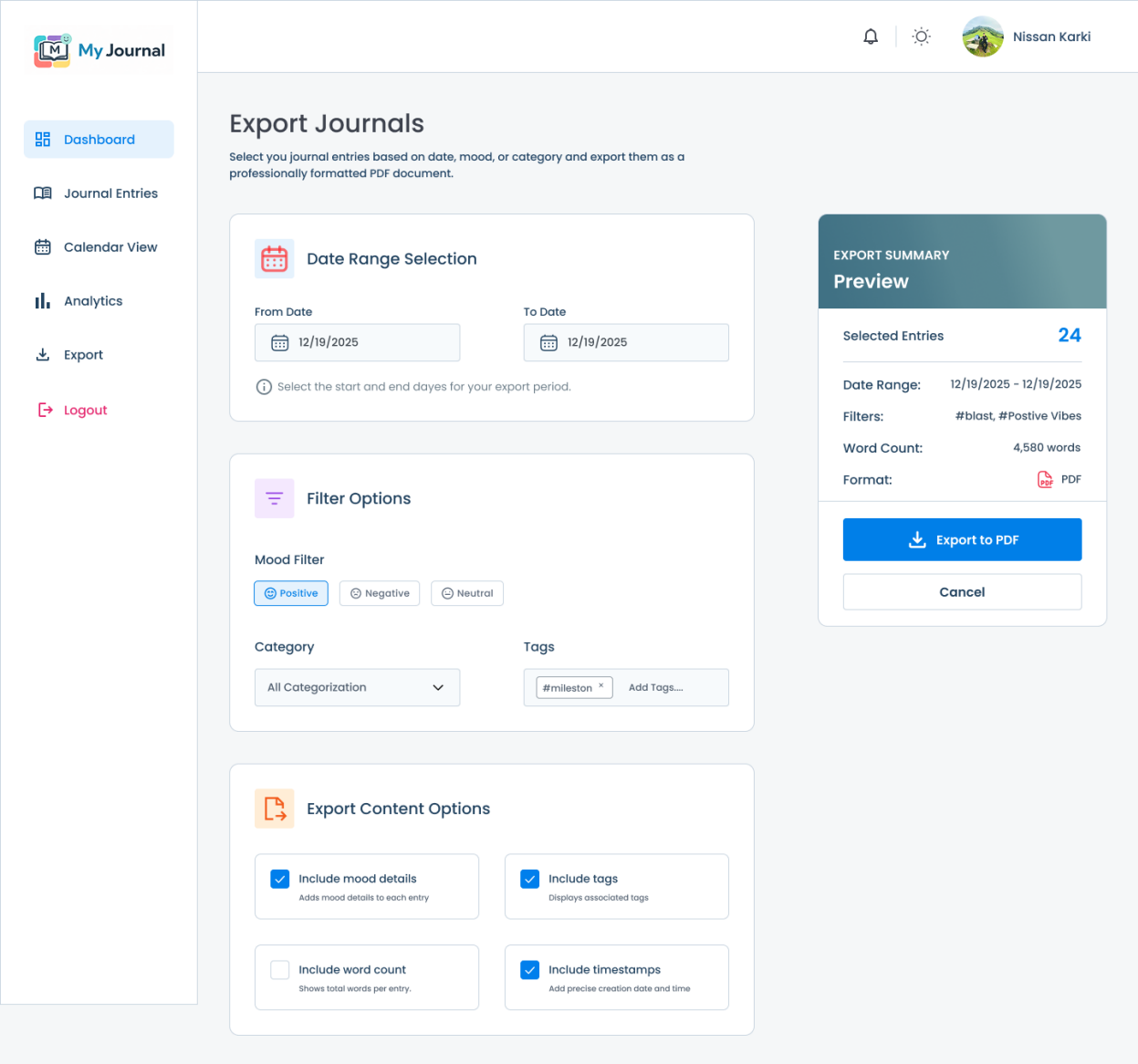


Figure 80: UI Design (Figma) of Export Page

3.2. Use Case Diagram

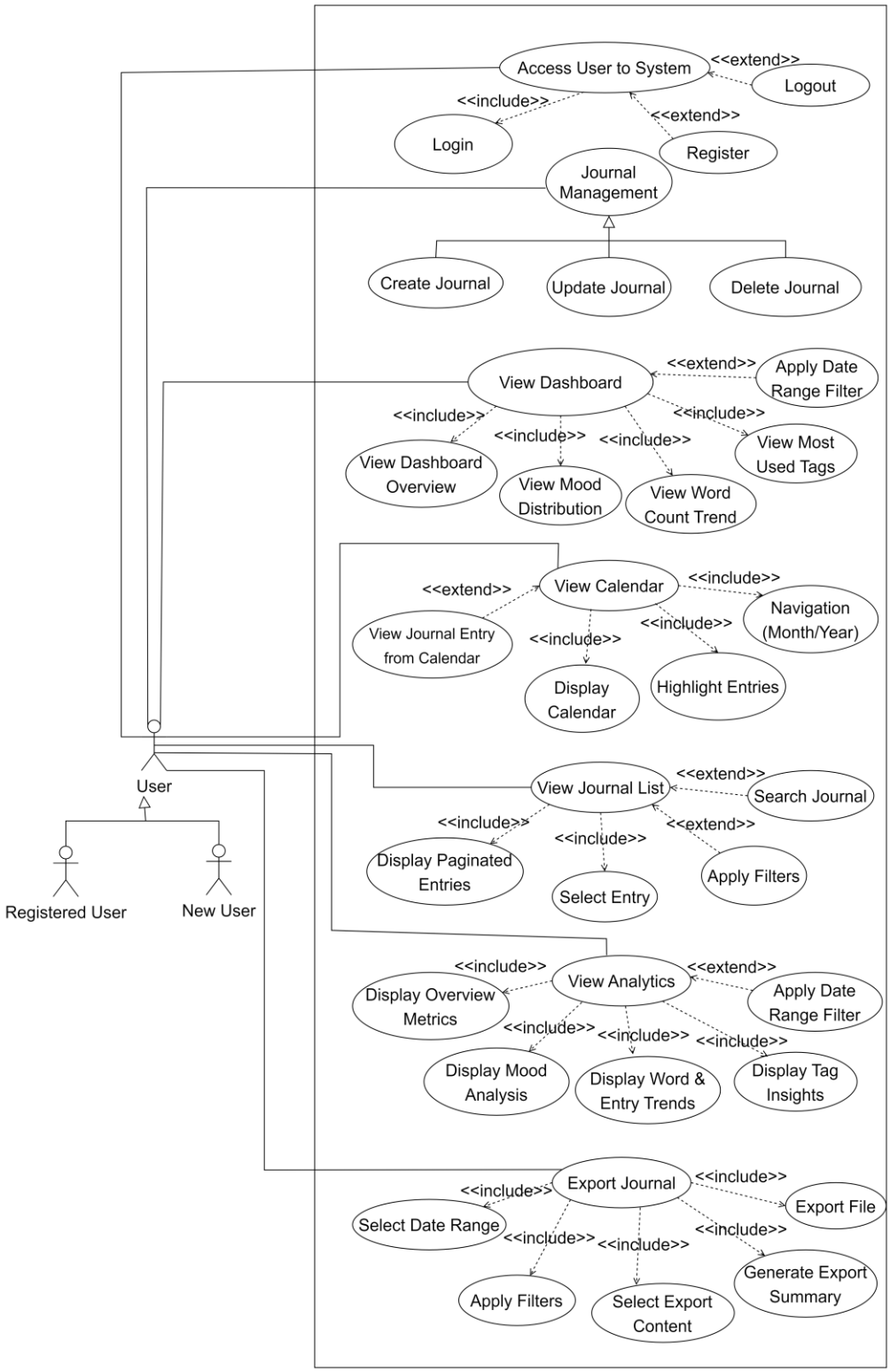


Figure 81: Use Case Diagram

3.3. Entity Relation Diagram

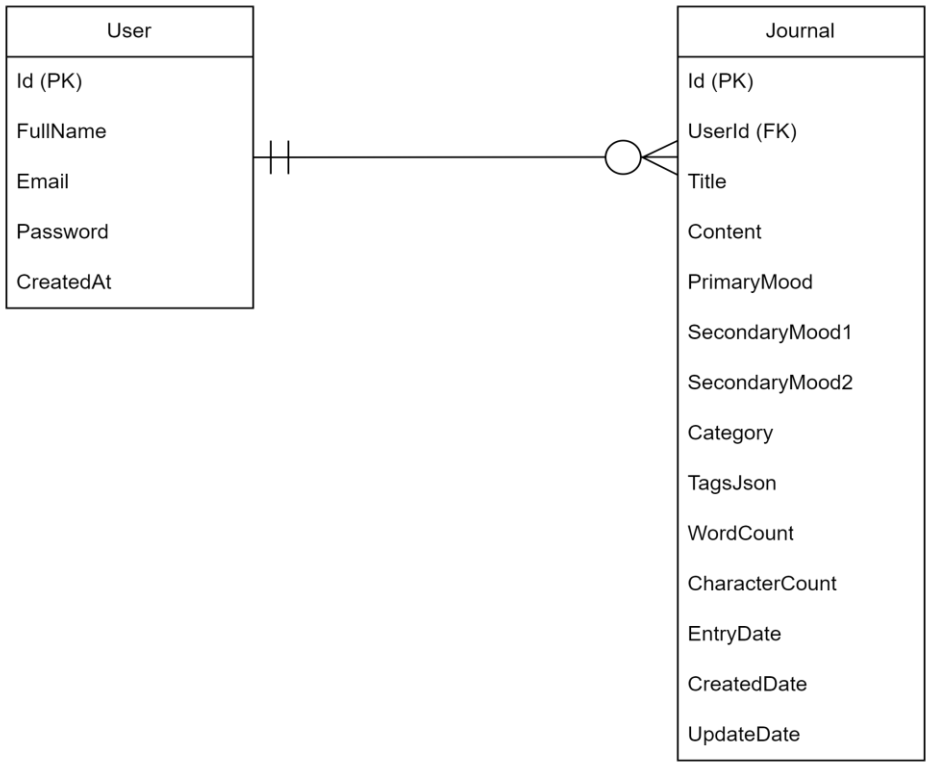


Figure 82: Entity Relation Diagram

Database Design Justification

The system uses a simplified database structure consisting of only two core tables User and Journals. This design was intentionally chosen to maintain simplicity, performance, and ease of maintenance while fully supporting all application features. User information is stores in the Users table, while all journaling-related data such as content, moods, tags, categories, dates, and analytics-related attributes are stored in the Journals table. Flexible data such as tags are stored using JSON format, reducing the need for additional relational tables. This approach minimizes database complexity, avoids unnecessary joins, improves query performance, and ensures a clean one-to-many relationship between users and journal entries. The design fully supports all applications functionalities while keeping the system scalable, efficient, and easy to manage.

3.4. Class Diagram

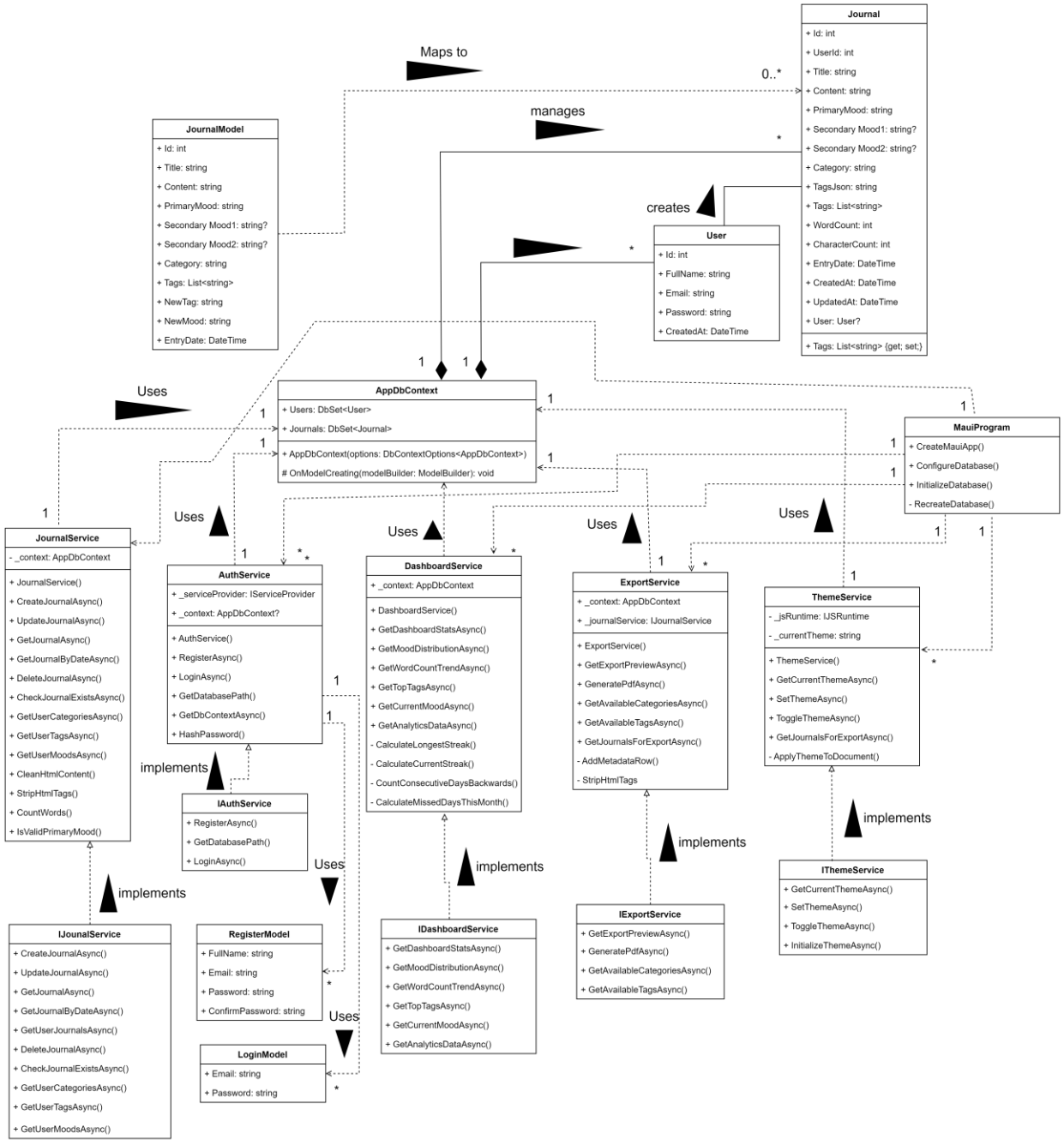


Figure 83: Class Diagram

3.5. VCS Repository

- Commits History

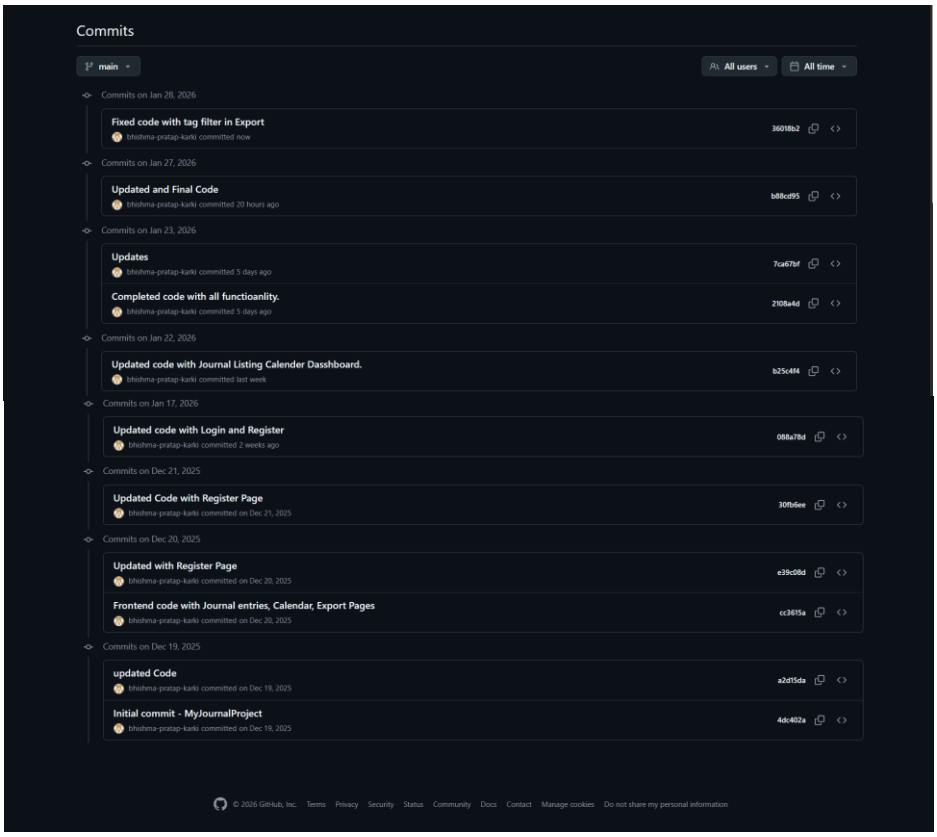


Figure 84: Commits History

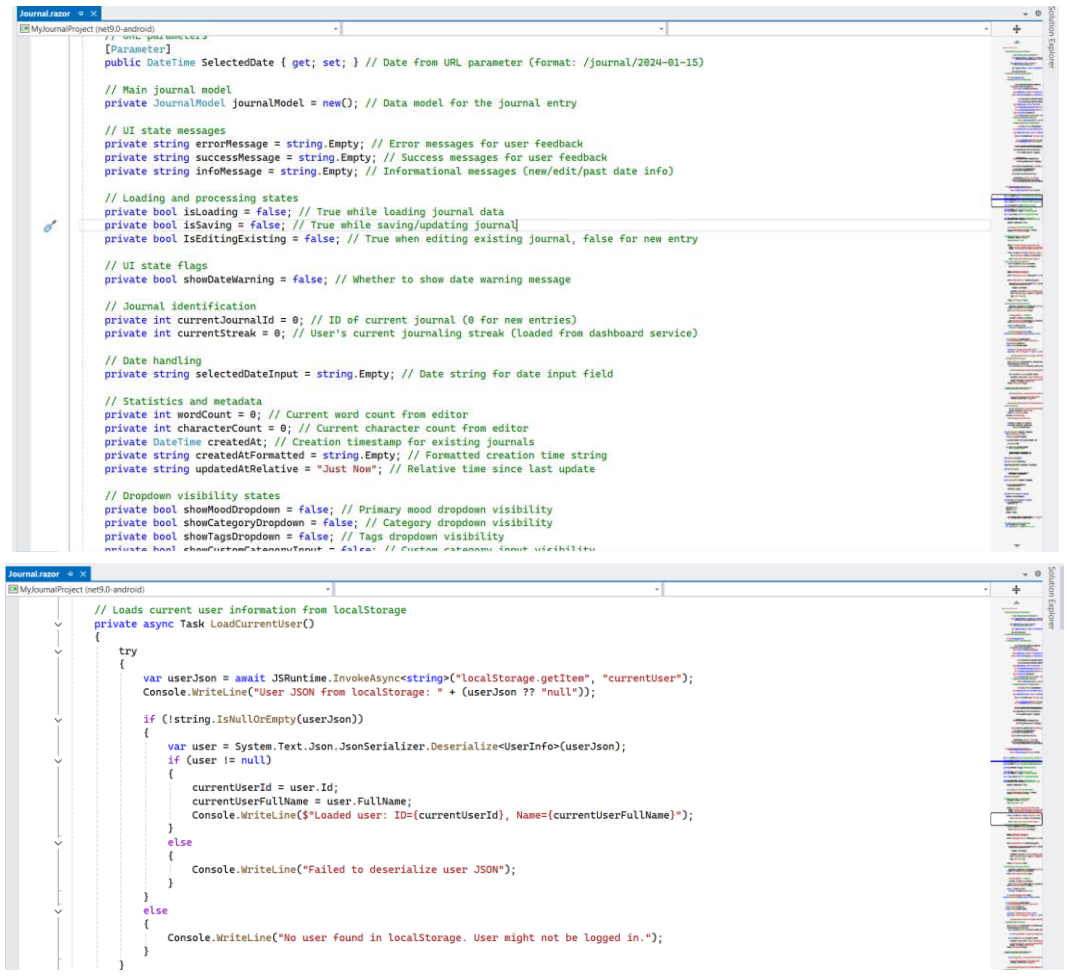
3.6. Quality

3.6.1. Code Readability

Code readability ensures that the system is easy to understand, maintain, and extend. The application achieves this through descriptive naming conventions, consistent formatting, structured logic, comprehensive documentation, and organized project structure.

Evidence:

- Descriptive variable and method naming improves clarity and understanding.
  Journal.razor



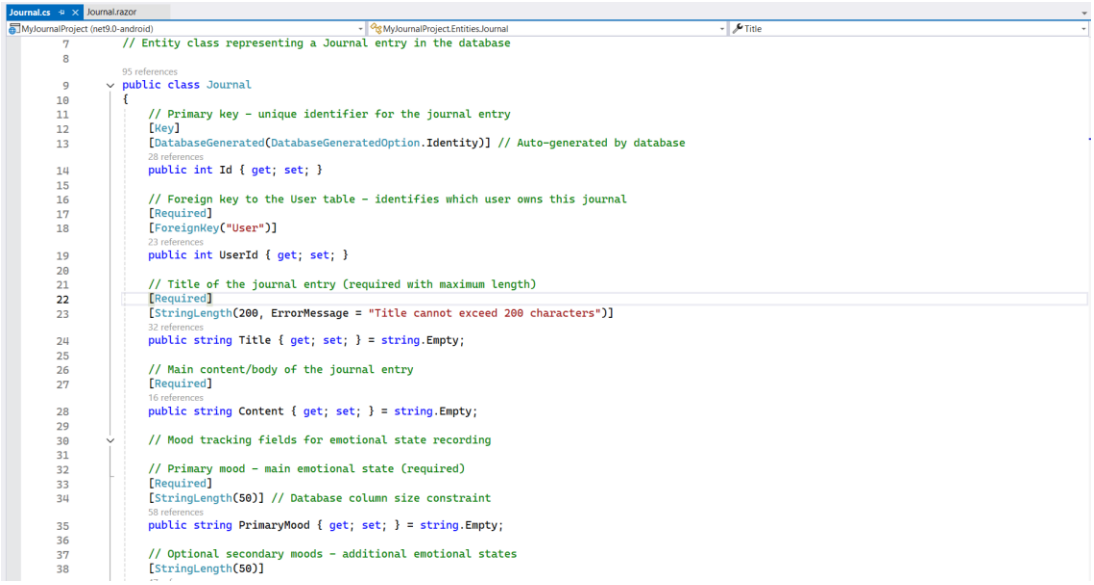
```
[Parameter]
public DateTime SelectedDate { get; set; } // Date from URL parameter (format: /journal/2024-01-15)

// Main journal model
private JournalModel journalModel = new(); // Data model for the journal entry

// UI state messages
private string errorMessage = string.Empty; // Error messages for user feedback
private string successMessage = string.Empty; // Success messages for user feedback
private string infoMessage = string.Empty; // Informational messages (new/edit/past date info)

// Loading and processing states
private bool isLoading = false; // True while loading journal data
private bool isSaving = false; // True while saving/updating journal
private bool IsEditingExisting = false; // True when editing existing journal, false for new entry

// UI state flags
private bool showDateWarning = false; // Whether to show date warning message

// Journal identification
private int currentJournalId = 0; // ID of current journal (0 for new entries)
private int currentStreak = 0; // User's current journaling streak (loaded from dashboard service)

// Date handling
private string selectedDateInput = string.Empty; // Date string for date input field

// Statistics and metadata
private int wordCount = 0; // Current word count from editor
private int characterCount = 0; // Current character count from editor
private DateTime createdAt; // Creation timestamp for existing journals
private string createdAtFormatted = string.Empty; // Formatted creation time string
private string updatedAtRelative = "Just Now"; // Relative time since last update

// Dropdown visibility states
private bool showMoodDropdown = false; // Primary mood dropdown visibility
private bool showCategoryDropdown = false; // Category dropdown visibility
private bool showTagsDropdown = false; // Tags dropdown visibility
```

```
// Loads current user information from localStorage
private async Task LoadCurrentUser()
{
    try
    {
        var userJson = await JSRuntime.InvokeAsync<string>("localStorage.getItem", "currentUser");
        Console.WriteLine("User JSON from localStorage: " + (userJson ?? "null"));

        if (!string.IsNullOrEmpty(userJson))
        {
            var user = System.Text.Json.JsonSerializer.Deserialize<UserInfo>(userJson);
            if (user != null)
            {
                currentUserId = user.Id;
                currentUserFullName = user.FullName;
                Console.WriteLine($"Loaded user: ID={currentUserId}, Name={currentUserFullName}");
            }
            else
            {
                Console.WriteLine("Failed to deserialize user JSON");
            }
        }
        else
        {
            Console.WriteLine("No user found in localStorage. User might not be logged in.");
        }
    }
```

Figure 85: Descriptive variable and method naming.

- Consistent Entity Property Organization.

  Journal.cs

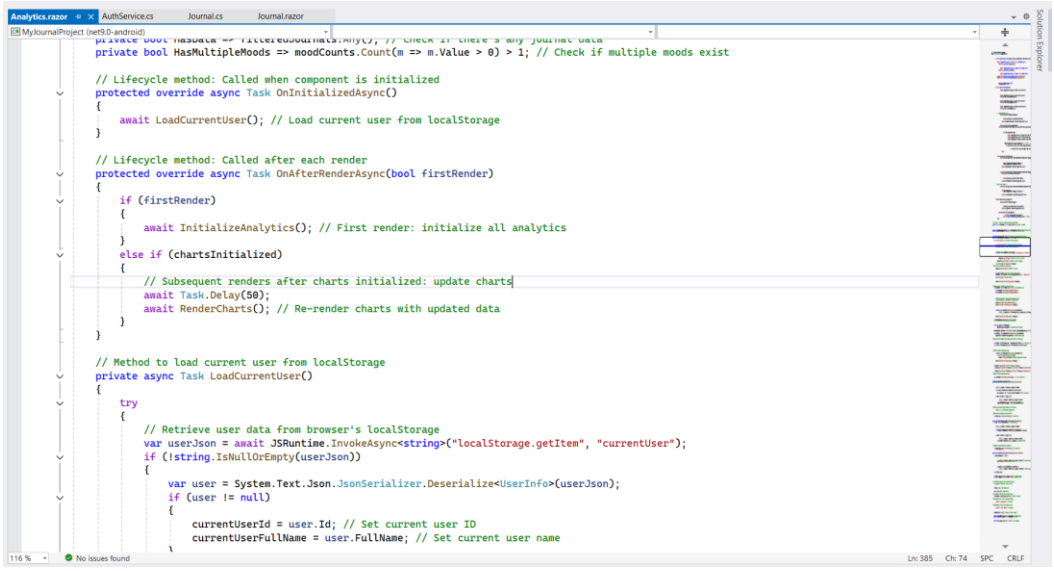


```
// Entity class representing a Journal entry in the database

95 references
public class Journal
{
    // Primary key - unique identifier for the journal entry
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)] // Auto-generated by database
    28 references
    public int Id { get; set; }

    // Foreign key to the User table - identifies which user owns this journal
    [Required]
    [ForeignKey("User")]
    23 references
    public int UserId { get; set; }

    // Title of the journal entry (required with maximum length)
    [Required]
    [StringLength(200, ErrorMessage = "Title cannot exceed 200 characters")]
    32 references
    public string Title { get; set; } = string.Empty;

    // Main content/body of the journal entry
    [Required]
    16 references
    public string Content { get; set; } = string.Empty;

    // Mood tracking fields for emotional state recording

    // Primary mood - main emotional state (required)
    [Required]
    [StringLength(50)] // Database column size constraint
    58 references
    public string PrimaryMood { get; set; } = string.Empty;

    // Optional secondary moods - additional emotional states
    [StringLength(50)]
```

Figure 86: Consistent Entity Property Organization.

- Inline comments and documentation explain system logic and workflows

  Analytics.razor



```
private bool HasMultipleMoods => moodCounts.Count(m => m.Value > 0) > 1; // Check if multiple moods exist

// Lifecycle method: Called when component is initialized
protected override async Task OnInitializedAsync()
{
    await LoadCurrentUser(); // Load current user from localStorage
}

// Lifecycle method: Called after each render
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        await InitializeAnalytics(); // First render: initialize all analytics
    }
    else if (chartsInitialized)
    {
        // Subsequent renders after charts initialized: update charts
        await Task.Delay(50);
        await RenderCharts(); // Re-render charts with updated data
    }
}

// Method to load current user from localStorage
private async Task LoadCurrentUser()
{
    try
    {
        // Retrieve user data from browser's localStorage
        var userJson = await JSRuntime.InvokeAsync<string>("localStorage.getItem", "currentUser");
        if (!string.IsNullOrEmpty(userJson))
        {
            var user = System.Text.Json.JsonSerializer.Deserialize<UserInfo>(userJson);
            if (user != null)
            {
                currentUserId = user.Id; // Set current user ID
                currentUserFullName = user.FullName; // Set current user name
```

Figure 87: Inline comments and documentation.

- Logical Folder and file structure improves navigation and maintainability

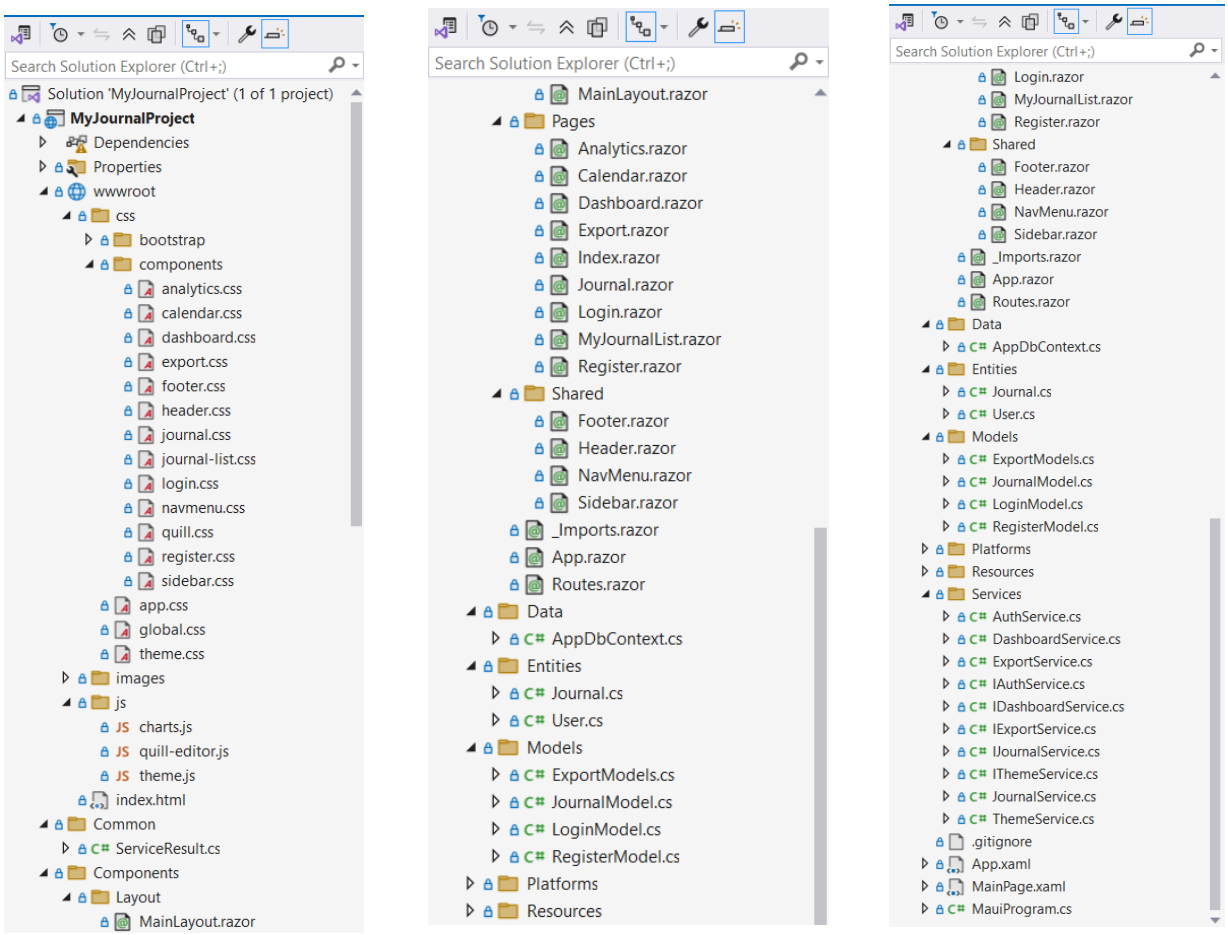


Figure 88: Logical Folder and file structure improves navigation and maintainability

3.6.2. Code Efficiency

The system is optimized for performance and resource usage through efficient data access, asynchronous operations, pagination, and optimized data processing.

Evidence:

- Efficient Database Querying with Filtering.

  JournalServics.cs

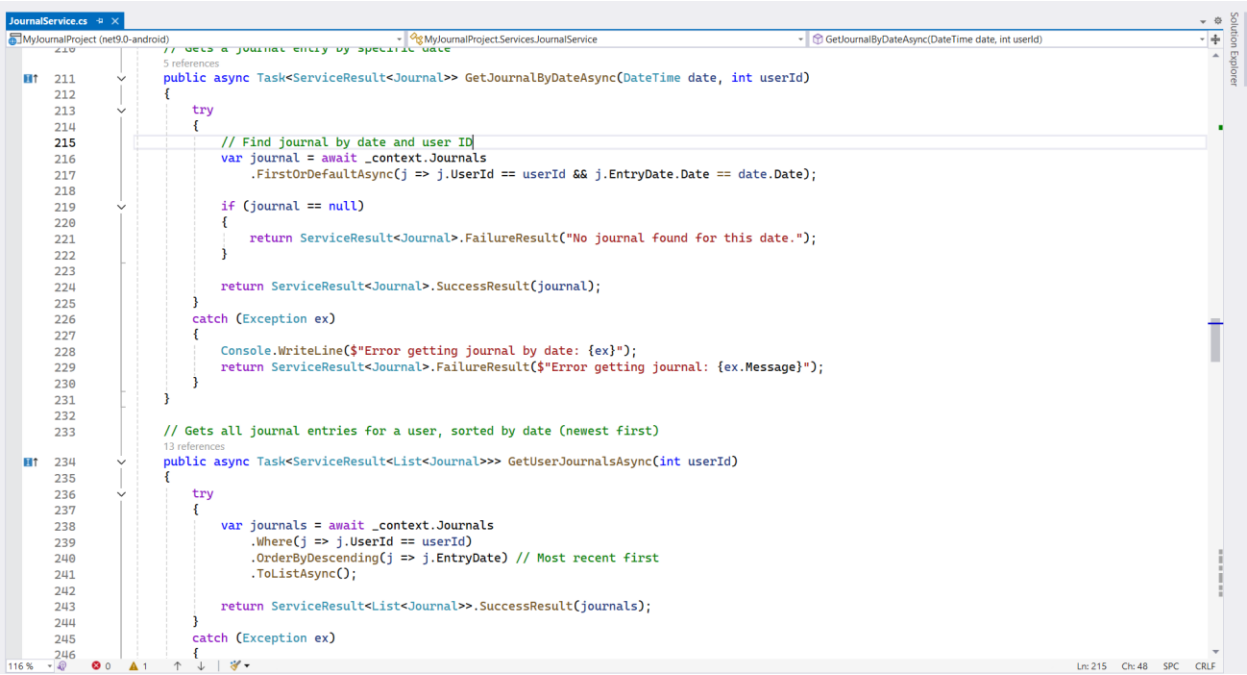
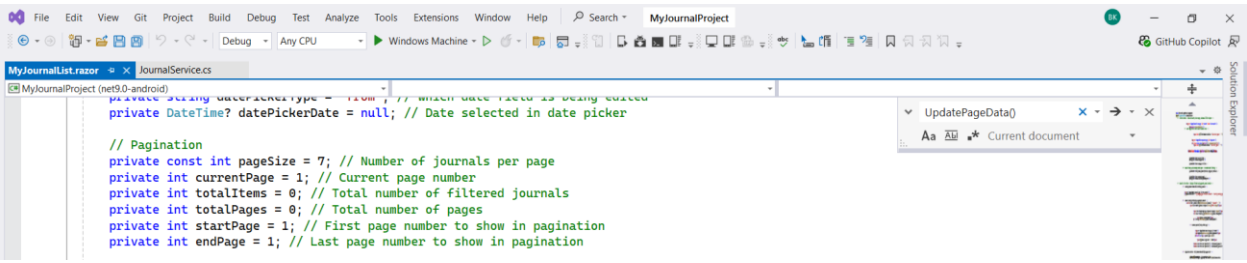
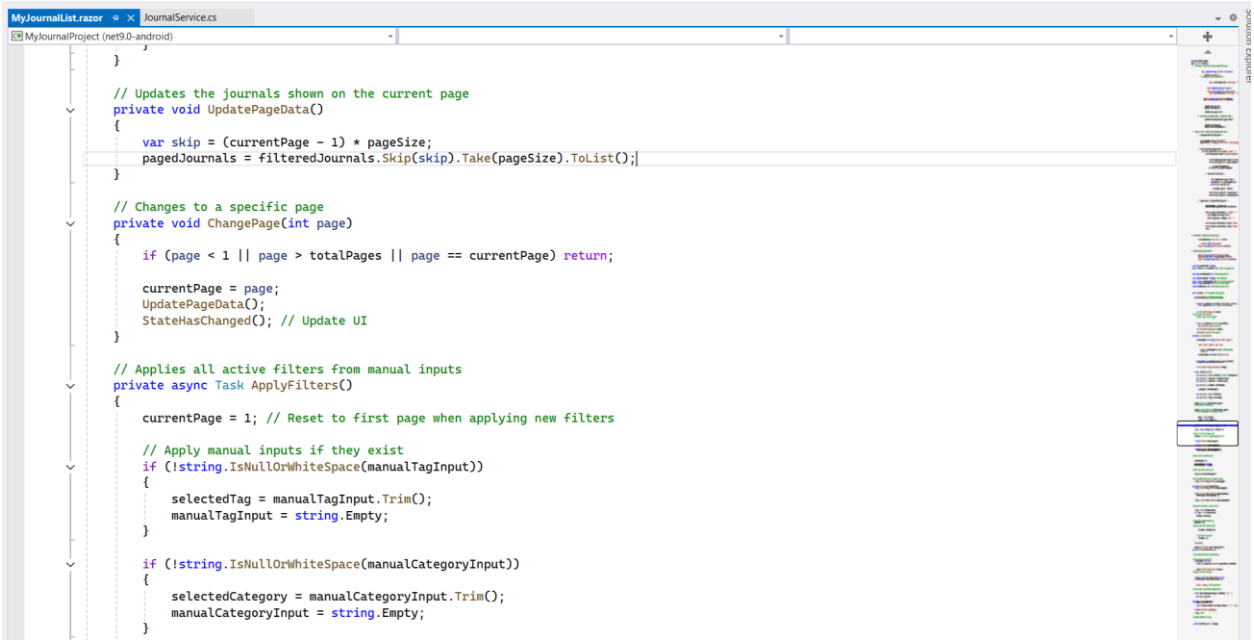


Figure 89: Efficient Database Querying with Filtering.

- Pagination loads only required records per page.

  MyJournalList.razor



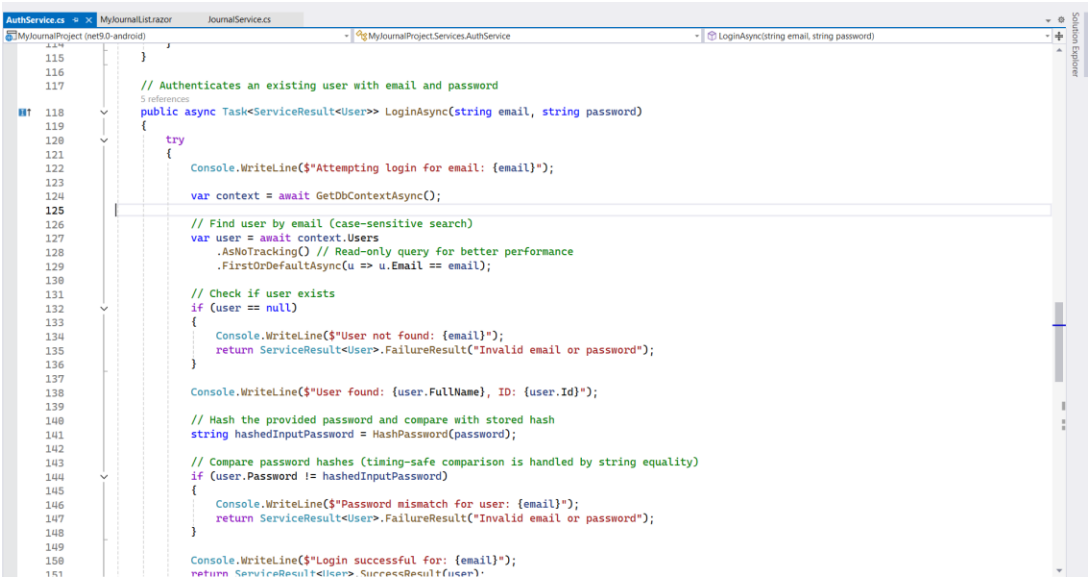


```
    }

    // Updates the journals shown on the current page
    private void UpdatePageData()
    {
        var skip = (currentPage - 1) * pageSize;
        pagedJournals = filteredJournals.Skip(skip).Take(pageSize).ToList();
    }

    // Changes to a specific page
    private void ChangePage(int page)
    {
        if (page < 1 || page > totalPages || page == currentPage) return;

        currentPage = page;
        UpdatePageData();
        StateHasChanged(); // Update UI
    }

    // Applies all active filters from manual inputs
    private async Task ApplyFilters()
    {
        currentPage = 1; // Reset to first page when applying new filters

        // Apply manual inputs if they exist
        if (!string.IsNullOrWhiteSpace(manualTagInput))
        {
            selectedTag = manualTagInput.Trim();
            manualTagInput = string.Empty;
        }

        if (!string.IsNullOrWhiteSpace(manualCategoryInput))
        {
            selectedCategory = manualCategoryInput.Trim();
            manualCategoryInput = string.Empty;
        }
```

Figure 90: Pagination loads only required records per page.

- Async/await ensures non-blocking operations and responsive UI.

AuthService.cs

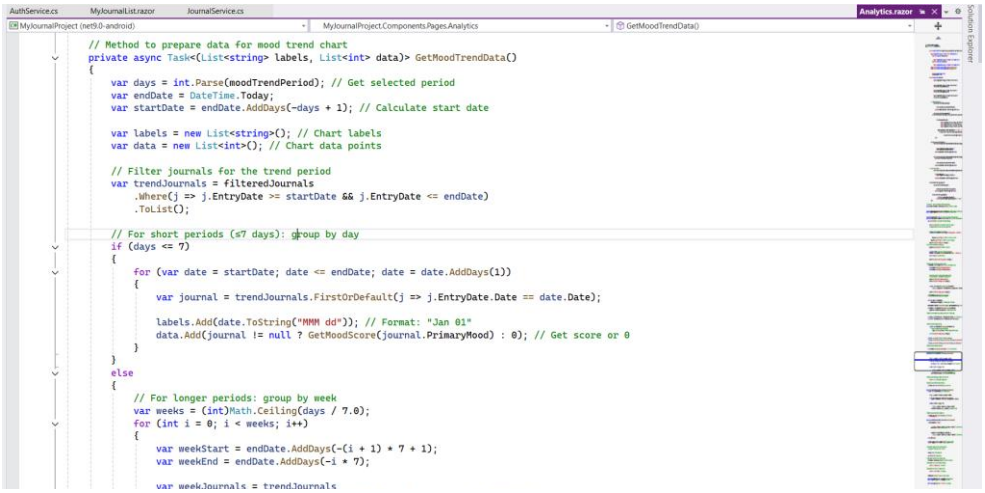


```
    }

    // Authenticates an existing user with email and password
    5 references
    public async Task<ServiceResult<User>> LoginAsync(string email, string password)
    {
        try
        {
            Console.WriteLine($"Attempting login for email: {email}");

            var context = await GetDbContextAsync();

            // Find user by email (case-sensitive search)
            var user = await context.Users
                .AsNoTracking() // Read-only query for better performance
                .FirstOrDefaultAsync(u => u.Email == email);

            // Check if user exists
            if (user == null)
            {
                Console.WriteLine($"User not found: {email}");
                return ServiceResult<User>.FailureResult("Invalid email or password");
            }

            Console.WriteLine($"User found: {user.FullName}, ID: {user.Id}");

            // Hash the provided password and compare with stored hash
            string hashedInputPassword = HashPassword(password);

            // Compare password hashes (timing-safe comparison is handled by string equality)
            if (user.Password != hashedInputPassword)
            {
                Console.WriteLine($"Password mismatch for user: {email}");
                return ServiceResult<User>.FailureResult("Invalid email or password");
            }

            Console.WriteLine($"Login successful for: {email}");
```

Figure 91: Async/await ensures non-blocking operations and responsive UI.

- Optimized Chart Data Processing.

  Analaytics.razor



```
// Method to prepare data for mood trend chart
private async Task<(List<string> labels, List<int> data)> GetMoodTrendData()
{
    var days = int.Parse(moodTrendPeriod); // Get selected period
    var endDate = DateTime.Today;
    var startDate = endDate.AddDays(-days + 1); // Calculate start date

    var labels = new List<string>(); // Chart labels
    var data = new List<int>(); // Chart data points

    // Filter journals for the trend period
    var trendJournals = filteredJournals
        .Where(j => j.EntryDate >= startDate && j.EntryDate <= endDate)
        .ToList();

    // For short periods (≤7 days): group by day
    if (days <= 7)
    {
        for (var date = startDate; date <= endDate; date = date.AddDays(1))
        {
            var journal = trendJournals.FirstOrDefault(j => j.EntryDate.Date == date.Date);

            labels.Add(date.ToString("MMM dd")); // Format: "Jan 01"
            data.Add(journal != null ? GetMoodScore(journal.PrimaryMood) : 0); // Get score or 0
        }
    }
    else
    {
        // For longer periods: group by week
        var weeks = (int)Math.Ceiling(days / 7.0);
        for (int i = 0; i < weeks; i++)
        {
            var weekStart = endDate.AddDays(-(i + 1) * 7 + 1);
            var weekEnd = endDate.AddDays(-i * 7);

            var weekJournals = trendJournals
```

Figure 92: Optimized Chart Data Processing.

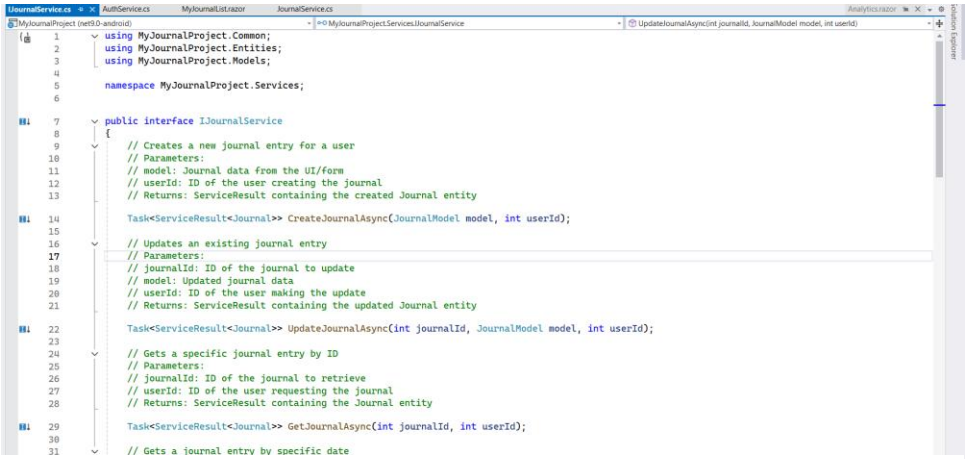
3.6.3. Code Modularity

The system follows a modular architecture using separation of concerns, service layer, interface-based design, dependency injection, and reusable components.

Evidence:

- Interface-Based Service Design.

  IJournalService.cs

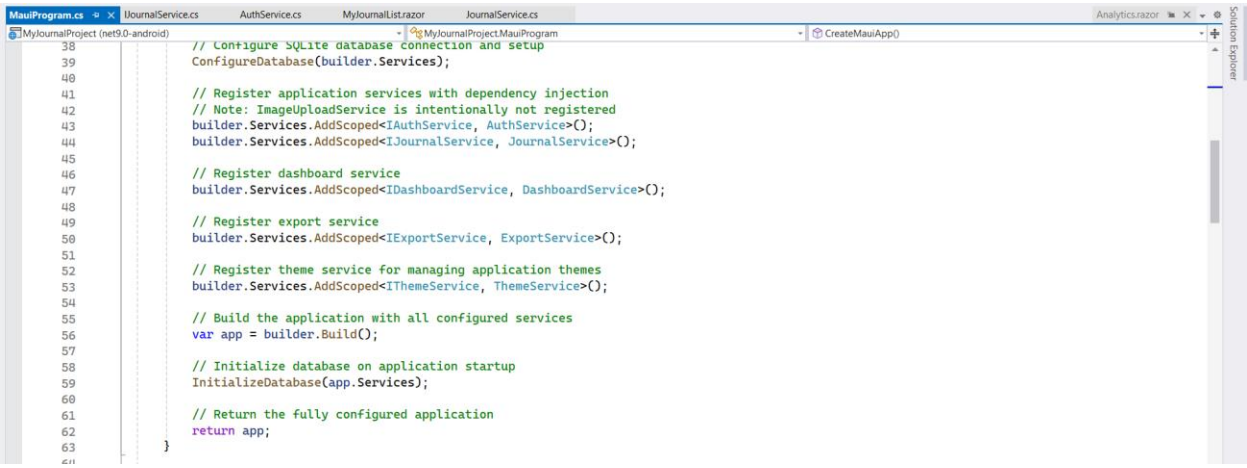


```
using MyJournalProject.Common;
using MyJournalProject.Entities;
using MyJournalProject.Models;

namespace MyJournalProject.Services;

public interface IJournalService
{
    // Creates a new journal entry for a user
    // Parameters:
    // model: Journal data from the UI/form
    // userId: ID of the user creating the journal
    // Returns: ServiceResult containing the created Journal entity

    Task<ServiceResult<Journal>> CreateJournalAsync(JournalModel model, int userId);

    // Updates an existing journal entry
    // Parameters:
    // journalId: ID of the journal to update
    // model: Updated journal data
    // userId: ID of the user making the update
    // Returns: ServiceResult containing the updated Journal entity

    Task<ServiceResult<Journal>> UpdateJournalAsync(int journalId, JournalModel model, int userId);

    // Gets a specific journal entry by ID
    // Parameters:
    // journalId: ID of the journal to retrieve
    // userId: ID of the user requesting the journal
    // Returns: ServiceResult containing the Journal entity

    Task<ServiceResult<Journal>> GetJournalAsync(int journalId, int userId);

    // Gets a journal entry by specific date
```

Figure 93: Interface-Based Service Design

- Dependency Injection Configuration.

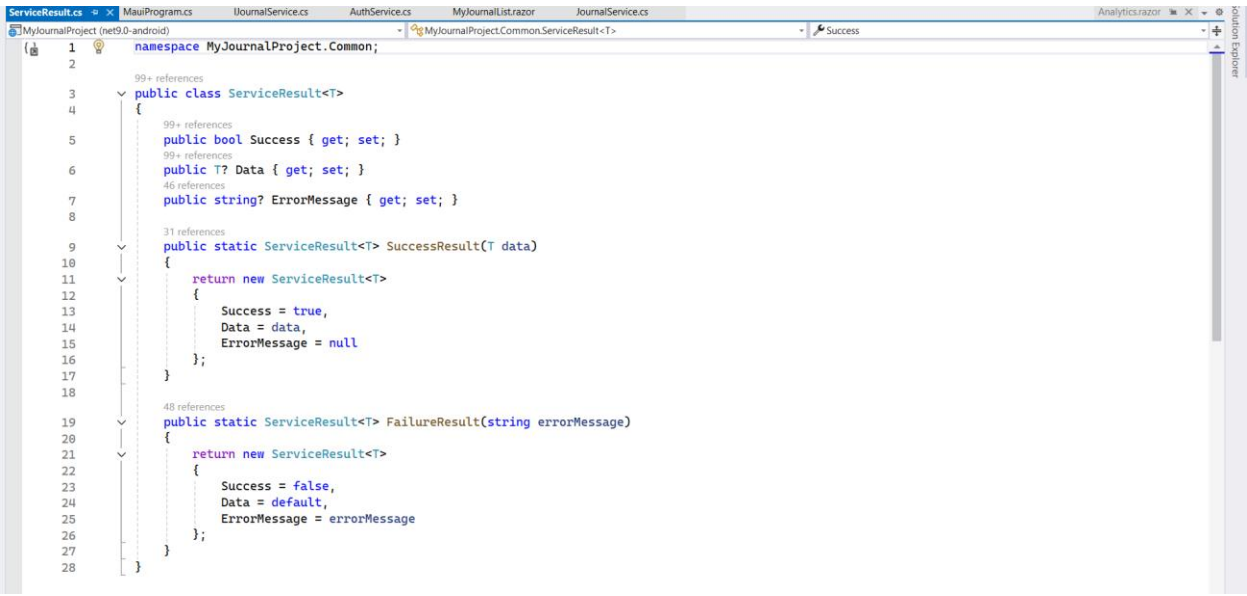
  MauiProgram.cs



```
// Configure SQLite database connection and setup
ConfigureDatabase(builder.Services);

// Register application services with dependency injection
// Note: ImageUploadService is intentionally not registered
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IJournalService, JournalService>();

// Register dashboard service
builder.Services.AddScoped<IDashboardService, DashboardService>();

// Register export service
builder.Services.AddScoped<IExportService, ExportService>();

// Register theme service for managing application themes
builder.Services.AddScoped<IThemeService, ThemeService>();

// Build the application with all configured services
var app = builder.Build();

// Initialize database on application startup
InitializeDatabase(app.Services);

// Return the fully configured application
return app;
}
```

Figure 94: Dependency Injection Configuration.

- Reusable ServiceResult Pattern.



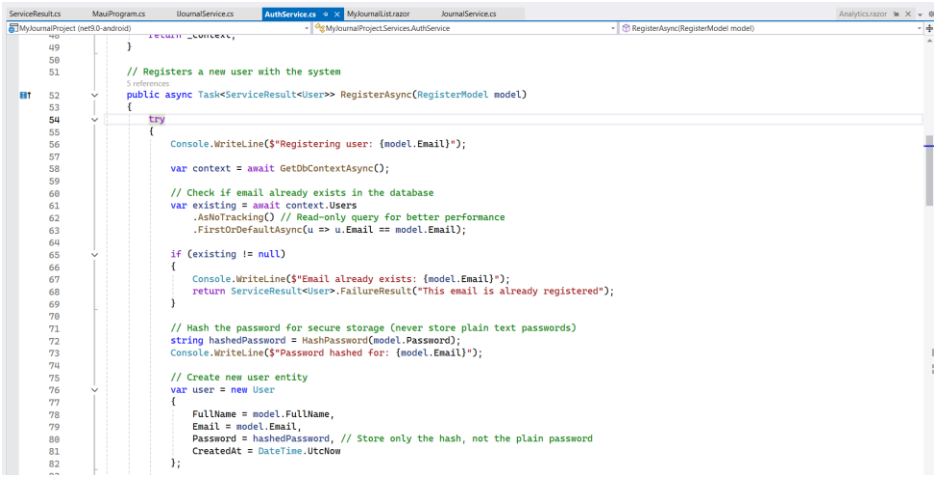
```
namespace MyJournalProject.Common;

99+ references
public class ServiceResult<T>
{
    99+ references
    public bool Success { get; set; }
    99+ references
    public T? Data { get; set; }
    46 references
    public string? ErrorMessage { get; set; }

    31 references
    public static ServiceResult<T> SuccessResult(T data)
    {
        return new ServiceResult<T>
        {
            Success = true,
            Data = data,
            ErrorMessage = null
        };
    }

    48 references
    public static ServiceResult<T> FailureResult(string errorMessage)
    {
        return new ServiceResult<T>
        {
            Success = false,
            Data = default,
            ErrorMessage = errorMessage
        };
    }
}
```

Figure 95: Reusable ServiceResult Pattern

3.6.4. Error Handling

The system implements strong and reliable error handling to ensure stability, user safety, and system reliability through validation, exception handling, fallbacks, and user-friendly feedback.

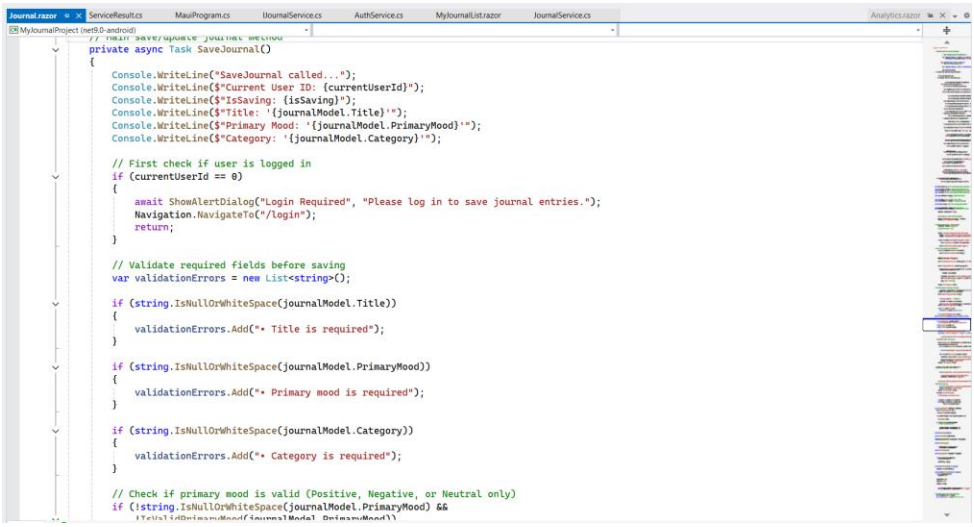
- Structured try-catch handling with specific exception handling.
  AuthService.cs



```
// Registers a new user with the system
public async Task<ServiceResult<User>> RegisterAsync(RegisterModel model)
{
    try
    {
        Console.WriteLine($"Registering user: {model.Email}");

        var context = await GetDbContextAsync();

        // Check if email already exists in the database
        var existing = await context.Users
            .AsNoTracking() // Read-only query for better performance
            .FirstOrDefaultAsync(u => u.Email == model.Email);

        if (existing != null)
        {
            Console.WriteLine($"Email already exists: {model.Email}");
            return ServiceResult<User>.FailureResult("This email is already registered");
        }

        // Hash the password for secure storage (never store plain text passwords)
        string hashedPassword = HashPassword(model.Password);
        Console.WriteLine($"Password hashed for: {model.Email}");

        // Create new user entity
        var user = new User
        {
            FullName = model.FullName,
            Email = model.Email,
            Password = hashedPassword, // Store only the hash, not the plain password
            CreatedAt = DateTime.UtcNow
        };
```

Figure 96: Structured try-catch handling with specific exception handling.

- Input Validation with User Feedback.
  Journal.razor

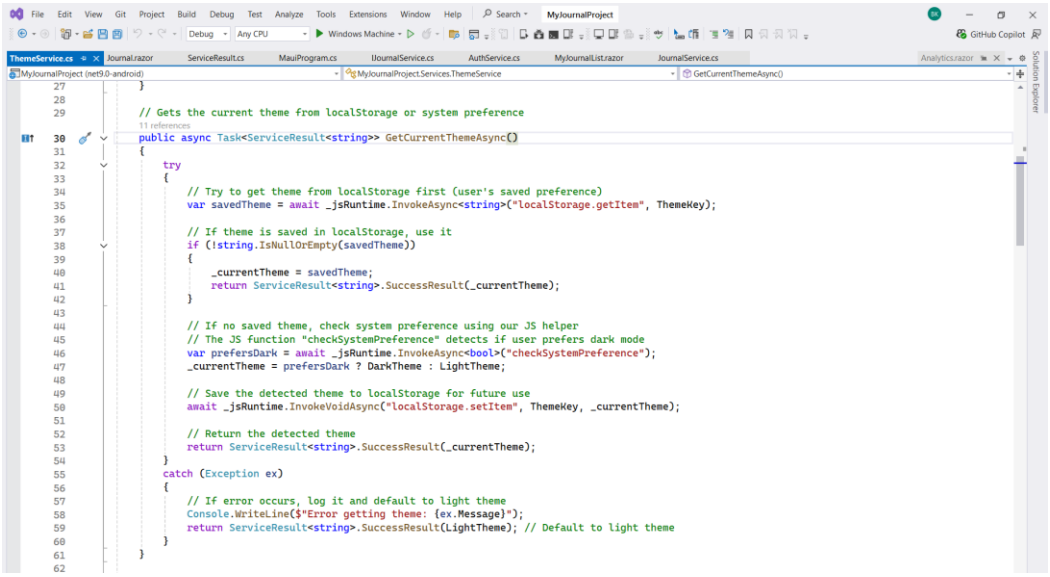


```
private async Task SaveJournal()
{
    Console.WriteLine("SaveJournal called...");
    Console.WriteLine($"Current User ID: {currentUserId}");
    Console.WriteLine($"IsSaving: {isSaving}");
    Console.WriteLine($"Title: '{journalModel.Title}'");
    Console.WriteLine($"Primary Mood: '{journalModel.PrimaryMood}'");
    Console.WriteLine($"Category: '{journalModel.Category}'");

    // First check if user is logged in
    if (currentUserId == 0)
    {
        await ShowAlertDialog("Login Required", "Please log in to save journal entries.");
        Navigation.NavigateTo("/login");
        return;
    }

    // Validate required fields before saving
    var validationErrors = new List<string>();

    if (string.IsNullOrWhiteSpace(journalModel.Title))
    {
        validationErrors.Add("• Title is required");
    }

    if (string.IsNullOrWhiteSpace(journalModel.PrimaryMood))
    {
        validationErrors.Add("• Primary mood is required");
    }

    if (string.IsNullOrWhiteSpace(journalModel.Category))
    {
        validationErrors.Add("• Category is required");
    }

    // Check if primary mood is valid (Positive, Negative, or Neutral only)
    if (!string.IsNullOrWhiteSpace(journalModel.PrimaryMood) &&
```

Figure 97: Input Validation with User Feedback.

- Degradation and Fallbacks.

  ThemeService.cs

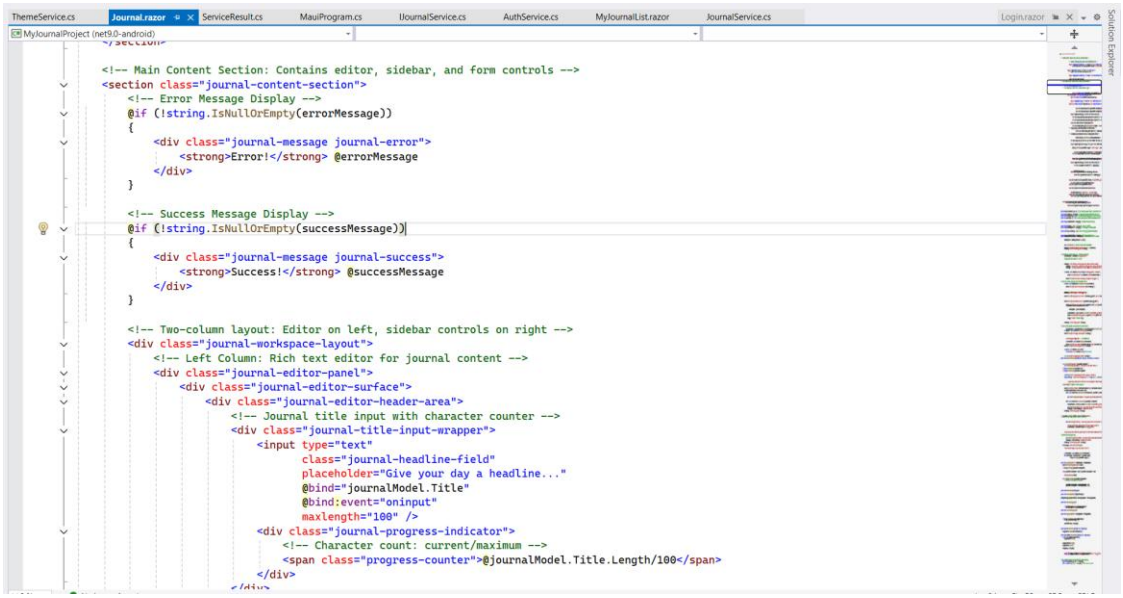


```
27          }
28
29          // Gets the current theme from localStorage or system preference
            11 references
30          public async Task<ServiceResult<string>> GetCurrentThemeAsync()
31          {
32              try
33              {
34                  // Try to get theme from localStorage first (user's saved preference)
35                  var savedTheme = await _jsRuntime.InvokeAsync<string>("localStorage.getItem", ThemeKey);
36
37                  // If theme is saved in localStorage, use it
38                  if (!string.IsNullOrEmpty(savedTheme))
39                  {
40                      _currentTheme = savedTheme;
41                      return ServiceResult<string>.SuccessResult(_currentTheme);
42                  }
43
44                  // If no saved theme, check system preference using our JS helper
45                  // The JS function "checkSystemPreference" detects if user prefers dark mode
46                  var prefersDark = await _jsRuntime.InvokeAsync<bool>("checkSystemPreference");
47                  _currentTheme = prefersDark ? DarkTheme : LightTheme;
48
49                  // Save the detected theme to localStorage for future use
50                  await _jsRuntime.InvokeVoidAsync("localStorage.setItem", ThemeKey, _currentTheme);
51
52                  // Return the detected theme
53                  return ServiceResult<string>.SuccessResult(_currentTheme);
54              }
55              catch (Exception ex)
56              {
57                  // If error occurs, log it and default to light theme
58                  Console.WriteLine($"Error getting theme: {ex.Message}");
59                  return ServiceResult<string>.SuccessResult(LightTheme); // Default to light theme
60              }
61          }
62
```

Figure 98: Graceful Degradation and Fallbacks.

- User-friendly error and success message display.

  Journal.razor

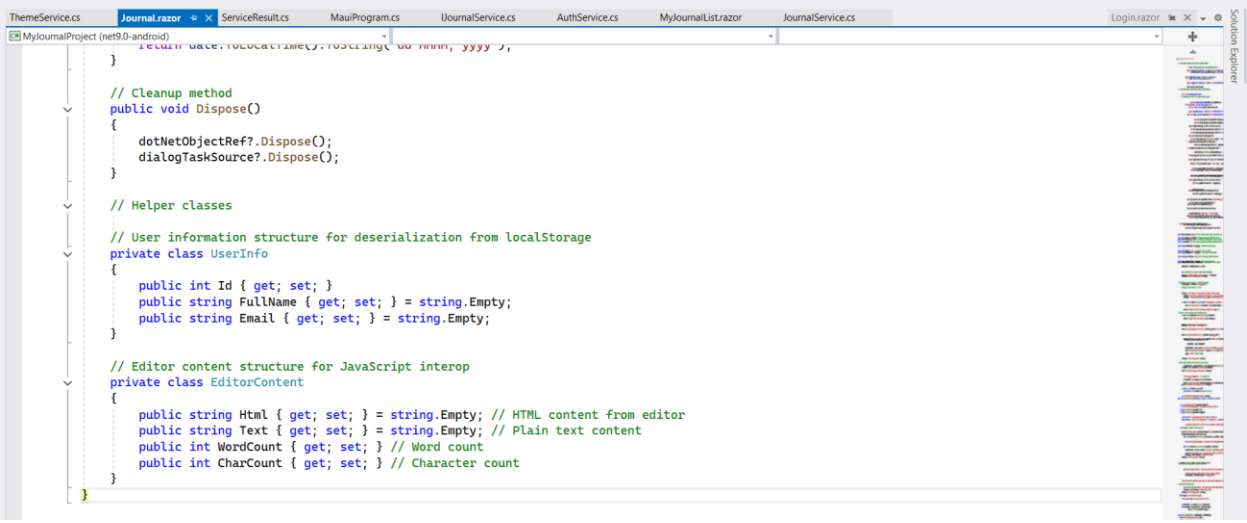


```
<!-- Main Content Section: Contains editor, sidebar, and form controls -->
<section class="journal-content-section">
    <!-- Error Message Display -->
    @if (!string.IsNullOrEmpty(errorMessage))
    {
        <div class="journal-message journal-error">
            <strong>Error!</strong> @errorMessage
        </div>
    }

    <!-- Success Message Display -->
    @if (!string.IsNullOrEmpty(successMessage))
    {
        <div class="journal-message journal-success">
            <strong>Success!</strong> @successMessage
        </div>
    }

    <!-- Two-column layout: Editor on left, sidebar controls on right -->
    <div class="journal-workspace-layout">
        <!-- Left Column: Rich text editor for journal content -->
        <div class="journal-editor-panel">
            <div class="journal-editor-surface">
                <div class="journal-editor-header-area">
                    <!-- Journal title input with character counter -->
                    <div class="journal-title-input-wrapper">
                        <input type="text"
                               class="journal-headline-field"
                               placeholder="Give your day a headline..."
                               @bind="journalModel.Title"
                               @bind:event="oninput"
                               maxlength="100" />
                        <div class="journal-progress-indicator">
                            <!-- Character count: current/maximum -->
                            <span class="progress-counter">@journalModel.Title.Length/100</span>
                        </div>
```

Figure 99: User-friendly error and success message display.

- Proper Resource Cleanup.

  Journal.razor




```
    }

    // Cleanup method
    public void Dispose()
    {
        dotNetObjectRef?.Dispose();
        dialogTaskSource?.Dispose();
    }

    // Helper classes

    // User information structure for deserialization from localStorage
    private class UserInfo
    {
        public int Id { get; set; }
        public string FullName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
    }

    // Editor content structure for JavaScript interop
    private class EditorContent
    {
        public string Html { get; set; } = string.Empty; // HTML content from editor
        public string Text { get; set; } = string.Empty; // Plain text content
        public int WordCount { get; set; } // Word count
        public int CharCount { get; set; } // Character count
    }
}
```

Figure 100: Proper Resource Cleanup.

3.7. Testing

3.7.1. Test Case 1: Successful User Registration with Password Hashing

Table 2: Test Case 1: Successful User Registration with Password Hashing

Test Case 1: Successful User Registration with Password Hashing	
Objective	To verify that a new user can register successfully when valid full name, email, and password are provided, and that the password is securely stored using hashing.
Action	i. Open the Journaling application. ii. Navigate to the Register form. iii. Enter the Full Name: John Doe. iv. Enter Email: john@example.com. v. Enter Password: Password@123. vi. Enter Confirm Password: Password@123 vii. Click the Register "Register" button
Expected Result	i. The system should create a new user account. ii. A confirmation message should be displayed: "Success! Account created successfully! Redirecting to login…". iii. User details should be stored correctly in the database. iv. Password should be stored in hashed format. v. No errors or warnings should occur. vi. User should be redirected to the Login Page.
Actual Result	i. User registered successfully. ii. Confirmation message displayed. iii. User entry John Doe created in SQLite database. iv. Password stored in hashed format in the database. iv. User redirected to the Login Page.
Conclusion	The test was successfully implemented. This confirms that the registration functionality works correctly for valid input, the system securely stores passwords using hashing, user data is saved

	accurately, and the user is redirected to the Login page after successful registration.

- Registration Form with User Details Entered.

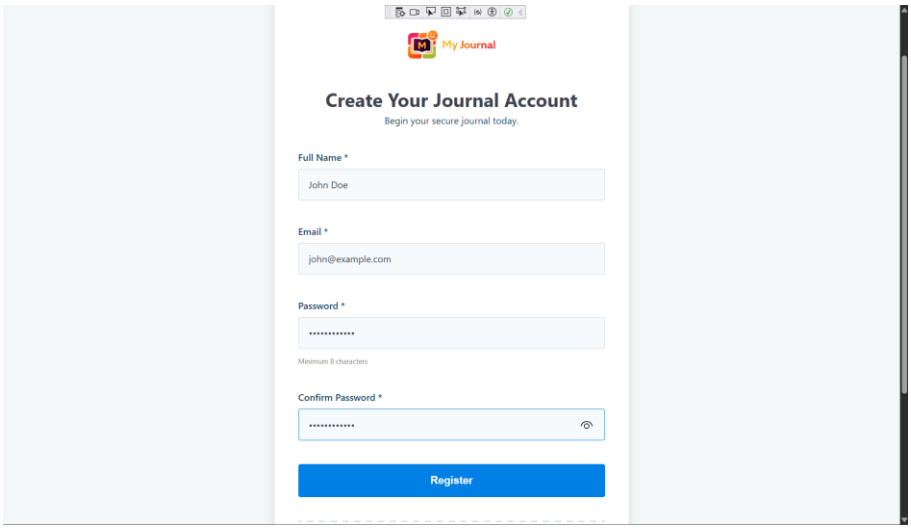


Figure 101: Entering Full Name, Email and Password

- Registration Success Confirmation Message and redirection to the Login Page.

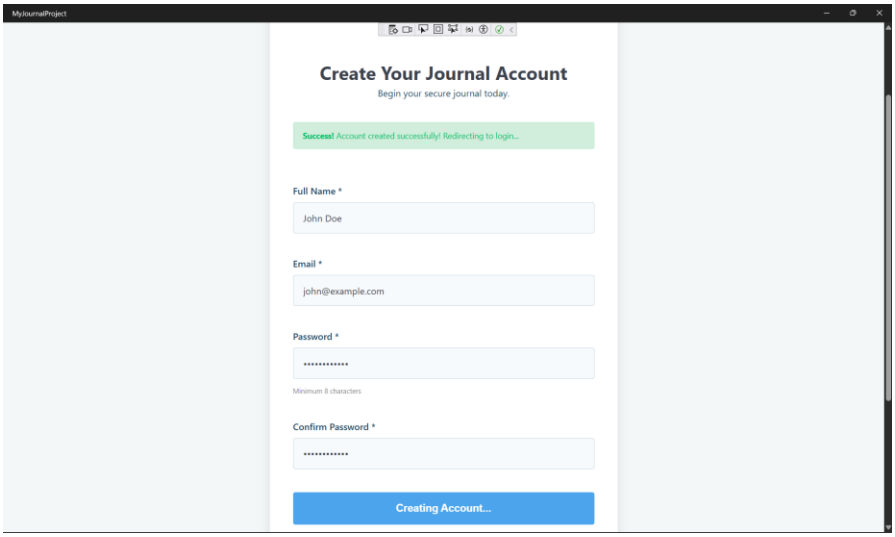


Figure 102: Click on the Register Button

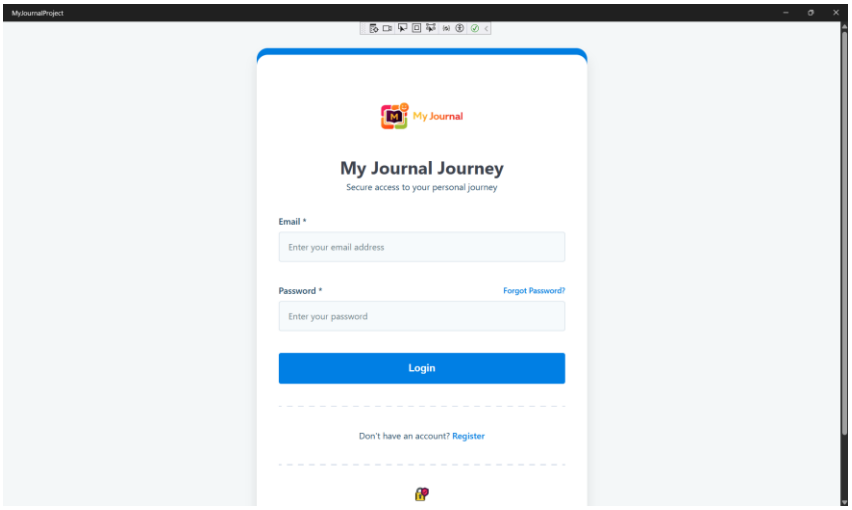


Figure 103: Redirecting to the Login Page

- SQLite Database Entry for Registered User.

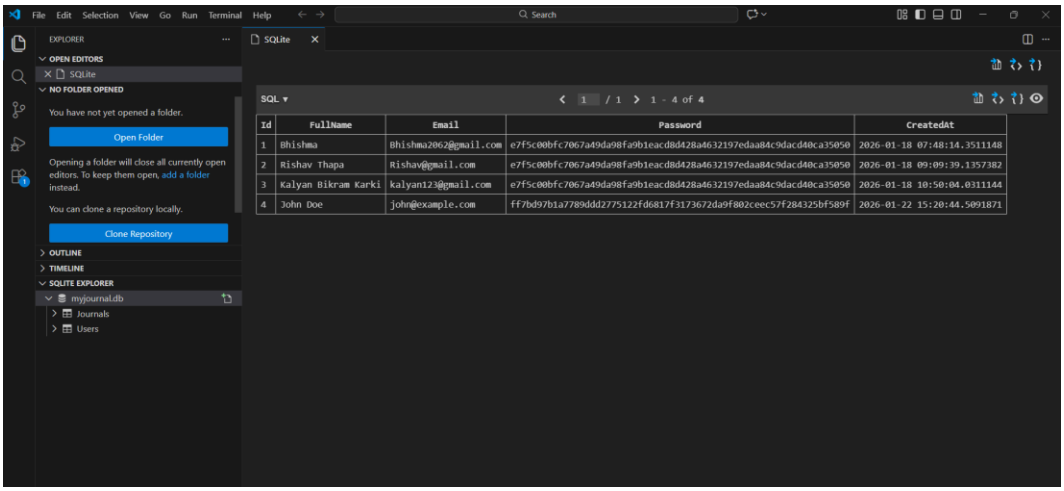


Figure 104: User entry John Doe created in SQLite database.

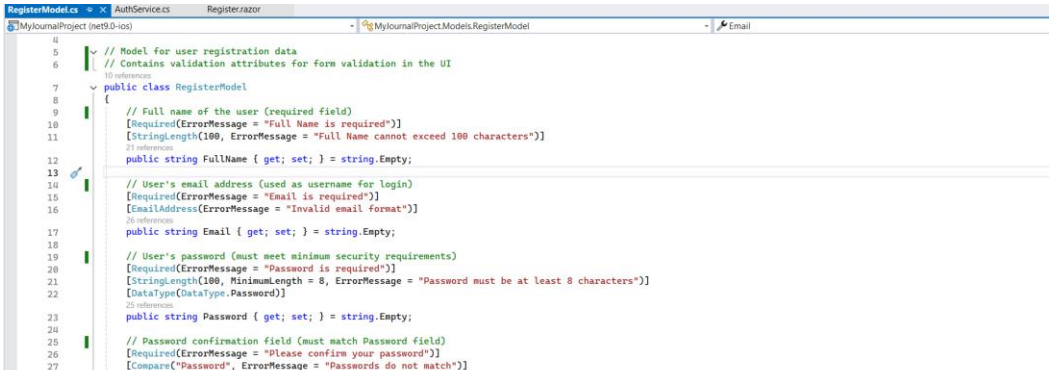
- Code Related to the Registration.

AuthService.cs

```
// Registers a new user with the system
public async Task<ServiceResult<User>> RegisterAsync(RegisterModel model)
{
    try
    {
        Console.WriteLine($"Registering user: {model.Email}");

        var context = await GetDbContextAsync();

        // Check if email already exists in the database
        var existing = await context.Users
            .AsNoTracking() // Read-only query for better performance
            .FirstOrDefaultAsync(u => u.Email == model.Email);

        if (existing != null)
        {
            Console.WriteLine($"Email already exists: {model.Email}");
            return ServiceResult<User>.FailureResult("This email is already registered");
        }

        // Hash the password for secure storage (never store plain text passwords)
        string hashedPassword = HashPassword(model.Password);
        Console.WriteLine($"Password hashed for: {model.Email}");

        // Create new user entity
```

Figure 105: AuthService.cs

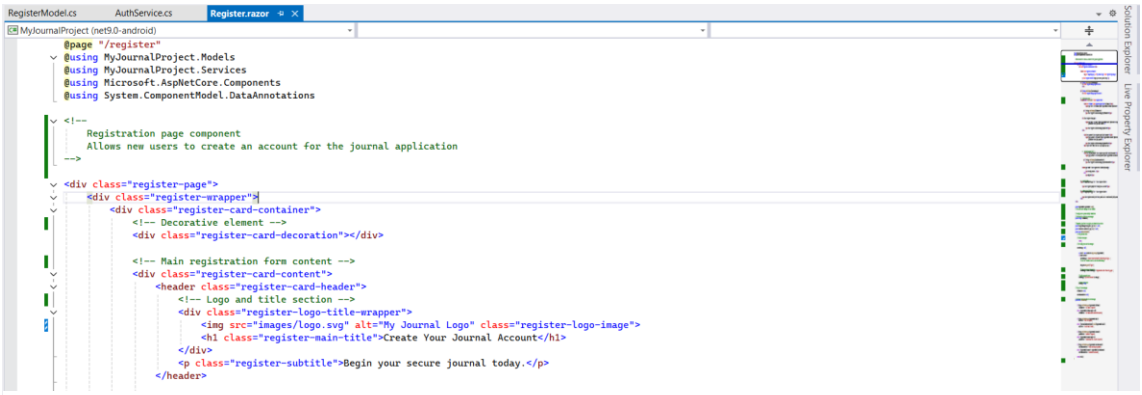
RegisterModel.cs



```
// Model for user registration data
// Contains validation attributes for form validation in the UI
public class RegisterModel
{
    // Full name of the user (required field)
    [Required(ErrorMessage = "Full Name is required")]
    [StringLength(100, ErrorMessage = "Full Name cannot exceed 100 characters")]
    public string FullName { get; set; } = string.Empty;

    // User's email address (used as username for login)
    [Required(ErrorMessage = "Email is required")]
    [EmailAddress(ErrorMessage = "Invalid email format")]
    public string Email { get; set; } = string.Empty;

    // User's password (must meet minimum security requirements)
    [Required(ErrorMessage = "Password is required")]
    [StringLength(100, MinimumLength = 8, ErrorMessage = "Password must be at least 8 characters")]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;

    // Password confirmation field (must match Password field)
    [Required(ErrorMessage = "Please confirm your password")]
    [Compare("Password", ErrorMessage = "Passwords do not match")]
```

Figure 106: RegisterModel.cs

Register.razor



```
@page "/register"
@using MyJournalProject.Models
@using MyJournalProject.Services
@using Microsoft.AspNetCore.Components
@using System.ComponentModel.DataAnnotations

<!--
    Registration page component
    Allows new users to create an account for the journal application
-->

<div class="register-page">
    <div class="register-wrapper">
        <div class="register-card-container">
            <!-- Decorative element -->
            <div class="register-card-decoration"></div>

            <!-- Main registration form content -->
            <div class="register-card-content">
                <header class="register-card-header">
                    <!-- Logo and title section -->
                    <div class="register-logo-title-wrapper">
                        <img src="images/logo.svg" alt="My Journal Logo" class="register-logo-image">
                        <h1 class="register-main-title">Create Your Journal Account</h1>
                    </div>
                    <p class="register-subtitle">Begin your secure journal today.</p>
                </header>
```

Figure 107: Register.razor

3.7.2. Test Case 2: Duplicate Email Registration

Table 3: Test Case 2: Duplicate Email Registration

Test Case 2: Duplicate Email Registration	
Objective	To verify that the system prevents user registration when an email address that already exists in the database is used.
Action	i. Open the Journaling application. ii. Navigate to the Register form. iii. Enter the Full Name: Kalyan Bikram Karki. iv. Enter Email: kalyan123@gmail.com. v. Enter Password: User@1234. vi. Enter Confirm Password: User@1234. vii. Click the Register “Register” button
Expected Result	i. The system should not create a new user account. ii. An error message should be displayed: “Error! This email is already registered”. iii. User should remain on the registration page. iv. No duplicate user entry should be created in the database.
Actual Result	i. Duplicate registration was not allowed. ii. Error message displayed: “Error! This email is already registered”. iii. No new user record created in SQLite database. iv. Use remained on the registration page.
Conclusion	The test was successfully implemented. This confirms that the system correctly prevents users from registering with an existing email, shows the correct error message, and does not create duplicate user records.

- User Kalyan Bikram Karki with email kalyan123@gmail.com already exist in the database

SQL ▾ 1 / 1 1 - 4 of 4

Id	FullName	Email	Password	CreatedAt
1	Bhishma	Bhishma2062@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 07:48:14.3511148
2	Rishav Thapa	Rishav@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 09:09:39.1357382
3	Kalyan Bikram Karki	kalyan123@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 10:50:04.0311144
4	John Doe	john@example.com	ff7bd97b1a7789ddd2775122fd6817f3173672da9f802ceec57f284325bf589f	2026-01-22 15:20:44.5091871

Figure 108: Database with User email kalyan123@gmail.com

- Registration From with Existing Email.

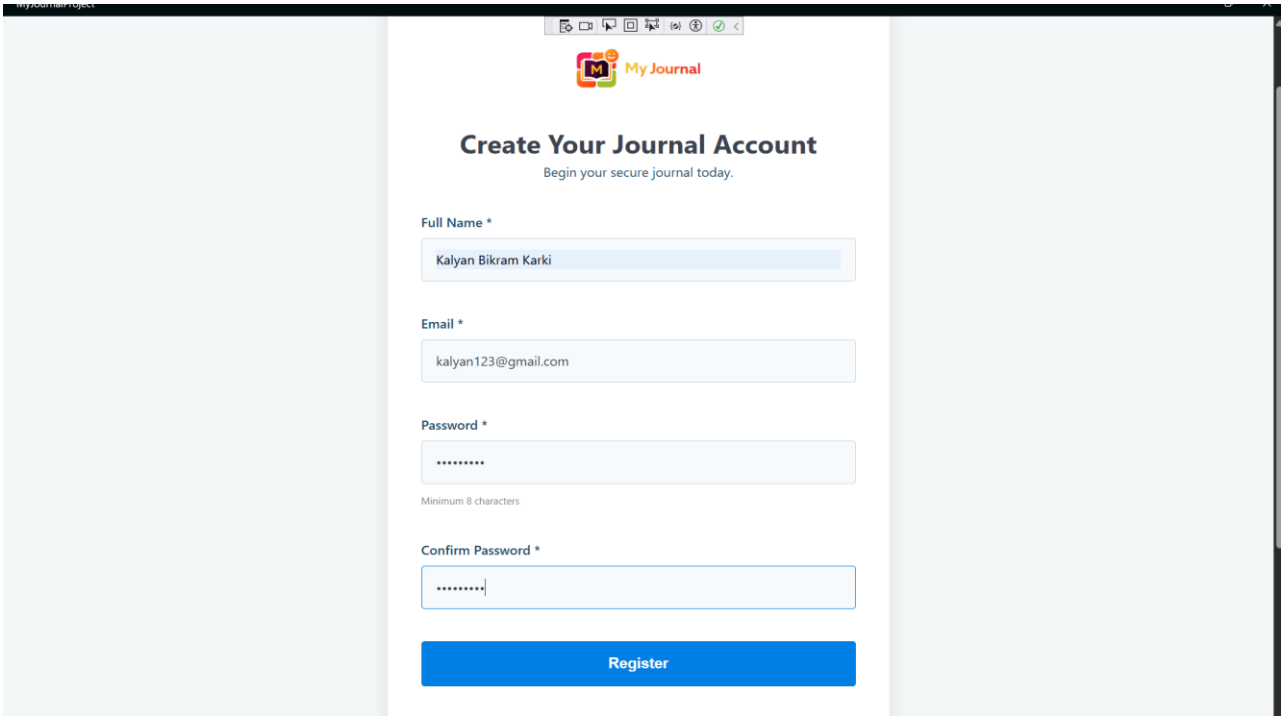


Figure 109: Registration From with Existing Email

- Duplicate Email Error Message: “Error! This email is already registered”.

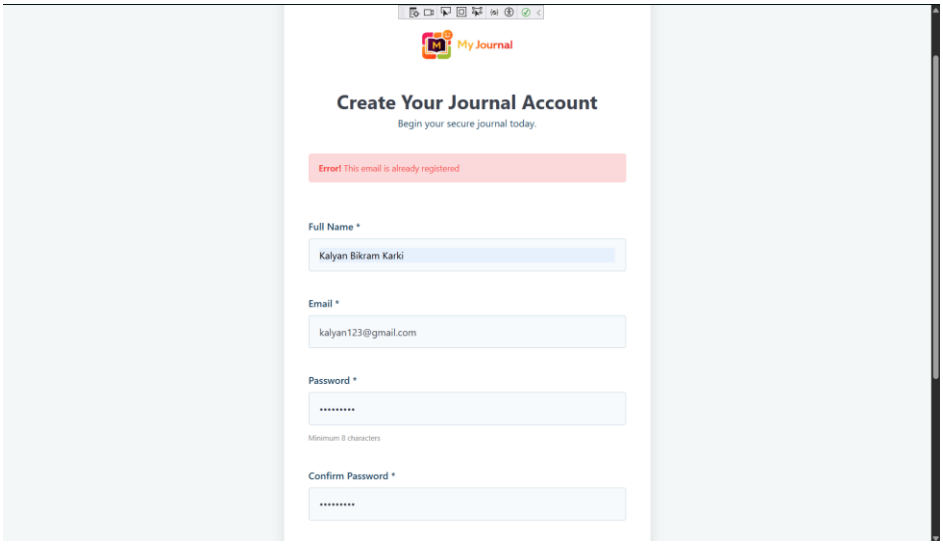


Figure 110: Duplicate Email Error Message.

- SQLite Database Check to confirm no duplicate entry for kalyan123@gmail.com.

SQL ▾ 1 / 1 1 - 4 of 4

Id	FullName	Email	Password	CreatedAt
1	Bhishma	Bhishma2062@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 07:48:14.3511148
2	Rishav Thapa	Rishav@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 09:09:39.1357382
3	Kalyan Bikram Karki	kalyan123@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 10:50:04.0311144
4	John Doe	john@example.com	ff7bd97b1a7789ddd2775122fd6817f3173672da9f802ceec57f284325bf589f	2026-01-22 15:20:44.5091871

Figure 111: SQLite Database Check to confirm no duplicate entry.

- AuthService.cs

Figure 112: AuthService.cs related to existing email check.

3.7.3. Test Case 3: Form Validation for Required Fields (Register Form)

Table 4: Test Case 3: Form Validation for Required Fields (Register Form)

Test Case 3: Form Validation for Required Fields	
Objective	To verify that the system prevents user registration when any required field left empty and displays appropriate validation messages.
Action	i. Open the Journaling application. ii. Navigate to the Register form. iii. Leave one or more required fields empty (Full Name, Email, Password, or Confirm Password). iv. Click the Register Button.
Expected Result	i. The system should not create a new user account. ii. Validation messages should be displayed for the empty fields. iii. The user should remain on the Register page. iv. No data should be stored in the database.
Actual Result	i. Registration was not allowed. ii. Validation messages were displayed for missing fields. iii. User remained on the Register page. iv. No data should be stored in the database.
Conclusion	The test was successfully implemented. This confirms that the system correctly validates required fields, prevents incomplete submissions, and ensures that user information is stored correctly by blocking invalid registration attempts.

- Registration form with missing required fields and validation message displayed.

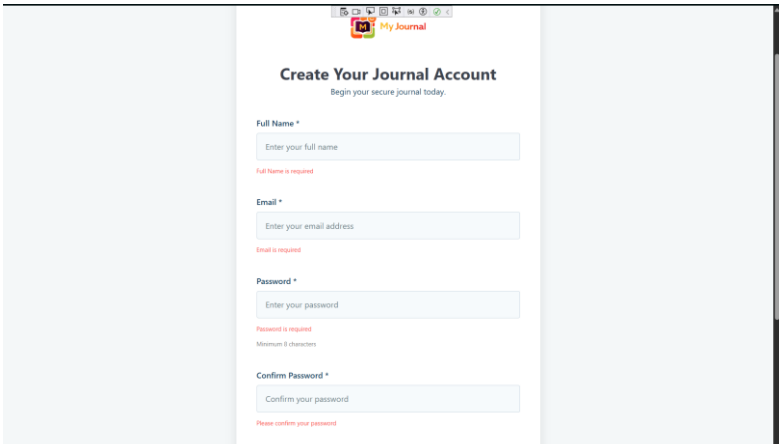


Figure 113: Registration form with missing required fields

- SQLite Database Verification.

Id	FullName	Email	Password	CreatedAt
1	Bhishma	Bhishma2062@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 07:48:14.3511148
2	Rishav Thapa	Rishav@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 09:09:39.1357382
3	Kalyan Bikram Karki	kalyan123@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 10:50:04.0311144
4	John Doe	john@example.com	ff7bd97b1a7789ddd2775122fd6817f3173672da9f802ceec57f284325bf589f	2026-01-22 15:20:44.5091871

Figure 114: SQLite Database Verification.

- RegisterModel.cs with Required fields

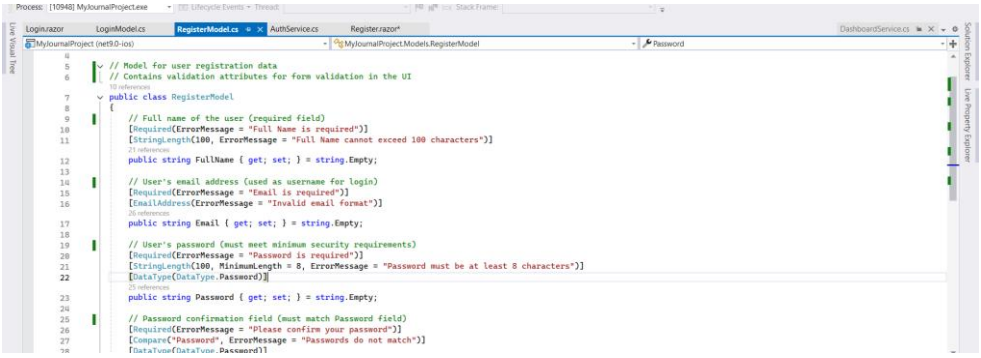


Figure 115: RegisterModel.cs with Required fields

3.7.4. Test Case 4: Successful Login

Table 5: Test Case 4: Successful Login

Test Case 4: Successful Login	
Objective	To verify that a registered user can login successfully using a valid email and password.
Action	i. Open the Journaling application. ii. Navigate to the Login form. iii. Enter Email: john@example.com. v. Enter Password: Password@123. vii. Click the “Login” button
Expected Result	i. The user should be logged in successfully. ii. The system should open the main dashboard. iii. No error message should be shown.
Actual Result	i. User logged in successfully. ii. Main dashboard page opened. iii. No error messages were displayed.
Conclusion	The test was successfully implemented. This confirms that the login functionality works correctly with valid user details and allows the user to access the system.

- Login with existing user details John Doe.

SQL ▾ 1 / 1 1 - 4 of 4

Id	FullName	Email	Password	CreatedAt
1	Bhishma	Bhishma2062@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 07:48:14.3511148
2	Rishav Thapa	Rishav@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 09:09:39.1357382
3	Kalyan Bikram Karki	kalyan123@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 10:50:04.0311144
4	John Doe	john@example.com	ff7bd97b1a7789ddd2775122fd6817f3173672da9f802ceec57f284325bf589f	2026-01-22 15:20:44.5091871

Figure 116: Login with existing user details John Doe.

- Login Form with Valid User Details.

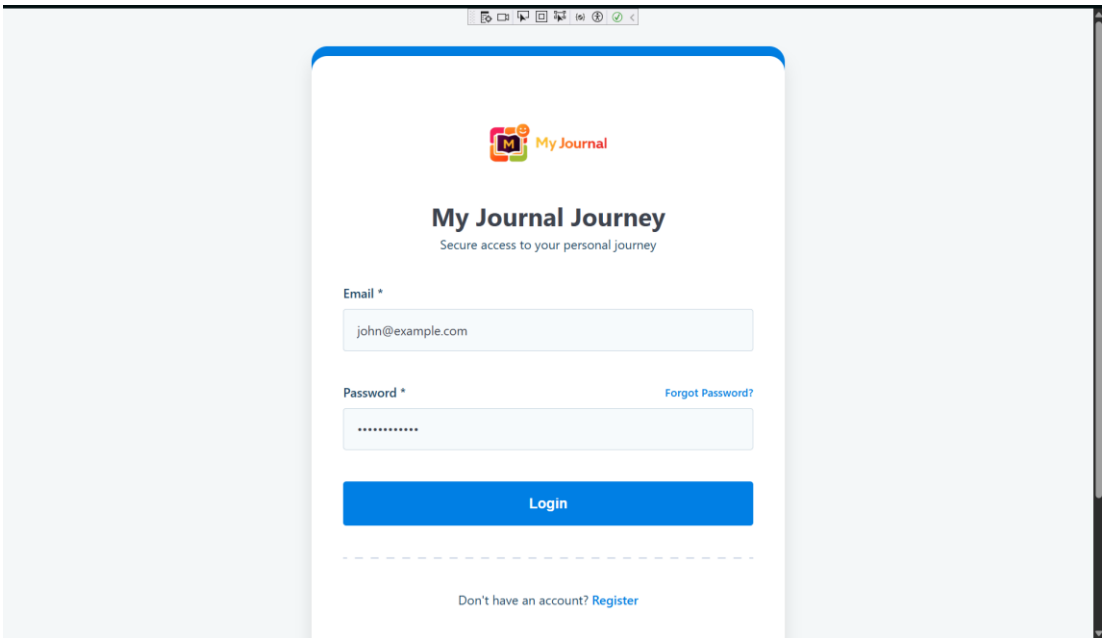


Figure 117: Login Form with Valid User Details.

- Successful Login and Dashboard View.

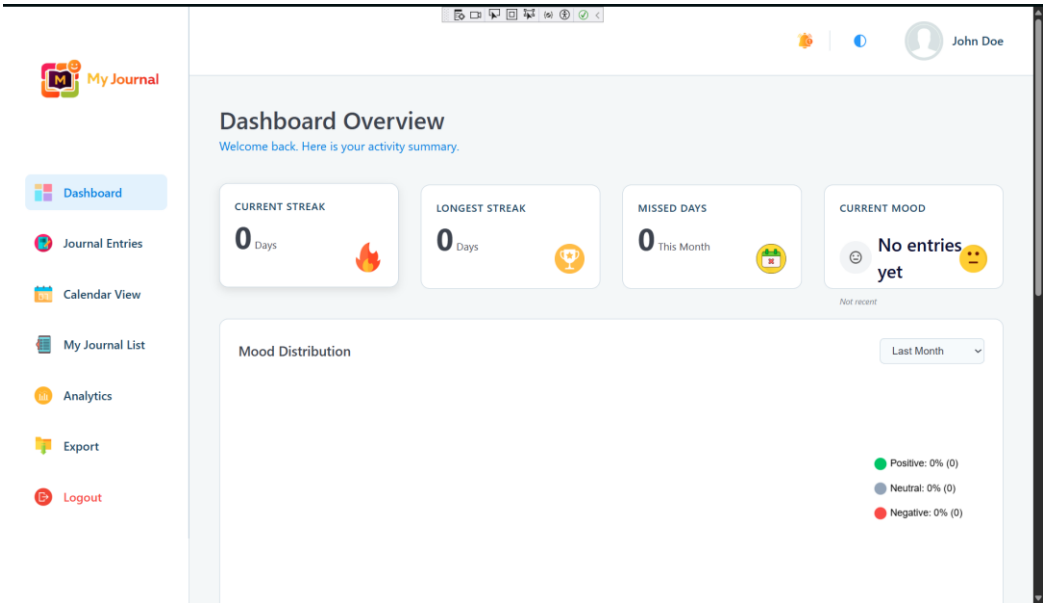
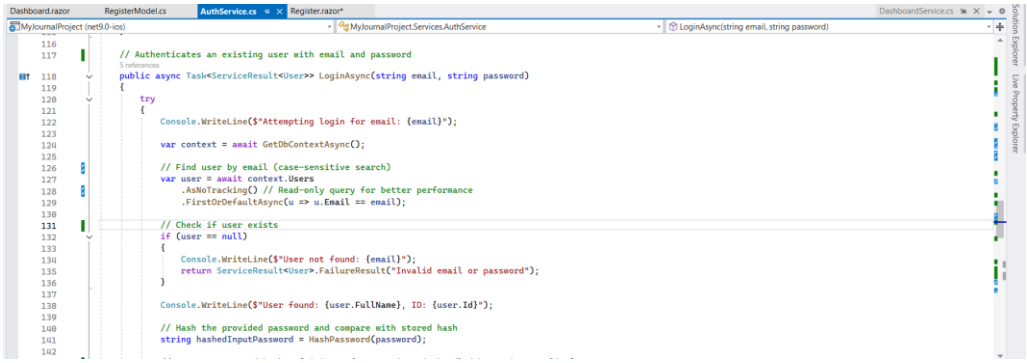


Figure 118: Successful Login and Dashboard View.

- Code Related to Login.

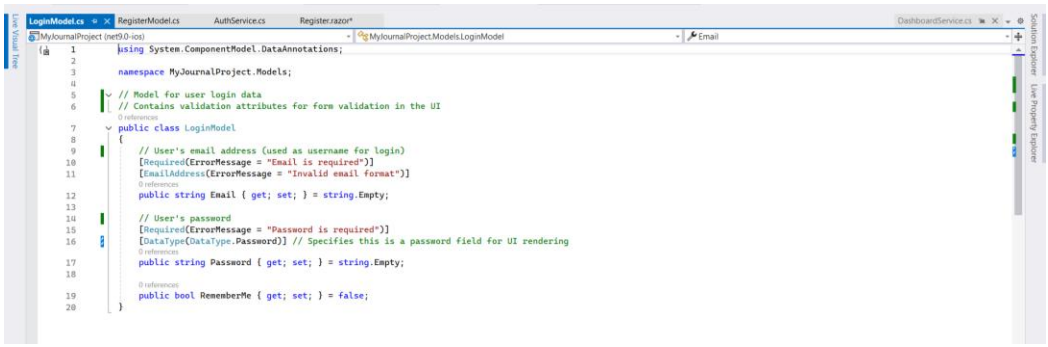
AuthService.cs



```
// Authenticates an existing user with email and password
public async Task<ServiceResult<User>> LoginAsync(string email, string password)
{
    try
    {
        Console.WriteLine($"Attempting login for email: {email}");

        var context = await GetDbContextAsync();

        // Find user by email (case-sensitive search)
        var user = await context.Users
            .AsNoTracking() // Read-only query for better performance
            .FirstOrDefaultAsync(u => u.Email == email);

        // Check if user exists
        if (user == null)
        {
            Console.WriteLine($"User not found: {email}");
            return ServiceResult<User>.FailureResult("Invalid email or password");
        }

        Console.WriteLine($"User found: {user.FullName}, ID: {user.Id}");

        // Hash the provided password and compare with stored hash
        string hashedInputPassword = HashPassword(password);
```

Figure 119: AuthService.cs related to Login

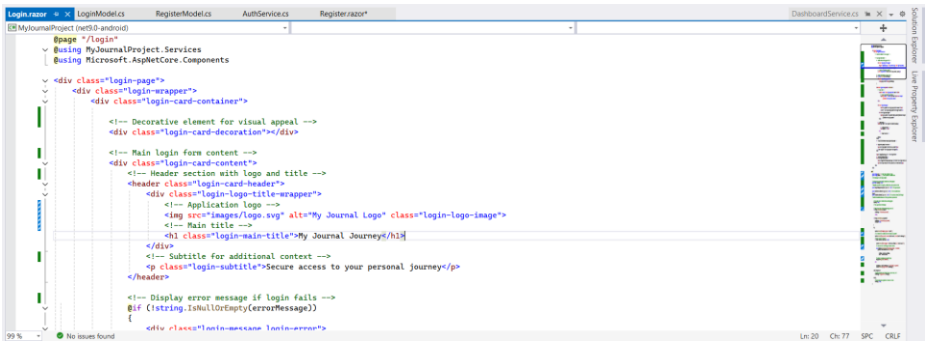
LoginModel.cs



```
using System.ComponentModel.DataAnnotations;

namespace MyJournalProject.Models;

// Model for user login data
// Contains validation attributes for form validation in the UI
public class LoginModel
{
    // User's email address (used as username for login)
    [Required(ErrorMessage = "Email is required")]
    [EmailAddress(ErrorMessage = "Invalid email format")]
    public string Email { get; set; } = string.Empty;

    // User's password
    [Required(ErrorMessage = "Password is required")]
    [DataType(DataType.Password)] // Specifies this is a password field for UI rendering
    public string Password { get; set; } = string.Empty;

    public bool RememberMe { get; set; } = false;
}
```

Figure 120: LoginModel.cs

Login.razor



```
@page "/login"
@using MyJournalProject.Services
@using Microsoft.AspNetCore.Components

<div class="login-page">
    <div class="login-wrapper">
        <div class="login-card-container">

            <!-- Decorative element for visual appeal -->
            <div class="login-card-decoration"></div>

            <!-- Main login form content -->
            <div class="login-card-content">
                <!-- Header section with logo and title -->
                <header class="login-card-header">
                    <div class="login-logo-title-wrapper">
                        <!-- Application logo -->
                        <img src="images/logo.svg" alt="My Journal Logo" class="login-logo-image">
                        <!-- Main title -->
                        <h1 class="login-main-title">My Journal Journey</h1>
                    </div>
                    <!-- Subtitle for additional context -->
                    <p class="login-subtitle">Secure access to your personal journey</p>
                </header>

                <!-- Display error message if login fails -->
                @if (!string.IsNullOrEmpty(errorMessage))
                {
```

Figure 121: Login.razor

3.7.5. Test Case 5: Login with Wrong Password

Table 6: Test Case 5: Login with Wrong Password

Test Case 5: Login with Wrong Password	
Objective	To verify that the system does not allow login when an incorrect password is entered for a registered email.
Action	i. Open the Journaling application. ii. Navigate to the Login form. iii. Enter Email: john@example.com. v. Enter Password: WrongPassword@123. vii. Click the “Login” button
Expected Result	i. The user should not be logged in. ii. An error message should be displayed: “Login Failed! Invalid email or password”. iii. The user should remain on the login page.
Actual Result	i. Login was not allowed. ii. Error message displayed: “Login Failed! Invalid email or password”. iii. User remained on the login page.
Conclusion	The test was successfully implemented. This confirms that the correctly handles wrong password input, shows the proper error message, and prevents unauthorized access.

- Login with existing user details John Doe.

SQL ▾ 1 / 1 1 - 4 of 4

Id	FullName	Email	Password	CreatedAt
1	Bhishma	Bhishma2062@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 07:48:14.3511148
2	Rishav Thapa	Rishav@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 09:09:39.1357382
3	Kalyan Bikram Karki	kalyan123@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 10:50:04.0311144
4	John Doe	john@example.com	ff7bd97b1a7789ddd2775122fd6817f3173672da9f802ceec57f284325bf589f	2026-01-22 15:20:44.5091871

Figure 122: Login with existing user details John Doe.

- Login Attempt with Wrong Password.

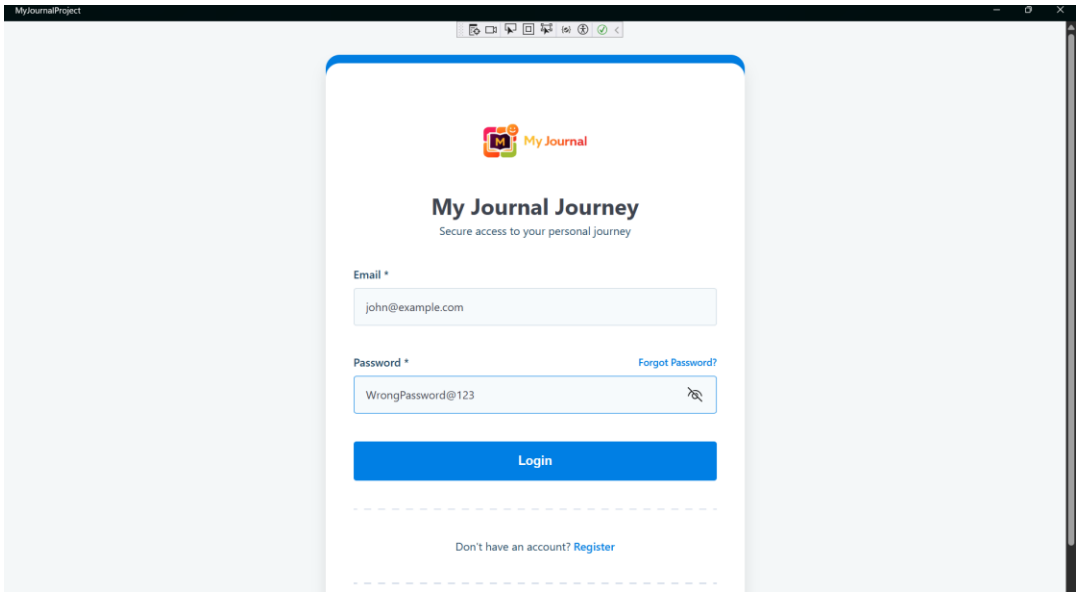


Figure 123: Login Attempt with Wrong Password.

- Error Message for Wrong Password: “Login Failed! Invalid email or password”.

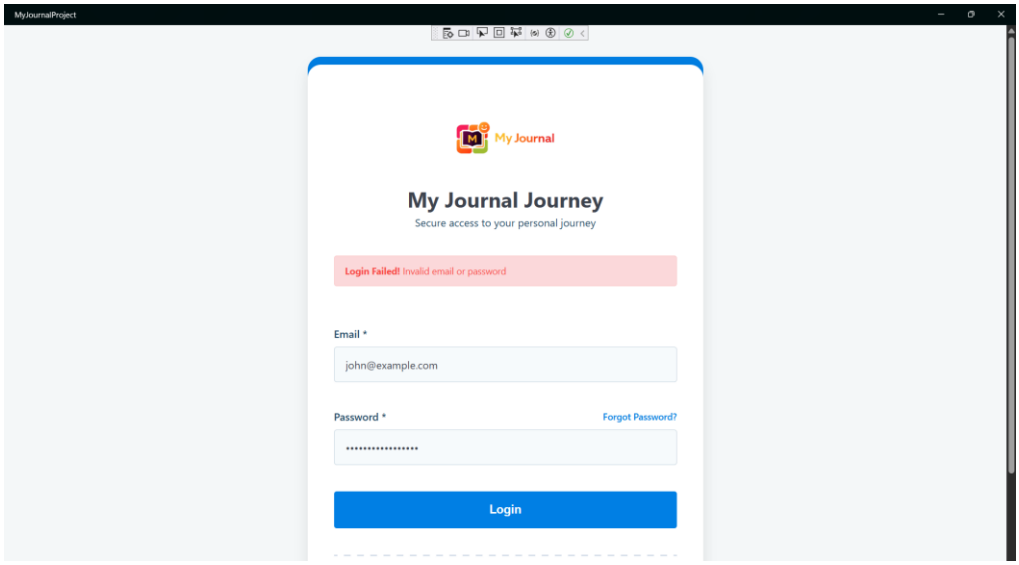


Figure 124: Error Message for Wrong Password.

- AuthService.cs

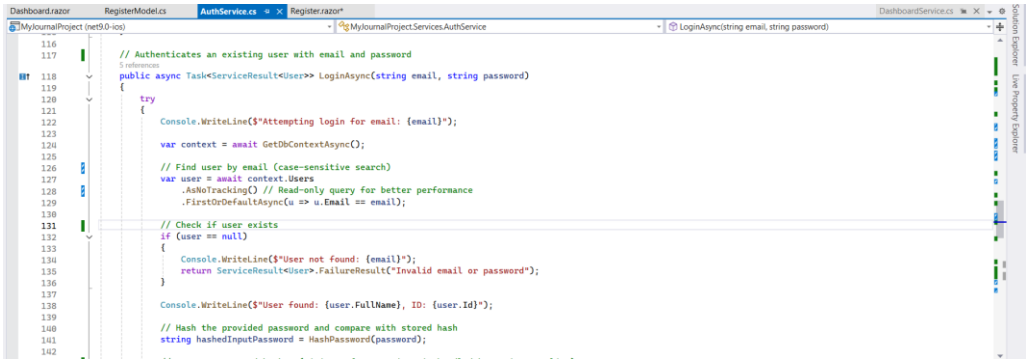


Figure 125: AuthService.cs related to Login with Wrong Password

3.7.6. Test Case 6: Form Validation for Required Fields (Login Form)

Table 7: Test Case 6: Form Validation for Required Fields (Login Form)

Test Case 6: Form Validation for Required Fields	
Objective	To verify that the system prevents login when any required login fields is left empty and display appropriate validation messages.
Action	i. Open the Journaling application. ii. Navigate to the Login form. iii. Leave one or more required fields empty (Email or Password). iv. Click the Login Button.
Expected Result	i. The user should not allow login. ii. Validation messages should be displayed for the empty fields. iii. The user should remain on the Login page.
Actual Result	i. Login was not allowed. ii. Validation message were displayed for the missing fields. iii. User remained on the Login Page.
Conclusion	The test was successfully implemented. This confirms that the correctly validates required login fields, prevent incomplete login attempts, and ensures only valid users can access the system.

- Login form with missing required fields and validation message displayed.

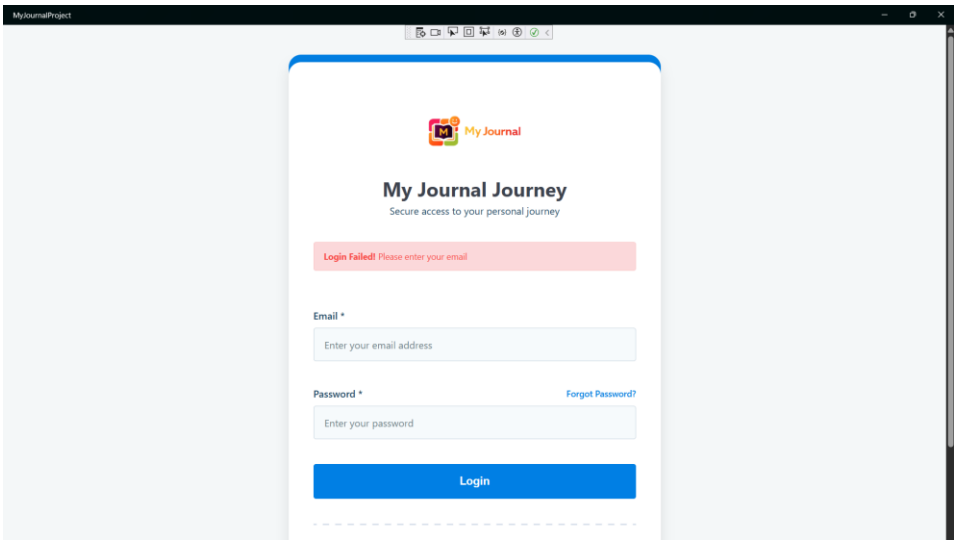


Figure 126: Login form with missing required fields.

- LoginModel.cs with required fields.

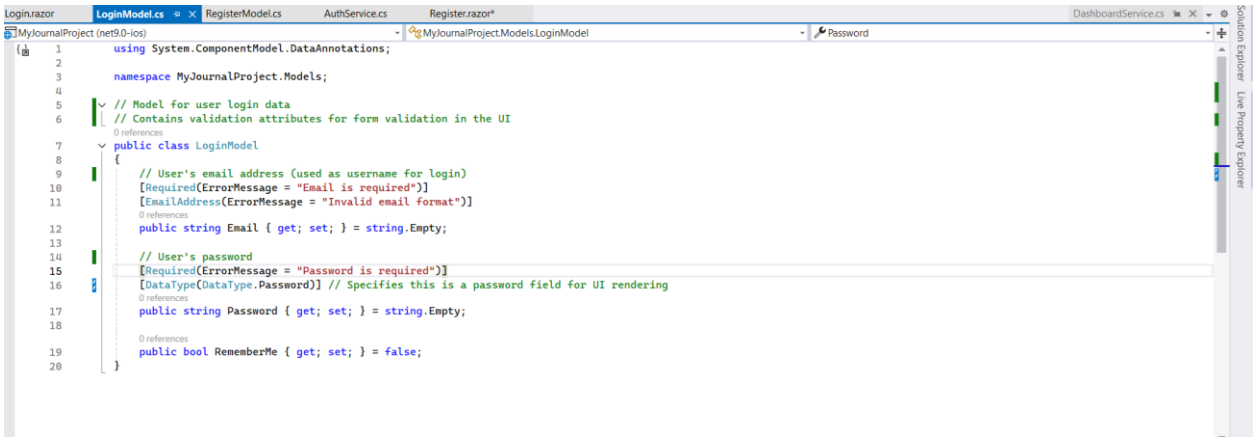


Figure 127: LoginModel.cs with required fields.

3.7.7. Test Case 7: Successful Journal Creation

Table 8: Test Case 7: Successful Journal Creation

Test Case 7: Successful Journal Creation	
Objective	To verify that a user can successfully create a new journal entry when valid information is provided.
Action	i. Open the Journaling application. ii. Login with a valid user account: Rishav Thapa. iii. Navigate to the Journal Entries. iv. Enter a Title for the journal entry. v. Enter Journal Content / Description. vi. Select Primary Mood (Positive, Negative or Neutral). vii. Select Secondary Mood but maximum 2 (Optional) vii. Select a Category. viii. Enter tags or select existing tags. ix. Click the “Save Entry” Button.
Expected Result	i. The journal entry should be created successfully. ii. A success dialog message should be displayed. iii. The journal entry should appear in the journal list page. iv. The journal data should be saved in the database. v. No errors or warnings should appear.
Actual Result	i. Journal entry created successfully. ii. Success dialog message displayed. iii. Journal entry visible in the journal list. iv. Journal entry stored in the SQLite database. v. No errors occurred.
Conclusion	The test was successfully implemented. This confirms that the system correctly allows user to create journal entries, saves the journal data properly, and displays the journal in the journal list page.

- Journal Creation Form for User Rishav Thapa with Entered Details.

  Login with User Id 2 Rishav Thapa.

Id	FullName	Email	Password	CreatedAt
1	Bhishma	Bhishma2062@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 07:48:14.3511148
2	Rishav Thapa	Rishav@gmail.com	e7f5c00bfc7067a49da98fa9b1eacd8d428a4632197edaa84c9dacd40ca35050	2026-01-18 09:09:39.1357382

Figure 128: Login with User Id 2 Rishav Thapa

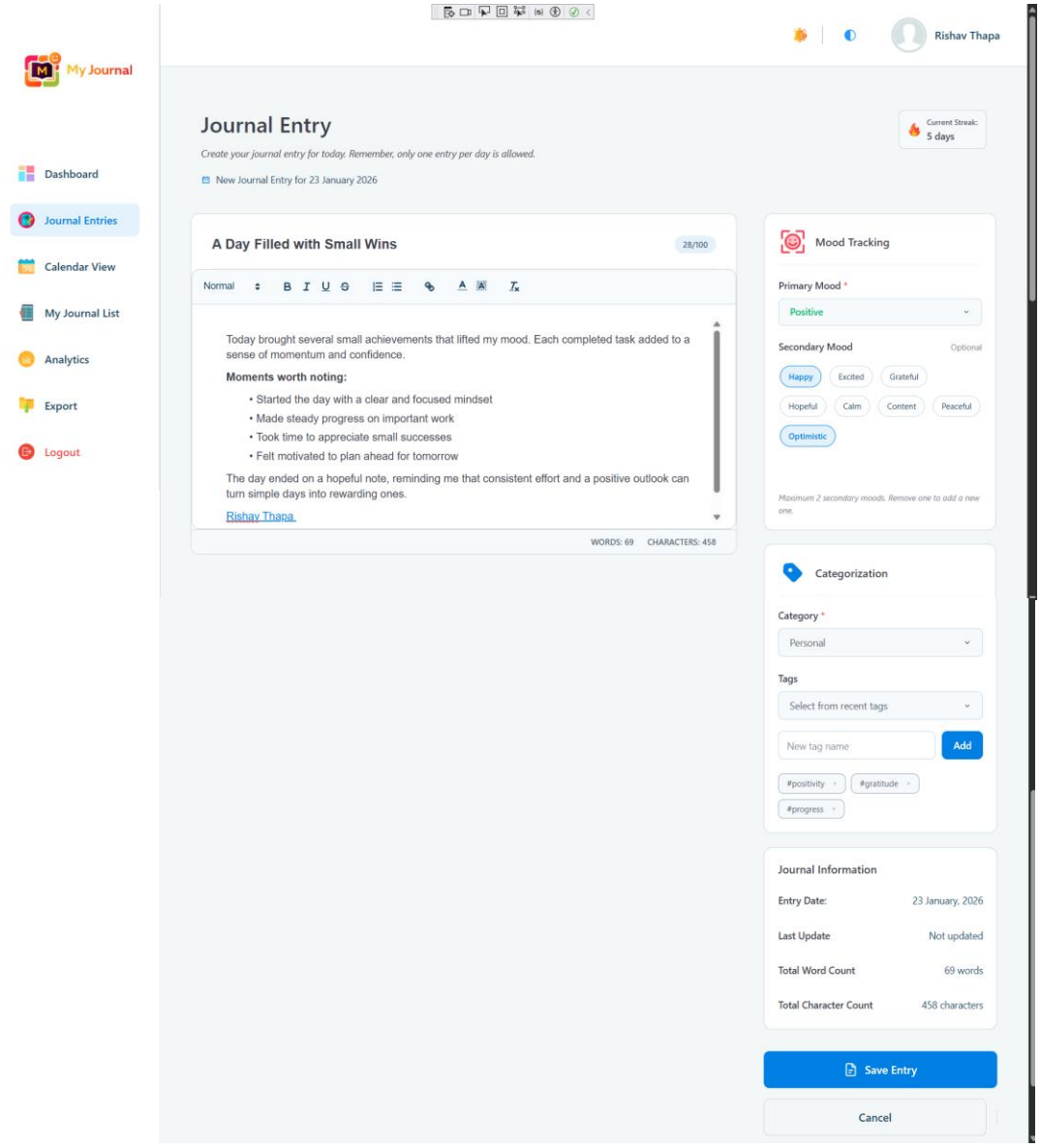


Figure 129: Journal Entries for User Rishav Thapa

- Journal Creation Success Dialog Message.

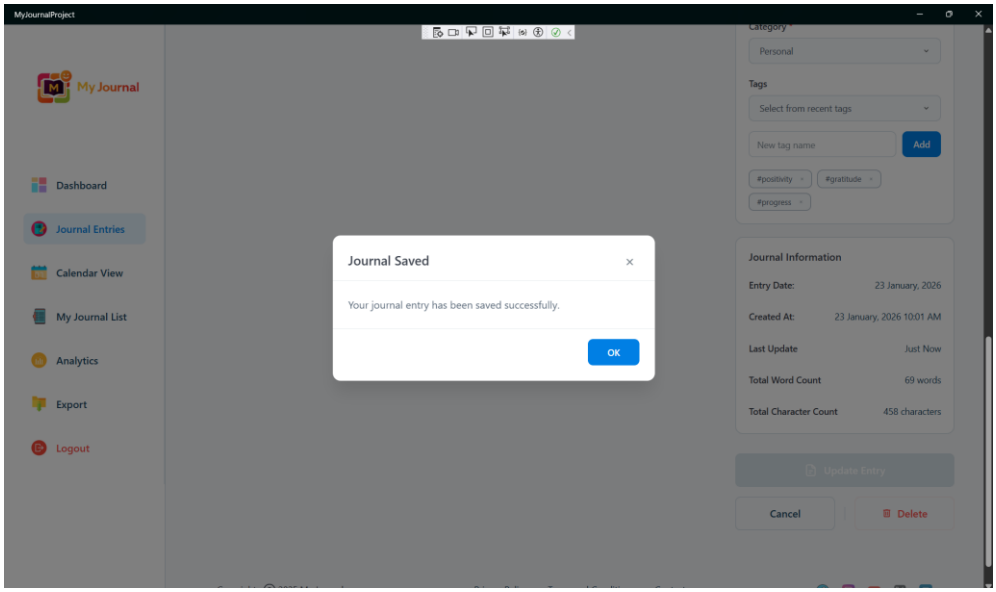


Figure 130: Journal Creation Success Dialog Message.

- Success Message also displayed in the Journal Entries page.

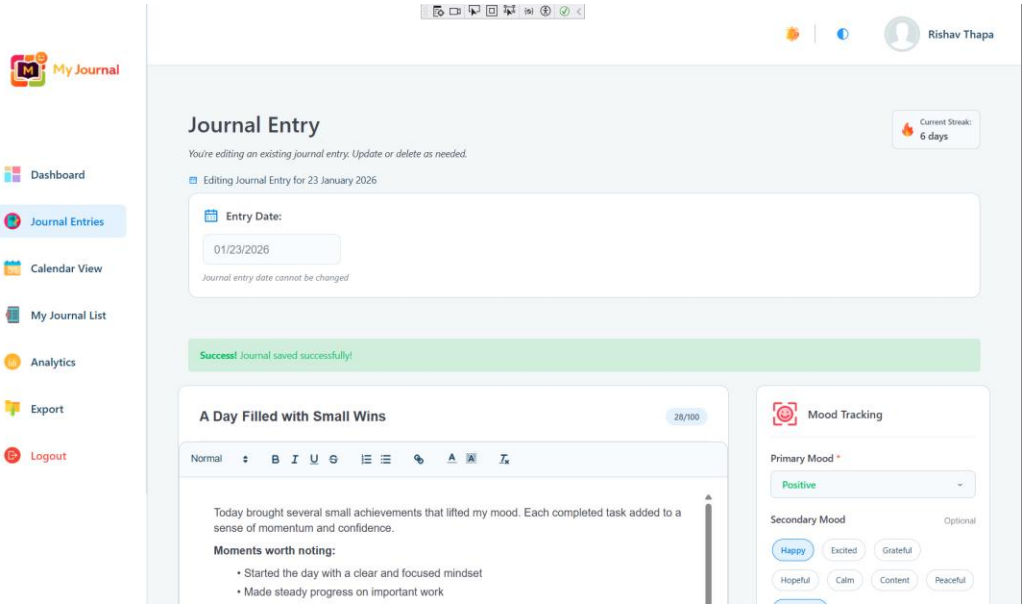


Figure 131: Success Message also displayed in the Journal Entries page.

- Journal Entry Displayed in Journal List page.

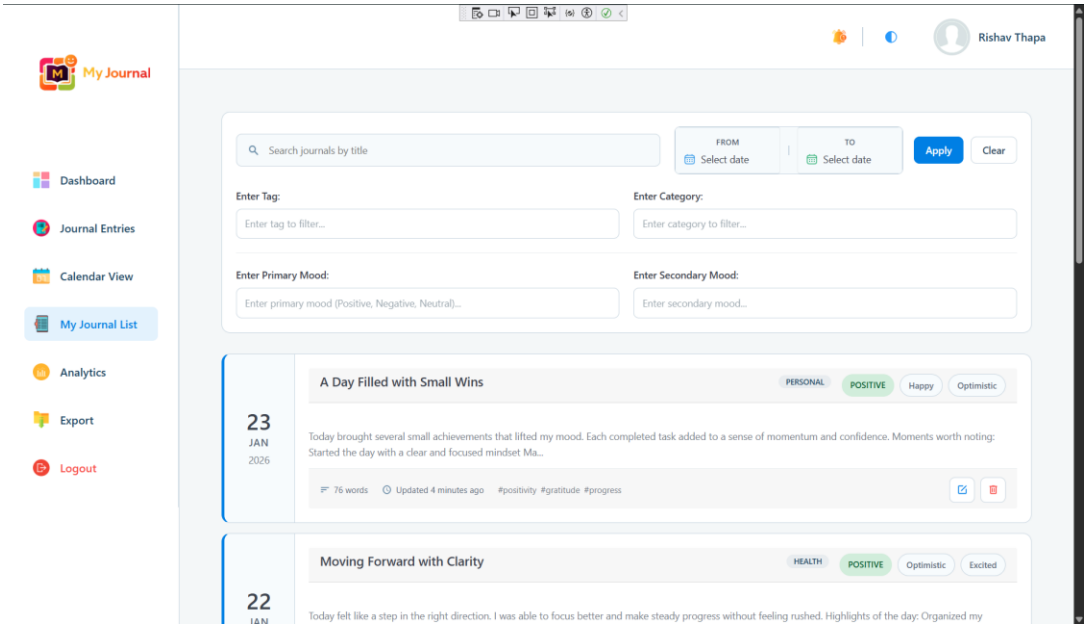


Figure 132: Journal Entry Displayed in Journal List page.

- SQLite Database Record for Journal Entries.

New journal stored successfully with Id 17 and UserId 2.

Figure 133: SQLite Database Record for Journal Entries

3.7.8. Test Case 8: Duplicate Journal Entries Not Allowed

Table 9: Test Case 8: Duplicate Journal Entries Not Allowed

Test Case 8: Duplicate Journal Entries Not Allowed	
Objective	To verify that the system does not allow a user to create more than one journal entry for the same date.
Action	i. Open the Journaling application. ii. Login with a valid user account: Rishav Thapa. iii. Navigate to the Journal Entries. iv. Create and save a journal entry for today's date. v. Navigate back to the Journal Entries page.
Expected Result	i. The first journal entry for the today's date should be created successfully. ii. The system should not allow a second journal entry for the same date. iii. The existing journal entry should be displayed instead. iv. The system should show Update, Delete, and Cancel buttons. v. No duplicate journal entry should be stored in the database.
Actual Result	i. The first journal entry was created successfully. ii. The system did not allow creating a second journal entry for the same date. iii. The existing journal entry was displayed. iv. Only Update, Delete and Cancel buttons with existing journal details were available. v. No duplicate journal entries was stored in the SQLite database.
Conclusion	The test was successfully implemented. This confirms that the system correctly allows the one journal entry per day rule, prevents duplicate journal creation, and provides proper options of updating or managing existing journal entries.

- Existing Journal Entry Displayed for Today's Date in Journal Entries page with Update, Delete and Cancel Buttons Available.

As the journal entry for today's date was already created in **Test Case 7,** the Journal Entries page displays the existing journal entry and does not allow creating a new journal for the same date. The page sows the existing journal with Update, Delete and Canel buttons. The user can only update or manage the existing journal entry, and no new entry can be created for today.

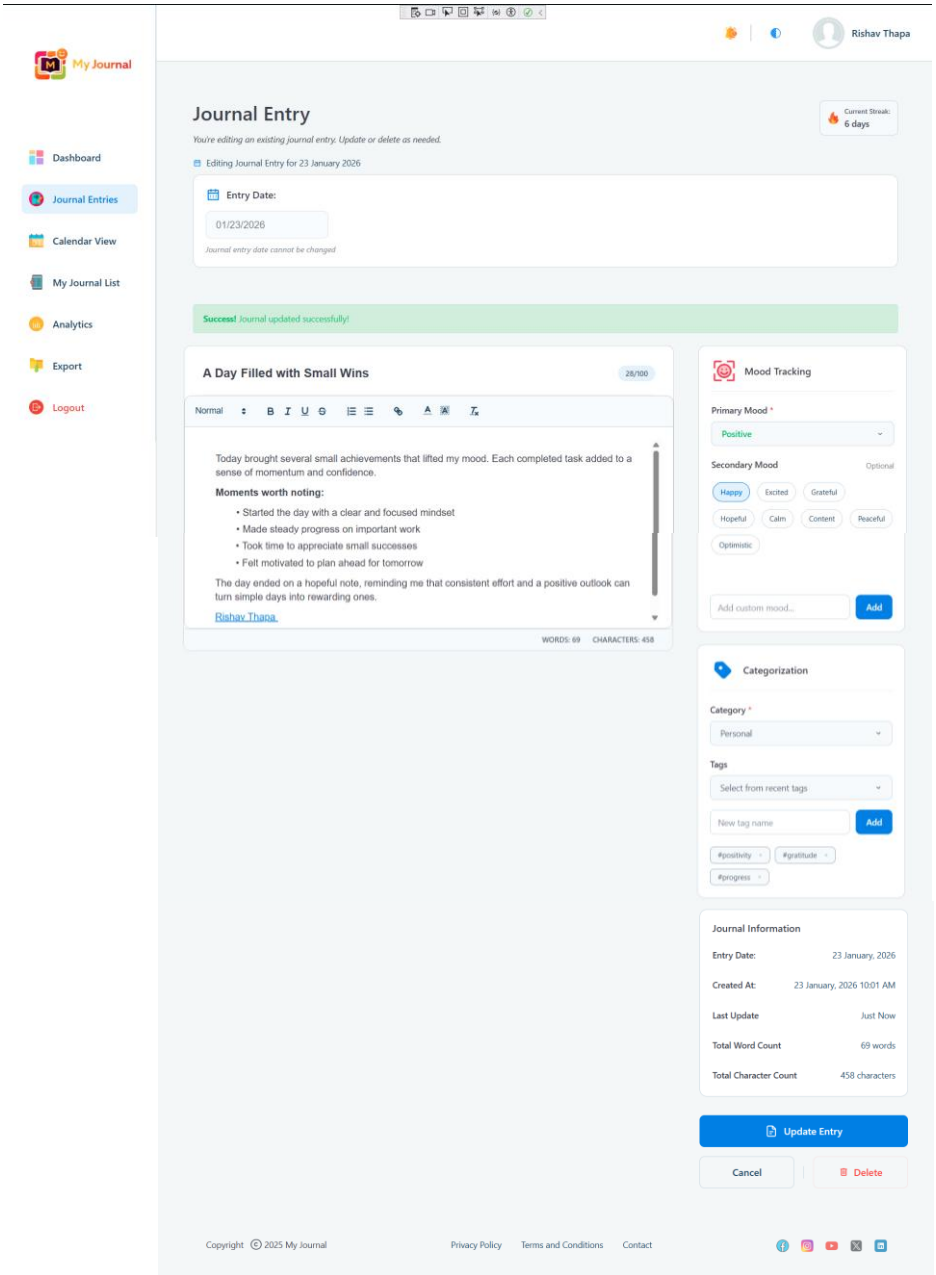
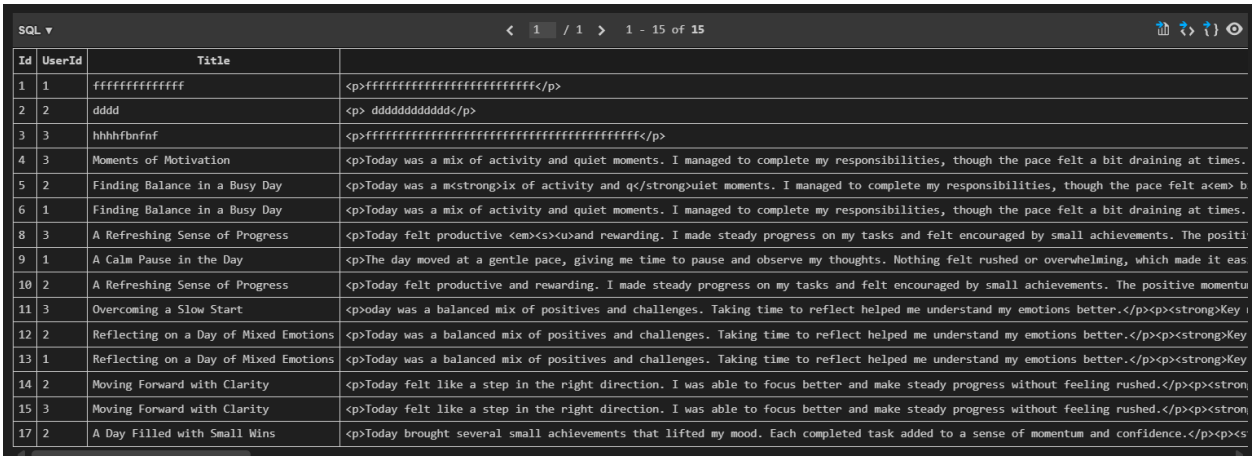


Figure 134: Existing Journal Entry Displayed for Today's Date

- SQLite Database Showing No Duplicate Journal Entries



SQL ▾ 1 / 1 1 - 15 of 15

Id	UserId	Title	
1	1	fffffffffffff	fffffffffffffffffffffffff
2	2	dddd	ddddddddddd
3	3	hhhhfbnfnf	fffffffffffffffffffffffffffffffffffffffff
4	3	Moments of Motivation	Today was a mix of activity and quiet moments. I managed to complete my responsibilities, though the pace felt a bit draining at times.
5	2	Finding Balance in a Busy Day	Today was a mix of activity and quiet moments. I managed to complete my responsibilities, though the pace felt a b
6	1	Finding Balance in a Busy Day	Today was a mix of activity and quiet moments. I managed to complete my responsibilities, though the pace felt a bit draining at times.
8	3	A Refreshing Sense of Progress	Today felt productive <s><u>and rewarding. I made steady progress on my tasks and felt encouraged by small achievements. The positi
9	1	A Calm Pause in the Day	The day moved at a gentle pace, giving me time to pause and observe my thoughts. Nothing felt rushed or overwhelming, which made it eas
10	2	A Refreshing Sense of Progress	Today felt productive and rewarding. I made steady progress on my tasks and felt encouraged by small achievements. The positive momentu
11	3	Overcoming a Slow Start	oday was a balanced mix of positives and challenges. Taking time to reflect helped me understand my emotions better. Key
12	2	Reflecting on a Day of Mixed Emotions	Today was a balanced mix of positives and challenges. Taking time to reflect helped me understand my emotions better. Key
13	1	Reflecting on a Day of Mixed Emotions	Today was a balanced mix of positives and challenges. Taking time to reflect helped me understand my emotions better. Key
14	2	Moving Forward with Clarity	Today felt like a step in the right direction. I was able to focus better and make steady progress without feeling rushed. <stron
15	3	Moving Forward with Clarity	Today felt like a step in the right direction. I was able to focus better and make steady progress without feeling rushed. <stron
17	2	A Day Filled with Small Wins	Today brought several small achievements that lifted my mood. Each completed task added to a sense of momentum and confidence. <s

Figure 135: SQLite Database Showing No Duplicate Journal Entries

3.7.9. Test Case 9: Update Journal from Existing Journal List

Table 10: Test Case 9: Update Journal from Existing Journal List

Test Case 9: Update Journal from Existing Journal List	
Objective	To verify that a user a can successfully update an existing journal entry selected from the journal listing page.
Action	i. Open the Journaling application. ii. Login with a valid user account: Rishav Thapa. iii. Navigate to the Journal Listing Page. iv. Select an existing journal entry from the list. v. Click the Edit option on the selected journal. vi. Modify the Title from “Confidence Growing with Every Step” to “ Consistent Progress, Positive Energy” vii. Update Primary Mood from “Positive” to “Neutral”. viii. Update Content from “Today felt uplifting as I noticed real progress in my efforts …” to “I noticed real progress in my efforts …” ix. Update Secondary Mood from “Hopeful” to “Calm”.

	x. Change Category from “Health” to “Personal”. xi. Add a new Tag “Achievement”. xii. Click the “Update Entry” button.
Expected Result	i. The journal entry should be updated successfully. ii. A success message should be displayed. iii. The updated journal details should be visible in the journal listing page. iv. The updated data should be saved in the database. v. The Update At timestamp should change. vi. No errors or warnings should occur.
Actual Result	i. Journal entry updated successfully. ii. Success message displayed. iii. Updated journal details visible on the journal list page with new title, content, mood, category and tags. iv. SQLite database shows updated values for Title, Primary Mood, Secondary Mood, Category, Tags and Updated At fields. v. Word count and character count updated based on content changes. vi. No errors occurred.
Conclusion	The test was successfully implemented. This confirms that the system correctly supports updating journal entries directly from the journal listing page, saves all modified fields accurately, updates timestamp automatically, and reflects changes consistently in both the UI and database.

- Journal Listing Page showing existing journal entries.

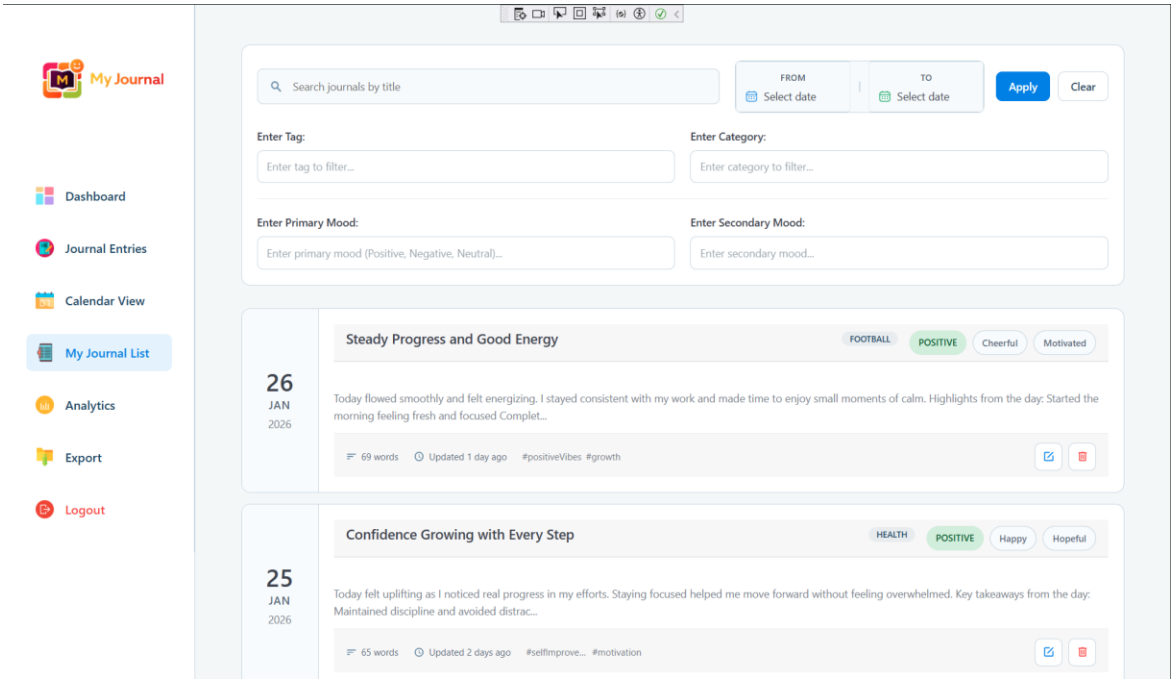
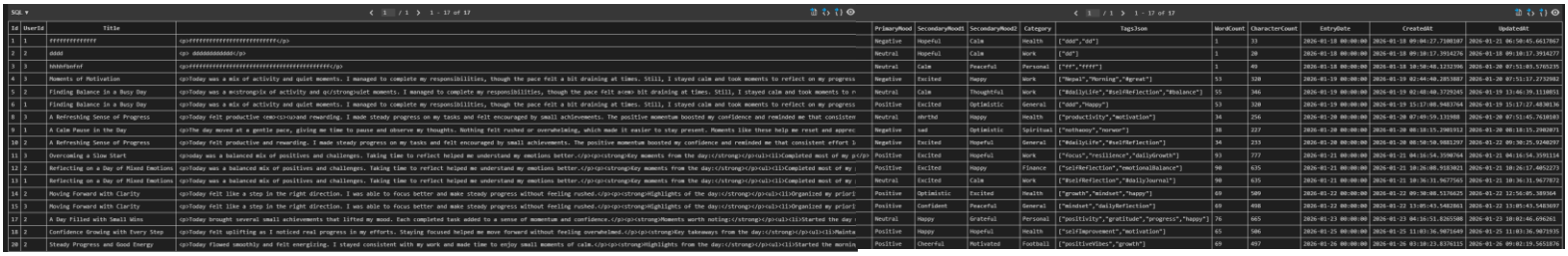


Figure 136: Journal Listing Page showing existing journal entries.

- Database storing the entry of 25 JAN 2026.



Id	UserId	Title	PrimaryMood	SecondaryMood1	SecondaryMood2	Category	TagsJson	WordCount	CharacterCount	EntryDate	CreatedAt	UpdatedAt
1	1	fffffffffffff	Negative	Hopeful	Calm	Health	["ddd","dd"]	1	33	2026-01-18 00:00:00	2026-01-18 09:04:27.7188187	2026-01-21 06:58:45.6617867
2	2	dddd	Neutral	Hopeful	Calm	Work	["dd"]	1	20	2026-01-18 00:00:00	2026-01-18 09:10:17.3914276	2026-01-18 09:10:17.3914277
3	3	hhhhfbnfnf	Neutral	Calm	Peaceful	Personal	["ff","ffff"]	1	49	2026-01-18 00:00:00	2026-01-18 10:58:48.1232396	2026-01-20 07:51:03.5765235
4	3	Moments of Motivation	Negative	Excited	Happy	Work	["Nepal","Morning","#great"]	53	328	2026-01-19 00:00:00	2026-01-19 02:44:40.2853887	2026-01-20 07:51:37.2732982
5	2	Finding Balance in a Busy Day	Neutral	Calm	Thoughtful	Work	["#DailyLife","#SelfReflection","#Balance"]	55	346	2026-01-19 00:00:00	2026-01-19 02:48:40.3729245	2026-01-19 13:46:39.1130851
6	1	Finding Balance in a Busy Day	Positive	Excited	Optimistic	General	["ddd","Happy"]	53	328	2026-01-19 00:00:00	2026-01-19 15:17:08.9483764	2026-01-19 15:17:27.4830136
8	3	A Refreshing Sense of Progress	Neutral	shrthd	Happy	Health	["productivity","motivation"]	34	256	2026-01-20 00:00:00	2026-01-20 07:49:59.1319888	2026-01-20 07:51:45.7610103
9	1	A Calm Pause in the Day	Negative	sad	Optimistic	Spiritual	["nothappy","nurwar"]	38	227	2026-01-20 00:00:00	2026-01-20 08:18:15.2901912	2026-01-20 08:18:15.2902073
10	2	A Refreshing Sense of Progress	Negative	Excited	Hopeful	General	["#DailyLife","#SelfReflection"]	34	233	2026-01-20 00:00:00	2026-01-20 08:50:50.9883297	2026-01-22 09:38:25.9240297
11	3	Overcoming a Slow Start	Positive	Excited	Hopeful	Work	["focus","resilience","dailyGrowth"]	93	577	2026-01-21 00:00:00	2026-01-21 04:16:54.3598764	2026-01-21 04:16:54.3598134
12	2	Reflecting on a Day of Mixed Emotions	Positive	Excited	Happy	Finance	["selfReflection","emotionalBalance"]	98	635	2026-01-21 00:00:00	2026-01-21 10:26:08.9183023	2026-01-21 10:26:17.4052273
13	1	Reflecting on a Day of Mixed Emotions	Neutral	Excited	Calm	Work	["#SelfReflection","#DailyJournal"]	98	635	2026-01-21 00:00:00	2026-01-21 10:36:31.9677565	2026-01-21 10:36:31.9677872
14	2	Moving Forward with Clarity	Positive	Optimistic	Excited	Health	["growth","mindset","happy"]	69	509	2026-01-22 00:00:00	2026-01-22 09:30:08.5178625	2026-01-22 12:56:05.389964
15	3	Moving Forward with Clarity	Positive	Confident	Peaceful	General	["mindset","DailyReflection"]	69	498	2026-01-22 00:00:00	2026-01-22 13:05:43.5483881	2026-01-22 13:05:43.5483697
17	2	A Day Filled with Small Wins	Neutral	Happy	Grateful	Personal	["positivity","gratitude","progress","happy"]	76	665	2026-01-23 00:00:00	2026-01-23 04:16:51.8265588	2026-01-23 10:02:46.696261
18	2	Confidence Growing with Every Step	Positive	Happy	Hopeful	Health	["selfImprovement","motivation"]	65	586	2026-01-25 00:00:00	2026-01-25 11:03:36.9871649	2026-01-25 11:03:36.9871935
20	2	Steady Progress and Good Energy	Positive	Cheerful	Motivated	Football	["positiveVibes","growth"]	69	497	2026-01-26 00:00:00	2026-01-26 03:10:23.8376115	2026-01-26 09:02:19.5651876

Figure 137: Database storing the entry of 25 JAN 2026.

- Edit icon or update icon clicked on an existing journal entry of 25 JAN 2026 the Journal Edit Form appears.

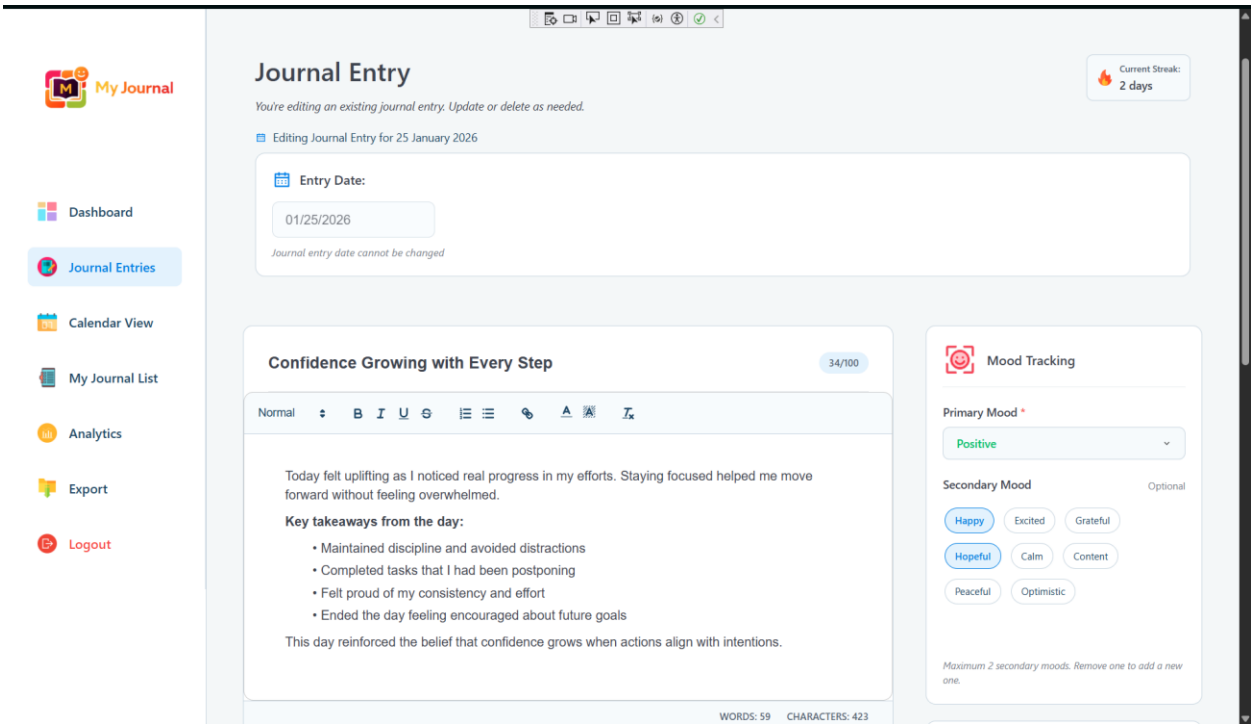


Figure 138: Update icon clicked on an existing journal entry of 25 JAN 2026

- Journal Edit Form with modified fields (title, mood, category, tags, content).

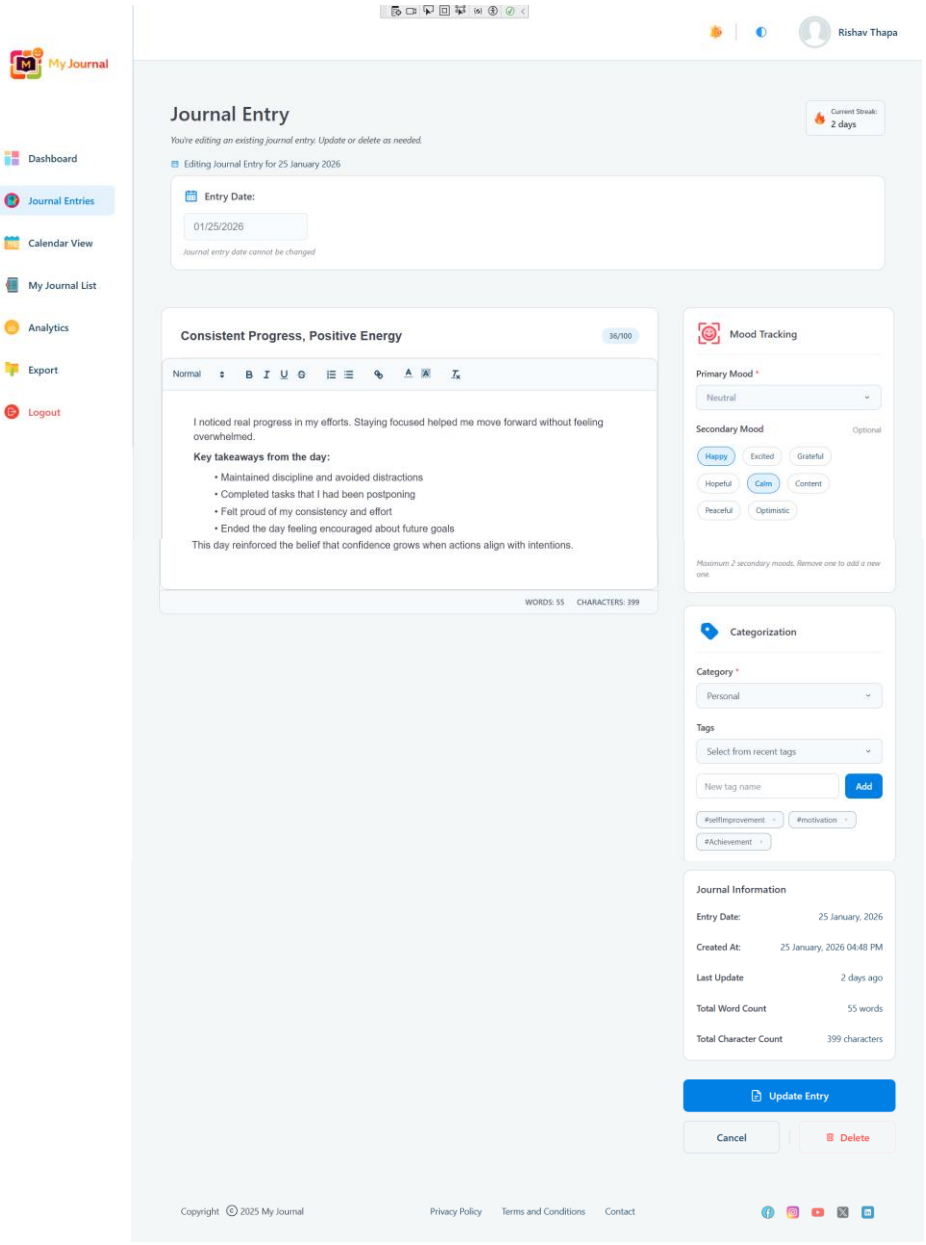


Figure 139: Journal Edit Form with modified fields.

- Success dialog message after clicking “Update Entry”

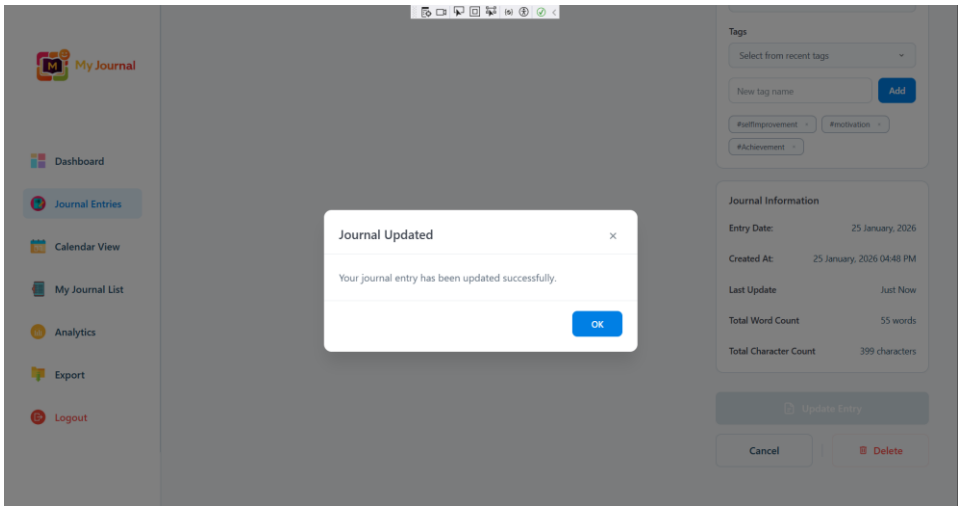


Figure 140: Success dialog message after clicking “Update Entry”

- Updated Journal Entry visible in the journal list with new title, category, moods and tags.

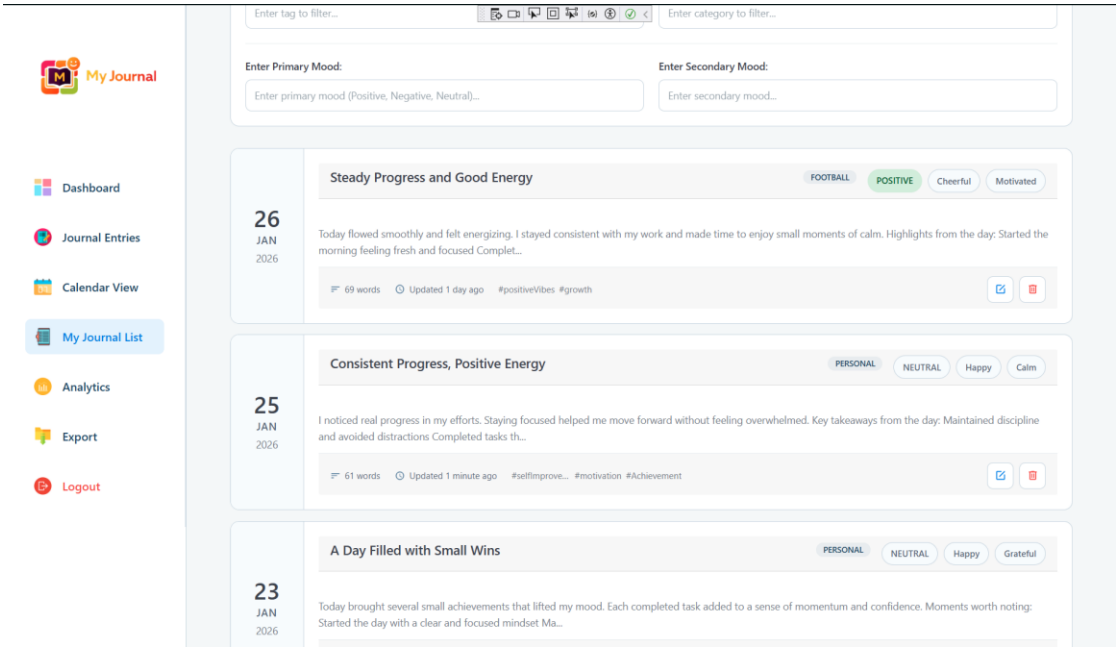
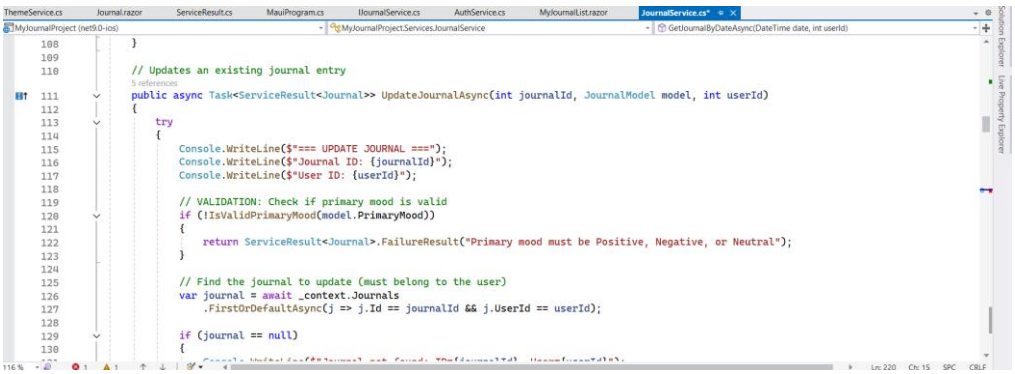


Figure 141: Updated Journal Entry visible in the journal list

- Database with updated Title, Primary Mood, Secondary Mood, Category, Tags and Update At timestamp.

Figure 142: Database with updated records

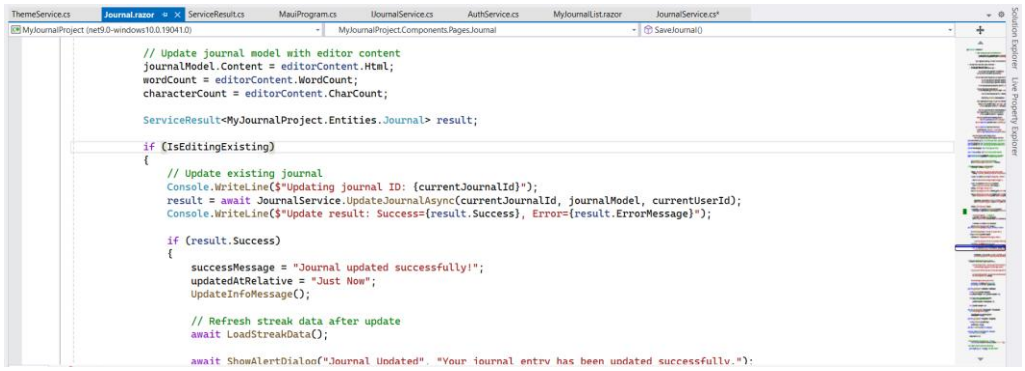
- JournalService.cs with Journal Update Code



```
108     }
109
110     // Updates an existing journal entry
        5 references
111     public async Task<ServiceResult<Journal>> UpdateJournalAsync(int journalId, JournalModel model, int userId)
112     {
113         try
114         {
115             Console.WriteLine($"=== UPDATE JOURNAL ===");
116             Console.WriteLine($"Journal ID: {journalId}");
117             Console.WriteLine($"User ID: {userId}");
118
119             // VALIDATION: Check if primary mood is valid
120             if (!IsValidPrimaryMood(model.PrimaryMood))
121             {
122                 return ServiceResult<Journal>.FailureResult("Primary mood must be Positive, Negative, or Neutral");
123             }
124
125             // Find the journal to update (must belong to the user)
126             var journal = await _context.Journals
127                 .FirstOrDefaultAsync(j => j.Id == journalId && j.UserId == userId);
128
129             if (journal == null)
130             {
```

Figure 143: JournalService.cs with Journal Update Code

- Journal.razor with Journal Update Code



```
            // Update journal model with editor content
            journalModel.Content = editorContent.Html;
            wordCount = editorContent.WordCount;
            characterCount = editorContent.CharCount;

            ServiceResult<MyJournalProject.Entities.Journal> result;

            if (IsEditingExisting)
            {
                // Update existing journal
                Console.WriteLine($"Updating journal ID: {currentJournalId}");
                result = await JournalService.UpdateJournalAsync(currentJournalId, journalModel, currentUserId);
                Console.WriteLine($"Update result: Success={result.Success}, Error={result.ErrorMessage}");

                if (result.Success)
                {
                    successMessage = "Journal updated successfully!";
                    updatedAtRelative = "Just Now";
                    UpdateInfoMessage();

                    // Refresh streak data after update
                    await LoadStreakData();

                    await ShowAlertDialog("Journal Updated", "Your journal entry has been updated successfully.");
```

Figure 144: Journal.razor with Journal Update Code

3.7.10. Test Case 10: Journal Deletion

Table 11: Test Case 10: Journal Deletion

Test Case 10: Journal Deletion	
Objective	To verify that a user a can successfully delete an existing journal entry from the journal list.
Action	i. Open the Journaling application. ii. Login with a valid user account: Rishav Thapa. iii. Navigate to the Journal Listing Page. iv. Click on the Delete button (trash icon) for an existing journal entry. v. A confirmation dialog appears. vi. Click “Delete” in the confirmation dialog.
Expected Result	i. A confirmation dialog should appear with the message: “Delete Journal. Are you sure you want to delete this journal entry?” ii. After confirmation, the journal entry should be deleted. iii. A success message should be displayed: “Journal Deleted. Your journal entry has been deleted successfully”. iv. The deleted journal should disappear from the journal list. v. The journal record should be removed from the SQLite database.
Actual Result	i. Confirmation dialog appeared correctly. ii. Journal entry deleted successfully after confirmation. iv. Journal entry removed from the journal list view. v. SQLite database no longer contains the deleted journal record.
Conclusion	The test was successfully implemented. This confirms that the system correctly handles journal deletion with the proper user confirmation, ensures complete data removal from both UI and database, and provides clear user feedback throughout the process.

- Journal Entry of 25 JAN 2026 in Journal List Page before deletion.

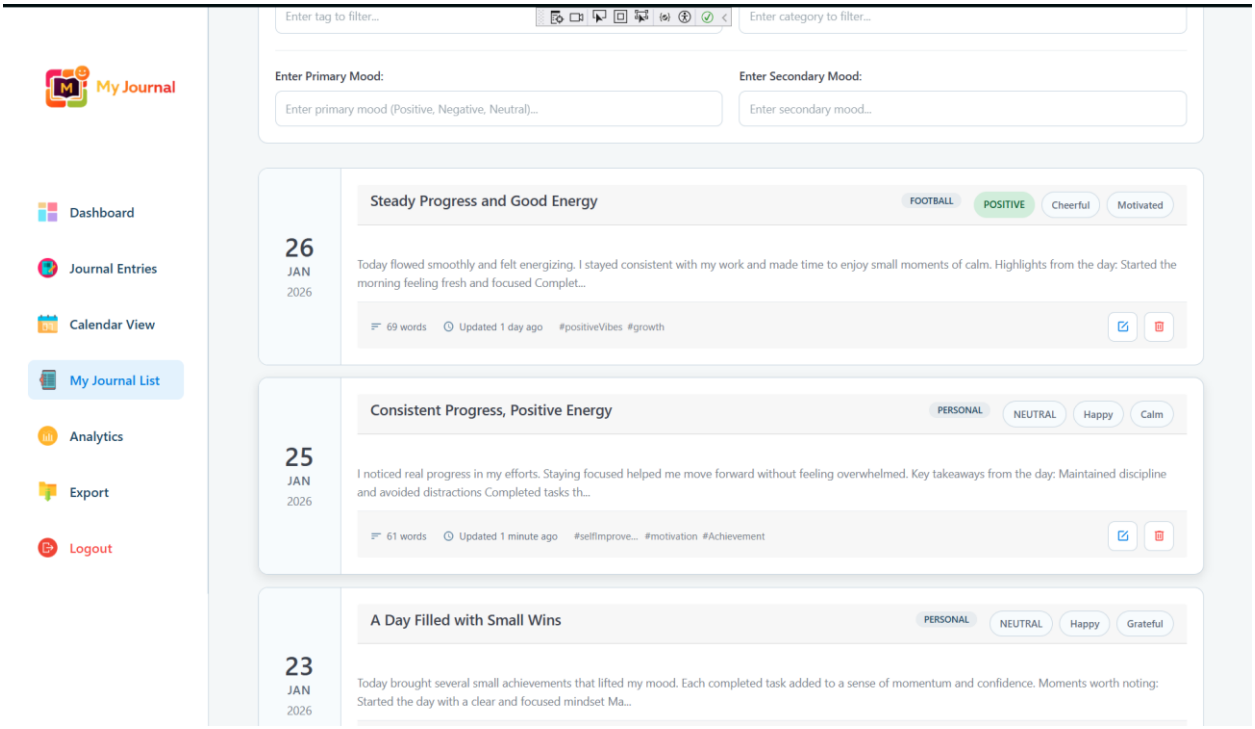


Figure 145: Journal Entry of 25 JAN 2026 in Journal List Page before deletion.

- Delete Confirmation dialog.

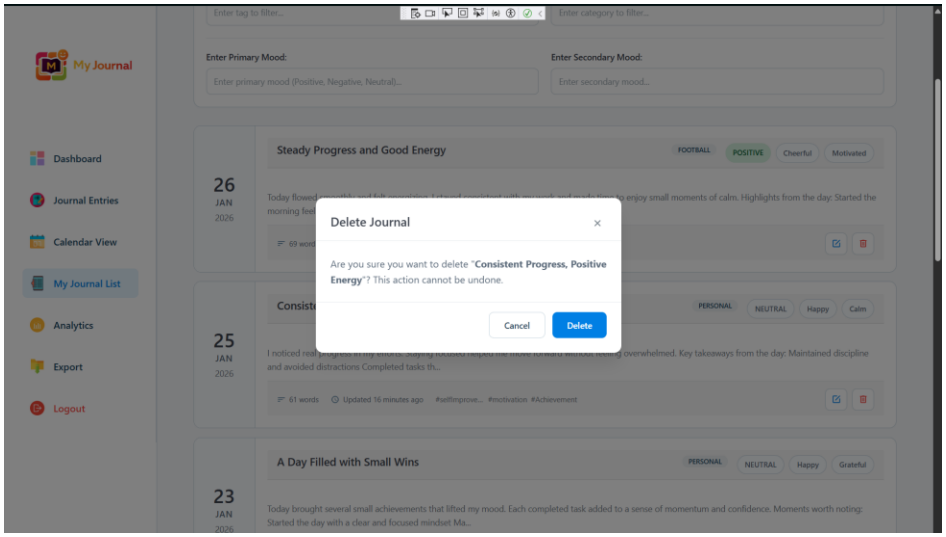


Figure 146: Delete Confirmation dialog.

- Updated journal list showing the entry removed of 25 JAN 2026

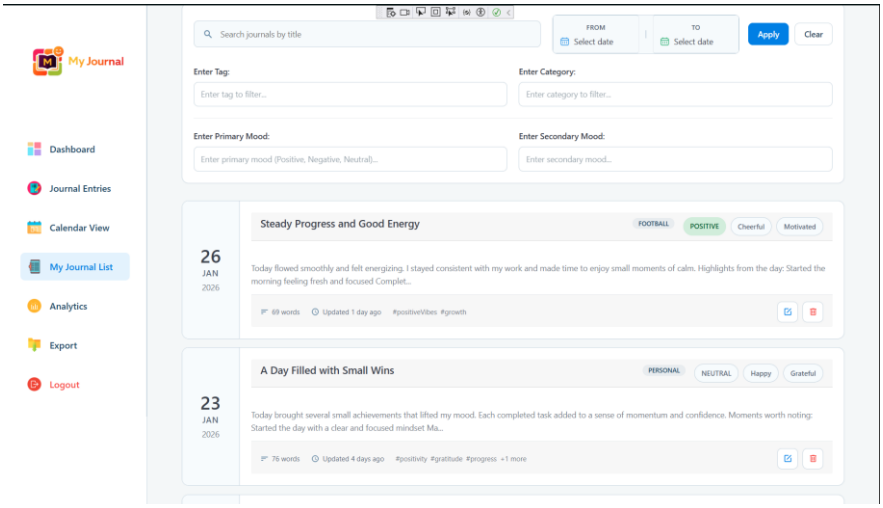


Figure 147: Updated journal list showing the entry removed

- SQLite database confirmation the journal record has been removed.

Figure 148: SQLite database confirmation the journal record has been removed.

- JournalService.cs with delete functionality.

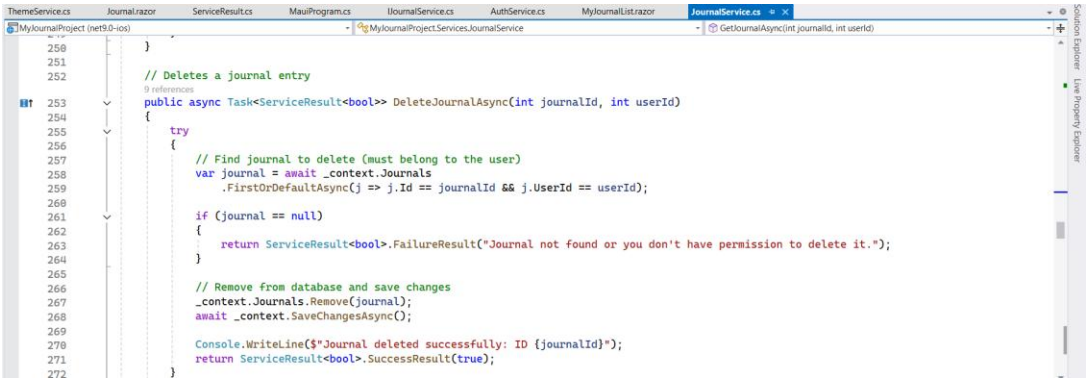


Figure 149: JournalService.cs with delete functionality.

3.7.11. Test Case 11: Calendar Navigation and View

Table 12: Test Case 11: Calendar Navigation and View

Test Case 11: Journal Deletion	
Objective	To verify that a user can successfully delete an existing journal entry from the journal list.
Action	i. Open the Journaling application. ii. Login with a valid user account: Rishav Thapa. iii. Navigate to the Journal Listing Page. iv. Click on the Delete button (trash icon) for an existing journal entry. v. A confirmation dialog appears. vi. Click “Delete” in the confirmation dialog.
Expected Result	i. A confirmation dialog should appear with the message: “Delete Journal. Are you sure you want to delete this journal entry?” ii. After confirmation, the journal entry should be deleted. iii. A success message should be displayed: “Journal Deleted. Your journal entry has been deleted successfully”. iv. The deleted journal should disappear from the journal list. v. The journal record should be removed from the SQLite database.
Actual Result	i. Confirmation dialog appeared correctly. ii. Journal entry deleted successfully after confirmation. iv. Journal entry removed from the journal list view. v. SQLite database no longer contains the deleted journal record.
Conclusion	The test was successfully implemented. This confirms that the system correctly handles journal deletion with the proper user confirmation, ensures complete data removal from both UI and database, and provides clear user feedback throughout the process.

- Calendar View with current monthly display showing mood-colour wise entries on calendar dates along with Missed days indicators.

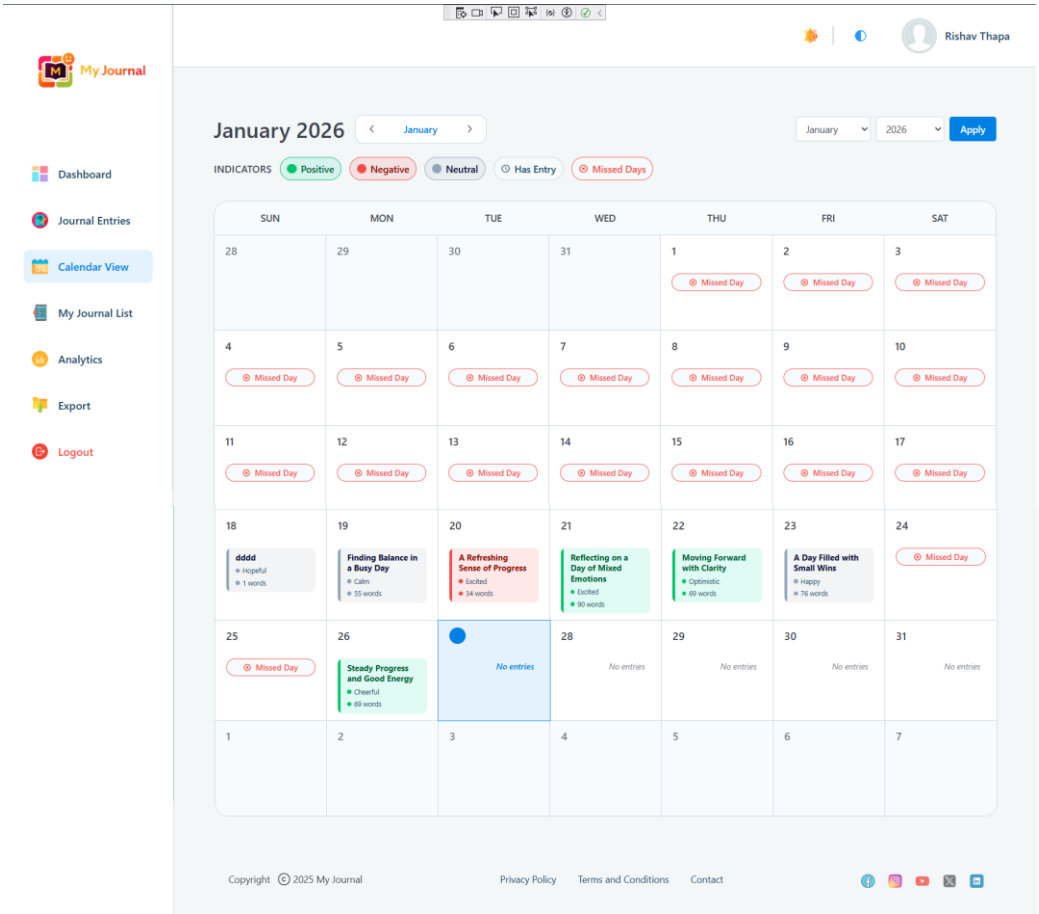


Figure 150: Calendar View

- Previous / Next Month navigation in action.

  Previous Month Navigation

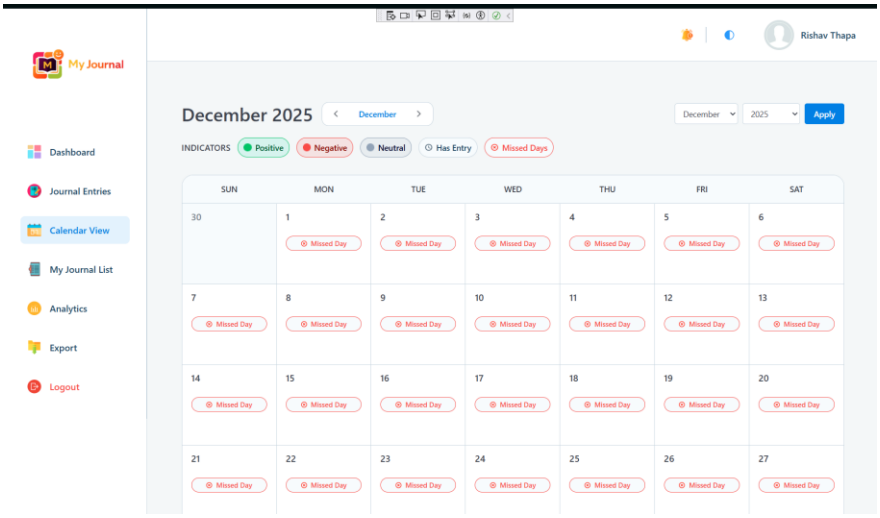


Figure 151: Previous Month Navigation

Next Month Navigation

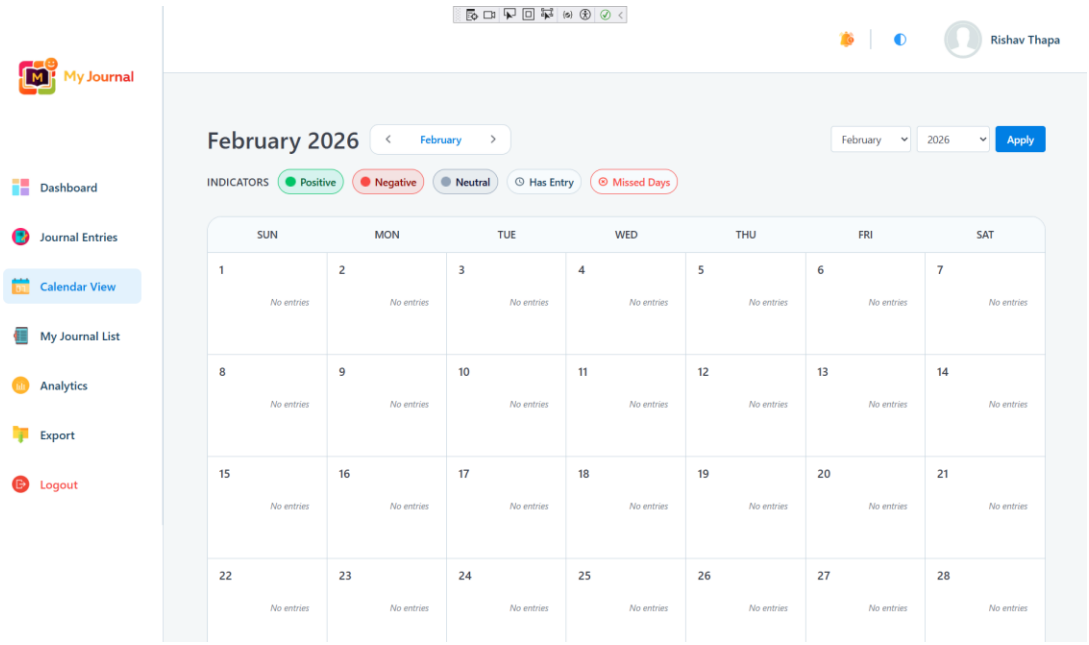


Figure 152: Next Month Navigation

- Month / Year dropdown selection.

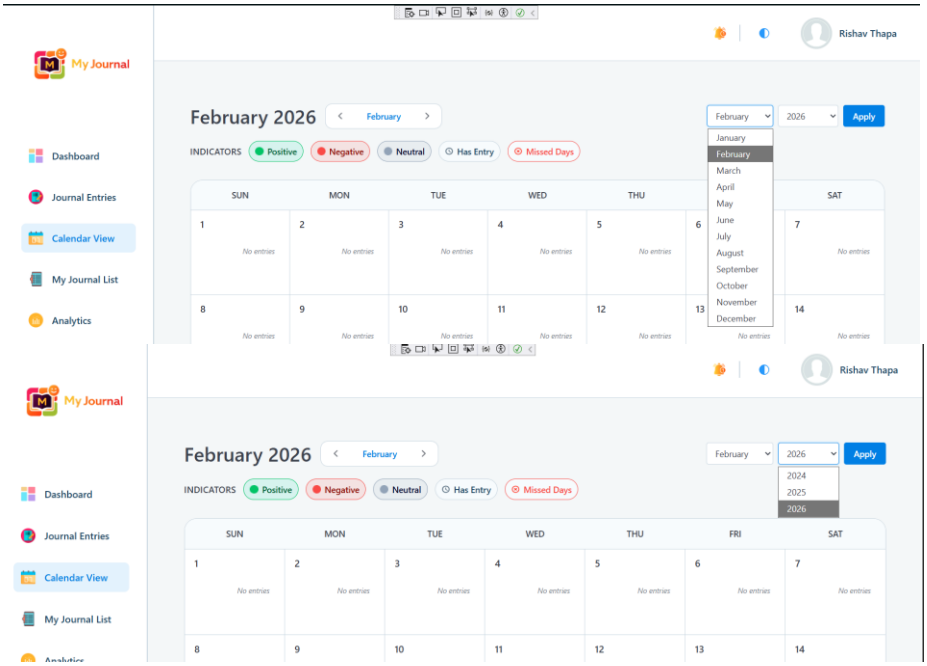


Figure 153: Month / Year dropdown selection.

- Journal details dialog after clicking a date.

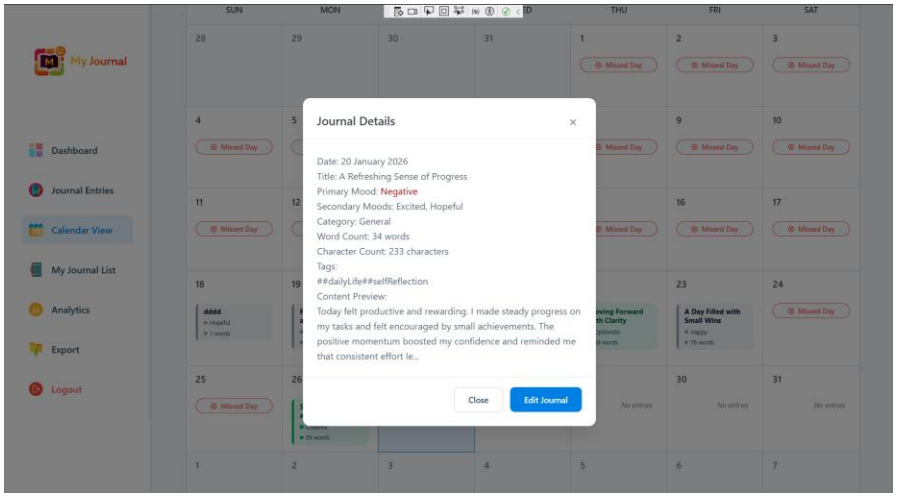


Figure 154: Journal details dialog after clicking a date.

- Calendar.razor

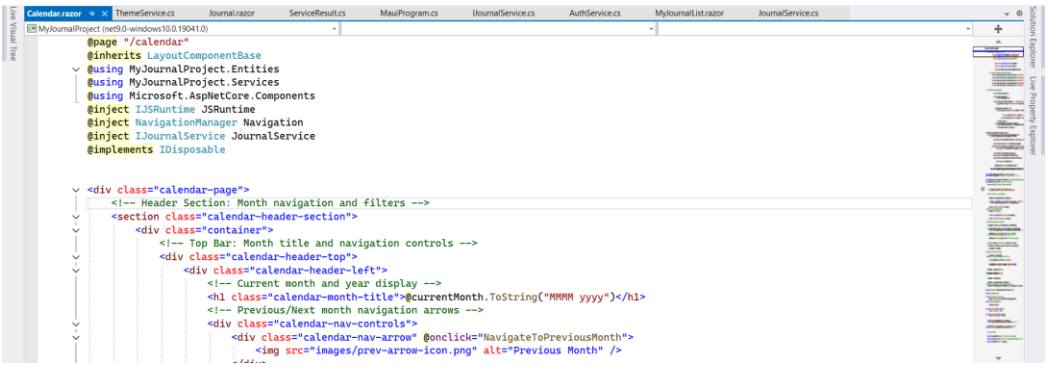


Figure 155: Calendar.razor with Calendar View implementation

3.7.12. Test Case 12: Search and Filter Journal Entries in Journal Listing Page.

Table 13: Test Case 12: Search and Filter Journal Entries in Journal Listing Page.

Test Case 12: Search and Filter Journal Entries in Journal Listing Page	
Objective	To verify that a user can search and filter journal entries using multiple criteria.
Action	i. Open the Journaling application. ii. Login with a valid user account containing multiple journal entries. iii. Navigate to the Journal Listing Page. iv. Enter title in the search box. v. Apply date range filters (From/To) vi. Apply mood filters. vii. Apply tag filters. viii. Apply Category filters.
Expected Result	i. Search should filter entries by journal title. ii. Date range filter should show entries within the selected dates. iii. Mood filters should show entries with entered moods. iv. Tag filters should show entries with entered tags. v. Category filters should show entries with entered categories.

	vi. Clear Filters button should reset all filters.
Actual Result	i. Search correctly filtered journal entries by title. ii. Date range filtering worked as expected. iii. Mood filters displayed correct results. iv. Tag filters displayed correct results. v. Category filters functioned correctly. vi. Manual inputs were accepted using the Enter key. vii. Clear filters successfully reset all filters criteria.
Conclusion	The test was successfully implemented. This confirms that the search and filter functionality works correctly across all filtering criteria, providing users with flexible and accurate way to find specific journal entries efficiently.

- My Journal List Page (default view)

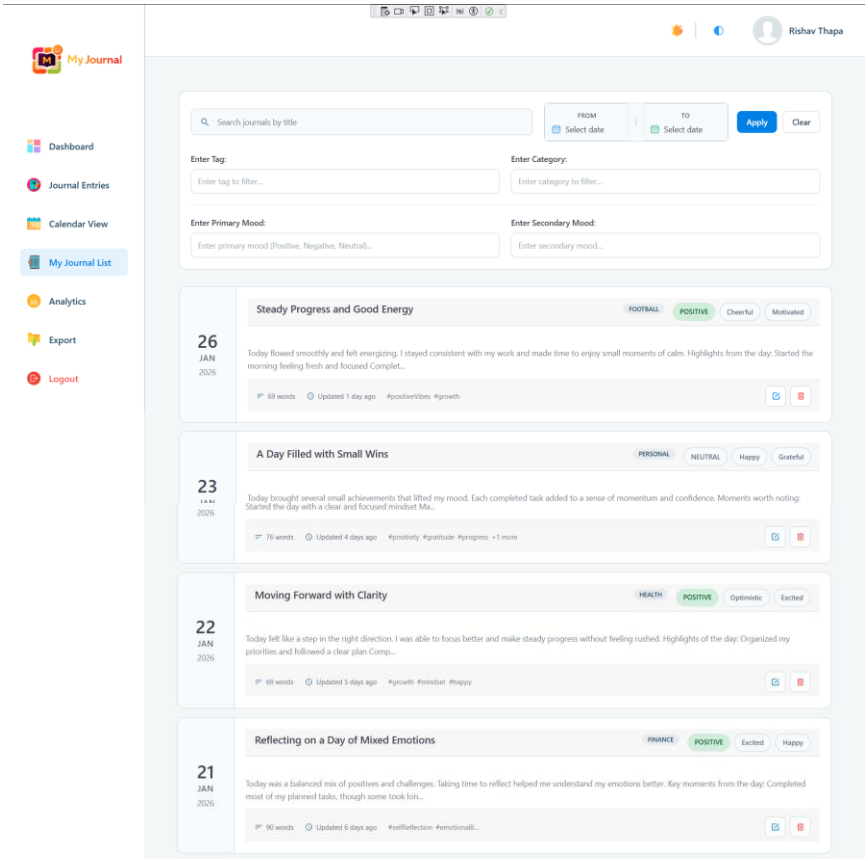


Figure 156: My Journal List Page (default view)

- Showing search results filter by title.

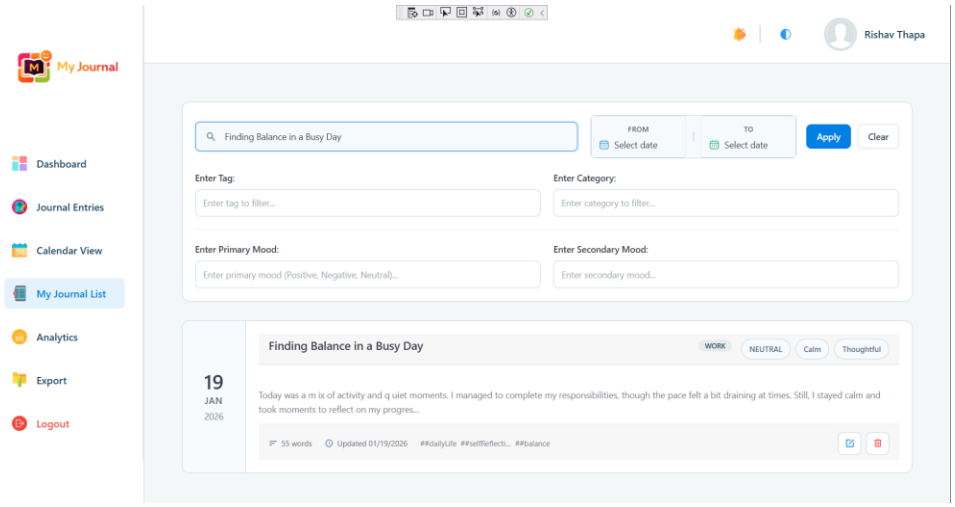


Figure 157: Showing search results filter by title.

- Showing date range filtered results.

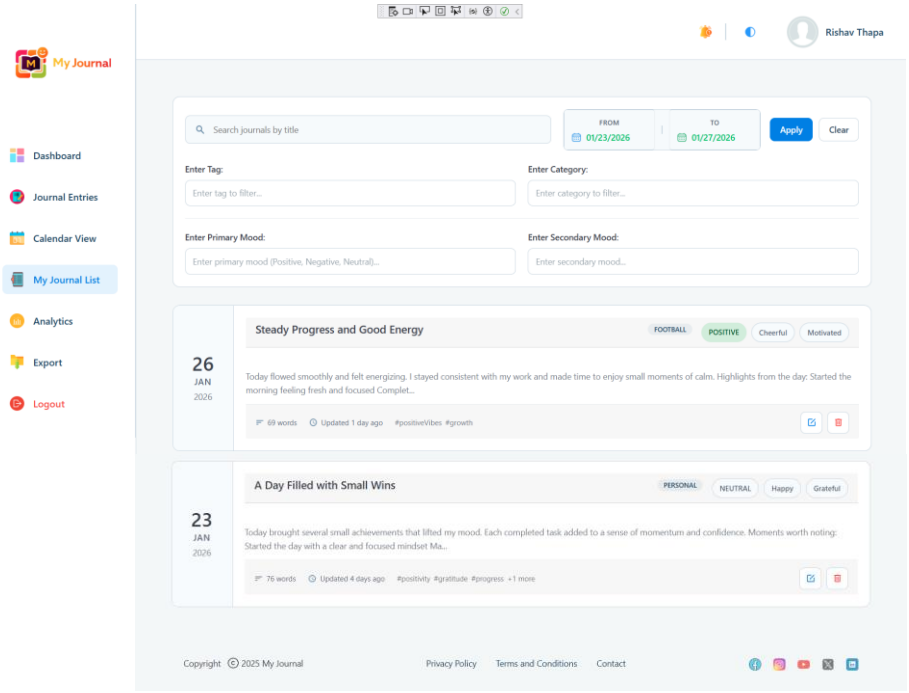


Figure 158: Showing date range filtered results.

- Mood-filtered results.

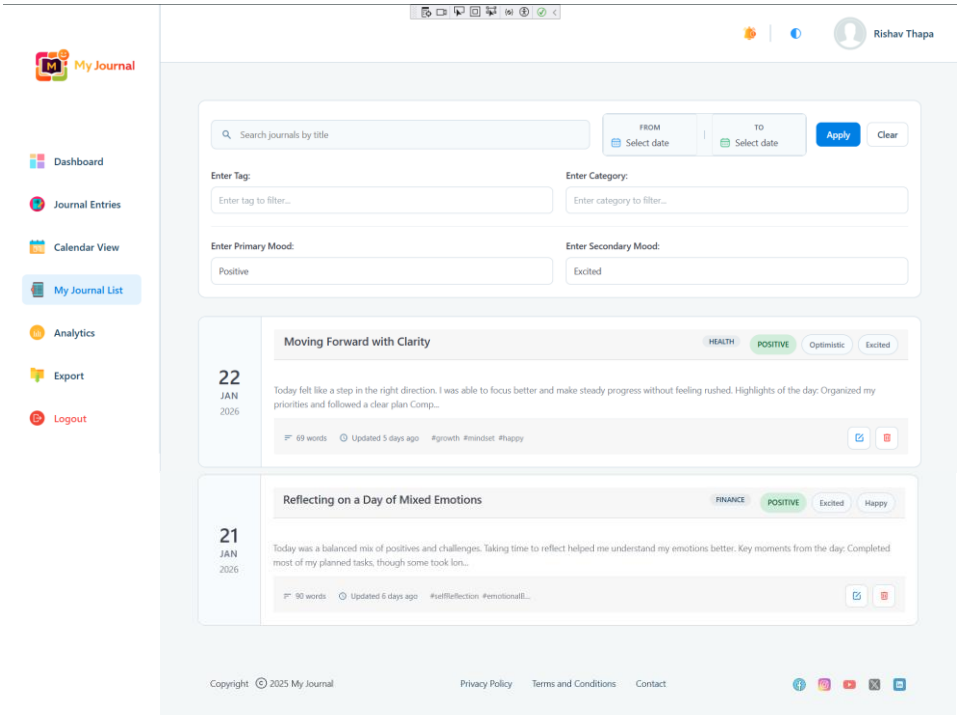


Figure 159: Mood-filtered results.

- Tag filtered journal listing.

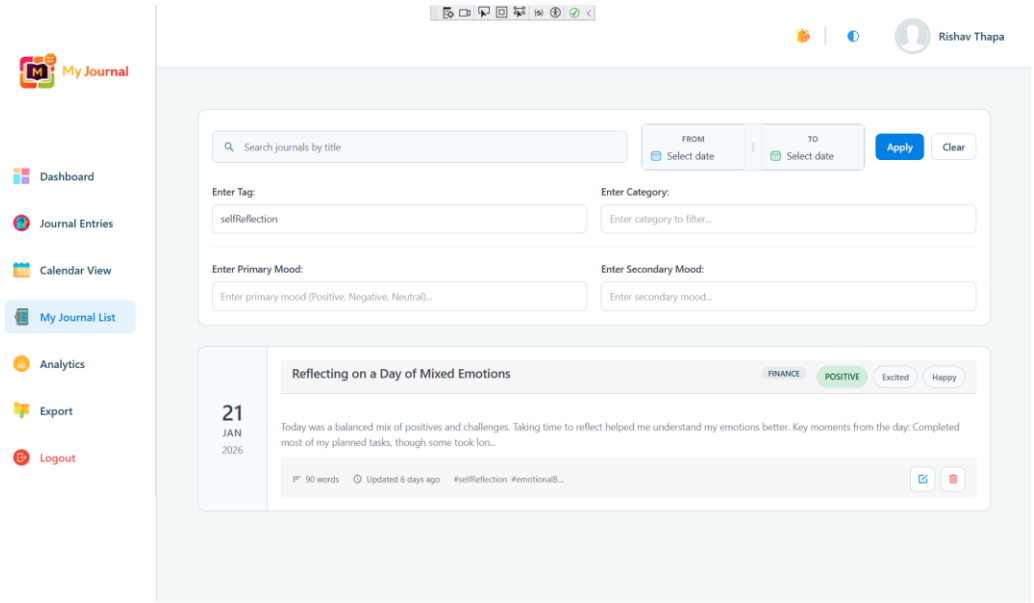


Figure 160: Tag filtered journal listing.

- Category filtered results.

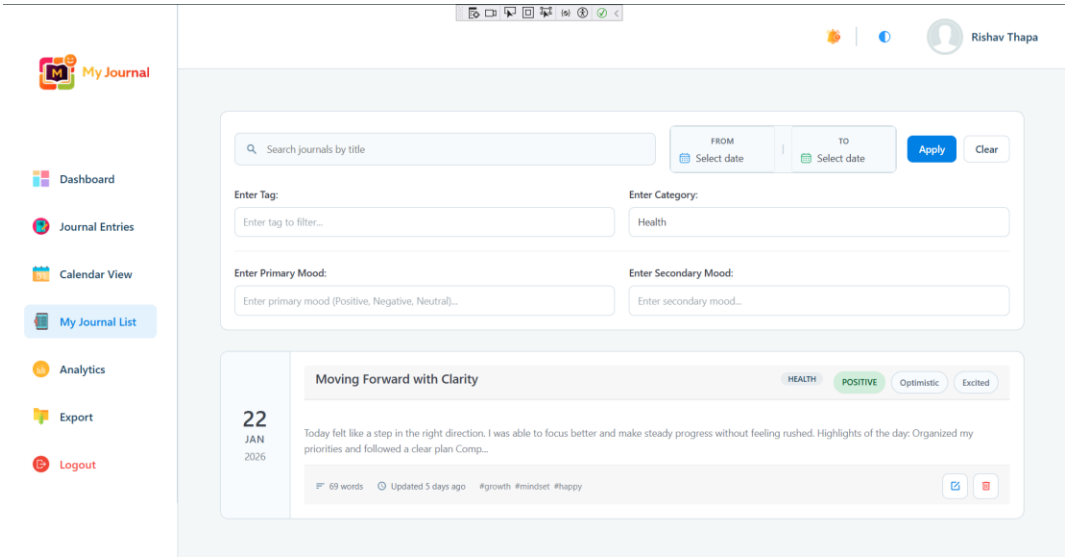


Figure 161: Category filtered results.

- Combined filters applied simultaneously.

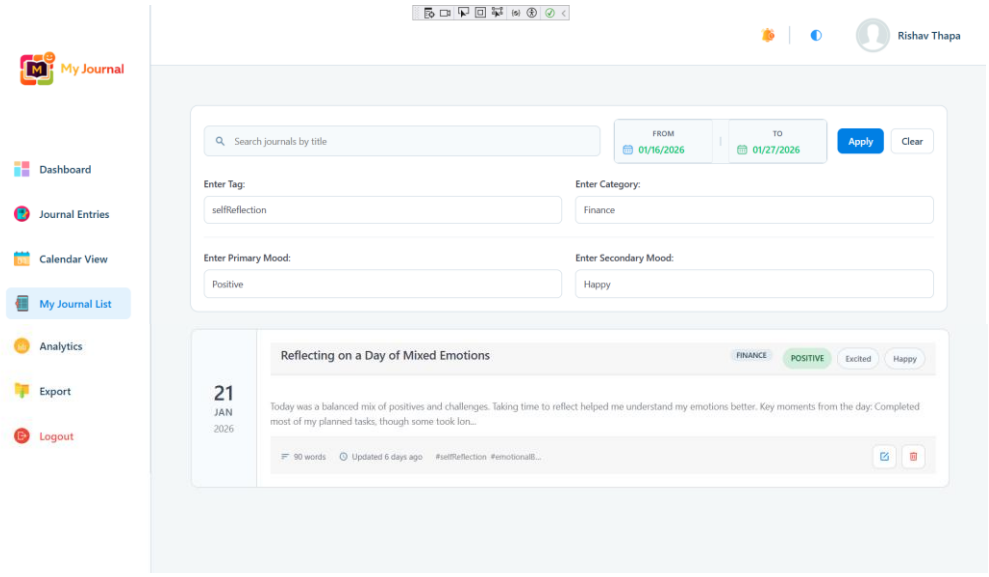
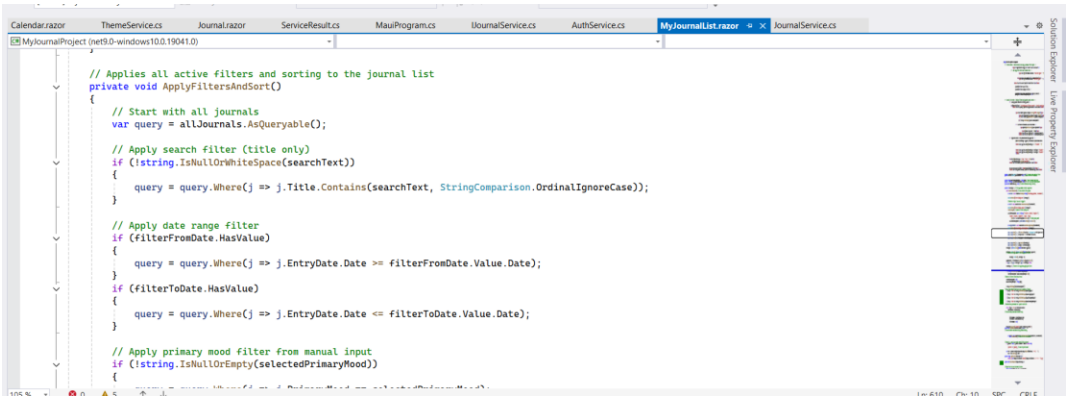


Figure 162: Combined filters applied simultaneously.

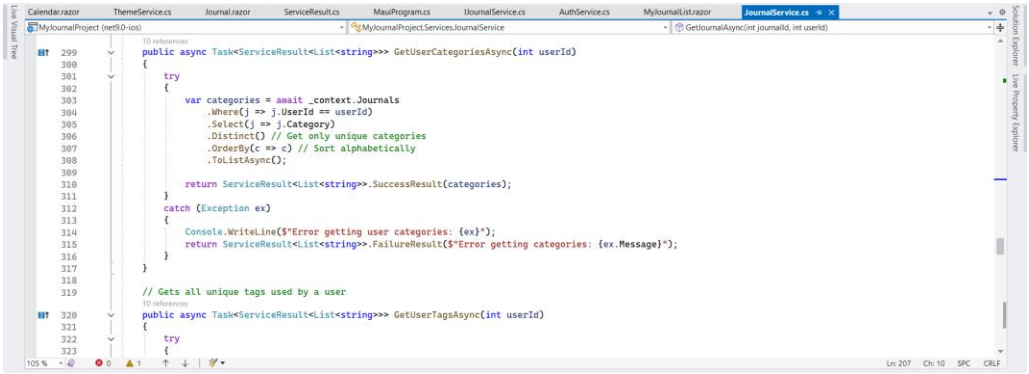
- MyJouranlList.razor with filter and search implementation.



```
// Applies all active filters and sorting to the journal list
private void ApplyFiltersAndSort()
{
    // Start with all journals
    var query = allJournals.AsQueryable();

    // Apply search filter (title only)
    if (!string.IsNullOrWhiteSpace(searchText))
    {
        query = query.Where(j => j.Title.Contains(searchText, StringComparison.OrdinalIgnoreCase));
    }

    // Apply date range filter
    if (filterFromDate.HasValue)
    {
        query = query.Where(j => j.EntryDate.Date >= filterFromDate.Value.Date);
    }
    if (filterToDate.HasValue)
    {
        query = query.Where(j => j.EntryDate.Date <= filterToDate.Value.Date);
    }

    // Apply primary mood filter from manual input
    if (!string.IsNullOrEmpty(selectedPrimaryMood))
    {
```

Figure 163: MyJouranlList.razor with filter and search implementation.

- JournalService.cs with filter and search implementation.



```
public async Task<ServiceResult<List<string>>> GetUserCategoriesAsync(int userId)
{
    try
    {
        var categories = await _context.Journals
            .Where(j => j.UserId == userId)
            .Select(j => j.Category)
            .Distinct() // Get only unique categories
            .OrderBy(c => c) // Sort alphabetically
            .ToListAsync();

        return ServiceResult<List<string>>.SuccessResult(categories);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error getting user categories: {ex}");
        return ServiceResult<List<string>>.FailureResult($"Error getting categories: {ex.Message}");
    }
}

// Gets all unique tags used by a user
public async Task<ServiceResult<List<string>>> GetUserTagsAsync(int userId)
{
    try
    {
```

Figure 164: JournalService.cs with filter and search implementation.

3.7.13. Test Case 13: Dashboard Analytics Display

Table 14: Test Case 13: Dashboard Analytics Display

Test Case 13: Dashboard Analytics Display	
Objective	To verify that the dashboard correctly calculates, displays, and update analytics and statistics.
Action	i. Open the Journaling application. ii. Login with a valid user account containing multiple journal entries. iii. Navigate to the Dashboard page. iv. Check current streak display. v. Check longest streak display. vi. Check missed days count. vii. Check current mood display. viii. Check mood distribution chart. ix. Check word count trend chart. x. Check top tags analytics section. xi. Change chart period dropdowns.
Expected Result	i. Current streak should display correctly. ii. Longest streak should show the highest consecutive days. iii. Missed days should count days without entries in the current month. iv. Current mood should show the latest journal's mood. v. Mood distribution chart should show correct mood proportions. vi. Word count chart should show trends over the selected period. vii. Top tags should display most frequently used tags. viii. Period dropdowns should dynamically update charts.
Actual Result	i. All statistics displayed correctly. ii. Charts rendered properly using Chart.js. iii. Period changes updated charts dynamically. iv. Moo icons and colour displayed correctly.

	v. Tag progress bars showed correct usage number.
Conclusion	The test was successfully implemented. This confirms that the dashboard analytics system accurately calculates and display all statistics, with interactive visualizations that respond correctly to user selections and filters.

- Dashboard main view with current streak, longest streak, missed days, current mood display, mood distribution chart, word count trend chart and top tags analytics.

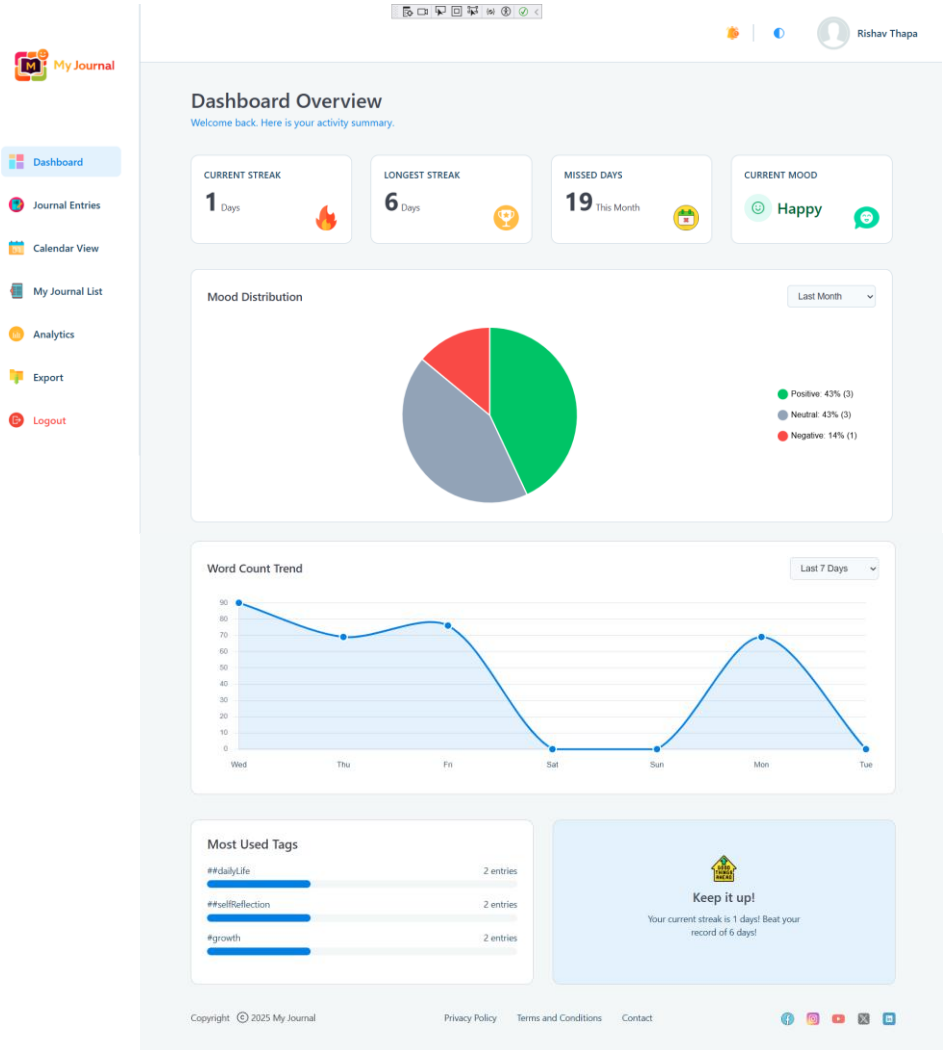


Figure 165: Dashboard main view

- Interactive charts with hovering.

Figure 166: Interactive charts with hovering.

- Dynamic chart updates after dropdown time interval change.

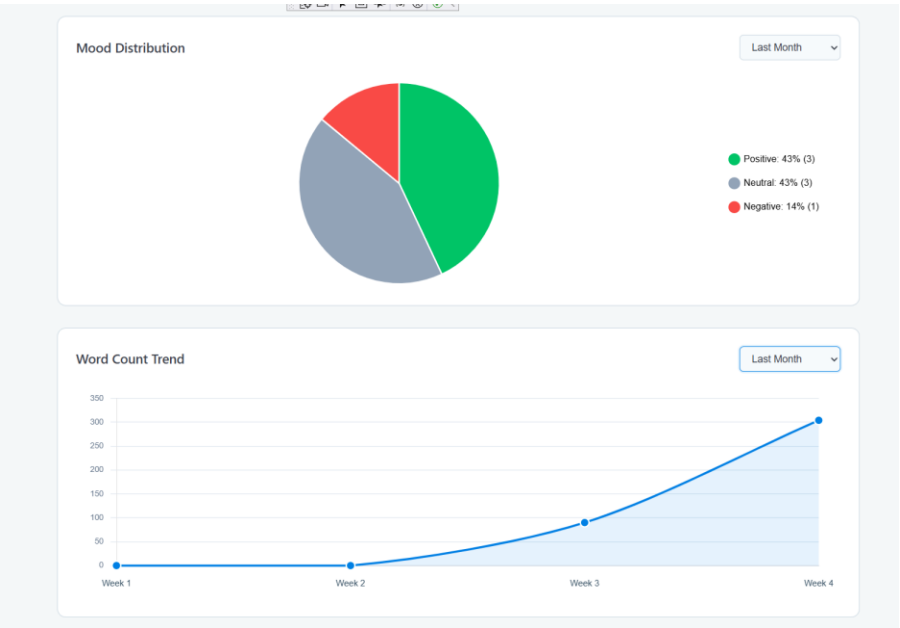
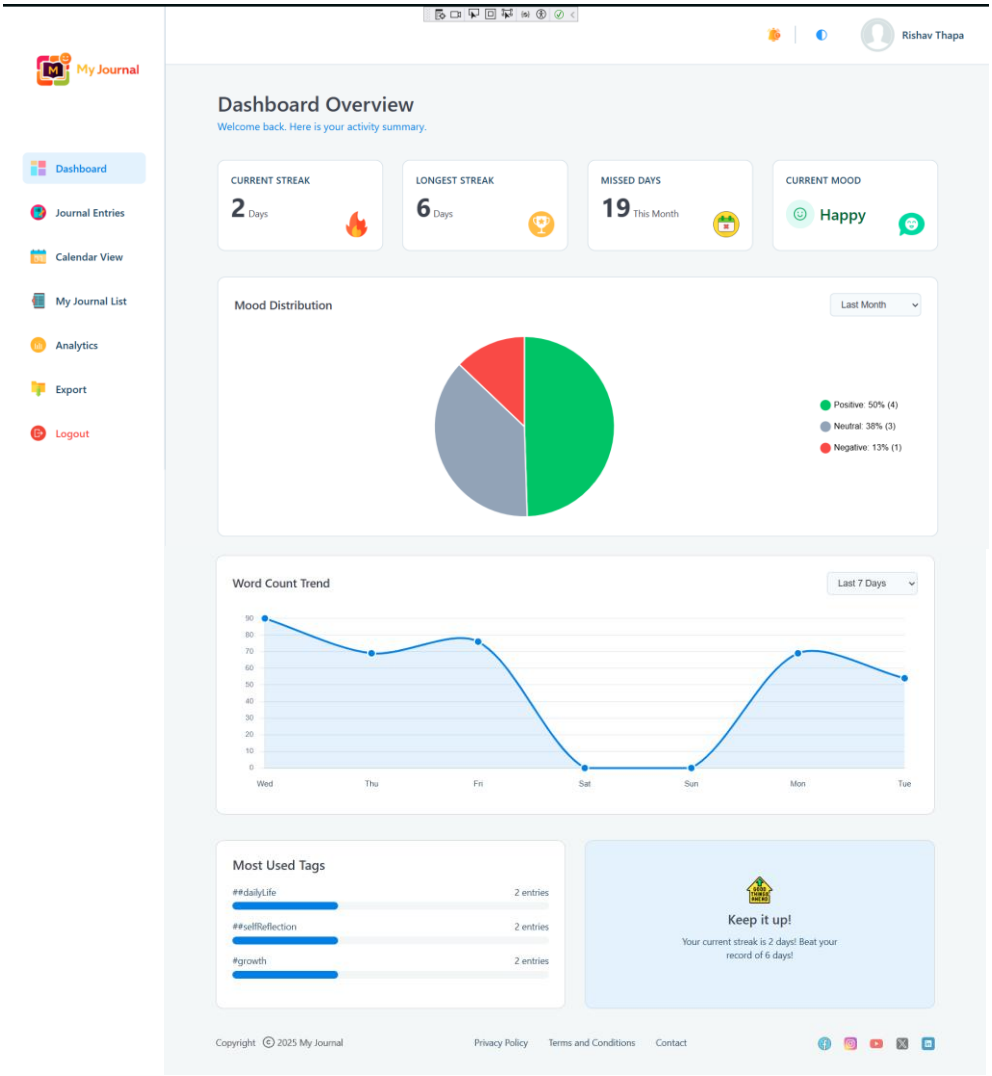
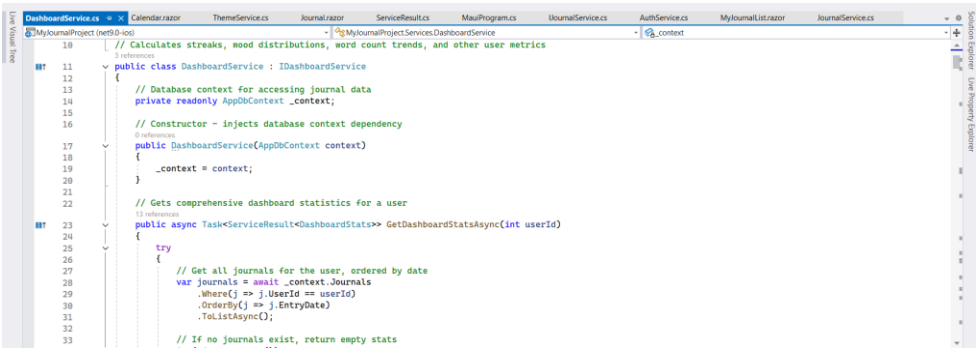


Figure 167: Dynamic chart updates after dropdown time interval change.

- Dashboard updates after new journal entry.



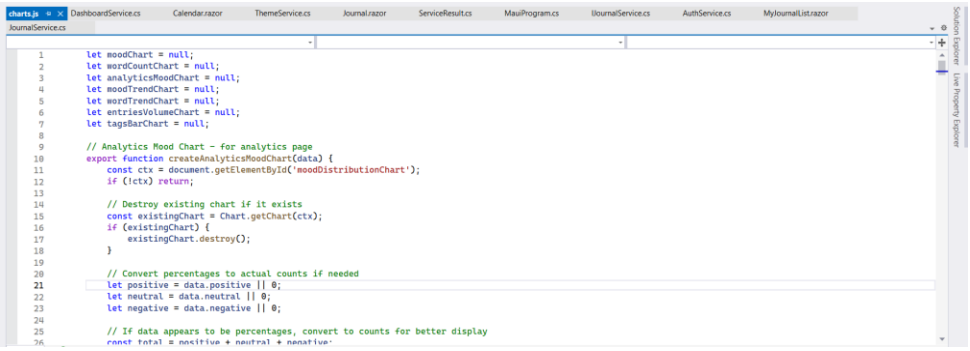
- DashboardService.cs with Dashboard View code.



```
// Calculates streaks, mood distributions, word count trends, and other user metrics
public class DashboardService : IDashboardService
{
    // Database context for accessing journal data
    private readonly AppDbContext _context;

    // Constructor - injects database context dependency
    public DashboardService(AppDbContext context)
    {
        _context = context;
    }

    // Gets comprehensive dashboard statistics for a user
    public async Task<ServiceResult<DashboardStats>> GetDashboardStatsAsync(int userId)
    {
        try
        {
            // Get all journals for the user, ordered by date
            var journals = await _context.Journals
                .Where(j => j.UserId == userId)
                .OrderBy(j => j.EntryDate)
                .ToListAsync();

            // If no journals exist, return empty stats
```

Figure 168: DashboardService.cs with Dashboard View code.

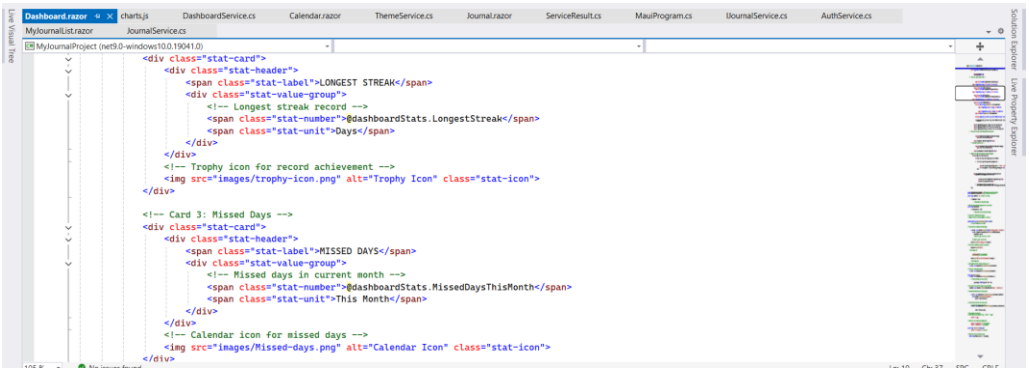
- charts.js configuration and rendering code.



```
let moodChart = null;
let wordCountChart = null;
let analyticsMoodChart = null;
let moodTrendChart = null;
let wordTrendChart = null;
let entriesVolumeChart = null;
let tagsBarChart = null;

// Analytics Mood Chart - for analytics page
export function createAnalyticsMoodChart(data) {
    const ctx = document.getElementById('moodDistributionChart');
    if (!ctx) return;

    // Destroy existing chart if it exists
    const existingChart = Chart.getChart(ctx);
    if (existingChart) {
        existingChart.destroy();
    }

    // Convert percentages to actual counts if needed
    let positive = data.positive || 0;
    let neutral = data.neutral || 0;
    let negative = data.negative || 0;

    // If data appears to be percentages, convert to counts for better display
```

Figure 169: charts.js configuration and rendering code.

- Dashboard.razor with Dashboard View



```
<div class="stat-card">
    <div class="stat-header">
        <span class="stat-label">LONGEST STREAK</span>
        <div class="stat-value-group">
            <!-- Longest streak record -->
            <span class="stat-number">@dashboardStats.LongestStreak</span>
            <span class="stat-unit">Days</span>
        </div>
    </div>
    <!-- Trophy icon for record achievement -->
    <img src="images/trophy-icon.png" alt="Trophy Icon" class="stat-icon">
</div>

<!-- Card 3: Missed Days -->
<div class="stat-card">
    <div class="stat-header">
        <span class="stat-label">MISSED DAYS</span>
        <div class="stat-value-group">
            <!-- Missed days in current month -->
            <span class="stat-number">@dashboardStats.MissedDaysThisMonth</span>
            <span class="stat-unit">This Month</span>
        </div>
    </div>
    <!-- Calendar icon for missed days -->
    <img src="images/Missed-days.png" alt="Calendar Icon" class="stat-icon">
</div>
```

Figure 170: Dashboard.razor with Dashboard View

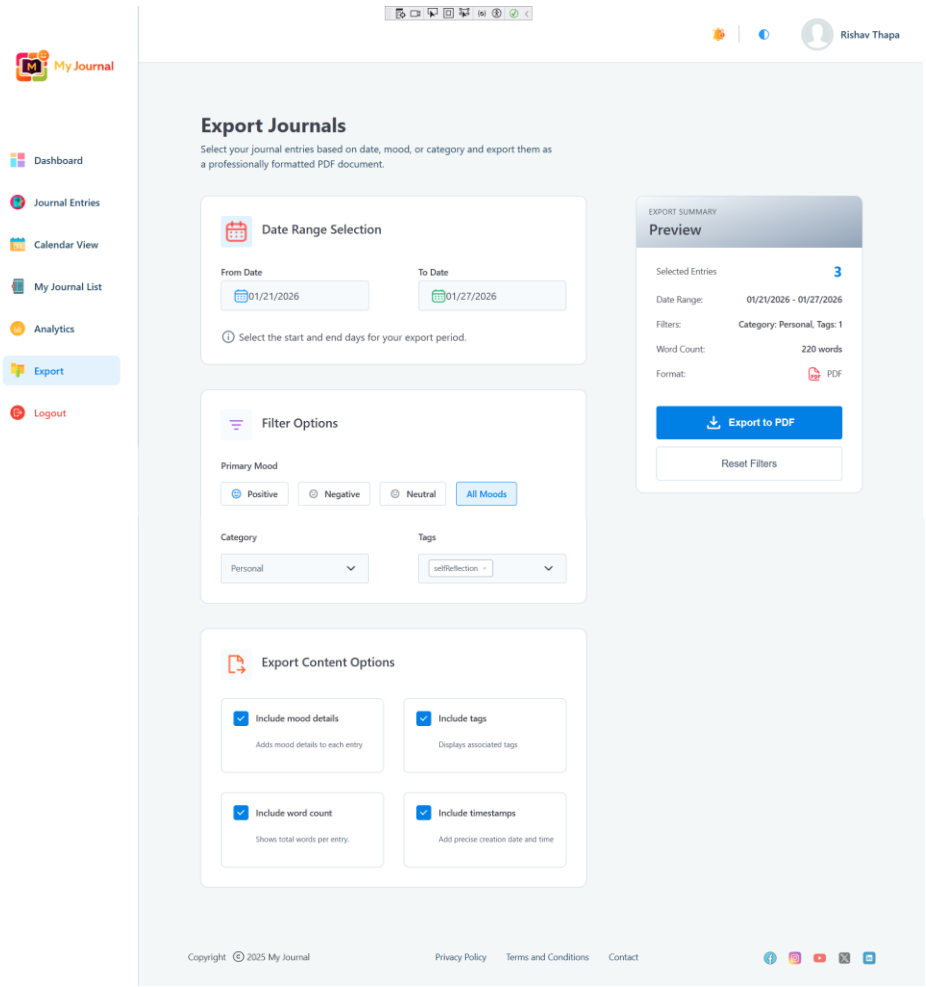
3.7.14. Test Case 14: Export Journal to PDF

Table 15: Export Journal to PDF

Test Case 14: Export Journal to PDF	
Objective	To verify that users can export journal entries to PDF with multiple filters and export options.
Action	i. Open the Journaling application. ii. Login with a valid user account containing multiple journal entries. iii. Navigate to the Export page. iv. Select a date range. v. Apply mood filter. vi. Apply category filter. vii. Apply tag filter. viii. Select export options: • Include Mood Details • Include Tags • Include Word Count • Include Timestamps ix. Click the “Export to PDF” button. x. Open the downloaded PDF file.
Expected Result	i. Preview should show correct entry count and word count. ii. PDF should download automatically. iii. PDF should contain all filtered journal entries. iv. PDF should include selected data (mood, tags, word count, timestamps). v. PDF should be properly formatted with headers, dates, and structured content. vi. PDF should contain page number.
Actual Result	i. Preview updated correctly when filters and options were applied. ii. PDF downloaded successfully.

	iii. PDF contained all filtered journal entries. iv. Selected data was included correctly. v. Professional formatting applied with headings and structured layout. vi. Page numbers were displayed in the footer.
Conclusion	The test was successfully implemented. This confirms that the PDF export functionality works correctly, allowing users to export filtered journal entries with selected options, generating well-structured, properly formatted, and complete PDF documents.

- Export page with date range and selected filter options, including preview showing Selected Entries, Date Range, Filters, Word Count, and Format.



- Export to PDF button click action.

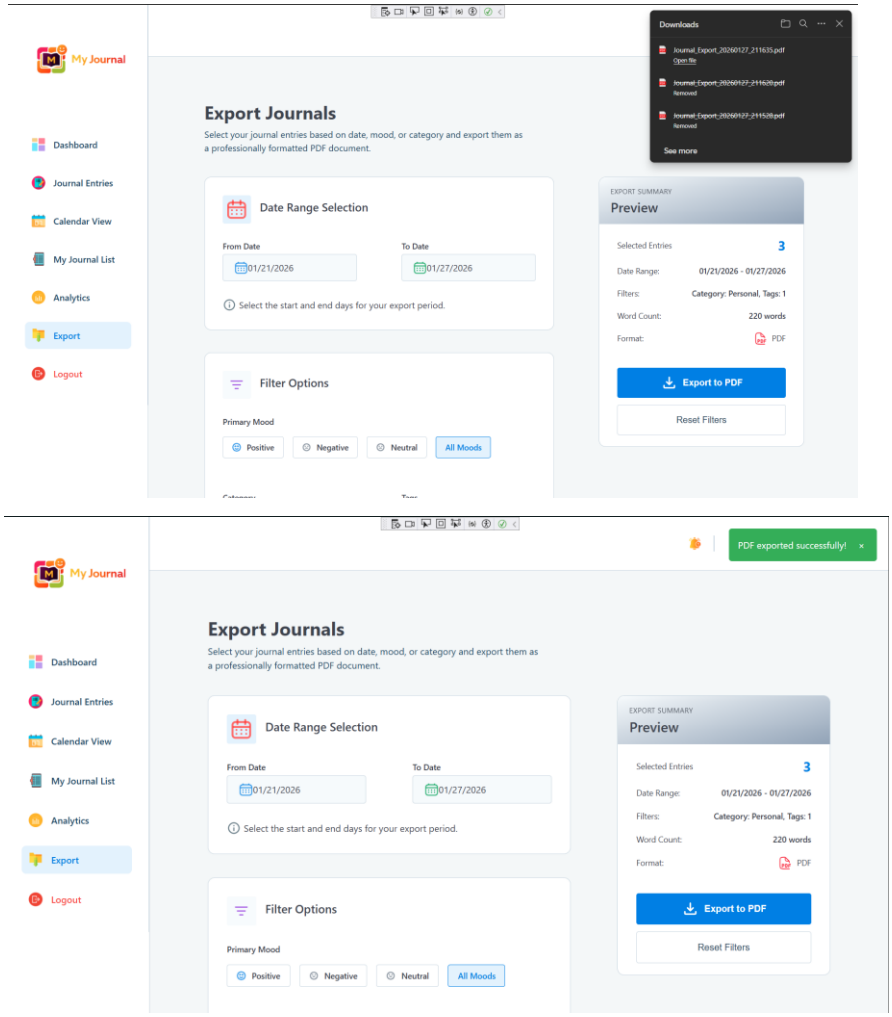


Figure 171: Export to PDF button click action.

- Opening PDF showing:
    - Journal entries
    - Headers and formatting
    - Data like mood, tags, timestamps, word count
    - Page numbers

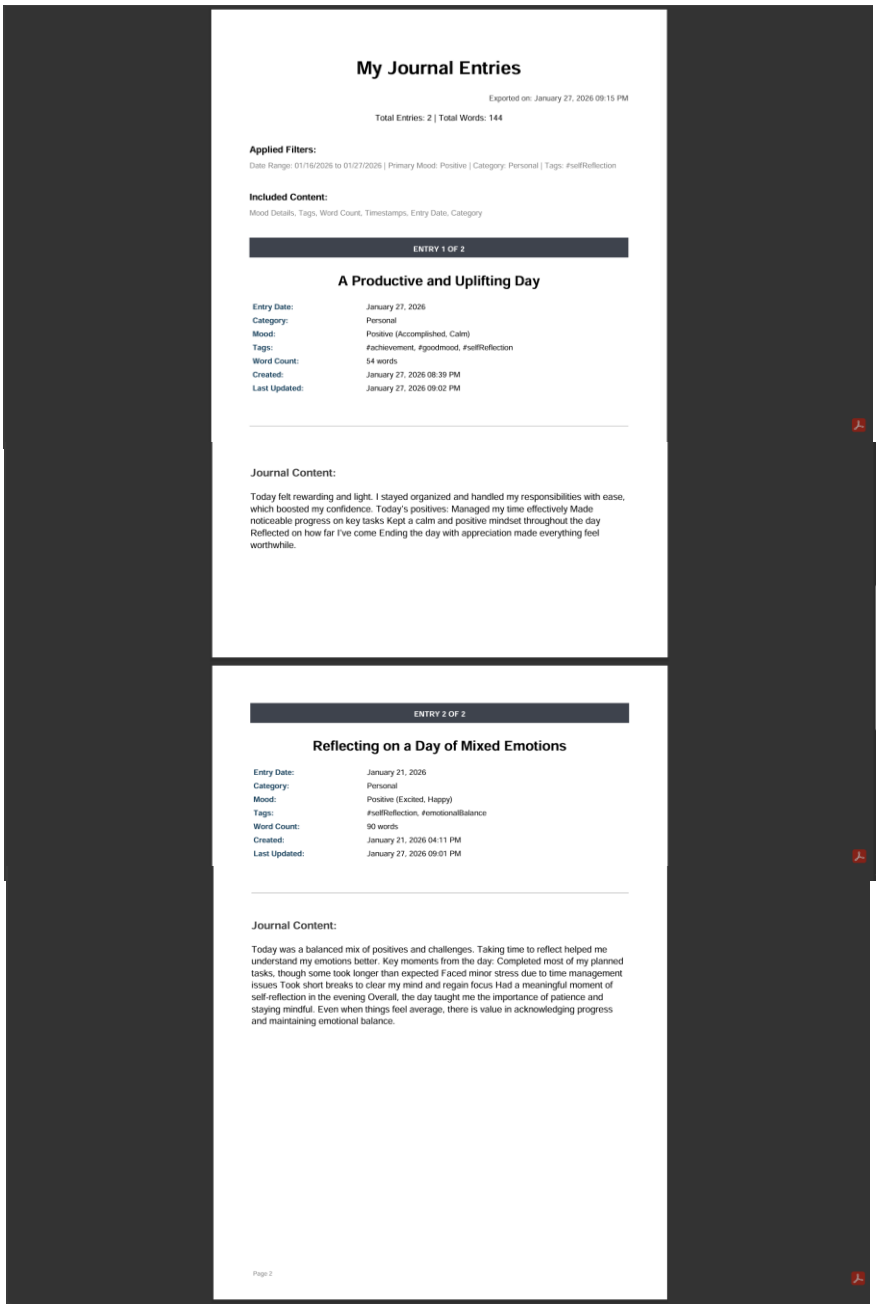


Figure 172: Opening PDF showing

- ExportService.cs with PDF generation logic.

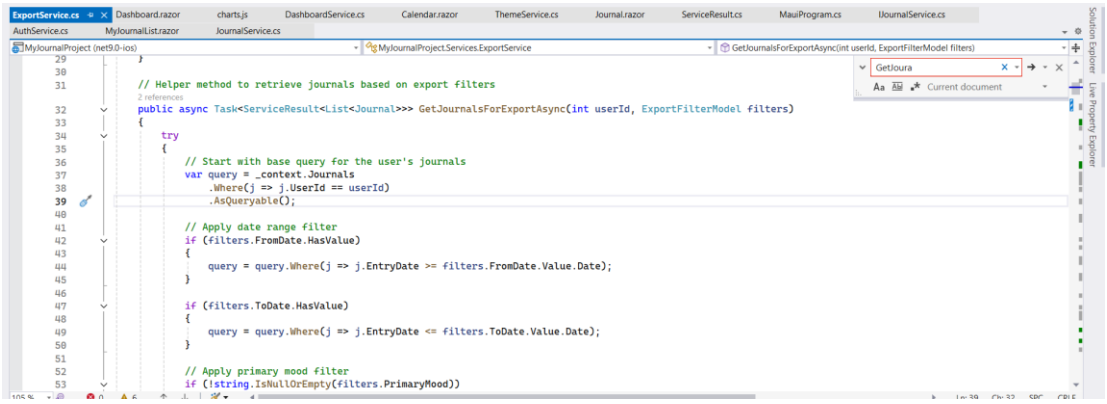


Figure 173: ExportService.cs with PDF generation logic.

- Export.razor UI and export handling.

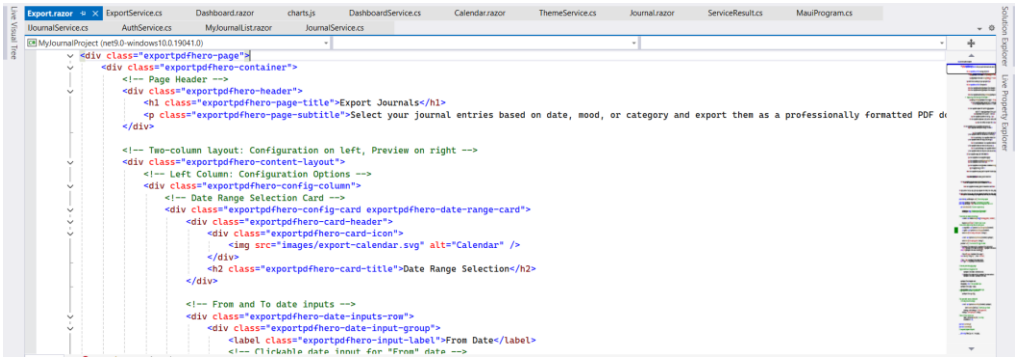


Figure 174: Export.razor UI and export handling.

- ExportModel.cs

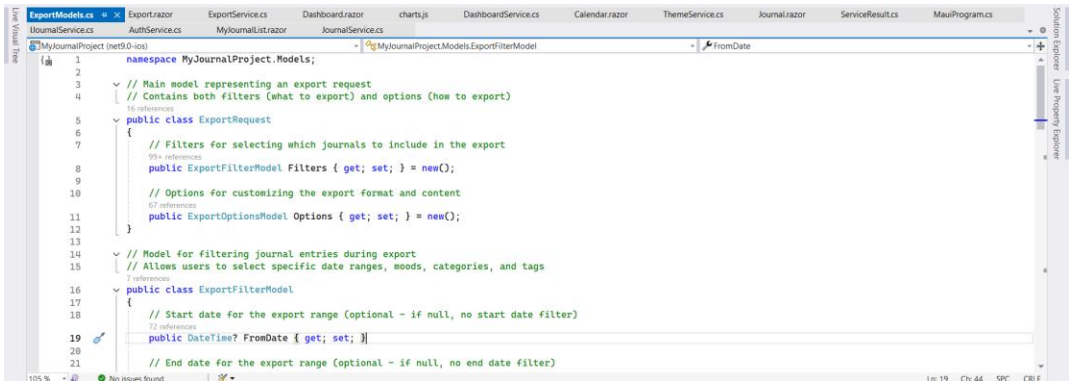


Figure 175: ExportModel.cs

3.7.15. Test Case 15: Theme Toggle Functionality

Table 16: Test Case 15: Theme Toggle Functionality

Test Case 15: Export Journal to PDF	
Objective	To verify that users can switch between light and dark themes and that the selected theme persists across navigation, refresh, and sessions.
Action	i. Open the Journaling application. ii. Login with a valid user account containing multiple journal entries. iii. Click the theme toggle button in the header. iv. Navigate to different pages (Dashboard, Journal, Calendar, Analytics, Export). v. Refresh the page. vi. Logout and login again.
Expected Result	i. Theme should toggle between light and dark modes. ii. Theme should remain across page navigation. iii. Theme should remain after page refresh. iv. Theme should remain after logout and login. v. Theme should apply to all UI elements consistently.
Actual Result	i. Theme toggled correctly between light and dark modes. ii. Theme remained across different pages. iii. Theme remained after a new login. iv. All UI elements adapted correctly to the selected theme.
Conclusion	The test was successfully implemented. This confirms that the theme toggle system works correctly, stores user preferences, and applies the selected theme consistently across all pages.

- Header showing theme toggle button (Light / Dark switch).

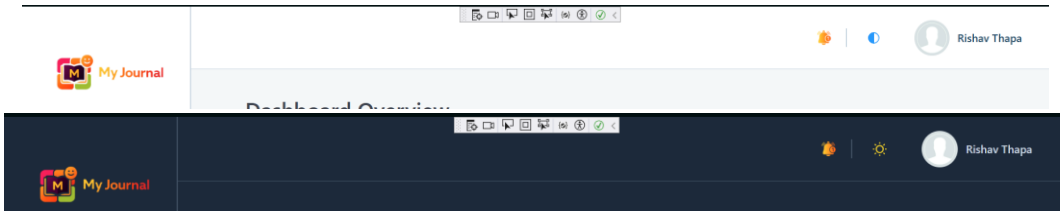


Figure 176: Header showing theme toggle button (Light / Dark switch).

- Light theme interface screenshot (Journal, Calendar, Analytics pages).

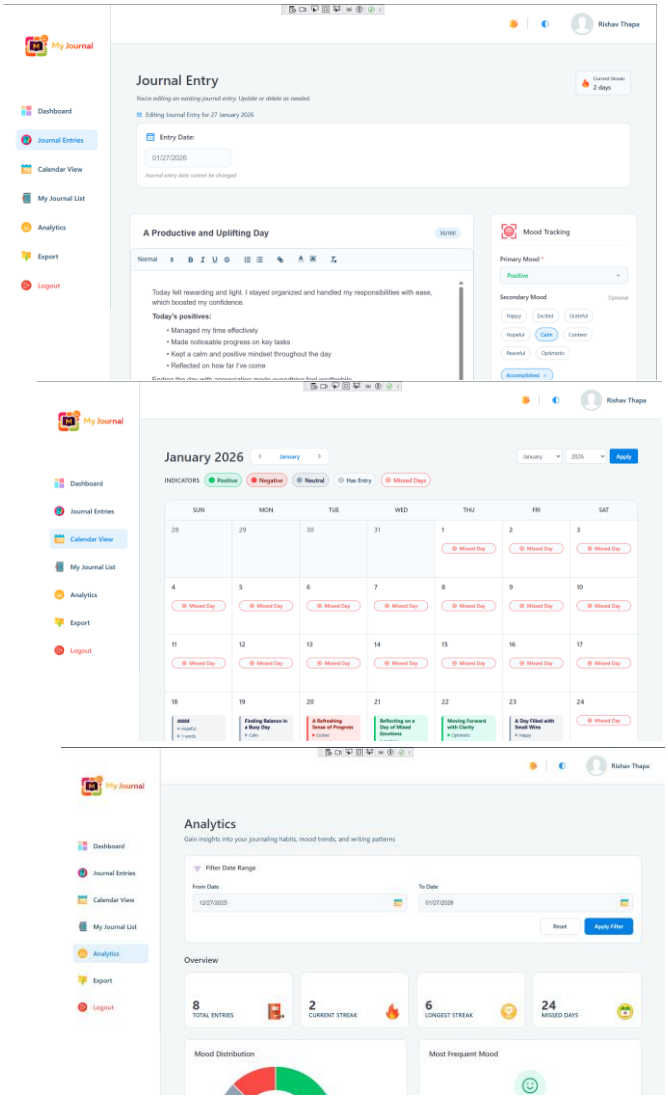


Figure 177: Light theme interface screenshot

- Dark theme interface screenshot (Journal, Calendar, Analytics pages).

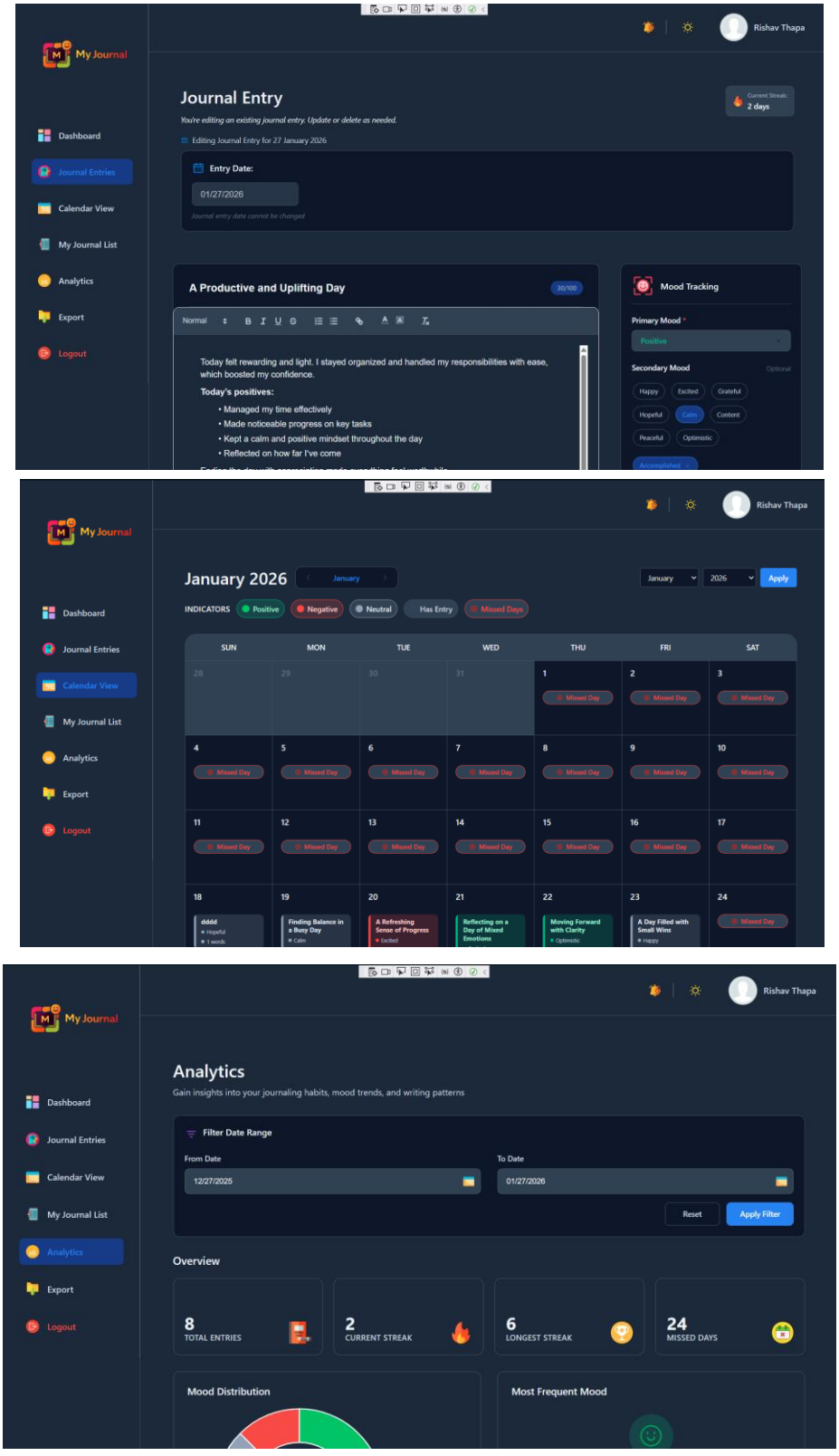


Figure 178: Dark theme interface screenshot.

- Register and login showing consistent theme.

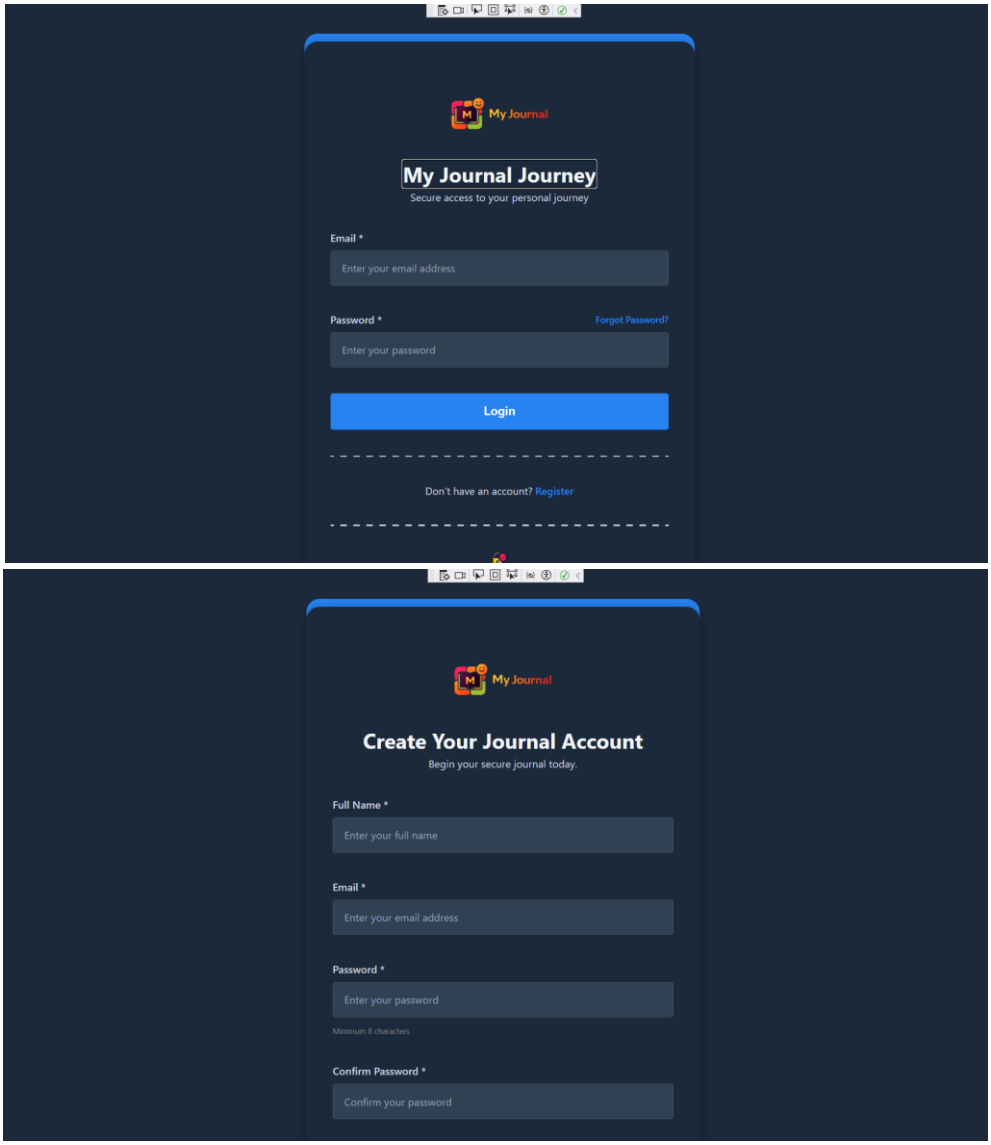
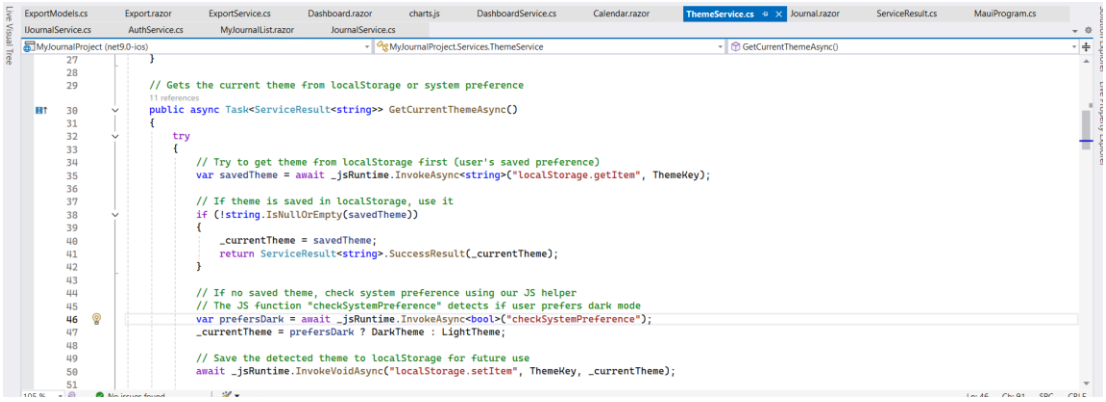


Figure 179: Register and login showing consistent theme.

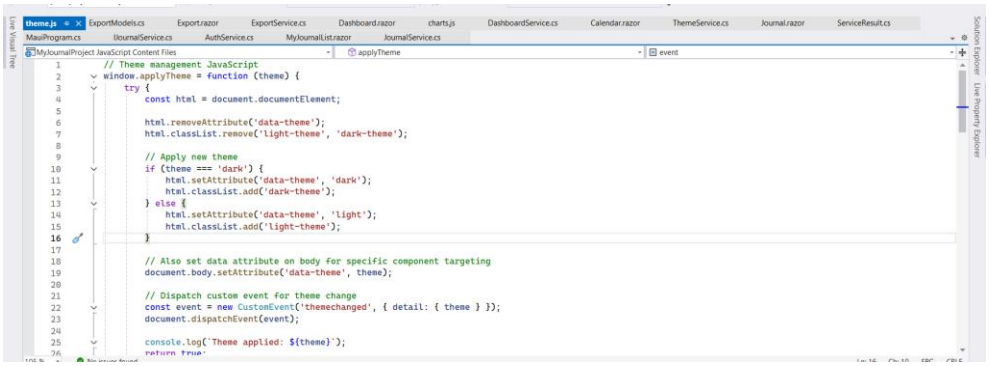
- ThemeService.cs with theme implementation.



```
    }

    // Gets the current theme from localStorage or system preference
    public async Task<ServiceResult<string>> GetCurrentThemeAsync()
    {
        try
        {
            // Try to get theme from localStorage first (user's saved preference)
            var savedTheme = await _jsRuntime.InvokeAsync<string>("localStorage.getItem", ThemeKey);

            // If theme is saved in localStorage, use it
            if (!string.IsNullOrEmpty(savedTheme))
            {
                _currentTheme = savedTheme;
                return ServiceResult<string>.SuccessResult(_currentTheme);
            }

            // If no saved theme, check system preference using our JS helper
            // The JS function "checkSystemPreference" detects if user prefers dark mode
            var prefersDark = await _jsRuntime.InvokeAsync<bool>("checkSystemPreference");
            _currentTheme = prefersDark ? DarkTheme : LightTheme;

            // Save the detected theme to localStorage for future use
            await _jsRuntime.InvokeVoidAsync("localStorage.setItem", ThemeKey, _currentTheme);
```

Figure 180: ThemeService.cs with theme implementation.

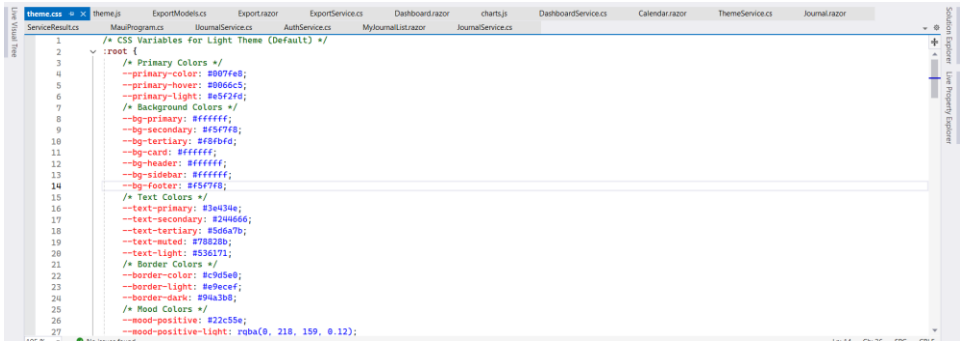
- theme.js



```
// Theme management JavaScript
window.applyTheme = function (theme) {
    try {
        const html = document.documentElement;

        html.removeAttribute('data-theme');
        html.classList.remove('light-theme', 'dark-theme');

        // Apply new theme
        if (theme === 'dark') {
            html.setAttribute('data-theme', 'dark');
            html.classList.add('dark-theme');
        } else {
            html.setAttribute('data-theme', 'light');
            html.classList.add('light-theme');
        }

        // Also set data attribute on body for specific component targeting
        document.body.setAttribute('data-theme', theme);

        // Dispatch custom event for theme change
        const event = new CustomEvent('themechanged', { detail: { theme } });
        document.dispatchEvent(event);

        console.log(`Theme applied: ${theme}`);
```

Figure 181: theme.js

- theme.css



```
/* CSS Variables for Light Theme (Default) */
:root {
    /* Primary Colors */
    --primary-color: #007fe8;
    --primary-hover: #0066c5;
    --primary-light: #e5f2fd;
    /* Background Colors */
    --bg-primary: #ffffff;
    --bg-secondary: #f5f7f8;
    --bg-tertiary: #f8fbfd;
    --bg-card: #ffffff;
    --bg-header: #ffffff;
    --bg-sidebar: #ffffff;
    --bg-footer: #f5f7f8;
    /* Text Colors */
    --text-primary: #3e434e;
    --text-secondary: #244666;
    --text-tertiary: #5d6a7b;
    --text-muted: #7882bb;
    --text-light: #536171;
    /* Border Colors */
    --border-color: #c9d5e0;
    --border-light: #e9ecef;
    --border-dark: #94a3b8;
    /* Mood Colors */
    --mood-positive: #22c55e;
    --mood-positive-light: rgba(0, 218, 159, 0.12);
```

Figure 182: theme.css

3.7.16. Test Case 16: Pagination in Journal List

Table 17: Test Case 16: Pagination in Journal List

Test Case 16: Pagination in Journal List	
Objective	To verify that pagination works correctly when a user has many journal entries in My Journal List page.
Action	i. Open the Journaling application. ii. Login with a valid user account that has more than 7 journal entries (page size). iii. Navigate to the My Journal List page. iv. Click the Next Page button. v. Click the Previous Page button. vi. Click specific page number. vii. Click the First and Last page buttons.
Expected Result	i. Only 7 entries should be displayed per page. ii. Page navigation buttons should work correctly. iii. The current page should be highlighted.
Actual Result	i. 7 entries were displayed per page. ii. All navigation buttons worked correctly. iii. The current page was highlighted.
Conclusion	The test was successfully implemented. This confirms that the pagination system works correctly, allowing smooth navigation through large sets of journal entries with clear UI feedback and proper page control behavior.

- First page showing only 7 journal entries.

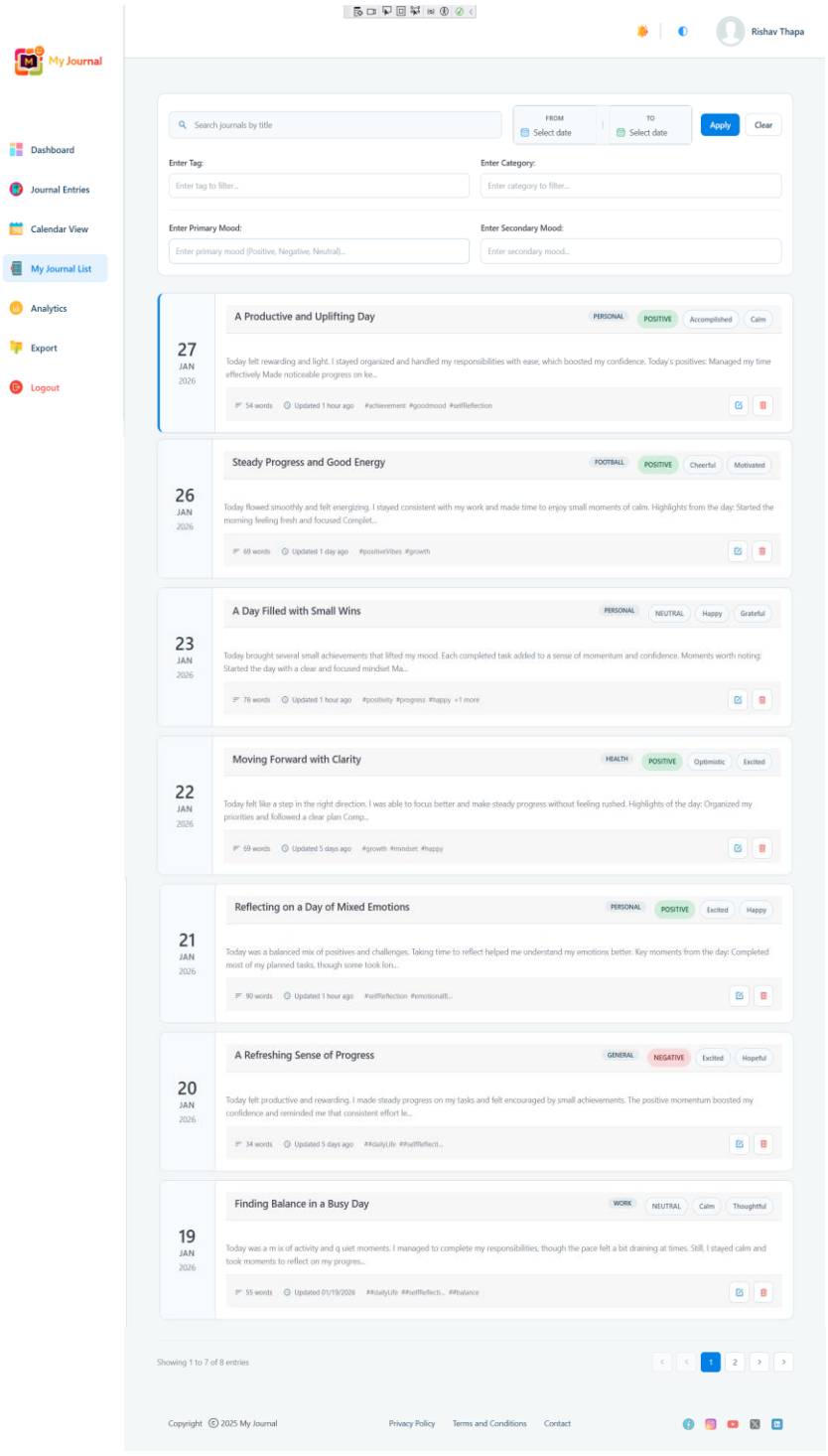


Figure 183: First page showing only 7 journal entries.

- Second page showing remaining journal entries.

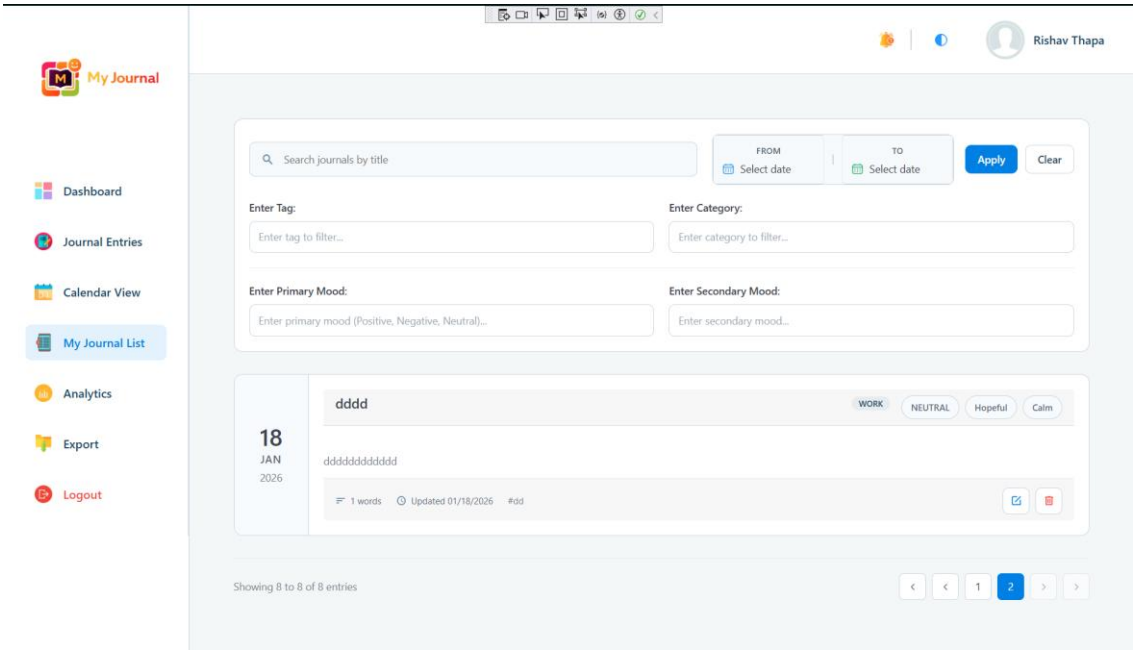


Figure 184: Second page showing remaining journal entries.

3.7.17. Test Case 17: Rich Text Editor Functionality

Table 18: Test Case 17: Rich Text Editor Functionality

Test Case 17: Rich Text Editor Functionality	
Objective	To verify that the rich text editor supports text formatting, content structuring, and content statistics correctly.
Action	i. Open the Journaling application. ii. Login with a valid user account. iii. Navigate to Create Journal Entry Page. iv. Enter sample text in the editor. v. Apply bold formatting to selected text. vi. Apply italic formatting to selected text. vii. Apply underline formatting to selected text. viii. Create an unordered list.

	ix. Create an ordered list. x. Insert a hyperlink. xi. Apply heading styles (H1, H2, H3). xii. Observe word count and character count display.
Expected Result	i. Bold, italic, and underline formatting should apply correctly. ii. Ordered and unordered lists should render properly. iii. Hyperlinks should be clickable and correctly formatted. iv. Headings should display with appropriate font sizes and styles. v. Word count and character count should update dynamically based on content.
Actual Result	i. Text formatting tools applied correctly. ii. Lists rendered properly. iii. Links inserted and functional. iv. Headings displayed correctly. v. Word and character counts updated in real time.
Conclusion	The test was successfully implemented. This confirms that the rich text editor fully supports text formatting, structure content creation, and real-time content statistics ensuring an enhanced and user-friendly journaling experience.

- Rich text editor UI showing formatting (bold/italic/underline), lists, links, headings and real-time word/character count.

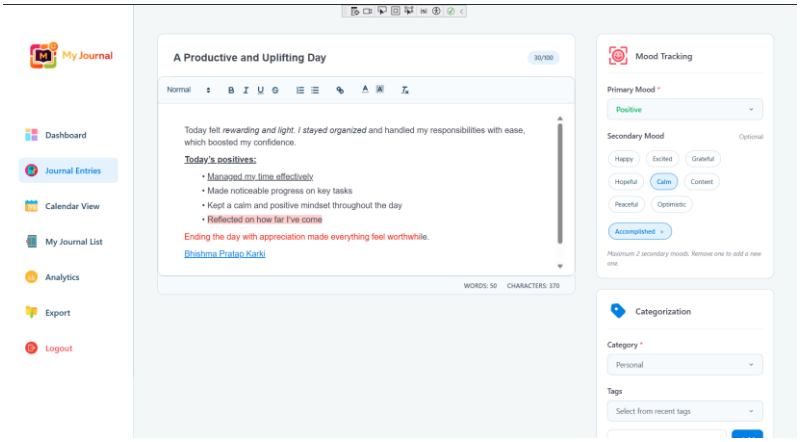


Figure 185: Rich Text Editor UI Showing Formatted Text

- SQLite database record showing formatted HTML content storage.

Id	UserId	Title	
1	1	fffffffffffff	fffffffffffffffffffffffffff
2	2	dddd	ddddddddddddd
3	3	hhhhfbnfnf	fffffffffffffffffffffffffffffffffffffffffffff
4	3	Moments of Motivation	Today was a mix of activity and quiet moments. I managed to complete my responsibilities, though the pace felt a bit draining at times. Still, I stayed calm and took moments to reflect on my progress.
5	2	Finding Balance in a Busy Day	Today was a mix of activity and quiet moments. I managed to complete my responsibilities, though the pace felt a bit draining at times. Still, I stayed calm and took moments to re
6	1	Finding Balance in a Busy Day	Today was a mix of activity and quiet moments. I managed to complete my responsibilities, though the pace felt a bit draining at times. Still, I stayed calm and took moments to reflect on my progress.
8	3	A Refreshing Sense of Progress	Today felt productive <s><u>and rewarding. I made steady progress on my tasks and felt encouraged by small achievements. The positive momentum boosted my confidence and reminded me that consistent
9	1	A Calm Pause in the Day	The day moved at a gentle pace, giving me time to pause and observe my thoughts. Nothing felt rushed or overwhelming, which made it easier to stay present. Moments like these help me reset and appreci
10	2	A Refreshing Sense of Progress	Today felt productive and rewarding. I made steady progress on my tasks and felt encouraged by small achievements. The positive momentum boosted my confidence and reminded me that consistent effort le
11	3	Overcoming a Slow Start	oday was a balanced mix of positives and challenges. Taking time to reflect helped me understand my emotions better. Key moments from the day: Completed most of my pl
12	2	Reflecting on a Day of Mixed Emotions	Today was a balanced mix of positives and challenges. Taking time to reflect helped me understand my emotions better. Key moments from the day: Completed most of my p
13	1	Reflecting on a Day of Mixed Emotions	Today was a balanced mix of positives and challenges. Taking time to reflect helped me understand my emotions better. Key moments from the day: Completed most of my p
14	2	Moving Forward with Clarity	Today felt like a step in the right direction. I was able to focus better and make steady progress without feeling rushed. Highlights of the day: Organized my priorit
15	3	Moving Forward with Clarity	Today felt like a step in the right direction. I was able to focus better and make steady progress without feeling rushed. Highlights of the day: Organized my priorit
17	2	A Day Filled with Small Wins	Today brought several small achievements that lifted my mood. Each completed task added to a sense of momentum and confidence. Moments worth noting: Started the day w
20	2	Steady Progress and Good Energy	Today flowed smoothly and felt energizing. I stayed consistent with my work and made time to enjoy small moments of calm. Highlights from the day: Started the morning
23	2	A Productive and Uplifting Day	Today felt rewarding and light. I stayed organized and handled my responsibilities with ease, which boosted my confidence. <u>Today's positives:</u> <u>Manag

Figure 186: SQLite database record showing formatted HTML content storage.

- quill-editor.js

Figure 187: quill-editor.js

3.7.18. Test Case 18: Secondary Mood Limitation

Table 19: Test Case 18: Secondary Mood Limitation

Test Case 18: Secondary Mood Limitation	
Objective	To verify that the system correctly controls secondary mood selection and customization.
Action	i. Open the Journaling application. ii. Login with a valid user account.

	iii. Navigate to the Journal Entry page. iv. Select more than two secondary moods. v. Add a custom secondary mood. vi. Select secondary moods from the predefined list.
Expected Result	i. User should only be able to select a maximum of 2 secondary moods. ii. Custom secondary moods should be added successfully. iii. Predefined moods should be selectable without errors.
Actual Result	i. System restricted selection to a maximum of 2 secondary moods. ii. Custom mood was added successfully. iii. Predefined mood selection worked correctly.
Conclusion	The test was successfully implemented. This confirms that the system correctly limits secondary moods, supports custom mood creation, and allows selection from predefined options without issues.

- UI showing error/limit message when selecting more than 2 moods

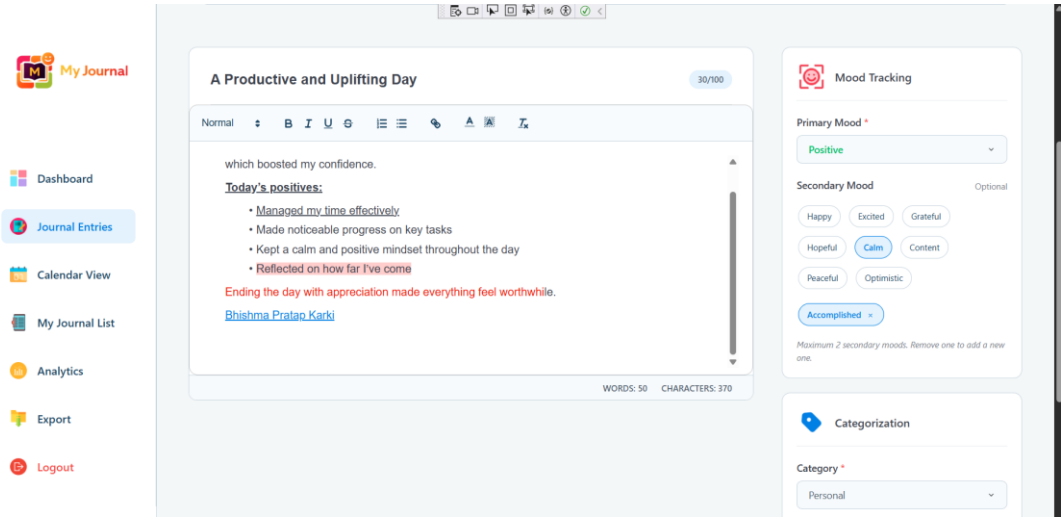


Figure 188: UI showing error/limit message when selecting more than 2 moods

- Custom secondary mood added and displayed.

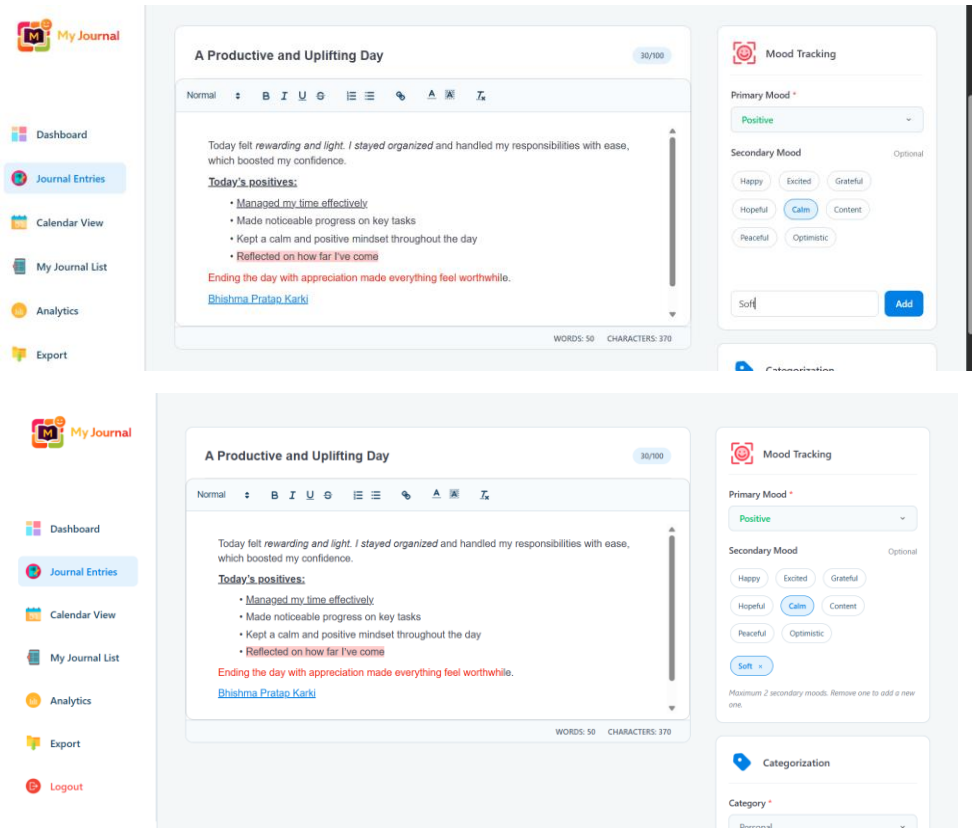


Figure 189: Custom secondary mood added and displayed.

- Saved Journal entry showing selected secondary moods.

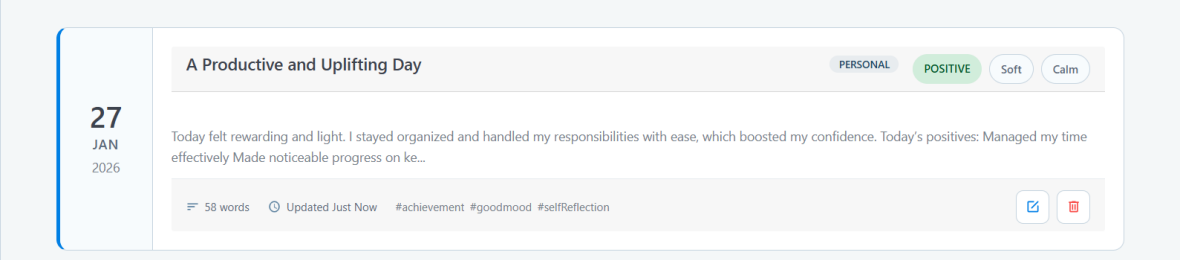


Figure 190: Saved Journal entry showing selected secondary moods.

3.7.19. Test Case 19: Analytics Page Loading and Data Display

Table 20: Test Case 19: Analytics Page Loading and Data Display

Test Case 19: Analytics Page Loading and Data Display	
Objective	To verify that the Analytics page loads correctly and displays all statistics and charts when the user has journal entries.
Action	i. Open the Journaling application. ii. Login with a valid user account. iii. Navigate to Analytics Page. iv. Wait for page for load completely. v. Check all sections: Stats Overview, Mood Distribution, Most Frequent Mood, Mood Trend, Word Cout Trend, Entries Volume, Tags Analysis.
Expected Result	i. Page loads without errors. ii. All data displayed correctly. iii. All Charts render properly. iv. No error messages shown.
Actual Result	i. Analytics page loaded successfully. ii. All data displayed with correct values. iii. Charts rendered using Chart.js. iv. No errors encountered.
Conclusion	The Analytics page functions correctly, loading and displaying all data properly when the user has journal entries.

- Full Analytics Page with stats overview section display, mood distribution chart, mood trend chart, word count trend chart, entries volume chart and tags analysis section.

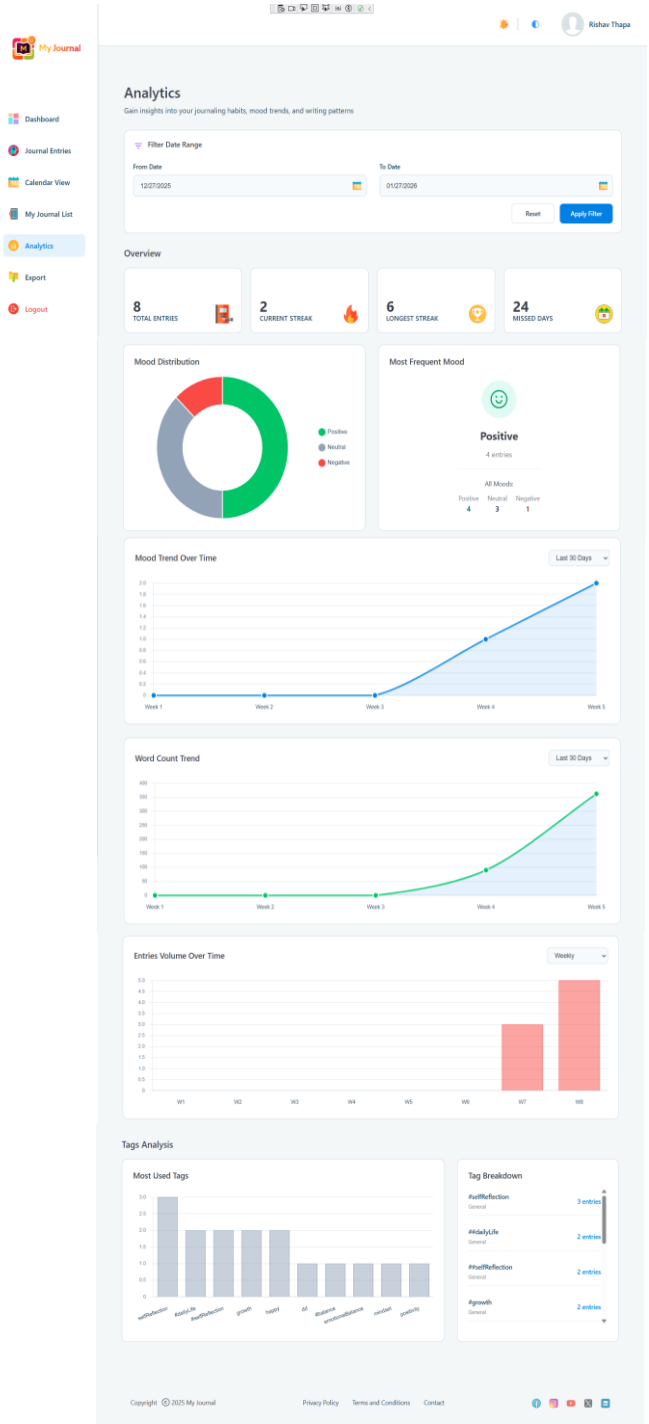


Figure 191: Full Analytics Page

- Analytics.razor

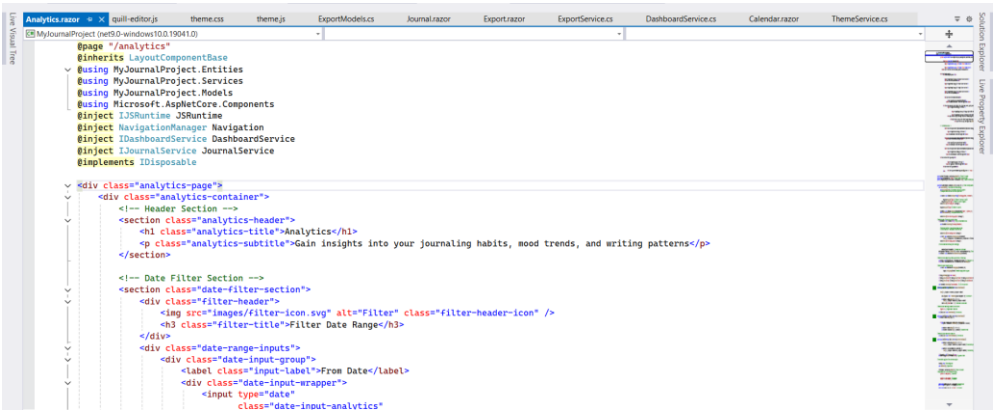


Figure 192: Analytics.razor

3.7.20. Test Case 20: Date Range Filtering in Analytics

Table 21: Test Case 20: Date Range Filtering in Analytics

Test Case 20: Date Range Filtering in Analytics	
Objective	To verify that date range filters work correctly on the Analytics page.
Action	i. Open the Journaling application. ii. Login with a valid user account. iii. Navigate to Analytics Page. iv. Set From Date to 01/21/2026. v. Set To Date to 01/27/2026. vi. Click the “Apply Filter” button. vii. Checks if the data and charts update to show only January 2026 data.
Expected Result	i. Date inputs accept valid dates. ii. Apply button triggers data refresh. iii. Statistics update to reflect filtered period. iv. Charts show only data from selected range.
Actual Result	i. Date filters applied correctly.

	ii. All statistics updated to show January 2026 data only. iii. Chart refreshed with filtered data.
Conclusion	Date range filtering works correctly, allowing users to analyse specific time periods with accurate and updated analytics.

- Analytics page with date range inputs visible and updated statistics section after filtering.

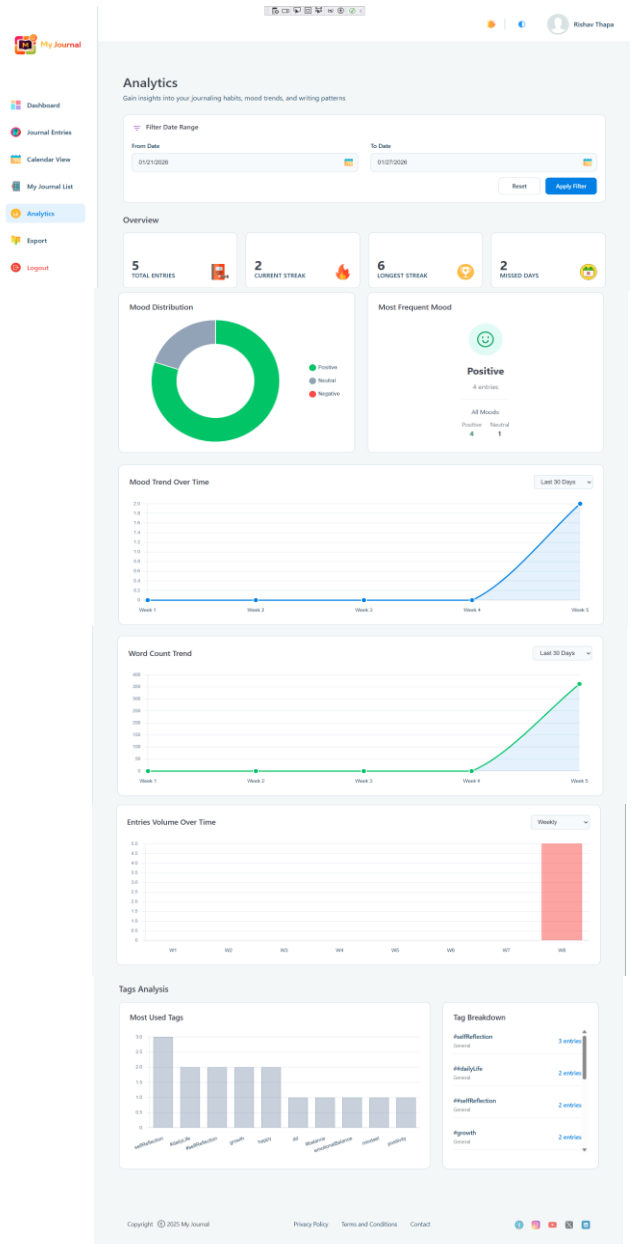


Figure 193: Analytics page with date range inputs visible

4. Individual Reflection

This project allowed me to take full responsibility for planning, developing, testing, and documenting the journaling application. I worked on designing the system structure and implementing core features such as journal management, user authentication, analytics, calendar view, filtering, pagination, and PDF export. I also conducted structured testing using defined test cases to ensure that all features worked correctly. This gave me a chance to experience the full development process from start to finish.

During development, I faced several challenges. Handling data validation, managing journal rules, implementing analytics calculations, and integrating multiple features like calendar view, dashboard filters, and export functionality was sometimes difficult. Debugging database issues, maintaining data consistency, and ensuring smooth interactions across different components required problem-solving skills and careful attention to detail. These challenges helped me learn how to approach and solve real-world coding problems efficiently.

Through this project, I learned how to turn theoretical knowledge into a complete, working system. I gained practical experience in CRUD operations, authentication, UI design, analytics processing, and integrating multiple features. I also improved my coding discipline, learned proper design thinking, and developed systematic testing practices by validating each feature with test cases. Overall, this project strengthened my technical skills, problem-solving abilities, and confidence to handle future software development work.

5. Conclusion

The main objective of this project was to design, develop, and test a fully functional journaling application that allows users to manage journals, track moods, view analytics, and export entries. The project successfully met this objective by implementing features such as journal creation, updating, deletion, calendar view, filtering, pagination, analytics charts, PDF export, and theme toggling.

The project demonstrates the importance of structured planning, careful design, and systematic testing. While the application performs all intended functions effectively, there were some limitations, such as not having full designed ERD and limited advanced analytics. Despite these limitations, the application provides a solid foundation for future improvements.

For future research and development, enhancements could include implementing more advanced analytics, integrating cloud-based storage, adding remainders or notifications, and improving mobile responsiveness. Overall, this project has provided valuable practical experience in software development, strengthened problem solving and coding skills, and resulted in a reliable journaling application ready for further growth and use.